FC Fluxo de Caixa

VOLUME ÚNICO

Documentação completa

Todos os entregáveis do desafio em um único volume

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

00 · Visão geral e guia de execução

Fluxo de Caixa 8

Documentação de entrega — Fluxo de Caixa 15

01 · Contexto e requisitos

01 — Contexto e requisitos 18

02 · Domínio e capacidades de negócio (DDD)

02 — Domínio (DDD) 29

03 · Arquitetura alvo, C4 e fluxos de dados

Arquitetura 39

Arquitetura-alvo 40

C4 — Nível 1: contexto 45

C4 — Nível 2: containers 46

C4 — Nível 3: componentes do lancamentos-service 48

C4 — Nível 3: componentes do consolidado-service 50

C4 — Nível 4: registrar lançamento 52

Fluxos de dados 54

Padrões de integração 60

Deployment na AWS 67

04 · Decisões arquiteturais (ADRs)

Decisões de arquitetura 73

Registrar decisões de arquitetura em ADR 75

Microsserviços por bounded context 77

CQRS e integração assíncrona orientada a eventos

Transactional outbox 81

SNS e SQS como mensageria 83

PostgreSQL no RDS, um banco por serviço 85

Java 25 e Spring Boot 4 87

Arquitetura hexagonal 89

ECS Fargate 91

Terraform como IaC 93

OAuth2/OIDC com Cognito e JWT validado no serviço

Cache Redis/ElastiCache no consolidado 97

Observabilidade com OpenTelemetry, ADOT, CloudWatch e X-Ray

Idempotência na escrita e no consumo 101

Lançamentos imutáveis, correção por estorno 103

Representação monetária 105

API REST versionada e problemas RFC 9457 107

Borda com ALB e AWS WAF 109

Região sa-east-1 e LGPD 111

Estratégia de deploy 113

Região us-east-1, com transferência internacional

Borda CloudFront, WAF e VPC Origin 119

Java 25, Spring Boot 4.0.9 e Resilience4j 2.4.0 122

Rotação de segredos 125

Rastreabilidade e propagação de contexto 128

ECS agora, EKS como evolução 130

Observabilidade: gerenciada agora, SaaS ou self-hosted depois

GitOps com ArgoCD, como evolução 135

Três ambientes e state remoto do Terraform 137

Réplica de leitura e estratégia de dados 140

API Gateway REST regional atrás do CloudFront 143

05 · Requisitos não funcionais

05 — Requisitos não funcionais 148

06 · Segurança

06 — Segurança 154

07 · Observabilidade e monitoramento

07 — Observabilidade 162

08 · Estimativa de custos

08 — Custos e licenças 172

09 · Arquitetura de transição

09 — Arquitetura de transição 183

10 · Evoluções futuras

10 — Evoluções futuras 189

11 · Estratégia de testes

11 — Testes 193

12 · Desempenho e dados

12 — Desempenho e dados 198

13 · ECS e EKS

13 — ECS agora, EKS quando couber 206

14 · CI/CD e qualidade

14 — CI/CD e qualidade 213

15 · Blueprint técnico

Blueprint Técnico de Referência — Fluxo de Caixa

16 · Serviços e ambiente local

lancamentos-service 232
Desempenho das consultas — lancamentos-service 238
consolidado-service 240
Desempenho das consultas do consolidado 247
Ambiente local 249
Carga do consolidado 251

17 · Infraestrutura como código (Terraform)

Terraform — fluxo de caixa na AWS 254
bootstrap 260
fluxo-caixa 262
network 264
kms 266
ecr 267
ecs-cluster 268
ecs-service 269
alb 272

edge 274

api-gateway 276

rds-postgres 278

elasticache 280

messaging 282

cognito 284

observability 286

PARTE 00

Visão geral e guia de execução

O que a solução faz, como rodar localmente e o mapa dos requisitos do desafio

FONTE: README.MD

Fluxo de Caixa

Sistema para um comerciante registrar créditos e débitos e consultar o saldo diário consolidado. A escrita do lançamento não depende do serviço de relatório: se o consolidado cair, o caixa continua aceitando movimento, e o saldo alcança o fato quando o consumidor volta.

O requisito que organiza a arquitetura é este: o controle de lançamentos permanece disponível se o consolidado diário cair; em pico, o consolidado atende **50 requisições por segundo** com **no máximo 5% de perda** (meta interna: 1%).

Documentos entregáveis (PDF)

A documentação final do desafio está em PDF, diagramada e com sumário, na pasta `documentos-entregaveis-desafio/`. Para ler tudo de uma vez, abra a documentação completa. Os documentos também estão separados por tema (requisitos, DDD, arquitetura e C4, ADRs, segurança, observabilidade, custos, transição, testes, entre outros).

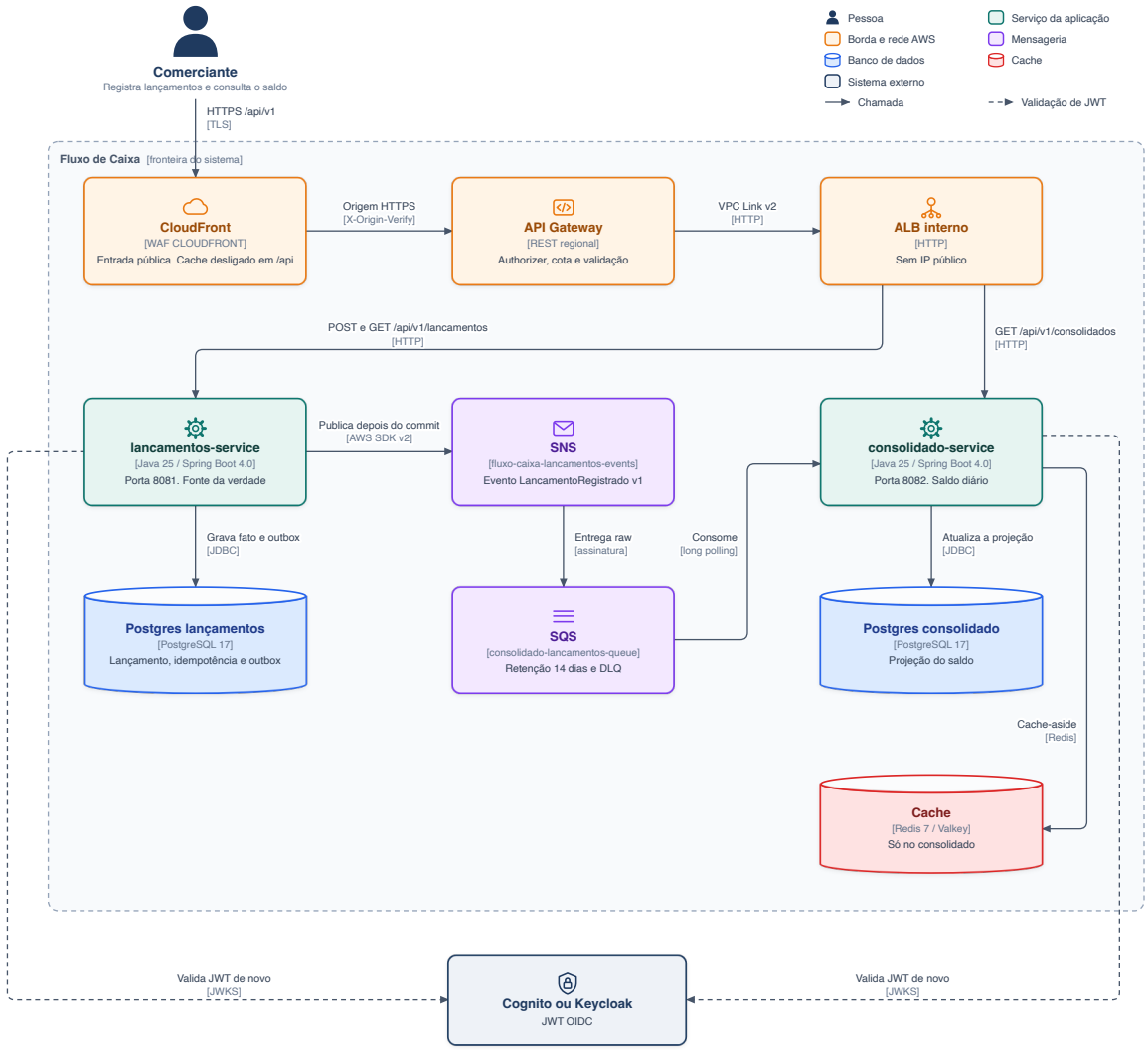
A pasta `documentos-brutos/` contém os **documentos brutos**: as fontes em Markdown, com os diagramas em `documentos-brutos/assets/diagramas/`, a partir das quais os PDFs são gerados (`make pdf`). Elas ficam versionadas junto com o código e prevalecem se houver divergência. A correspondência entre cada PDF e sua fonte está na seção Documentação.

Pasta	O que é	Para quem
`documentos-entregaveis-desafio/`	Entregáveis finais em PDF, com capa, sumário e diagramas	Avaliação e leitura
`documentos-brutos/`	Fontes em Markdown, ADRs, blueprint, Structurizr e diagramas	Manutenção e revisão no GitHub

Como o requisito é atendido

O `lancamentos-service` não chama o `consolidado-service`. O POST grava o lançamento e o evento na mesma transação (tabela `outbox_eventos`) e responde. Um relay publica no tópico cujo ARN vem de `LANCAMENTOS_TOPIC_ARN`. O consolidado consome a fila em `CONSOLIDADO_QUEUE_URL` (retenção de 14 dias, DLQ após 5 tentativas), aplica a soma uma única vez por `eventId` e serve a leitura por uma projeção em Postgres mais cache. No laboratório os nomes são `fluxo-caixa-lancamentos-events` e `consolidado-lancamentos-queue`. Na AWS o nome leva o prefixo do ambiente (`fluxo-caixa-dev-` ou `fluxo-caixa-prod-`); o serviço não tem nome de fila fixo no código. A conta de capacidade (Little: 2,8 requisições em voo no pico, duas tasks, cerca de 17% de utilização, 0,16 conexão de banco ativa) está em documentos-brutos/01-contexto-e-requisitos.md.

Fluxo de Caixa — containers



Stack

Peça	Escolha
Aplicação	Java 25, Spring Boot 4.0.9, Resilience4j 2.4.0, Maven wrapper, um projeto por serviço
Arquitetura	Dois microsserviços, hexagonal, CQRS, outbox, consumidor idempotente
Dados	PostgreSQL 17, um banco por serviço, Flyway. RDS Multi-AZ em produção. Réplica de leitura desligada até gatilho
Mensageria	SNS + SQS + DLQ. LocalStack no laboratório. AWS SDK v2. Trace propagado por atributo
Cache	Redis 7 local, Valkey no ElastiCache, só no consolidado
Segurança	OAuth2/OIDC. Cognito na AWS, Keycloak no local. JWT validado no serviço. Rotação de senha do RDS
Borda	CloudFront + WAF + API Gateway REST regional + VPC Link v2 + ALB interno, região `us-east-1` (VPC Origin direto como alternativa)
Execução	ECS Fargate (ARM). EKS é evolução, não o deploy de hoje

Peça	Escolha
Observabilidade	OpenTelemetry. ADOT, CloudWatch e X-Ray na AWS. Grafana LGTM no local
Infra	Terraform (provider AWS 6.x), ambientes `dev`, `hml` e `prod`, state em S3

Licença de runtime: nenhuma paga. A estimativa em `us-east-1`, com os três ambientes ligados o mês inteiro e o API Gateway REST ligado, fica em cerca de **USD 147/mês** (dev), **USD 359/mês** (hml) e **USD 778/mês** (prod). A mesma produção em `sa-east-1`, com o mesmo unitário de request da REST API (não reconsultado naquela região nesta passagem), sairia cerca de **USD 1.468/mês**. Local continua **USD 0**. Detalhe em documentos-brutos/08-custos.md.

Estrutura do repositório

```
fluxo-caixa/
├── README.md
├── Makefile
├── documentos-entregaveis-desafio/ # entregáveis finais em PDF
├── documentos-brutos/              # documentos brutos: fontes Markdown, ADRs e diagramas
├── services/
│   ├── lancamentos-service/        # porta 8081
│   └── consolidado-service/        # porta 8082
├── infra/
│   ├── terraform/                  # dev, hml e prod
│   └── local/                      # docker-compose
├── tests/
│   ├── load/                       # k6
│   └── e2e/                        # smoke.sh e resiliencia.sh
├── tools/pdf/                      # gerador dos PDFs (pandoc + Chrome)
└── .github/workflows/
```

O contrato compartilhado (paths, evento, nomes de fila, variáveis, portas) está em documentos-brutos/blueprint-tecnico.md. A narrativa de arquitetura começa em documentos-brutos/README.md.

Pré-requisitos

- Java 25 (Temurin).
- Docker com Compose v2.
- Make.
- Opcional: k6 (`make load`), Terraform e AWS CLI (deploy).

O Maven não precisa estar instalado: cada serviço traz `mvnw`.

Subir no local

Na raiz do repositório:

```
make infra-up
make up
```

`make infra-up` sobe a infraestrutura local. `make up` sobe os dois serviços.

Serviço	URL
Keycloak	http://localhost:8080 (realm `fluxo-caixa`)
Lançamentos	http://localhost:8081
Consolidado	http://localhost:8082
Swagger lançamentos	http://localhost:8081/swagger-ui.html
Swagger consolidado	http://localhost:8082/swagger-ui.html
OpenAPI	http://localhost:8081/v3/api-docs e http://localhost:8082/v3/api-docs
Grafana (otel-lgtm)	http://localhost:3000
Postgres	localhost:5432, bancos `lancamentos` e `consolidado`
Redis	localhost:6379
LocalStack	http://localhost:4566

Usuário de laboratório: `comerciante1` / `comerciante1`, com `comerciante_id` `11111111-1111-1111-1111-111111111111`. Quando o Grafana pedir senha, a padrão da imagem `grafana/otel-lgtm` é `admin` / `admin`.

Para derrubar: `make down`.

Token

```
TOKEN=$(make token)
```

`make token` imprime o access token do `comerciante1` no stdout (client `fluxo-caixa-app`, direct access grant no Keycloak local).

Registrar um crédito

```
curl -sS -X POST http://localhost:8081/api/v1/lancamentos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{"tipo":"CREDITO","valor":150.00,"dataLancamento":"2026-11-10","descricao":"Venda
      balcão","categoria":"VENDAS"}'
```

A primeira resposta é **201**. Repetir com a mesma `Idempotency-Key` e o mesmo corpo devolve **200** e o mesmo recurso, sem segundo lançamento. O `comercianteId` não vai no JSON: o serviço lê o claim do token.

Consultar o lançamento e o dia

```
curl -sS http://localhost:8081/api/v1/lancamentos/COLE_O_ID \
  -H "Authorization: Bearer $TOKEN"

curl -sS "http://localhost:8081/api/v1/lancamentos?data=2026-11-10&page=0&size=50" \
  -H "Authorization: Bearer $TOKEN"
```

Estornar

```
curl -sS -X POST http://localhost:8081/api/v1/lancamentos/COLE_O_ID/estornos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"motivo":"Valor digitado errado"}'
```

O estorno é um lançamento novo, de tipo oposto, na mesma data de competência. O original não muda.

Saldo consolidado

O saldo é eventual (meta p95 abaixo de 5 s). Se o GET vier zerado logo depois do POST, espere um instante e repita.

```
curl -sS http://localhost:8082/api/v1/consolidados/diario/2026-11-10 \
  -H "Authorization: Bearer $TOKEN"

curl -sS "http://localhost:8082/api/v1/consolidados/diario?inicio=2026-11-01&fim=2026-11-10" \
  -H "Authorization: Bearer $TOKEN"
```

Dia sem movimento responde **200** com zeros. Período maior que 366 dias responde **400** `application/problem+json`.

Escopos exigidos: `lancamentos:write`, `lancamentos:read`, `consolidado:read`. No local o prefixo é vazio. Na AWS a autoridade leva o prefixo `fluxo-caixa/`.

Testes

```
make test          # unidade, ArchUnit e integração (Testcontainers)
make smoke         # jornada ponta a ponta
make resiliencia   # consolidado parado, POST segue, fila recupera
make load          # k6, opcional
make tf-validate   # Terraform fmt e validate
```

O que cada camada cobre está em documentos-brutos/11-testes.md.

Deploy na AWS

Produção em `us-east-1`: CloudFront com WAF, API Gateway REST regional com WAF próprio e VPC Link v2 até um ALB interno, VPC em três zonas, ECS Fargate com no mínimo duas tasks por serviço, RDS PostgreSQL separado e Multi-AZ para cada serviço, ElastiCache com réplica, SNS/SQS/DLQ, Cognito, Secrets Manager com rotação e ADOT. Há também `dev` e `hml`, menores. `enable_api_gateway = false` tira o gateway e volta o CloudFront à VPC Origin. O passo a passo de apply, state e variáveis fica em infra/terraform/README.md. A topologia e o DR estão em documentos-brutos/03-arquitetura/deployment-aws.md. O deploy da aplicação é rolling com circuit breaker e rollback (ADR 0020), promovendo a mesma imagem de dev para hml e prod.

O Cognito do ambiente é um user pool Essentials com domínio hospedado `fluxo-caixa-<env>-<account_id>` e app client público. O Terraform não cria usuário. O pool só aceita usuário criado por admin, e a senha do laboratório local não passa na policy (12 caracteres, com maiúscula, minúscula e número). O atributo developer-only entra como `dev:custom:comerciante_id`, um UUID de 36 caracteres. A Lambda de pre-token copia esse valor para o claim e acrescenta os escopos `fluxo-caixa/lancamentos:write`, `fluxo-caixa/lancamentos:read` e `fluxo-caixa/consolidado:read`, então o `USER_PASSWORD_AUTH` passa no authorizer.

```
cd infra/terraform/envs/dev
POOL=$(terraform output -raw cognito_user_pool_id)
CLIENT=$(terraform output -raw cognito_user_pool_client_id)
CF=$(terraform output -raw cloudfront_domain_name)

aws cognito-idp admin-create-user \
  --user-pool-id "$POOL" --username comerciante1 --message-action SUPPRESS \
  --user-attributes Name=dev:custom:comerciante_id,Value=11111111-1111-1111-1111-111111111111
aws cognito-idp admin-set-user-password \
  --user-pool-id "$POOL" --username comerciante1 --password 'SENHA_QUE_PASSA_NA_POLICY' --
        permanent

TOKEN=$(aws cognito-idp initiate-auth \
  --auth-flow USER_PASSWORD_AUTH --client-id "$CLIENT" \
  --auth-parameters USERNAME=comerciante1,PASSWORD='SENHA_QUE_PASSA_NA_POLICY' \
  --query AuthenticationResult.AccessToken --output text)

curl -sS -H "Authorization: Bearer $TOKEN" \
  "https://${CF}/api/v1/consolidados/diario/$(date +%F)"
```

A URL pública é o CloudFront. Sem token a borda responde 401. O `execute-api` direto, sem o header `X-Origin-Verify`, responde 403. O primeiro apply pode usar `image_tag = "bootstrap"` para criar os repositórios; a task só fica de pé depois do push da imagem ARM64 e de um apply com essa tag. O ECR é imutável.

Documentação

Cada tema tem o PDF entregável, em `documentos-entregaveis-desafio/`, e o documento bruto em Markdown que o origina, em `documentos-brutos/`. O volume único com tudo é a documentação completa em PDF.

Tema	PDF entregável	Documento bruto	Conteúdo
Visão geral	00	README e índice	Solução, execução local e mapa dos requisitos
Requisitos	01	01-contexto-e-requisitos.md	RF, RNF e capacidade
Domínio (DDD)	02	02-dominio-ddd.md	Capacidades, context map, evento
Arquitetura e C4	03	03-arquitetura/	C4, fluxos, integração, AWS, Structurizr
ADRs	04	04-adr/	Decisões de 0001 a 0031
RNF operacionais	05	05-requisitos-nao-funcionais.md	SLO, error budget, k6
Segurança	06	06-seguranca.md	STRIDE, OWASP API, LGPD, rotação
Observabilidade	07	07-observabilidade.md	Trace ponta a ponta, alarmes, runbooks
Custos	08	08-custos.md	USD e BRL, dev, hml e prod
Transição	09	09-arquitetura-de-transicao.md	Strangler a partir do legado
Evoluções	10	10-evolucoes-futuras.md	O que ficou de fora e por quê
Testes	11	11-testes.md	Pirâmide e comandos
Desempenho	12	12-desempenho-e-dados.md	Índices, réplica, cache, lake
ECS e EKS	13	13-ecs-vs-eks.md	Critério, stack e preço de observar
CI/CD	14	14-cicd-e-qualidade.md	Pipeline e ferramentas
Blueprint	15	blueprint-tecnico.md	Contrato: nomes, portas, evento, SLOs
Serviços e local	16	READMEs de lançamentos, consolidado e infra local	Guia dos serviços e laboratório
Terraform	17	infra/terraform/README.md e módulos	Stack, ambientes e módulos

Depois de alterar um documento bruto, rode `make pdf` para atualizar os PDFs.

Saúde dos processos

O ALB usa readiness. No local:

- http://localhost:8081/actuator/health/liveness
- http://localhost:8081/actuator/health/readiness
- http://localhost:8082/actuator/health/readiness

Métrica Prometheus: `/actuator/prometheus` em cada serviço (não publicada na internet em produção).

FONTE: DOCUMENTOS-BRUTOS/README.MD

Documentação de entrega — Fluxo de Caixa

Esta pasta guarda os **documentos brutos** (fontes em Markdown). A versão final em PDF, diagramada, está em `documentos-entregaveis-desafio/` e é gerada a partir daqui com `make pdf`.

Índice da documentação de arquitetura do desafio de Arquiteto de Soluções. O contrato executável (nomes, portas, eventos, recursos AWS e metas de SLO) está no blueprint técnico. Esta pasta explica o porquê, os trade-offs e como operar a solução. Em caso de divergência de contrato, vale o blueprint.

Documento	O que responde
01 — Contexto e requisitos	Negócio, glossário, RF com aceite, RNF mensuráveis, cálculo de capacidade do pico de 50 req/s
02 — Domínio (DDD)	Capacidades, subdomínios, context map, event storming, agregados, modelos de dados, contrato do evento
03 — Arquitetura	C4, Structurizr, estilos, fluxos e implantação AWS
04 — ADRs	Decisões no formato MADR, de 0001 a 0031, datadas em 2026-11-10
05 — Requisitos não funcionais	SLI/SLO, error budget, escala, resiliência, plano k6, rastreabilidade
06 — Segurança	STRIDE, OAuth2, multi-tenant, OWASP API, criptografia, LGPD, rotação de segredos
07 — Observabilidade	Trace W3C ponta a ponta, logs, RED/USE, alarmes, runbooks, Grafana e X-Ray
08 — Custos	dev, hml e prod em `us-east-1`, a mesma conta em `sa-east-1`, EKS e SaaS
09 — Arquitetura de transição	Strangler Fig a partir de planilha ou ERP monolítico
10 — Evoluções futuras	Roadmap: multi-moeda, fechamento, Open Finance, multi-região, Kafka
11 — Testes	Pirâmide, como executar smoke, resiliência e carga
12 — Desempenho e dados	Índices, keyset, réplica, cache, Aurora, DynamoDB, lake
13 — ECS e EKS	Quando o EKS passa a valer, stack open source, Karpenter, preço de observar
14 — CI/CD e qualidade	Pipeline, ferramentas e o que muda com o EKS

Como ler se o tempo for curto

1. README da raiz — como subir e o que a solução faz.
2. Arquitetura-alvo — estilos e o isolamento do RNF crítico.
3. ADR 0002, ADR 0003 e ADR 0004 — o núcleo da decisão.
4. Custos — o número mensal e o que realmente o compõe (NAT, endpoints, RDS), em `us-east-1`.

Mapa dos requisitos do desafio

Exigência	Onde está atendida
Mapeamento de domínios e capacidades	02
Refinamento de requisitos funcionais e não funcionais	01, 05
Desenho da solução (arquitetura-alvo)	03
Justificativa de ferramentas, tecnologias e estilo	04, 03 — arquitetura-alvo
Testes	11, `tests/e2e`, `tests/load`
README com execução local	README da raiz
Documentação no repositório	esta pasta
Arquitetura de transição (diferencial)	09
Estimativa de custos e licenças (diferencial)	08
Monitoramento e observabilidade (diferencial)	07
Critérios de segurança para integração (diferencial)	06
RNF crítico: lançamentos independe do consolidado; 50 req/s com perda ≤ 5%	01 § análise de capacidade, 05

PARTE 01

Contexto e requisitos

Requisitos funcionais e não funcionais refinados e análise de capacidade do pico

FONTE: DOCUMENTOS-BRUTOS/01-CONTEXTO-E-REQUISITOS.MD

01 — Contexto e requisitos

1. Problema de negócio

Um comerciante precisa controlar o caixa do dia: cada entrada é um crédito, cada saída é um débito, e no fim do dia (ou a qualquer momento) ele precisa ver o saldo daquele dia e o saldo acumulado até a data. O valor para o negócio está em duas capacidades distintas:

1. **Não perder um lançamento.** O registro do movimento é o fato contábil. Se o relatório cair, a venda ainda tem de ser lançada.
2. **Enxergar o caixa consolidado.** O saldo diário é uma leitura derivada. Pode atrasar alguns segundos. Não pode contradizer, no estado estável, a soma dos lançamentos.

O enunciado separa isso em dois serviços e fixa o requisito que orienta a arquitetura: o controle de lançamentos permanece disponível se o consolidado diário cair; em pico, o consolidado atende **50 requisições por segundo** com **no máximo 5% de perda**.

2. Premissas

Premissas explícitas, porque o enunciado não fixa volume de escrita, número de comerciantes nem moeda.

ID	Premissa	Uso
P1	Moeda única: BRL, duas casas decimais.	Modelo `Dinheiro` e evento
P2	Um access token representa exatamente um comerciante. O cadastro do comerciante vive no provedor de identidade, não neste sistema.	Multi-tenant por claim
P3	`dataLancamento` é data de competência (`LocalDate`), informada pelo cliente. Pode ser retroativa. `criadoEm` é o instante do registro (UTC).	Estorno corrige o dia original
P4	Pico de leitura do consolidado = 50 req/s, sustentado pelo menos durante a janela de fechamento.	Dimensionamento
P5	Pico de escrita = 20 lançamentos/s. O enunciado não define esse número. Vinte por segundo cobre ~72 mil lançamentos numa hora de pico e é a premissa de outbox, SQS e banco de escrita. Abaixo disso, a folga aumenta.	Capacidade da escrita
P6	Hit ratio de cache no pico ≥ 80%. O dia corrente e a semana recente concentram as consultas.	Pool JDBC e SLO de latência
P7	Há um operador (ou integração) por comerciante. Não há, nesta versão, papel de contador enxergando vários comerciantes.	Fora de escopo de autorização
P8	Consistência do saldo é eventual, com meta p95 < 5 s entre o commit do lançamento e o reflexo na leitura.	SLO do blueprint

ID	Premissa	Uso
P9	Ambiente diário de desenvolvimento é local (Docker). A conta AWS tem `dev`, `hml` e `prod`; dev não é o lugar onde se codifica o dia.	Custo, ADR 0029
P10	Região: `us-east-1`. Transferência internacional com salvaguarda (cláusulas-padrão e DPA da AWS). Residência no Brasil volta a ser requisito só por contrato ou norma, com o custo do documento 08.	LGPD art. 33, ADR 0021

3. Glossário (linguagem ubíqua)

Termo	Significado neste sistema
Comerciante	Titular do caixa. Identificado pelo claim `comerciante_id` (UUID). Nunca é aceito no corpo ou na URL.
Lançamento	Fato imutável de crédito ou débito. Uma vez persistido, não se altera nem se apaga.
Crédito	Entrada de dinheiro. Aumenta o saldo do dia de competência.
Débito	Saída de dinheiro. Reduz o saldo do dia de competência.
Valor	Quantia positiva, em BRL, com no máximo duas casas. Zero e negativo são rejeitados.
Data de competência	Dia a que o movimento pertence (`dataLancamento`). É a chave do saldo diário.
Data de registro	Instante em que o sistema aceitou o lançamento (`criadoEm`).
Estorno	Novo lançamento, de tipo oposto, mesmo valor e mesma data de competência, apontando para o lançamento original. É a única correção admitida.
Saldo diário	`totalCreditos − totalDebitos` de um comerciante em uma data de competência.
Saldo acumulado	Soma dos saldos diários do comerciante em todas as datas `≤` a data consultada.
Consolidação	Aplicação do evento `LancamentoRegistrado` na projeção de saldo. Comutativa: a ordem de chegada não altera o resultado.
Consistência eventual	A leitura do consolidado pode não conter um lançamento recém-aceito. O atraso alvo é p95 < 5 s. O lançamento em si já é durável.
Chave de idempotência	Header `Idempotency-Key` (UUID) no POST. A mesma chave, do mesmo comerciante, com o mesmo corpo, devolve o mesmo recurso.
Outbox	Tabela `outbox_eventos` gravada na mesma transação do lançamento. O relay publica no SNS depois do commit.
Perda de requisição	Resposta de erro (5xx ou timeout visto pelo cliente) sobre o total de requisições HTTP ao consolidado. Não significa descarte de lançamento.

4. Requisitos funcionais

Critérios no formato Given/When/Then. O `comercianteId` dos exemplos é o do realm local: `11111111-1111-1111-1111-111111111111`.

RF-01 — Registrar crédito

Dado um token com escopo `lancamentos:write` e claim `comerciante_id`
Quando `POST /api/v1/lancamentos` com `tipo=CREDITO`, `valor=150.00`, `dataLancamento` válida, `descricao` de 1 a 255 caracteres e `categoria` opcional até 60
Então o sistema persiste o lançamento e o evento na mesma transação, responde **201** com o recurso (sem `comercianteId` vindo do cliente) e o saldo daquele dia passa a incluir 150,00 depois da consolidação.

RF-02 — Registrar débito

Dado o mesmo contexto de escrita
Quando o tipo é `DEBITO`
Então o comportamento é o do RF-01, e a consolidação soma o valor em `totalDebitos`.

RF-03 — Rejeitar valor inválido

Dado um token de escrita
Quando `valor` é zero, negativo, tem mais de duas casas, ou não é numérico; ou `descricao` é vazia ou maior que 255; ou `tipo` não é `CREDITO` nem `DEBITO`
Então nada é persistido, a resposta é **400** `application/problem+json` (RFC 9457) e nenhum evento é publicado.

RF-04 — Idempotência do POST

Dado um `Idempotency-Key` UUID ainda não usado por aquele comerciante
Quando o mesmo POST é repetido com a mesma chave e o mesmo corpo
Então existe um único lançamento; a primeira resposta é **201** e a repetição é **200** com o mesmo corpo.

Dado a mesma chave
Quando o corpo é diferente
Então a resposta é **409** e o lançamento original permanece.

Sem o header, cada POST é um lançamento novo. A chave é opcional, como no blueprint.

RF-05 — Consultar um lançamento

Dado um token com `lancamentos:read`
Quando `GET /api/v1/lancamentos/{id}` e o id pertence ao comerciante do token
Então a resposta é **200** com tipo, valor, moeda `BRL`, data, descrição, categoria, `lancamentoEstornadoId` e `criadoEm`.

Quando o id não existe ou pertence a outro comerciante
Então a resposta é **404** nos dois casos (não se revela existência cruzada).

RF-06 — Listar lançamentos do dia

Dado um token de leitura
Quando `GET /api/v1/lancamentos?data=YYYY-MM-DD&page=0&size=50`
Então a resposta traz somente lançamentos daquele comerciante e daquela data de competência, paginados. `size` máximo efetivo: 50.

RF-07 — Estornar

Dado um lançamento do comerciante que ainda não foi estornado e que não é ele próprio um estorno
Quando `POST /api/v1/lancamentos/{id}/estornos` com escopo `lancamentos:write` e corpo opcional `{ "motivo": "..." }`
Então nasce um novo lançamento de tipo oposto, mesmo valor, mesma `dataLancamento`, `lancamentoEstornadoId` preenchido, e um evento `LancamentoRegistrado` v1. O original não é alterado.

Lacuna do blueprint, resolvida aqui e registrada no ADR 0015: o agregado não tem campo `motivo`, e o POST de estorno não envia `descricao`. A descrição do novo lançamento é o `motivo` (até 255 caracteres) ou, na ausência, `Estorno do lançamento {id}`. O motivo não entra no evento — o consolidado só precisa do tipo oposto, do valor e da data.

RF-08 — Estorno único

Dado um lançamento já estornado
Quando chega outro POST de estorno
Então a resposta é **409** e não nasce segundo estorno.

RF-09 — Estorno de estorno é proibido

Dado um lançamento cujo `lancamentoEstornadoId` não é nulo
Quando chega POST de estorno para ele
Então a resposta é **409**. A correção de um estorno errado é um lançamento novo, deliberado, não um estorno do estorno.

RF-10 — Lançamento sobrevive à queda do consolidado

Dado o `consolidado-service` indisponível, a fila acumulando e o banco do consolidado fora
Quando o comerciante registra um lançamento
Então o POST responde 201 no tempo do SLO de escrita, o registro está commitado em `lancamentos` e em `outbox_eventos`, e nenhuma chamada síncrona ao consolidado ocorre.

RF-11 — Saldo do dia

Dado um token com `consolidado:read`
Quando `GET /api/v1/consolidados/diario/{data}`
Então a resposta é **200** com `data`, `totalCreditos`, `totalDebitos`, `saldo`, `saldoAcumulado`, `quantidadeLancamentos` e `atualizadoEm`.

RF-12 — Dia sem movimento

Dado nenhuma projeção para aquela data
Quando o GET do dia é feito
Então a resposta é **200** com zeros, não 404. `saldoAcumulado` continua sendo a soma dos dias anteriores.

RF-13 — Relatório de período

Dado `inicio` e `fim` com `fim − inicio ≤ 366 dias`
Quando `GET /api/v1/consolidados/diario?inicio=&fim=`
Então a resposta lista cada dia do intervalo (inclusive dias zerados) e o saldo acumulado coerente com a

soma.

RF-14 — Período excessivo

Quando o intervalo supera 366 dias ou `inicio` > `fim`
Então a resposta é **400** e o banco não executa a consulta larga.

RF-15 — Isolamento entre comerciantes

Dado dois tokens com `comerciante_id` distintos
Quando um consulta id, lista, saldo ou estorno do outro
Então o resultado é o mesmo de "não existe": 404 no recurso, lista vazia, saldo zerado. Não há parâmetro de tenant na URL.

RF-16 — Escopos

Quando falta o escopo da operação (`lancamentos:write` , `lancamentos:read` ou `consolidado:read`)
Então a resposta é **403**. Token ausente ou inválido: **401**.

Na AWS a autoridade é `SCOPE_fluxo-caixa/lancamentos:write` (prefixo `fluxo-caixa/`). No Keycloak local o prefixo é vazio: `SCOPE_lancamentos:write` .

RF-17 — Consolidação do evento

Dado um `LancamentoRegistrado` v1 novo
Quando o consumidor processa
Então `totalCreditos` ou `totalDebitos` , `saldo` , `quantidadeLancamentos` e `atualizadoEm` mudam na linha `(comercianteId, dataLancamento)` , e `eventos_processados` guarda o `eventId` na mesma transação.

RF-18 — Estorno corrige o dia original

Dado um crédito de 150,00 em 2026-11-10 já consolidado
Quando o estorno é consolidado
Então o dia 2026-11-10 recebe um débito de 150,00. O saldo daquele dia volta ao valor anterior. Dias posteriores, no acumulado, acompanham essa correção. O dia "de hoje" não absorve o estorno se a competência era outra.

RF-19 — Reprocessamento não duplica saldo

Dado um `eventId` já presente em `eventos_processados`
Quando o SQS reentrega a mensagem (at-least-once)
Então o saldo não muda e a mensagem é removida da fila.

RF-20 — Erro no formato problema

Quando qualquer rejeição da API ocorre
Então o corpo é `application/problem+json` com `type` , `title` , `status` , `detail` e `instance` . Falha de domínio não vaza stack trace.

5. Requisitos não funcionais

Metas herdadas do blueprint e tornadas mensuráveis. O detalhamento de SLI, error budget e testes está no documento 05.

ID	Atributo	Meta	Como se mede
RNF-01	Disponibilidade de lançamentos	99,9% ao mês, com o consolidado no ar ou caído	Probes de readiness no ALB; minutos de 5xx/timeout no POST e GET de lançamentos
RNF-02	Disponibilidade do consolidado	99,5% ao mês	Idem para as rotas `/api/v1/consolidados*`
RNF-03	Latência de escrita	p95 < 200 ms, p99 < 500 ms	Histograma HTTP do POST, server-side
RNF-04	Latência de leitura	p95 < 100 ms com cache; p99 < 300 ms	Histograma do GET diário
RNF-05	Throughput e perda	≥ 50 req/s sustentados no consolidado; erro ≤ 5%; meta interna ≤ 1%	k6 `http_req_failed` e `http_reqs`
RNF-06	Atraso de consolidação	p95 < 5 s do commit até o saldo refletir	Diferença `atualizadoEm` / timestamp do evento vs. `criadoEm`
RNF-07	Durabilidade do lançamento	Zero perda após 201	Commit único lançamento+outbox; PITR do RDS
RNF-08	Retenção do buffer	Evento espera a volta do consumidor por até 14 dias	Retenção da fila `consolidado-lancamentos-queue`
RNF-09	RPO / RTO	RPO ≤ 5 min; RTO ≤ 1 h	PITR do RDS; runbook de restauração
RNF-10	Isolamento de falha	Falha do consolidado não aumenta o erro de lançamentos	Teste `tests/e2e/resiliencia.sh`
RNF-11	Segurança	OAuth2/OIDC, escopo, tenant no token, TLS, criptografia em repouso	06
RNF-12	Observabilidade	Trace único atravessando HTTP, outbox e SQS; alarme de DLQ e de lag	07
RNF-13	Manutenibilidade	Domínio sem framework; ArchUnit; API `/api/v1`; evento com `eventVersion`	ADR 0008, ADR 0017
RNF-14	Custo	Produção enxuta para o volume real, região `us-east-1`	08: cerca de USD 778/mês em prod
RNF-15	LGPD	Transferência internacional com salvaguarda, minimização, trilha imutável	ADR 0021
RNF-16	Escalabilidade	Horizontal, no mínimo 2 tasks em AZs distintas, auto scaling por CPU e requisições	05

Error budget mensal (30 dias = 43.200 min):

- Lançamentos, 0,1% → **43,2 minutos**.
- Consolidado, 0,5% → **216 minutos**.

"Perda de 5%" não é o error budget mensal. É o teto de erro **durante o pico de 50 req/s**. São coisas diferentes: o serviço pode estar dentro do SLO mensal e ainda assim reprovar o pico, ou o inverso.

6. Restrições

- Dois processos de deploy independentes: `lancamentos-service` (porta 8081) e `consolidado-service` (porta 8082).
- PostgreSQL 17, um banco por serviço. Em produção, duas instâncias RDS. Migração por Flyway.
- Integração somente pelo evento `LancamentoRegistrado` versão 1, tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ `consolidado-lancamentos-dlq` (`maxReceiveCount` 5).
- Java 25 e Spring Boot 4. Cliente AWS SDK v2, sem Spring Cloud AWS.
- JWT validado em cada serviço. Cognito na AWS, Keycloak no local.
- Sem update e sem delete de lançamento.
- `valor` no evento é string decimal com duas casas.

7. Fora de escopo desta entrega

- Interface de usuário. O contrato é a API HTTP.
- Cadastro, convite e recuperação de senha do comerciante (ficam no Cognito/Keycloak).
- Múltiplas moedas, contas bancárias, PIX, boleto, conciliação e Open Finance.
- Fechamento de período, trava contábil e reabertura.
- Papel multi-comerciante (contador, franqueadora).
- Event sourcing como persistência do lançamento. O estado atual é uma linha; o evento é de integração.
- Active-active em duas regiões.
- BFF e API Gateway. A borda desta versão é CloudFront, WAF e ALB interno (ADR 0022).

Esses itens estão priorizados em evoluções futuras.

8. Análise do RNF crítico

Há duas obrigações, e elas não se resolvem com o mesmo mecanismo.

Obrigações. (1) Disponibilidade de lançamentos independente do consolidado. (2) O consolidado absorve 50 req/s com no máximo 5% de respostas perdidas.

Interpretação de "perda". O enunciado fala em perda de *requisições* ao serviço de consolidado. Isso é taxa de erro HTTP (5xx, timeout, conexão recusada) ≤ 5% no pico. Meta interna do blueprint: ≤ 1%. Lançamento aceito com 201 não é "perdido" se o saldo atrasar: ele está no Postgres e o evento espera na fila por até 14 dias. Tratar atraso de projeção como perda contábil seria o desenho errado.

8.1 Por que o lançamento não cai junto

Não existe chamada síncrona entre os serviços. O POST grava lançamento e outbox na mesma transação local e responde. O relay publica no SNS em outro ciclo. Se o SNS estiver fora, o 201 já foi dado e a linha permanece com `publicado_em` nulo. Se o consolidado, o Redis ou o RDS dele caírem, a fila segura a mensagem. Bancos, tasks ECS, target groups e auto scaling são separados. O teste que evidencia isso é `make resiliencia`.

O custo desse isolamento é consistência eventual e um componente a mais (relay + fila). O custo de não isolá-lo seria o caixa parar toda vez que o relatório falha — exatamente o que o enunciado proíbe.

8.2 Lei de Little no caminho de leitura

Notação: λ taxa de chegada, S tempo médio de serviço dentro do `consolidado-service`, L = λ × S número médio de requisições em voo.

Símbolo	Valor	Origem
λ	50 s^{-1}	RNF do enunciado
S com cache (S_hit)	0,040 s	JWT com JWKS em cache + GET no Redis + serialização. Premissa a confirmar no k6
S sem cache (S_miss)	0,120 s	Miss + duas queries indexadas (linha do dia e `SUM` do acumulado) + SET
Hit ratio h	0,80	Premissa P6

Tempo médio:

```
S = 0,80 × 0,040 + 0,20 × 0,120 = 0,056 s
L = 50 × 0,056 = 2,8 requisições em voo
```

Um burst de 3× (150 req/s) por alguns segundos, com S estável:

```
L_burst = 150 × 0,056 = 8,4 requisições em voo
```

O caminho é limitado por I/O, não por thread. Com virtual threads (Java 25), um pool de plataforma de 200 threads não é o teto. O risco de perder requisição está em task de menos durante um scale-out lento, em stampede de cache, em rate limit do WAF calibrado abaixo do pico e em timeout — não em “acabou a thread”.

8.3 Quantas tasks

Meta de capacidade, a ser comprovada no k6 antes de virar número de produção: **uma task de 1 vCPU / 2 GiB** (aplicação + sidecar ADOT) sustenta **150 req/s** com erro < 1% e p95 < 100 ms quando h ≥ 0,80.

Utilização ρ = λ / (n × μ), com μ = 150 req/s por task:

Situação	n	ρ
Pico nominal, mínimo de HA	2	50 / 300 = **17%**
Uma AZ ou uma task perdida (N−1)	1	50 / 150 = **33%**

Situação	n	ρ
Teto do auto scaling do consolidado	10	50 / 1.500 = 3% no pico de requisito. O Terraform para em 10 tasks, não em 6

Com ρ = 0,33 e variação moderada, a fila interna ainda não está no joelho (o crescimento áspero de latência aparece perto de 70%). Duas tasks em AZs distintas atendem o requisito com folga de N−1. O mínimo de duas tasks existe por disponibilidade, não porque uma task seja incapaz de 50 req/s.

Auto scaling: desired 2, mínimo 2, máximo 6, por CPU (alvo 60%) e `ALBRequestCountPerTarget`. O scale-out precisa começar antes de ρ = 0,70. Como o pico de 50 req/s já nasce confortável em 2 tasks, o scaling cobre crescimento e a perda de uma task, não o requisito nominal.

8.4 Conexões de banco

No miss há duas sentenças: a linha `(comerciante_id, data)` e o `SUM(saldo)` até a data. Índice único em `(comerciante_id, data)`. Um ano de caixa é ~365 linhas por comerciante; o `SUM` é um range scan curto. Por isso a projeção **não** materializa saldo acumulado em todas as linhas futuras — um estorno antigo reescreveria o ano inteiro.

```
λ_stmt = 50 × (1 − 0,80) × 2 = 20 sentenças/s
S_stmt = 0,008 s
L_db   = 20 × 0,008 = 0,16 conexões ativas
```

Pior caso desenhado (cache inutilizado, h = 0, query de acumulado em 20 ms, ainda em 2 tasks):

```
λ_stmt = 50 × 2 = 100 sentenças/s
L_db   = 100 × 0,020 = 2 conexões ativas
```

Pool Hikari por task: `minimumIdle = 2`, `maximumPoolSize = 10`.

Tasks do consolidado	Conexões reservadas
2 (mínimo)	20
6 (teto)	60

`max_connections` aproximado do RDS, fórmula `DBInstanceClassMemory / 9.531.392`:

Classe	RAM	Conexões típicas
`db.t4g.micro`	1 GiB	~100
`db.t4g.small`	2 GiB	~225
`db.t4g.medium`	4 GiB	~450

Sessenta conexões cabem num `db.t4g.small` com margem para o operador, o Flyway e o monitoramento. A classe medium seria folga de memória, não exigência do pico. A estimativa de custo usa **small Multi-AZ** por esse cálculo; medium está na sensibilidade do documento de custos.

O mesmo raciocínio na escrita, com a premissa P5:

```
λ = 20 s⁻¹
S_tx = 0,030 s   (INSERT do lançamento + INSERT da outbox + commit)
L = 20 × 0,030 = 0,6 conexões ativas
```

Pool de 10 em 2 tasks de lançamentos reserva 20 conexões. O banco de escrita também cabe em `db.t4g.small`.

8.5 Outbox, fila e a hora em que o consolidado fica fora

Relay: a cada 500 ms, lote de 100, `SELECT … FOR UPDATE SKIP LOCKED`. Uma réplica publica até 100 / 0,5 = **200 eventos/s**. Duas réplicas, cerca de 400 eventos/s, vinte vezes o pico de escrita da premissa P5. O `SKIP LOCKED` é o que permite duas tasks sem publicar o mesmo registro.

Se o consolidado ficar 1 hora fora, no pico de escrita:

```
20 evt/s × 3.600 s = 72.000 mensagens na fila
```

SQS Standard absorve isso sem teto prático nessa ordem de grandeza. Na volta, se cada task aplica ~100 eventos/s, duas tasks drenam 72 mil mensagens em **6 minutos**. A retenção de 14 dias, no mesmo pico sustentado (um teto irreal, útil só como limite), guarda 20 × 86.400 × 14 ≈ 24 milhões de mensagens. O alarme de idade da mensagem mais antiga dispara muito antes disso (ver observabilidade).

Orçamento de erro HTTP no pico, para não confundir com a fila:

```
50 × 0,05 = 2,5 respostas com falha por segundo  (teto do enunciado)
50 × 0,01 = 0,5 respostas com falha por segundo  (meta interna)
```

Numa hora de pico o enunciado admitiria 9.000 erros; a meta interna, 1.800. O threshold do k6 trava a meta interna (`http_req_failed: rate<0.01`), não o teto frouxo de 5%.

8.6 O que faria a solução reprovar o RNF

- Chamar o consolidado no meio do POST (acoplamento síncrono).
- Dual-write: commit no Postgres e publish no SNS sem outbox. Falha entre os dois perde o evento ou exige 201 sem garantia.
- Uma única task, ou as duas tasks agendadas na mesma AZ.
- Rate-based rule do WAF abaixo de 50 req/s por cliente legítimo que concentre o fechamento.
- Pool JDBC grande o bastante para derrubar o `max_connections` num stampede, se alguém “proteger” o banco abrindo 100 conexões por task.
- Tratar fila cheia como erro do POST. A fila cheia é problema do consumidor; o POST já terminou.

9. Consequência para o desenho

A arquitetura que cai fora dessa conta é a de dois serviços com bancos separados, integração por outbox + SNS/SQS, leitura em projeção com cache, e duas tasks por serviço. O restante dos documentos descreve esse desenho sem reabrir a conta, exceto quando um trade-off muda capacidade ou custo.

PARTE 02

Domínio e capacidades de negócio (DDD)

Capacidades, subdomínios, context map, event storming, agregados e modelos de dados

FONTE: DOCUMENTOS-BRUTOS/02-DOMINIO-DDD.MD

02 — Domínio (DDD)

Complementa o blueprint §4 a §6. Onde o blueprint já fixou nome, invariante ou JSON, este texto não inventa outro contrato: explica o mapa, o storming e o modelo físico.

1. Capacidades de negócio

Nível 1 é o que a organização precisa saber fazer. Nível 2 é o que um sistema pode automatizar.



Capacidade nível 1	Nível 2	Contexto que realiza
Gestão de caixa	Registrar crédito e débito, estornar, consultar extrato	Lançamentos
Informação de caixa	Projetar saldo diário, consultar dia e período	Consolidação diária
Acesso	Autenticar, autorizar por escopo e por comerciante	Identidade e acesso

A cadeia de valor cabe em uma frase: o comerciante registra o fato, o fato é durável mesmo sem relatório, e o relatório alcança o fato em segundos.

2. Subdomínios

O blueprint classifica Consolidação como **Core/Supporting**. As duas palavras estão certas em camadas diferentes, e misturá-las sem dizer isso confunde o investimento.

Subdomínio	Classe no produto	Classe DDD	Por quê
Gestão de lançamentos	Core	**Core**	Imutabilidade, estorno único, dinheiro e competência são a regra que diferencia um caixa auditável de uma planilha editável. Fonte da verdade.
Consolidação diária	Core para a proposta de valor (o comerciante compra o saldo)	**Supporting**	É projeção determinística do core. A regra nova é pouca: somar, não duplicar, calcular acumulado. Não decide se um lançamento existe.
Identidade e acesso	Higiene	**Generic**	OAuth2/OIDC não é diferencial. Cognito na AWS, Keycloak no laboratório.
Observabilidade, mensageria, execução	Plataforma	**Generic**	SNS/SQS, ECS, OTel. Comprados ou adotados, não modelados como domínio.

Consequência prática: o tempo de design vai para as invariantes do `Lancamento` e para o contrato do evento. O consolidado não ganha regra paralela de "interpretação" do estorno. Um estorno é só outro `LancamentoRegistrado` com tipo oposto.

3. Bounded contexts

Contexto	Serviço	Modelo que protege
Lançamentos (Gestão de Caixa)	`lancamentos-service`	Agregado `Lancamento`, VO `Dinheiro`, estorno
Consolidação diária	`consolidado-service`	Agregado `SaldoDiario`, read models de dia e período
Identidade e acesso	Cognito / Keycloak	Usuário, client, escopos, claim `comerciante_id`

Pacotes: `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`.

Não há contexto de "Comerciante" neste sistema. O identificador chega pronto no token. Criar um serviço de cadastro agora seria um terceiro deploy sem requisito.

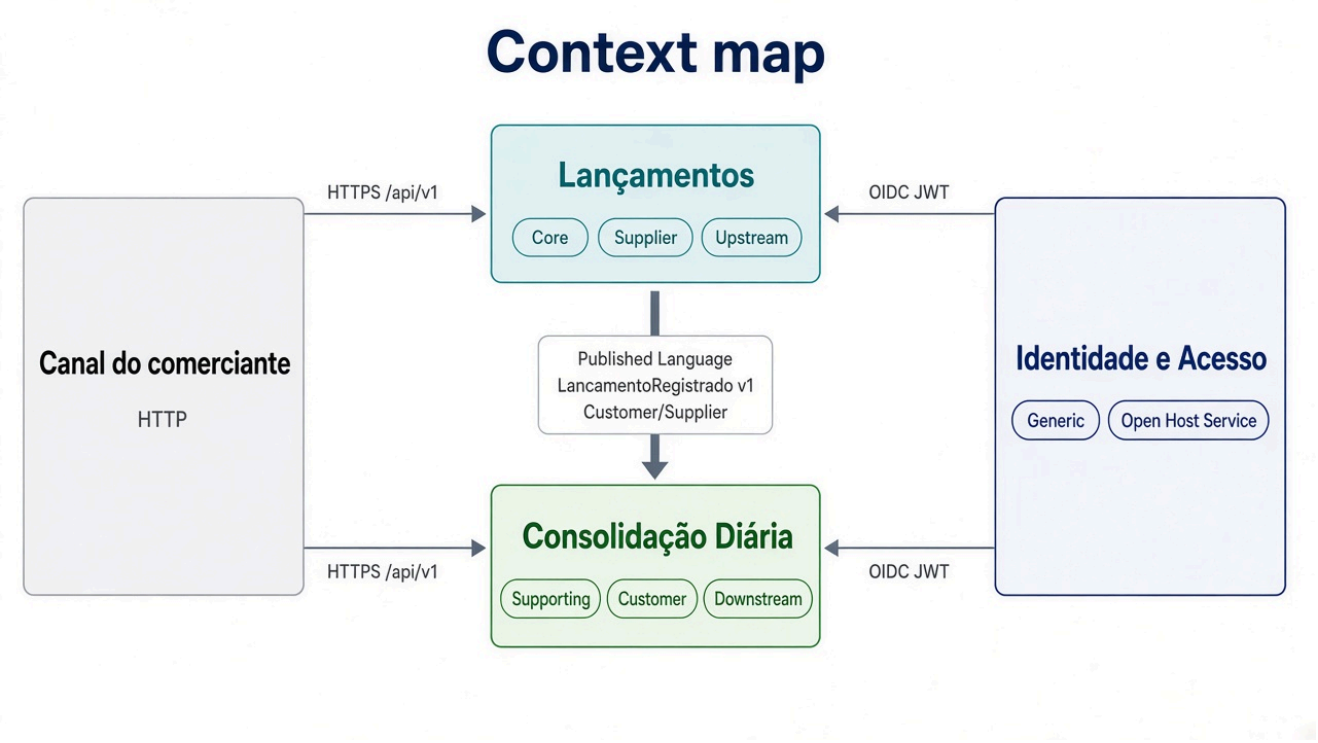
4. Context map

Padrões usados de propósito:

- **Customer/Supplier.** O consolidado é o customer: precisa do evento e não pode exigir mudança silenciosa no core. Lançamentos é o supplier: não consulta o consolidado e não falha se ele estiver fora.
- **Published Language.** O JSON versionado da seção 6 do blueprint é a linguagem publicada. Não é o modelo interno de nenhum dos dois.
- **Conformista, no sentido do relacionamento.** O downstream aceita a semântica do evento (soma comutativa, estorno = lançamento oposto). Não renegocia o fato.
- **ACL no adaptador.** O listener SQS traduz JSON em comando de aplicação. A entidade `SaldoDiario` não conhece `eventVersion`, SNS nem o formato string do valor. Conformista no negócio, anticorruptor

na borda técnica — é o que o blueprint pede ao dizer “downstream conformista com uma camada anti-corrupção no adaptador de mensageria”.

- **Open Host Service.** Cada API REST `/api/v1` é um host aberto para o canal do comerciante (e para integrações futuras). O provedor de identidade é OHS de token. Não há protocolo privado entre os dois serviços além do evento publicado.

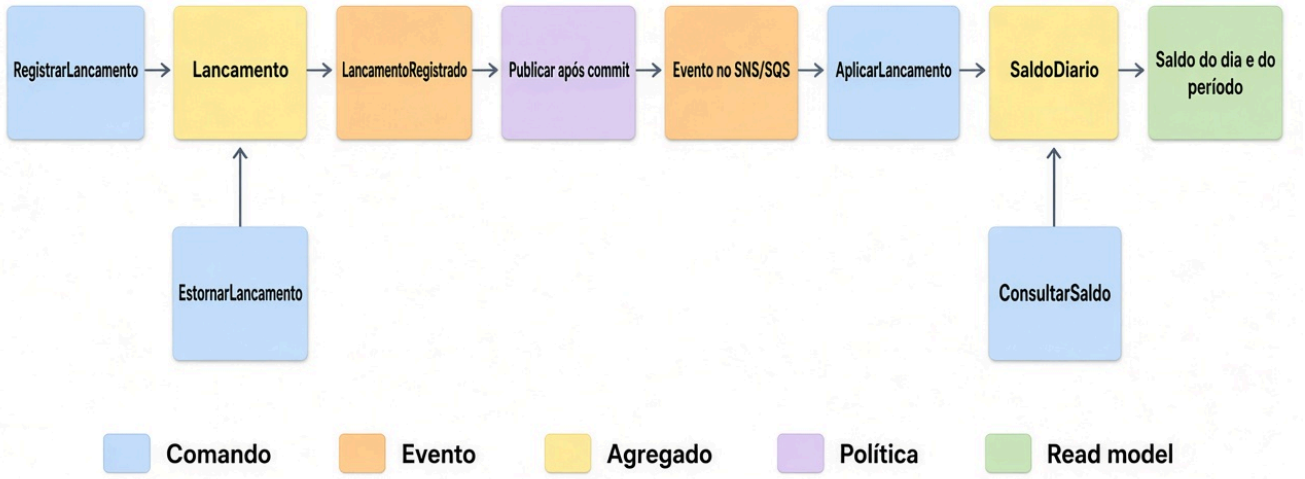


O que este mapa recusa: Partnership (os dois times não coevoluem o mesmo modelo), Shared Kernel (não há biblioteca de domínio compartilhada; duplicar o parse do valor na ACL é mais barato do que um acoplamento de release) e Conformist puro sem ACL (o JSON não entra no núcleo do consolidado).

5. Event storming

Fluxo do lançamento até a leitura. Cores: comando azul, evento laranja, agregado amarelo, política roxa, read model verde.

Event storming



Elemento	Tipo	Contexto	Notas
`RegistrarLancamento`	Comando	Lançamentos	POST `/api/v1/lancamentos`. Carrega tipo, valor, data, descrição, categoria, chave de idempotência opcional.
`EstornarLancamento`	Comando	Lançamentos	POST `.../{id}/estornos`. Falha se já estornado ou se o alvo já é estorno.
`ConsultarLancamento` / `ListarLancamentosDoDia`	Comando de leitura	Lançamentos	Leitura no próprio contexto. Não passa pelo consolidado.
`Lancamento`	Agregado	Lançamentos	Imutável após criado.
`LancamentoRegistrado`	Evento de domínio e de integração	Lançamentos → publicado	Mesmo nome nos dois papéis. A versão de integração é o JSON v1.
Publicar após o commit	Política	Lançamentos	Outbox + relay. Não é parte da transação visível ao cliente.
`AplicarLancamento`	Comando interno	Consolidação	Disparado pelo listener, não por HTTP.
`SaldoDiario`	Agregado	Consolidação	Chave `(comercianteId, data)`.
`EventoJaProcessado`	Política	Consolidação	Se `eventId` existe, o comando vira no-op.
`SaldoDoDia` / `RelatorioPeriodo`	Read models	Consolidação	Servidos pela API. Cache na frente.
`ConsultarSaldoDiario` / `ConsultarPeriodo`	Comando de leitura	Consolidação	GET. Dia vazio responde zeros.

Não há evento `LancamentoEstornado`. O estorno é um `LancamentoRegistrado` com `lancamentoEstornadoId` preenchido. Um segundo tipo de evento obrigaria o consumidor a conhecer uma máquina de estados que o domínio de escrita já resolveu.

6. Modelo tático — Lançamentos

Agregado `Lancamento`

Raiz de agregado. Identidade `id` (UUID; preferência por UUID v7 pela ordenação temporal, v4 aceito pelo blueprint). Não há coleção interna: um lançamento não contém outro. O vínculo de estorno é uma referência por id, não um grafo carregado junto.

Atributo	Tipo	Regra
`id`	UUID	Gerado pelo serviço.
`comercianteId`	UUID	Vem do token.
`tipo`	`CREDITO` ou `DEBITO`	Fechado.
`valor`	VO `Dinheiro`	`BigDecimal` escala 2, `> 0`, moeda `BRL`.
`dataLancamento`	`LocalDate`	Competência.
`descricao`	texto 1..255	Obrigatória.
`categoria`	texto até 60 ou nulo	Opcional. Não entra no evento v1.
`lancamentoEstornadoId`	UUID ou nulo	Preenchido somente em estorno.
`criadoEm`	`Instant`	UTC.

Value object `Dinheiro`

Imutável. Construtor rejeita escala maior que 2, sinal errado e moeda diferente de BRL. Igualdade por valor e moeda. A borda HTTP converte o número JSON do request (exemplo do blueprint: `150.00`) para `Dinheiro` via `BigDecimal`, nunca via `double`. A borda de mensagem serializa com `toPlainString` de duas casas (`"150.00"`).

Invariantes

1. Valor positivo com no máximo duas casas, moeda BRL.
2. Descrição presente e dentro do tamanho.
3. Um lançamento é estornado no máximo uma vez. A unicidade é restrição de persistência (índice único parcial em `lancamento_estornado_id`), não só checagem em memória.
4. Lançamento que já é estorno não pode ser estornado.
5. O estorno copia valor, moeda e `dataLancamento` do original e inverte o tipo.
6. Não há operação de alteração nem de exclusão.

Evento de domínio

`LancamentoRegistrado` carrega o mínimo para um consumidor recalcular saldo: ids, tipo, valor, moeda, data de competência, id do estornado, `eventId`, `eventVersion`, `occurredAt`. Descrição e categoria ficam de fora de propósito: são dado operacional do extrato, não da soma, e reduzem o acoplamento e a superfície de dado pessoal replicado.

7. Modelo tático — Consolidação

Agregado `SaldoDiario`

Identidade natural `(comercianteId, data)`. Nasce na primeira aplicação de evento daquele dia.

Atributo	Regra
`totalCreditos`, `totalDebitos`, `saldo`	`saldo = totalCreditos − totalDebitos`. Mantidos juntos para o GET do dia ser uma linha.
`quantidadeLancamentos`	Incrementa 1 por evento novo, inclusive estorno.
`versao`	Lock otimista. Dois eventos do mesmo dia em paralelo: um faz retry e reaplica. A soma é comutativa, então a ordem não muda o resultado.
`atualizadoEm`	Instante da última aplicação.

`saldoAcumulado` **não é coluna**. É calculado na consulta: `SUM(saldo)` das datas `≤` a data pedida. Materializar o acumulado em cada linha futura transformaria um estorno antigo em atualização em cadeia. Com centenas de linhas por comerciante, o `SUM` indexado é mais simples e cabe no orçamento de latência da análise de capacidade.

Idempotência

Tabela `eventos_processados(event_id PK)` escrita na mesma transação que faz o UPSERT de `saldos_diarios`. Entrega at-least-once do SQS deixa de ser problema de negócio.

Read models

- **Saldo do dia:** uma linha de `saldos_diarios` mais o `SUM` do saldo acumulado. Ausência da linha vira zeros e o acumulado dos dias anteriores. O saldo do dia não é coluna: é `total_creditos − total_debitos`.
- **Período:** uma linha por dia do intervalo, limite 366 dias, dias sem movimento preenchidos com zero na aplicação.

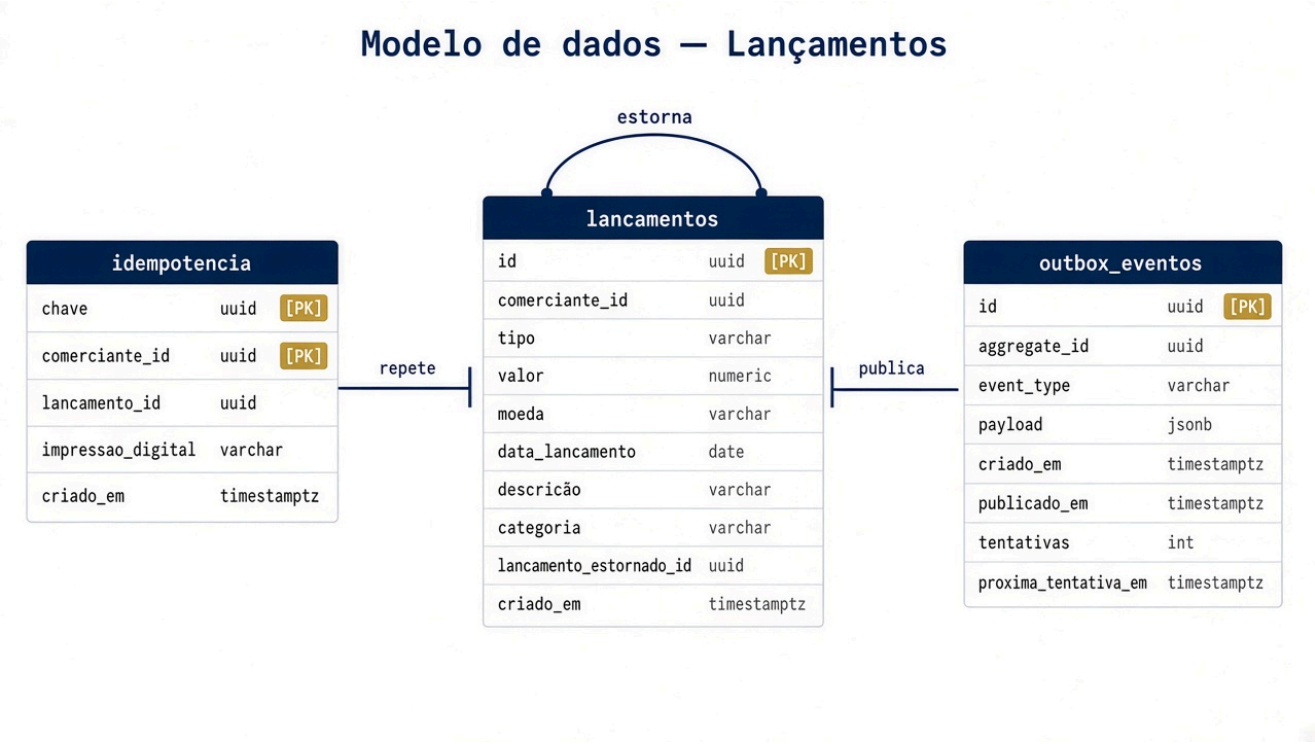
Cache

O cache não é fonte da verdade. Chave com geração por comerciante:

```
consolidado:versao:{comercianteId}
consolidado:dia:{comercianteId}:{versao}:{data}
consolidado:relatorio:{comercianteId}:{versao}:{inicio}:{fim}
```

Cada evento aplicado faz `INCR` na versão daquele comerciante depois do commit. Chaves velhas expiram por TTL de 30 s. Isso invalida o dia alterado, os acumulados posteriores e os relatórios de período sem uma lista de deletes. Dia corrente e dia antigo seguem a mesma regra, porque estorno retroativo existe (RF-18).

8. Modelo de dados — Lançamentos

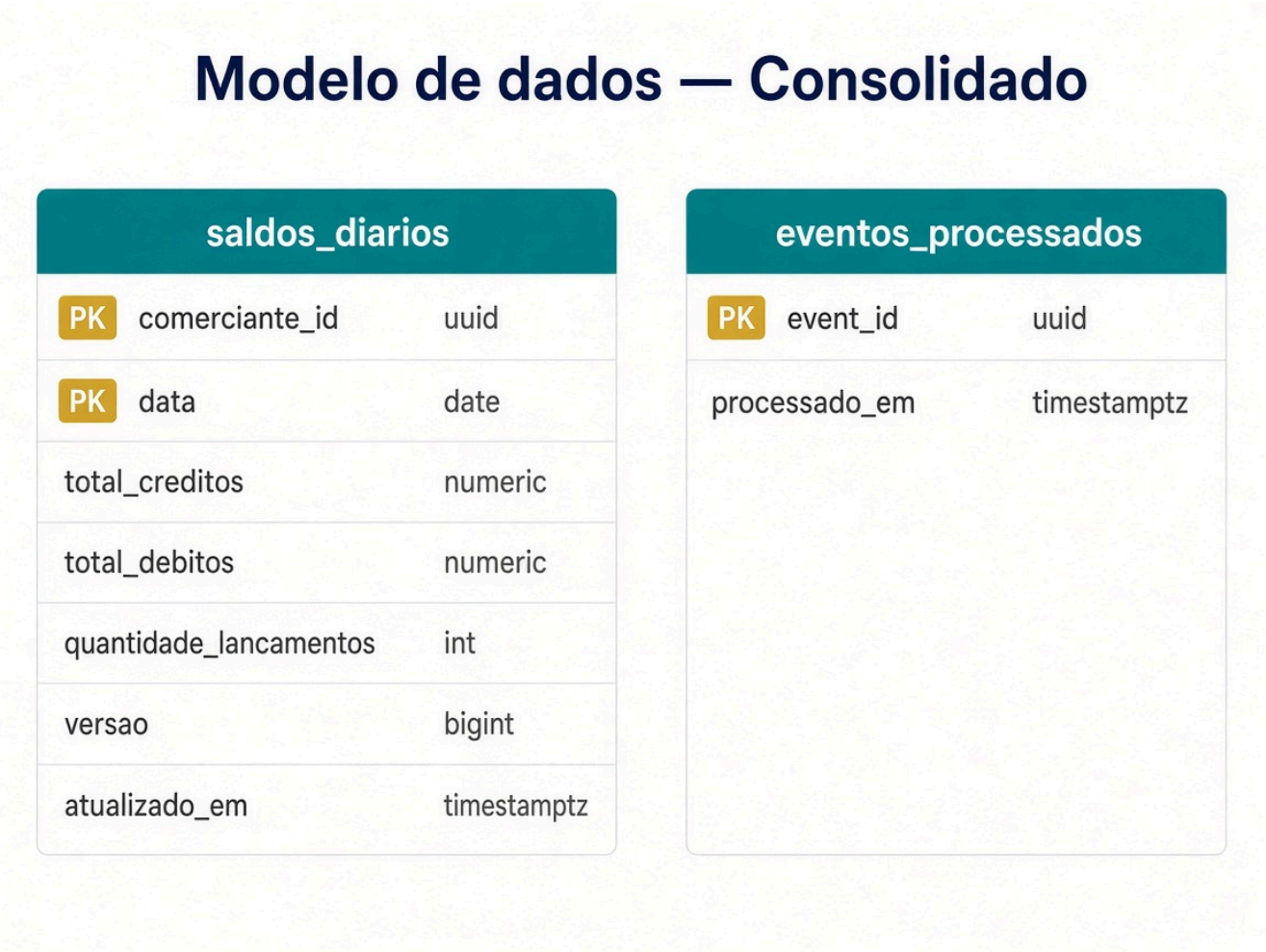


Índices e restrições da migração `V1__criar_tabelas.sql`:

- `idx_lancamentos_comerciante_data` em `(comerciante_id, data_lancamento)`.
- `UNIQUE (lancamento_estornado_id)`. No PostgreSQL vários nulos coexistem; um original só pode ser referenciado uma vez.
- `idx_outbox_eventos_pendentes` em `(proxima_tentativa_em, criado_em)` onde `publicado_em` é nulo.
- `idempotencia` chave primária `(chave, comerciante_id)`. `impressao_digital` é `VARCHAR(64)`, o SHA-256 hex do corpo.

`valor` é `NUMERIC(19,2)`. `moeda` é `VARCHAR(3)` com check `BRL`.

9. Modelo de dados — Consolidado



Não há coluna `saldo`. A consulta calcula `total_creditos - total_debitos`. O `versao` da linha é um contador do UPSERT, diferente da versão do cache Redis.

Não há chave estrangeira para o banco de lançamentos. A referência é o `lancamentoId` dentro do payload já aplicado; ele não é coluna obrigatória do saldo, porque o saldo é soma, não extrato. Guardar o id na projeção seria um extrato duplicado.

10. Contrato do evento

No local o tópico SNS é `fluxo-caixa-lancamentos-events`. Na AWS o nome leva o prefixo do ambiente e o serviço só conhece `LANCAMENTOS_TOPIC_ARN`. Assinatura da fila com **raw message delivery**. O corpo no SQS é o JSON, sem envelope SNS. Atributo de mensagem `eventType` = `LancamentoRegistrado`.

```
{
  "eventId": "0b8f6c1e-6d1a-4b8e-9a43-0f1b7e2f9c11",
  "eventType": "LancamentoRegistrado",
  "eventVersion": 1,
  "occurredAt": "2026-11-10T18:00:00.123Z",
  "lancamentoId": "5f0e1c7a-1d2b-4c3d-8e9f-0a1b2c3d4e5f",
  "comercianteId": "11111111-1111-1111-1111-111111111111",
  "tipo": "CREDITO",
```

```
  "valor": "150.00",
  "moeda": "BRL",
  "dataLancamento": "2026-11-10",
  "lancamentoEstornadoId": null
}
```

Regras de evolução:

- Campo novo é aditivo e opcional. Consumidor antigo ignora.
- Quebra (remover campo, mudar unidade, mudar semântica de estorno) exige `eventVersion` 2 e um período em que o consumidor aceita as duas versões.
- `valor` permanece string. Número JSON no evento reintroduziria binário de ponto flutuante em alguma linguagem do outro lado.
- Entrega at-least-once. Ordenação não é garantida e não é necessária.
- O consumidor descarta versão que não conhece para a DLQ com erro explícito, em vez de aplicar pela metade.

11. Limites que o modelo ainda não tem

Categoria não segmenta o saldo. Dois créditos de categorias diferentes somam no mesmo `SaldoDiario`. Um relatório por categoria exigiria outro read model e o campo `categoria` no evento — evolução, não omissão acidental, descrita em evoluções.

Não há conceito de “caixa fechado”. Um lançamento retroativo é sempre aceito. Travar competência é regra de um contexto futuro de fechamento, que hoje não existe; colocá-la dentro de `Lancamento` agora acoplaria o core a um processo que o enunciado não pediu.

PARTE 03

Arquitetura alvo, C4 e fluxos de dados

Modelos C4 de contexto a código, fluxos, padrões de integração e implantação na AWS

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/README.MD

Arquitetura

Visão de solução do Fluxo de Caixa. Os nomes de serviços, filas, portas e variáveis são os do blueprint.

Documento	Conteúdo
C4 nível 1 — contexto	Pessoas e sistemas externos
C4 nível 2 — containers	Serviços, bancos, mensageria, cache, identidade
C4 nível 3 — lançamentos	Componentes hexagonais do `lancamentos-service`
C4 nível 3 — consolidado	Componentes hexagonais do `consolidado-service`
C4 nível 4 — registrar lançamento	Código-alvo do caso de uso de escrita
Arquitetura-alvo	Estilos, padrões e trade-off de cada um
Fluxos de dados	Sequências: escrita, outbox, consolidação, cache, estorno, falha, DLQ
Padrões de integração	EIP entre a borda e os serviços, e entre lançamentos e consolidado
Deployment AWS	CloudFront, API Gateway, ALB interno, VPC, três ambientes e DR em `us-east-1`
Structurizr DSL	Workspace com context, containers, components, dynamic e deployment

Os diagramas Mermaid usam a sintaxe C4 (`C4Context` , `C4Container` , `C4Component`), que o GitHub renderiza. O workspace Structurizr é a fonte editável. A imagem `structurizr/cli:latest` de novembro de 2026 só imprime o aviso de migração; a tag `2025.03.28` valida e exporta este arquivo:

```
docker run --rm \
  -v "$PWD/docs/03-arquitetura/structurizr":/usr/local/structurizr \
  structurizr/cli:2025.03.28 \
  validate -w /usr/local/structurizr/workspace.dsl

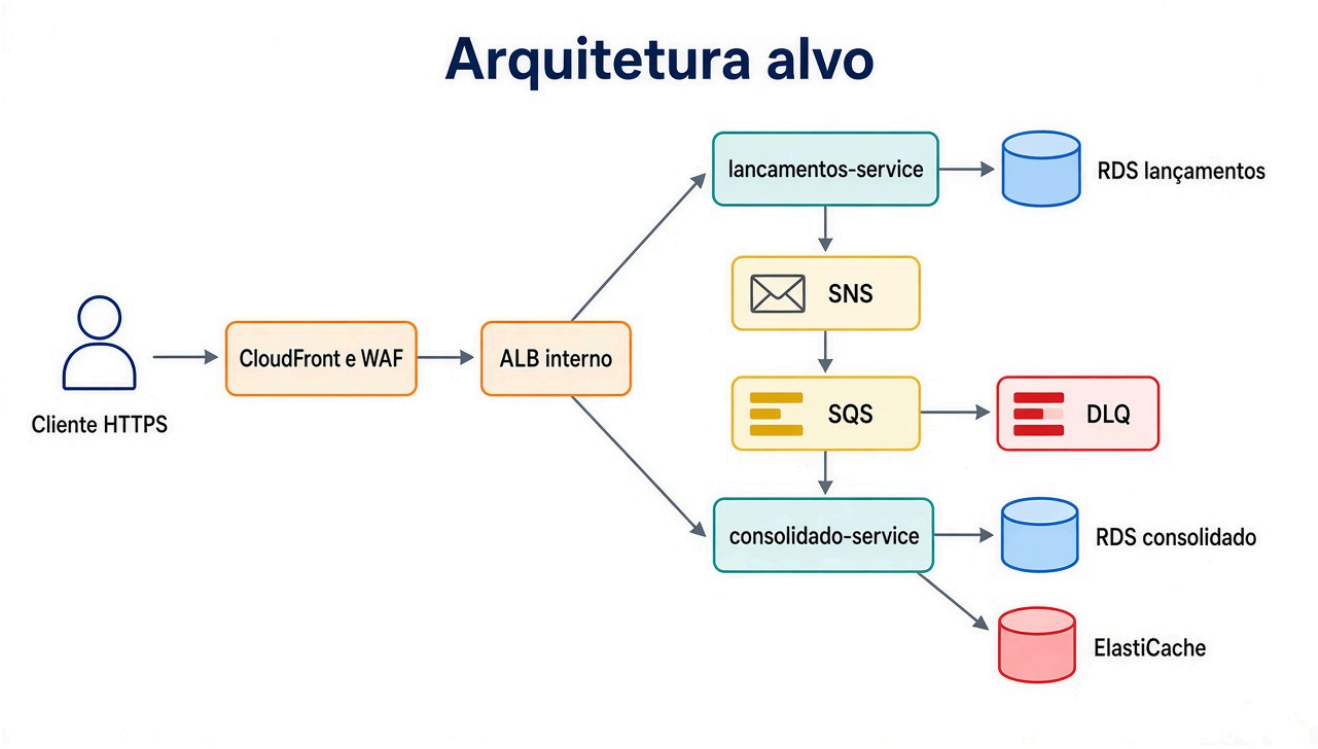
docker run --rm \
  -v "$PWD/docs/03-arquitetura/structurizr":/usr/local/structurizr \
  structurizr/cli:2025.03.28 \
  export -w /usr/local/structurizr/workspace.dsl \
  -format plantuml -o /usr/local/structurizr
```

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/ARQUITETURA-ALVO.MD

Arquitetura-alvo

1. Visão

Dois microsserviços alinhados a bounded contexts, com CQRS entre a escrita de lançamentos e a leitura do saldo. A integração é assíncrona. O POST não espera o relatório. A durabilidade do fato fica no Postgres de lançamentos; a disponibilidade do relatório em pico fica numa projeção pequena, cache e duas tasks.



O RNF crítico cabe neste desenho: a seta entre os serviços aponta só para a direita e termina numa fila, não numa chamada bloqueante. A conta de 50 req/s, 2 tasks e 0,16 conexões ativas está em contexto e requisitos.

2. Estilos e padrões

Cada padrão abaixo está adotado. O trade-off é o preço que aceitamos pagar.

Microsserviços por bounded context

Dois deploys, dois bancos, dois ciclos de vida. Lançamentos pode ser implantado sem tocar no consolidado.

Trade-off. Operação duplicada (pipeline, alarme, RDS). Um monolito modular daria a mesma fronteira de código com um deploy só, e atenderia o isolamento lógico se o módulo de escrita não chamasse o de leitura. Não atenderia o isolamento de *processo e de banco* que o enunciado pede quando "o sistema de consolidado cair": no mesmo processo, um vazamento de pool ou um crash derruba os dois. Serverless foi descartado pelo perfil de conexões longas com Postgres e pelo custo fixo de VPC em Lambda. Detalhe no ADR 0002.

CQRS

A escrita persiste o fato. A leitura serve uma projeção com outro esquema (`saldos_diarios` não é a tabela `lancamentos`).

Trade-off. Dois modelos para manter e uma janela de consistência eventual (p95 < 5 s). Em troca, o GET de pico não faz join no extrato, e o esquema da leitura pode ganhar índice e cache sem migrar a trilha de auditoria. ADR 0003.

Event-driven

O fato vira `LancamentoRegistrado` v1. Novos consumidores (notificação, auditoria externa, categoria) assinam o tópico com fila própria, sem o de escrita conhecê-los.

Trade-off. Depuração deixa de ser uma stack única. Exige correlation id, outbox e consumidor idempotente. Sem isso, evento é perda silenciosa.

Transactional outbox

Lançamento e linha em `outbox_eventos` no mesmo commit. Relay com `FOR UPDATE SKIP LOCKED`, lote 100, intervalo 500 ms.

Trade-off. Latência extra de até um ciclo do relay no caminho do evento (não no caminho do POST) e uma tabela para operar. Dual-write (commit e publish separados) perde evento ou publica evento de transação revertida. CDC com Debezium entrega o log, com um cluster a mais que este volume não paga. ADR 0004.

Consumidor idempotente

`eventos_processados(event_id PK)` na mesma transação da soma. A fila é at-least-once e Standard (sem ordem). A soma é comutativa, então a ausência de FIFO é aceitável.

Trade-off. Toda mensagem custa um insert de id. É a alternativa barata a “processar exatamente uma vez”, que o SQS não oferece.

Hexagonal

Domínio em Java puro, casos de uso nas portas, AWS e JPA nos adaptadores. ArchUnit no build.

Trade-off. Mais pacotes e mapeamentos. O ganho é poder testar estorno e dinheiro sem subir Spring, e trocar SNS por LocalStack sem editar regra. ADR 0008.

Cache-aside

O consolidado consulta Redis no local e Valkey no ElastiCache e, no miss, o Postgres, depois preenche o cache. A chave carrega a versão corrente do comerciante (`consolidado:versao:{id}`), incrementada a cada evento. O TTL de 30 s (`app.cache.ttl`) só apaga chave órfã; a leitura seguinte já usa a versão nova.

Trade-off. Memória e um modo de falha a mais. Se o Redis cair, a leitura segue no Postgres: em 50 req/s com miss total são ~2 conexões ativas, dentro do `db.t4g.small`. O cache existe para o p95 < 100 ms, não para a sobrevivência. ADR 0012.

Database per service

Cada serviço é dono do seu esquema e, em produção, da sua instância RDS. Não há `JOIN` entre lançamento e saldo, nem usuário de banco compartilhado.

Trade-off. Duas instâncias Multi-AZ custam mais do que um RDS com dois databases (a conta está no documento 08). O ganho é o de falha: esgotar conexão, disco ou IOPS do consolidado não tira o POST do ar. Em dev AWS as instâncias podem ser menores e Single-AZ; no Docker, um Postgres com dois databases basta.

Retry com backoff e jitter, timeout, DLQ

- Relay: retenta o publish, incrementa `tentativas`, não segura a thread do POST.
- SQS: visibility 60 s, até 5 recebimentos, depois `consolidado-lancamentos-dlq`.
- Redis e JWKS: timeout curto e retry com jitter. Falha de Redis degrada para o banco.
- Não se retenta 4xx de domínio. Estorno inválido não melhora na segunda tentativa.

Trade-off. Retry sem idempotência duplicaria saldo. Os dois padrões andam juntos. ADR 0014.

Circuit breaker

Há dois mecanismos com o mesmo nome, e eles não se misturam.

- **De deploy (ECS).** Circuit breaker com rollback automático se as tasks novas não ficam saudáveis. ADR 0020.
- **De aplicação (Resilience4j 2.4.0).** CircuitBreaker, Retry com jitter e TimeLimiter no cliente SNS do relay. Circuito aberto adia o lote; o evento fica na outbox. No consolidado, CircuitBreaker no Redis com fallback no Postgres, e Bulkhead no consumidor. Não existe circuito entre lançamentos e consolidado porque não existe chamada entre eles. ADR 0023.

Trade-off. O POST continua sem chamar o SNS. O circuito protege o relay, não a latência do 201. Timeout mal calibrado no Redis transforma lentidão em fallback, que é o comportamento desejado. Boot 4.1 fica bloqueado até o starter do Resilience4j declarar suporte.

Bulkhead

Pools JDBC separados, serviços ECS separados, filas separadas, bancos separados. Um consumidor lento não ocupa conexão do POST. O teto do pool é 10 no lançamentos e 20 no consolidado, por task.

Trade-off. Conexão ociosa (`minimumIdle` 2) custa slot no `max_connections`. O número foi escolhido na análise de capacidade para sobrar folga, não para “usar o banco inteiro”.

DLQ

Quinta falha de processamento vai para `consolidado-lancamentos-dlq`. A fila principal segue. Reprocessar é uma ação operada (corrigir o consumidor e reenviar), descrita no fluxo de DLQ e no runbook.

Trade-off. Mensagem na DLQ é saldo atrasado para aquele evento até alguém agir. O alarme é imediato. Sem DLQ, um poison message seguraria a partição lógica atrás de retries infinitos — no Standard não bloqueia as outras, mas queimaria receive e esconderia o erro.

API versionada e RFC 9457

Base `/api/v1`. Erro em `application/problem+json`. Evolução do evento por `eventVersion`. ADR 0017.

Trade-off. URIs de problema e versões são contrato para sempre. O ganho é cliente que distingue 409 de estorno de 400 de payload sem parsear texto livre.

Borda CloudFront, WAF e ALB interno, JWT no serviço

CloudFront é a única entrada pública. O WAF é de escopo `CLOUDFRONT`. O ALB é interno, alcançado por VPC Origin, e roteia `/api/v1/lancamentos*` e `/api/v1/consolidados*` para target groups diferentes. Health check em `/actuator/health/readiness`. `/api/*` não é cacheado. O JWT é validado no serviço, igual no Keycloak local e no Cognito.

Trade-off. O CloudFront não reduz carga de API, porque a resposta autenticada não se cacheia. O ganho é TLS no edge, Shield Standard e ALB sem IP público. API Gateway continua um hop e uma conta a mais, e um authorizer só na borda deixaria o serviço nu se alguém alcançasse a task. ADR 0022.

BFF / API Gateway — considerado e não adotado

Um BFF faria sentido com um frontend agregando saldo e extrato numa tela, escondendo os dois serviços. Não há frontend nesta entrega. Agregar no BFF agora criaria um terceiro serviço sem dono de negócio. O cliente chama os dois hosts paths no mesmo ALB. Quando existir canal web, o BFF entra na frente sem mudar o domínio — está no roadmap.

Strangler Fig

Usado na transição a partir de planilha ou ERP, não no dia a dia dos dois serviços novos. Os dois serviços já nascem no contorno certo; não há monolito nosso para estrangular. O padrão está especificado no documento 09 para o legado do comerciante.

Idempotência na borda HTTP

`Idempotency-Key` opcional, escopada pelo comerciante. Mesma chave e mesmo corpo: 201 na primeira vez, 200 na repetição. Corpo diferente: 409. ADR 0014.

Lançamentos imutáveis

Correção é estorno. A trilha é a própria tabela, append-only. ADR 0015.

Dinheiro

`BigDecimal` e `NUMERIC(19,2)` dentro; string de duas casas no evento. ADR 0016.

3. O que a arquitetura-alvo deliberadamente não é

Tentação	Por que não
Event sourcing do lançamento	O estado cabe numa linha. O evento de integração não precisa reconstruir o agregado. Reprocessar saldo a partir do tópico é evolução, não requisito.
Saga	Não há transação distribuída para compensar. O estorno é um fato novo, não um rollback do POST.

Tentação	Por que não
Service mesh	Dois serviços atrás de um ALB não pagam o custo de Istio/App Mesh. mTLS pode ser feito na evolução com VPC Lattice ou mesh quando houver dezenas de serviços.
Kafka / MSK	O volume da premissa (20 evt/s, 2 milhões/mês) é ruído para um cluster. SNS+SQS entrega DLQ, retenção e fan-out. ADR 0005.

4. Como as qualidades caem nos padrões

Qualidade	Padrão que a carrega
Lançamentos sobrevive ao consolidado	Assíncrono + outbox + fila + banco separado
50 req/s com perda baixa	Projeção + cache + 2 tasks com ρ ≈ 17% + WAF acima do pico
Auditoria	Append-only e estorno
Isolamento de tenant	Claim no token, filtro em toda query
Evolução do contrato	`/api/v1` e `eventVersion`
Custo proporcional	Fargate, RDS small, fila gerenciada; NAT e endpoint são o custo fixo a vigiar

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL1-CONTEXTO.MD

C4 — Nível 1: contexto

O comerciante usa o Fluxo de Caixa para registrar movimentos e ler o saldo. O sistema delega a autenticação a um provedor de identidade e, na implantação AWS, se apoia em serviços gerenciados de mensageria, dados e observabilidade. Esses serviços aparecem no nível 2 e na implantação; aqui ficam de fora para o diagrama mostrar só quem fala com quem no negócio.



Ator ou sistema	Responsabilidade na fronteira
Comerciante	Pessoa ou sistema dele (PDV, app) que chama a API. Não escolhe o tenant: o token já o traz.
Fluxo de Caixa	Dois serviços. Escrita durável de lançamentos e leitura eventual do saldo.
Provedor de identidade	Autentica, emite escopos `lancamentos:write`, `lancamentos:read`, `consolidado:read` e o claim `comerciante_id`.
Caixa legado	Fora do caminho de produção do dia zero. Entra na arquitetura de transição.

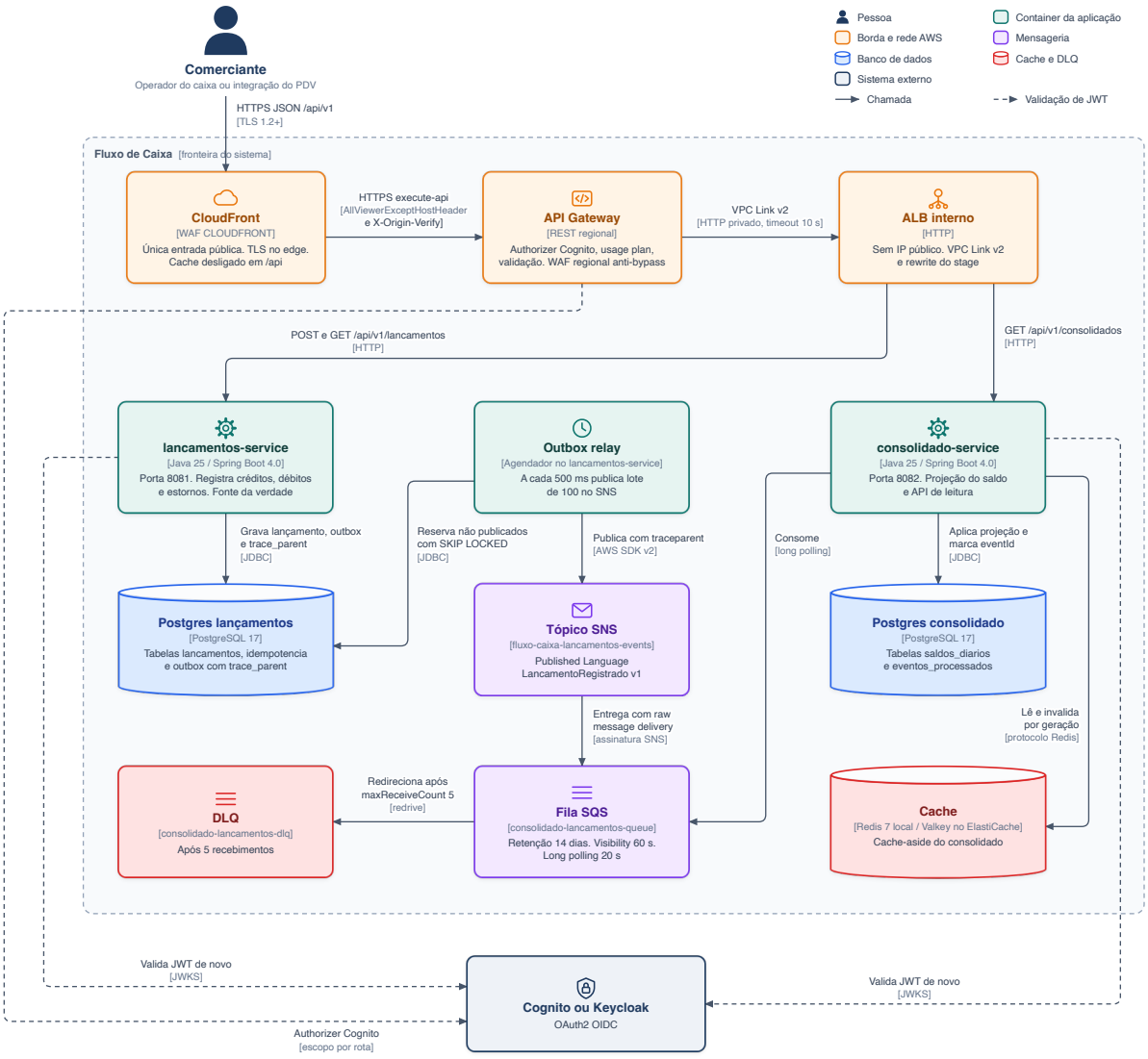
O diagrama de contexto de propósito não mostra SNS, RDS nem fila. Se eles aparecem aqui, o leitor confunde mecanismo com capacidade.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL2-CONTAINERS.MD

C4 — Nível 2: containers

Dois processos, dois bancos, um tópico, uma fila e uma DLQ. Cache só no consolidado. Identidade fora dos dois serviços.

C4 — Nível 2: containers do Fluxo de Caixa



O relay está desenhado como container lógico. No deploy ele é um agendador dentro do processo do `lancamentos-service`, não um terceiro serviço. Separá-lo no diagrama deixa visível que a publicação não está no caminho do POST. Subir um processo exclusivo de relay só vale se o agendamento competir com a API por CPU — não é o caso em 20 escritas/s.

Container	Escala	Estado
CloudFront + WAF	Distribuição global, PoP GRU	Sem estado de negócio. `/api/*` não é cacheado.

Container	Escala	Estado
API Gateway REST	Regional, stage `v1`	Sem estado de negócio. Rejeita JWT inválido antes da VPC.
ALB interno	Um, nas subnets privadas	Sem IP público. Dois target groups. Aceita o VPC Link.
`lancamentos-service`	2+ tasks ECS em prod	Stateless. Estado no RDS de lançamentos.
`consolidado-service`	2+ tasks ECS em prod	Stateless. Estado no RDS de consolidado e no cache.
Postgres (cada um)	RDS Multi-AZ em produção	Fonte durável.
SNS, SQS, DLQ	Gerenciados	Buffer entre os contextos.
Redis / Valkey	Réplica com failover em produção	Descartável. Perda do cache só aumenta latência.
Cognito / Keycloak	Gerenciado / container local	Fora do domínio.

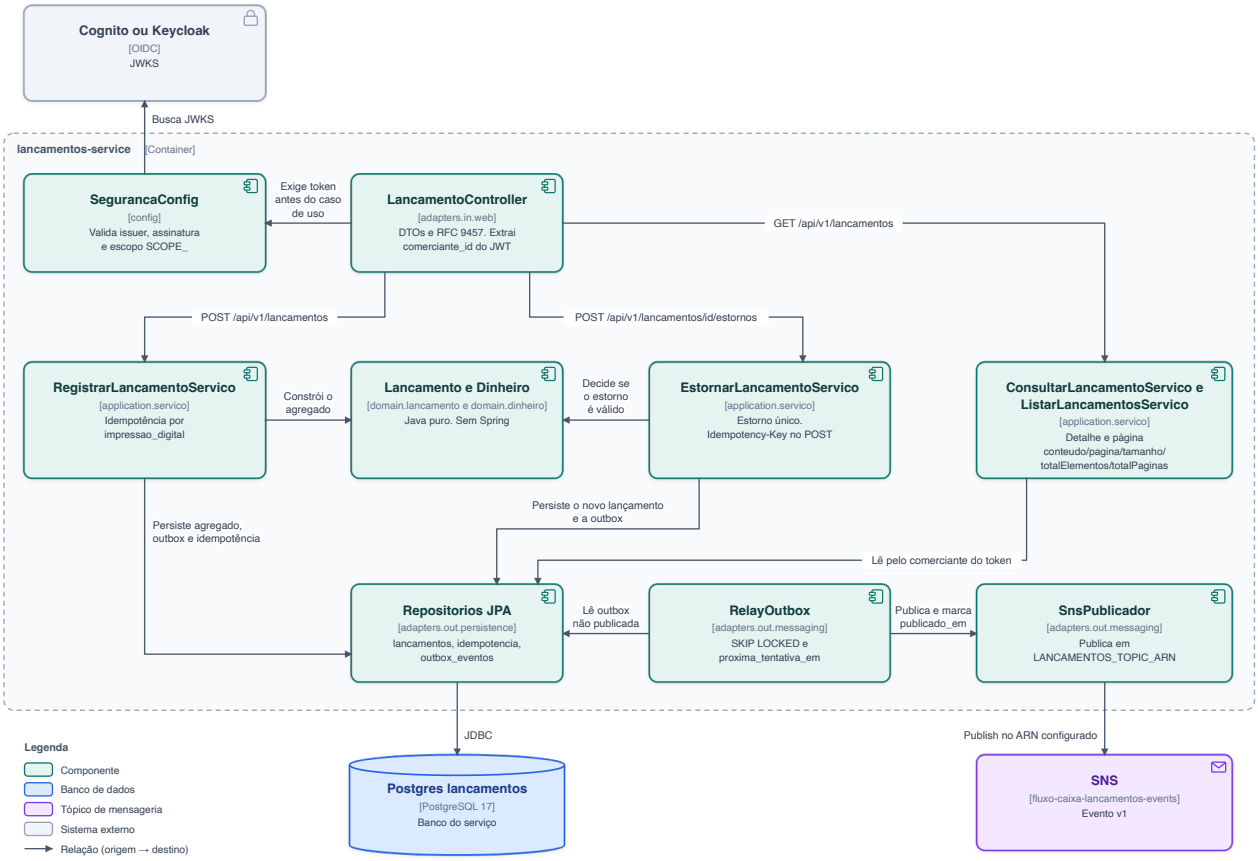
Portas locais: lançamentos 8081, consolidado 8082, Postgres 5432, Redis 6379, LocalStack 4566, Keycloak 8080, Grafana 3000, OTLP 4317 e 4318. Na AWS o cliente fala só com o CloudFront em 443. O API Gateway é regional e não é a URL que se publica. O ALB é interno, sem IP público, e as portas dos serviços ficam na rede privada. O relay continua sendo um agendador dentro do `lancamentos-service`, desenhado à parte para deixar visível que a publicação não está no caminho do POST. Com `enable_api_gateway = false`, o CloudFront volta a falar com o ALB pela VPC Origin e o container do gateway sai do caminho.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL3-LANCAMENTOS.MD

C4 — Nível 3: componentes do lancamentos-service

Arquitetura hexagonal no pacote `br.com.fluxocaixa.lancamentos`. Regra verificada por ArchUnit: `domain` não depende de Spring nem de `adapters`; `application` não depende de `adapters`.

C4 — Nível 3: componentes do lancamentos-service



Porta	Implementação	Observação
Entrada HTTP	`adapters/in/web`	Única porta síncrona. Não há listener de mensagem neste serviço.
Entrada de aplicação	`application.port.in` implementado em `application.servico`	O controller não calcula estorno.
Saída de persistência	JPA em `adapters.out.persistence`	Uma transação cobre lançamento, idempotência e outbox.
Saída de mensagem	`RelayOutbox` e `SnsPublicador`	Fora da transação do POST. Falha aqui não desfaz o 201. O nome do tópico não está no código.

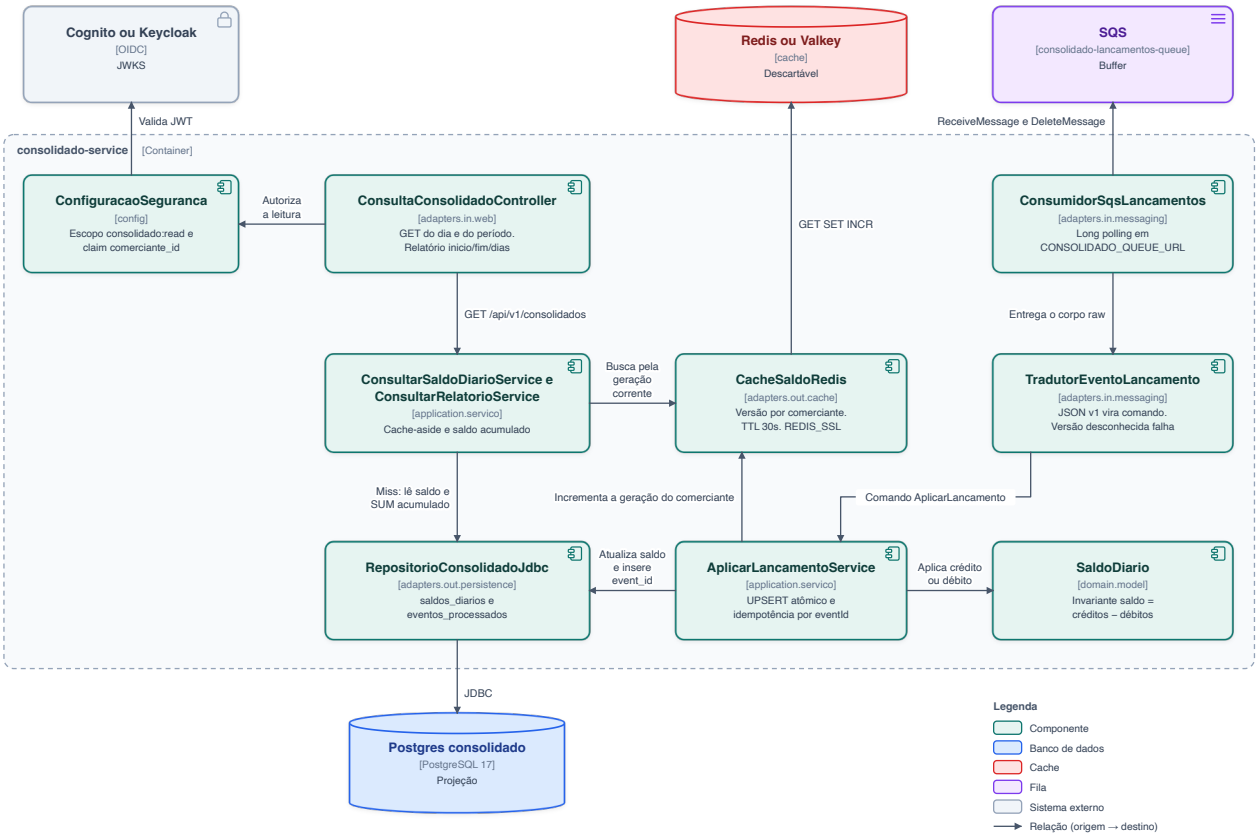
O domínio não importa `software.amazon.awssdk` nem `org.springframework`. O teste de arquitetura quebra o build se isso acontecer.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL3-CONSOLIDADO.MD

C4 — Nível 3: componentes do consolidado-service

Pacote `br.com.fluxocaixa.consolidado`. O HTTP só lê. Quem escreve a projeção é o listener SQS.

C4 — Nível 3: componentes do consolidado-service



Decisão local	Efeito
ACL no `TradutorEventoLancamento`, não no agregado	`SaldoDiario` não conhece JSON.
`eventos_processados` na mesma transação do UPSERT	`ON CONFLICT DO NOTHING` no evento e `ON CONFLICT DO UPDATE` somando em `saldos_diarios`. Reentrega não duplica saldo. Não há lock otimista.
Falha de parse ou versão ignorada	A mensagem volta após o visibility timeout e, na quinta vez, cai na DLQ. Não se aplica “no melhor esforço”.
Redis fora	O caso de uso lê o Postgres. Latência piora, disponibilidade da leitura permanece, dentro do que o RDS aguenta (ver capacidade).
Geração de cache	Um estorno em data antiga invalida acumulado e período daquele comerciante sem varrer chave por chave.

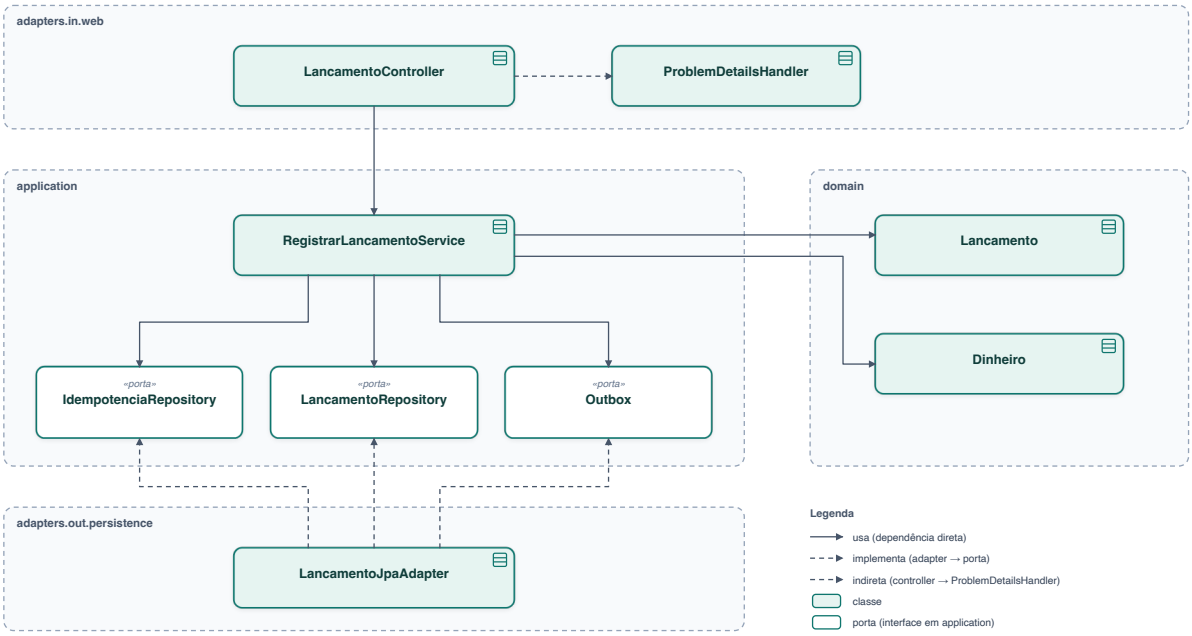
Não há porta de saída para o `lancamentos-service`. O consolidado não pergunta “qual é o lançamento?”. Se o evento não basta, o contrato está incompleto e a correção é versão nova do evento, não uma chamada síncrona de volta.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/C4-NIVEL4-REGISTRAR-LANCAMENTO.MD

C4 — Nível 4: registrar lançamento

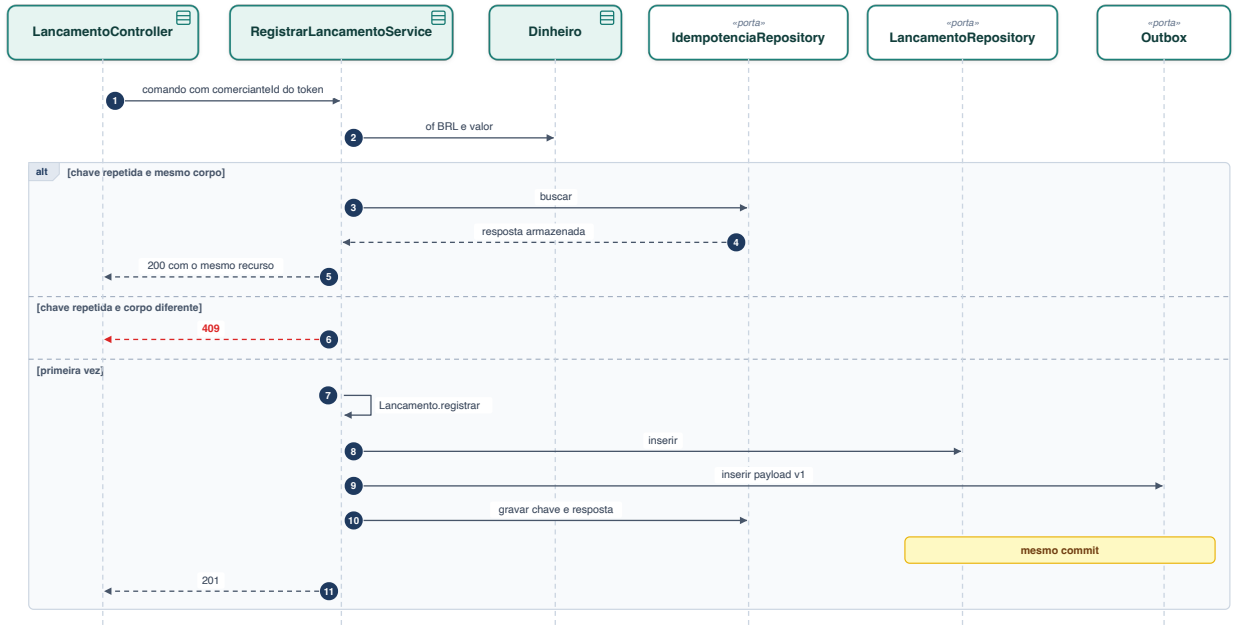
Nível de código do caso de uso que segura o RNF de independência. As classes abaixo são o alvo da implementação em `services/lancamentos-service`; os nomes de pacote são os do blueprint. Não há framework no miolo.

C4 — Nível 4: classes do registrar lançamento



Sequência interna, já com a transação no lugar certo:

Sequência interna do registrar lançamento



Pontos que o código não pode “simplificar”:

- O controller lê `comerciante_id` do `Jwt` e não de um campo do JSON. Se o JSON trouxer o campo, ele é ignorado.
- `Dinheiro.of` rejeita escala e sinal antes de qualquer `INSERT`.
- `Outbox.inserir` participa da transação do repositório. Publicar SNS dentro desse método quebraria o desenho: o SDK é chamado só pelo relay, depois do commit.
- A resposta 201 devolve `valor` como string de duas casas, alinhada ao evento e ao ADR 0016. O request continua aceitando número JSON, como no exemplo do blueprint.

O estorno segue o mesmo desenho, com `Lancamento.estornar(original)` no domínio devolvendo ou uma falha de invariante ou um novo `Lancamento`. A checagem “já estornado” é repetida pelo índice único: duas requisições paralelas não criam dois estornos.

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/FLUXOS-DE-DADOS.MD

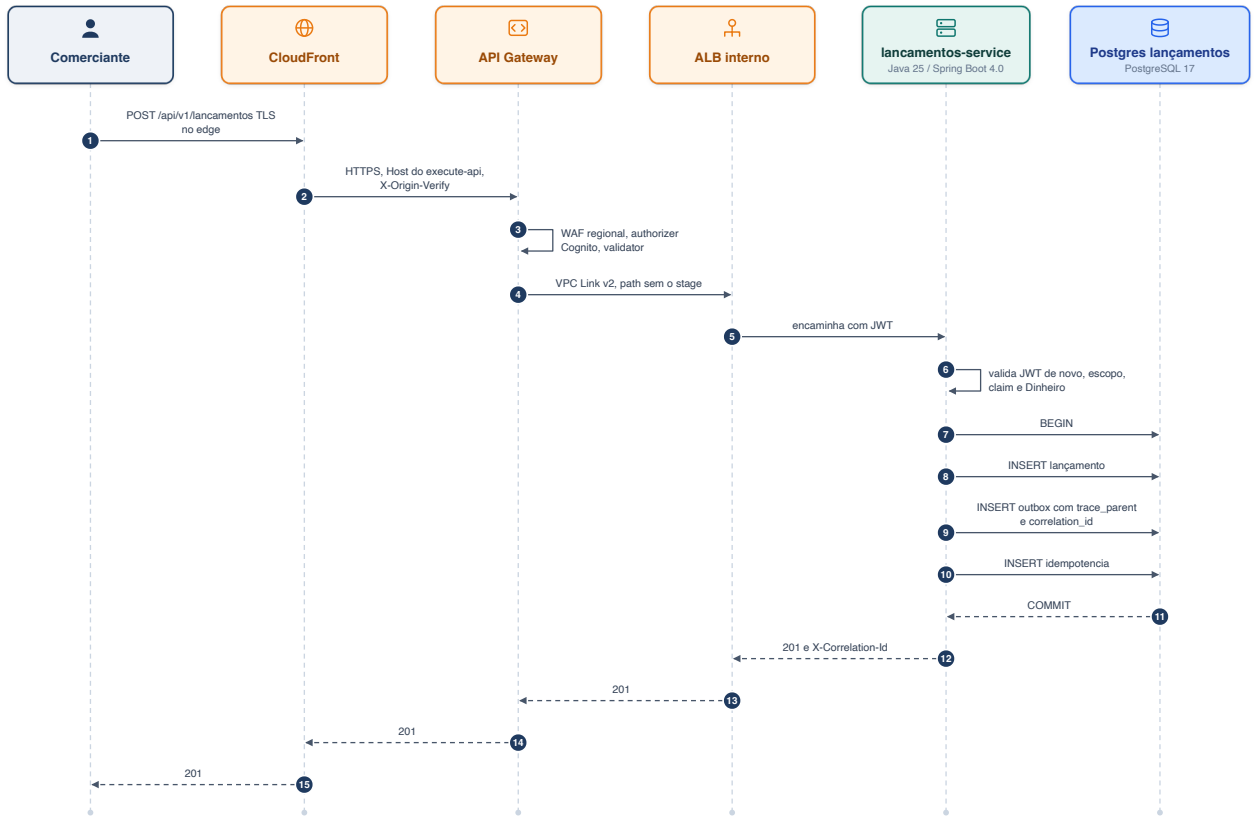
Fluxos de dados

Nomes e tempos seguem o blueprint: tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ `consolidado-lancamentos-dlq`, relay a cada 500 ms, visibility 60 s, `maxReceiveCount` 5, atraso de consolidação p95 < 5 s.

1. Registrar lançamento

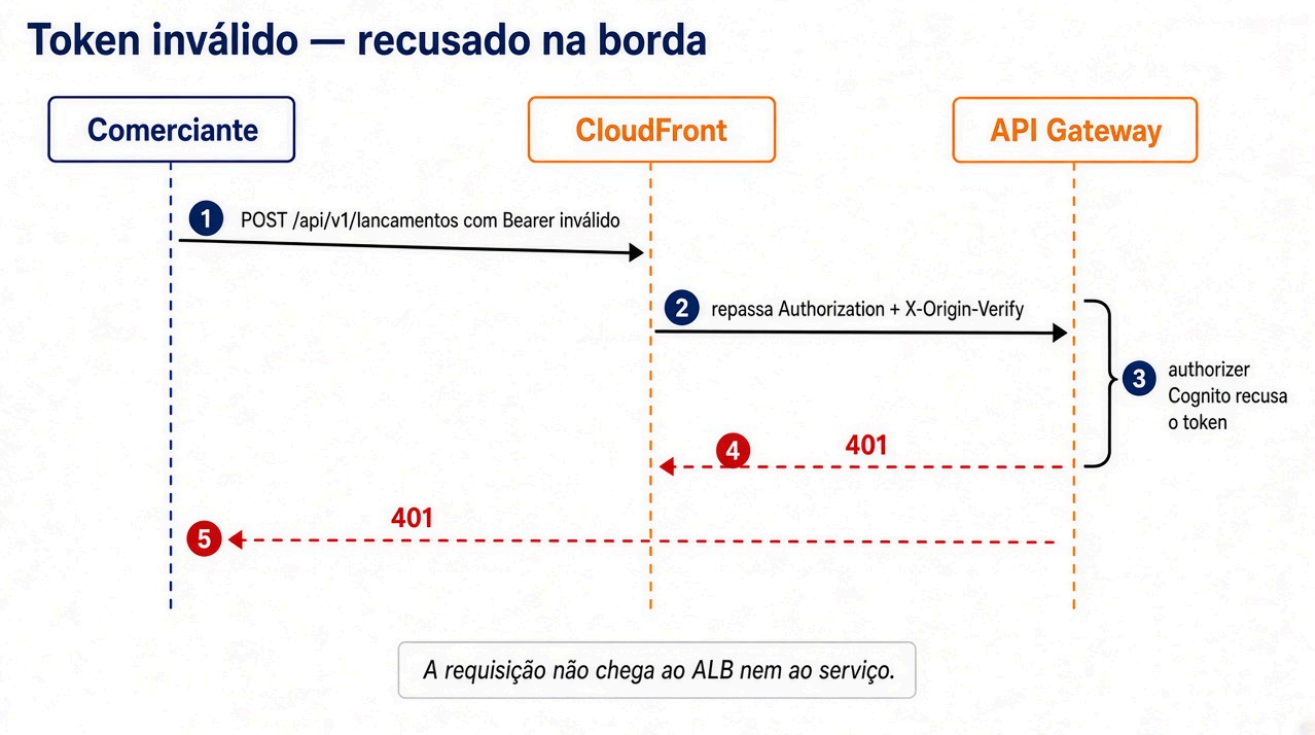
O POST termina no commit local. A publicação é outro fluxo.

Registrar lançamento



Se o header `Idempotency-Key` repetir a mesma requisição, o fluxo para no `SELECT` da idempotência e devolve 200, sem segundo insert.

Token inválido não chega na task:



O WAF regional responde 403 se o header `X-Origin-Verify` não for o atual nem o anterior. Esse caso é chamada direta ao `execute-api`, não o comerciante passando pelo CloudFront. O serviço continua validando o JWT de quem passou: o gateway não substitui o resource server.

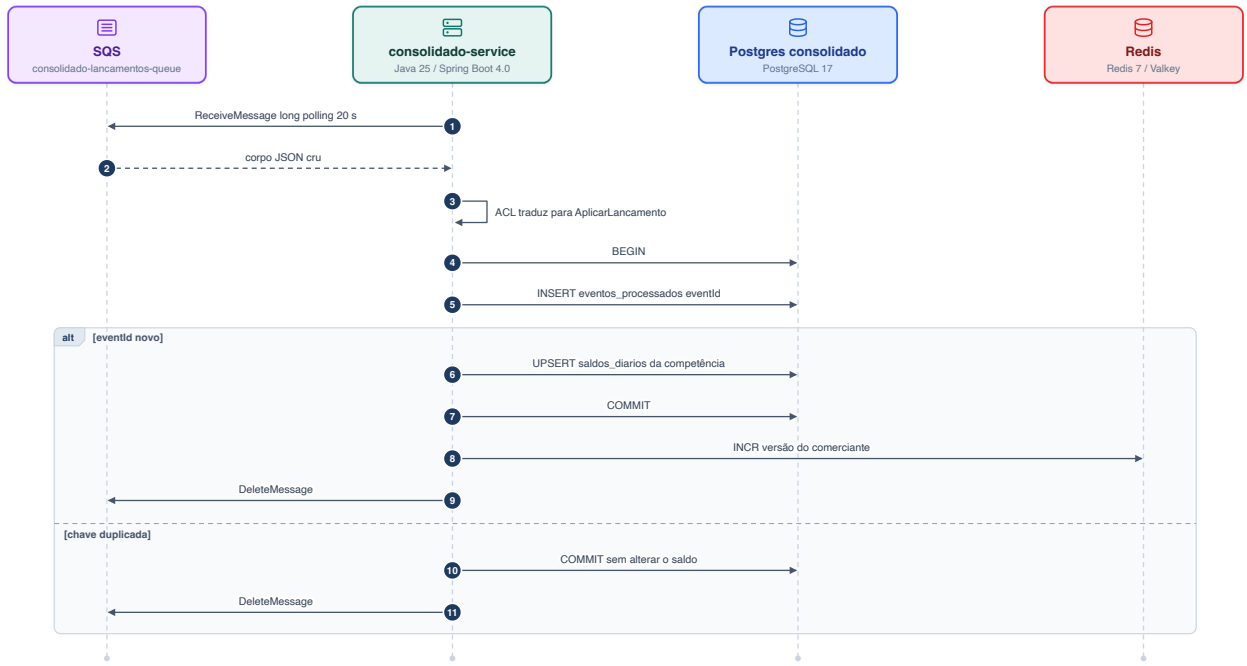
2. Outbox relay



Falha no `Publish`: a transação do lote não marca `publicado_em`, `tentativas` sobe, o próximo ciclo tenta de novo. Duas tasks não pegam a mesma linha por causa do `SKIP LOCKED`. O POST da seção anterior já respondeu ao comerciante antes deste loop.

3. Consolidação

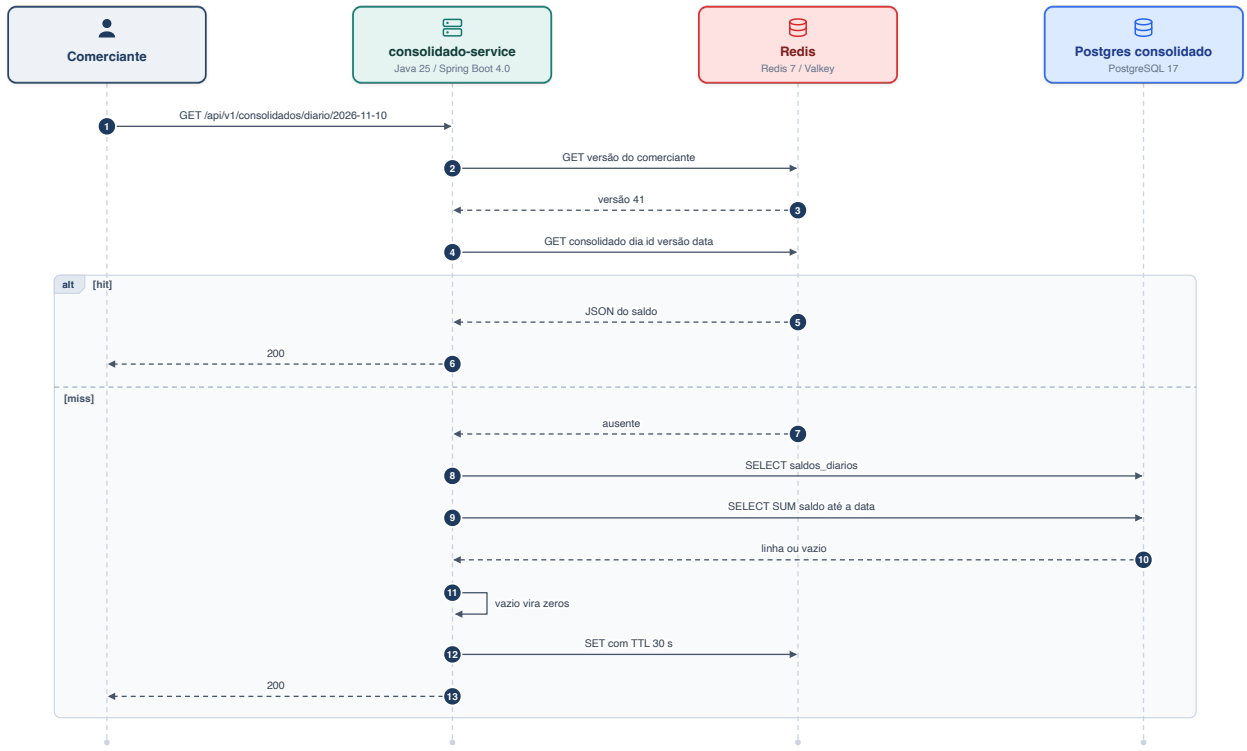
Consolidação



O `INCR` da versão acontece depois do commit do saldo. Se o Redis falhar nesse momento, a versão antiga continua válida até o TTL de 30 s. Se o commit do banco falhar, a mensagem não é apagada e volta após o visibility timeout.

4. Consulta com cache hit e cache miss

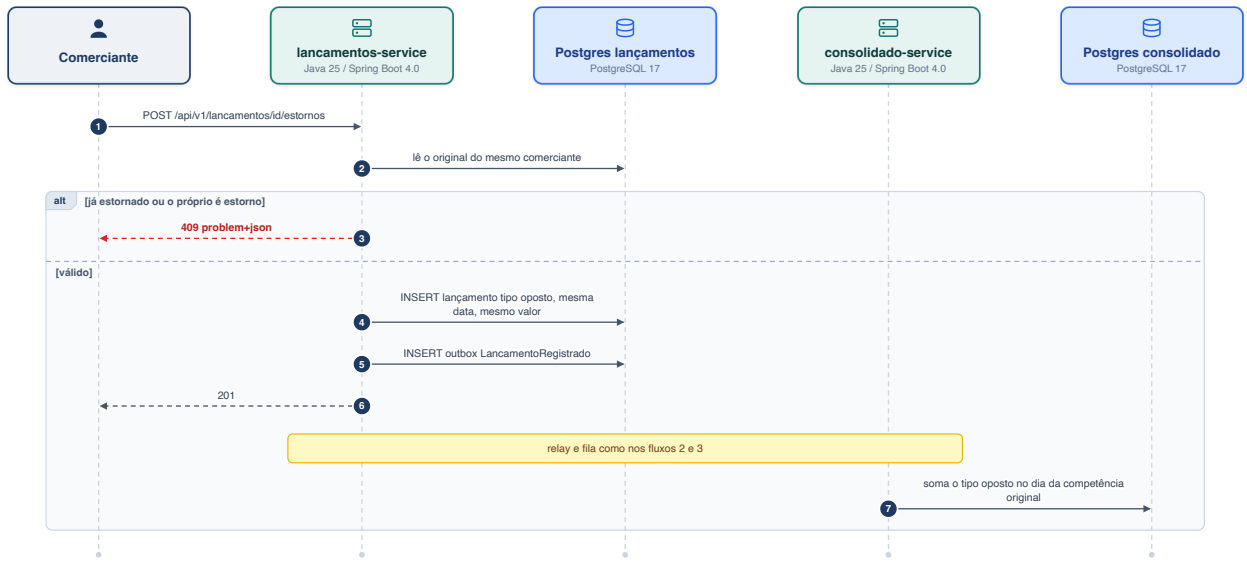
Consulta com cache hit e cache miss



O dia sem movimento não é miss eterno contra a tabela vazia: o serviço cacheia também a resposta zerada, para um cliente que insiste no mesmo dia não martelar o `SUM`.

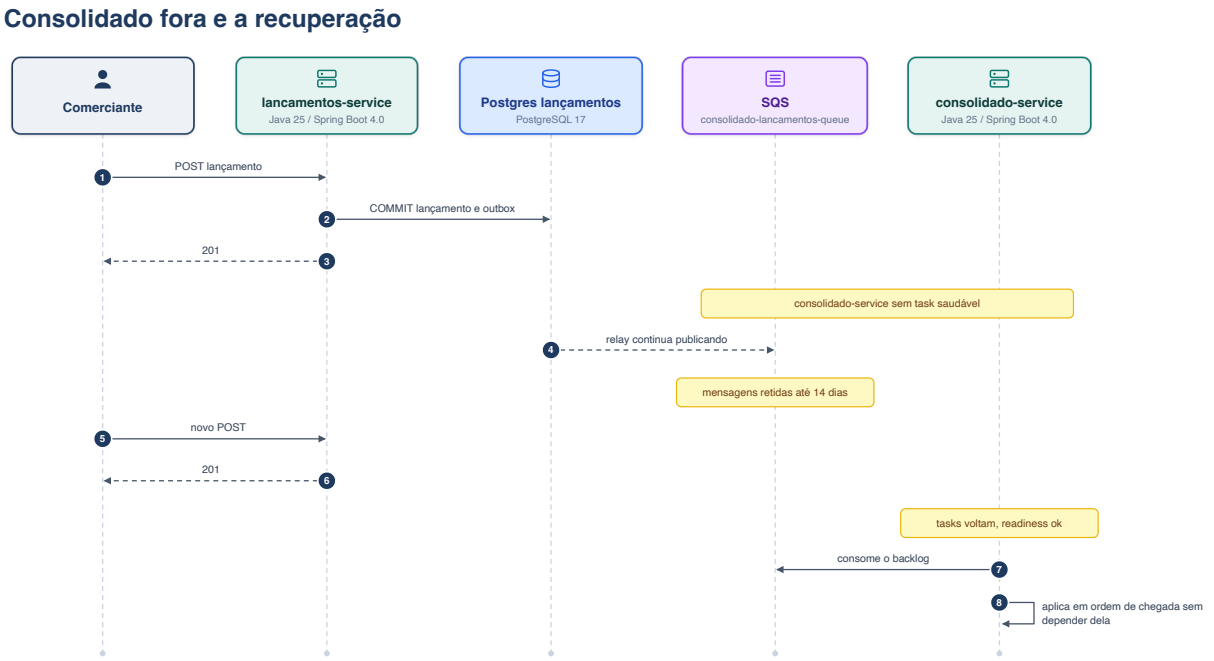
5. Estorno

Estorno



O saldo do dia corrente só muda se a competência do original for o dia corrente. Estorno de um crédito de ontem corrige ontem.

6. Consolidado fora e a recuperação



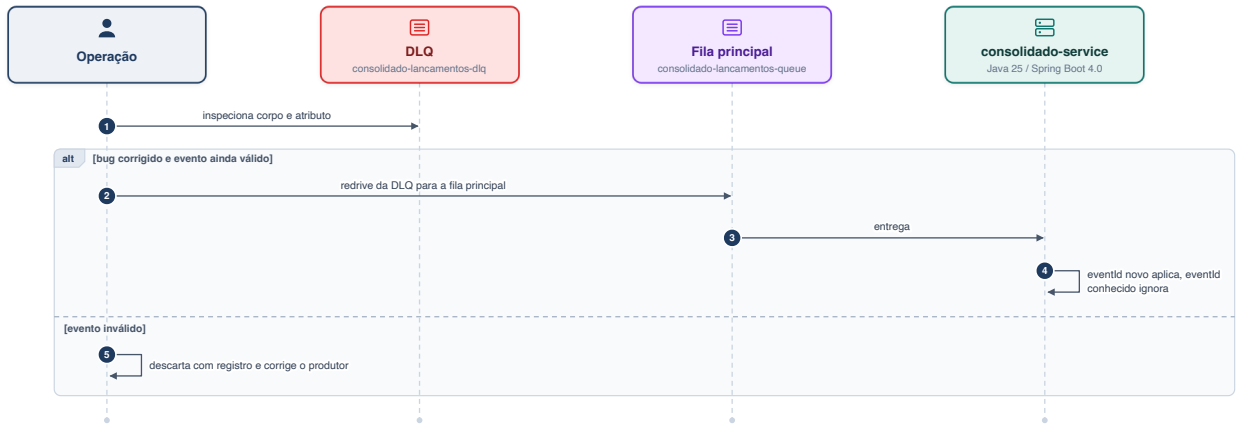
Durante a janela, `GET` no consolidado falha (entra no error budget de 99,5%). `POST` e `GET` de lançamentos seguem (SLO de 99,9%). Não há retry do cliente contra o consolidado embutido no POST.

Drenagem da premissa de 20 evt/s durante 1 h: 72.000 mensagens. Duas tasks a ~100 aplicações/s levam cerca de 6 minutos para alcançar o fim da fila. O p95 de 5 s vale para o regime estável, não para o catch-up depois de uma hora parado — o runbook trata esse catch-up como degradação esperada e medida pela idade da mensagem.

7. Reprocessar a DLQ

Uma mensagem chega na DLQ depois de 5 recebimentos. Causas típicas: JSON incompatível, `eventVersion` desconhecida, bug no `UPSERT`, poison message.

Reprocessar a DLQ



Redrive é seguro por causa do `eventId`. Não se edita a mensagem para "fazer passar". Se o contrato estiver errado, a correção é `eventVersion` nova e consumidor que a entende.

8. O que cada fluxo garante

Fluxo	Garantia	O que não garante
POST	Durabilidade do lançamento no commit	Saldo já visível
Relay	Publicação eventual com retentativa	Latência fixa se o SNS estiver degradado
Consumo	Soma única por `eventId`	Ordem entre eventos
GET com cache	Leitura rápida da última geração cacheada	Leitura monotonicamente fresca em todo miss concorrente
Queda do consolidado	POST segue	GET do saldo segue
DLQ	A fila principal não trava num poison message	O saldo daquele evento, até o redrive

FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/PADROES-DE-INTEGRACAO.MD

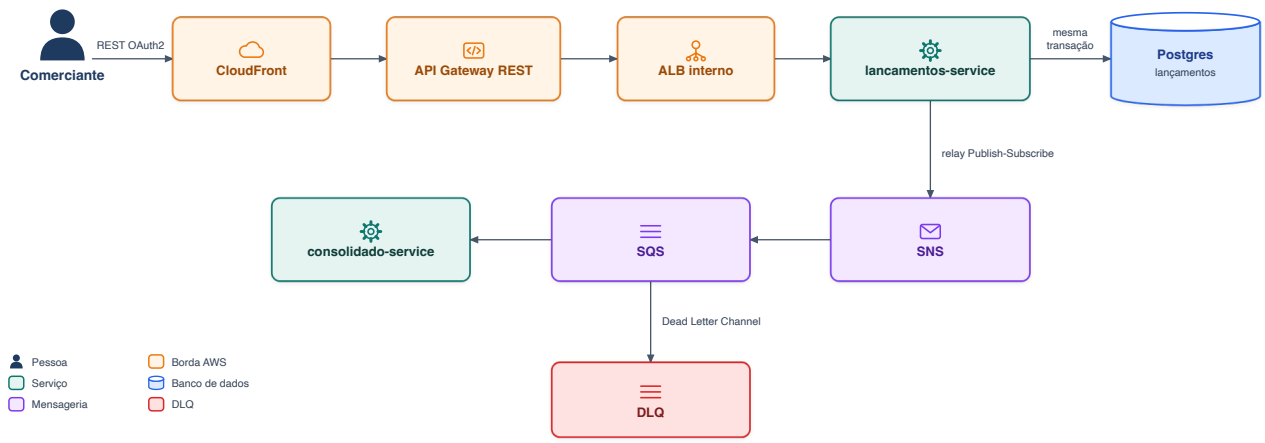
Padrões de integração

Catálogo aplicado, não um glossário de Enterprise Integration Patterns. Cada padrão abaixo existe num trecho concreto do caminho comerciante → borda → serviço, ou do caminho lançamentos → consolidado. O contrato (JSON, paths, atributos) continua no blueprint. O porquê da fila está no ADR 0005; o porquê da outbox, no ADR 0004.

Dois canais, e só dois:

1. **Síncrono**, comerciante para cada serviço, REST com OAuth2.
2. **Assíncrono**, lançamentos para consolidado, SNS e SQS. Não há HTTP entre os dois serviços. Essa ausência é o RNF crítico, não uma omissão.

Padrões de integração — os dois canais



1. Request-Reply síncrono com OAuth2

O comerciante chama `POST /api/v1/lancamentos` ou `GET /api/v1/consolidados/...` e espera a resposta na mesma conexão. Na AWS o caminho é CloudFront → API Gateway REST regional → VPC Link v2 → ALB → serviço (ADR 0031). O token é Bearer JWT, emitido pelo Cognito (ou pelo Keycloak no laboratório). O authorizer do gateway recusa token inválido ou escopo errado antes da VPC. O serviço valida o mesmo JWT de novo, com o JWKS do issuer, e lê `comerciante_id` do claim. O laboratório não tem gateway: o binário continua sendo o resource server (ADR 0011).

O gateway é o API Gateway pattern desta borda: um contrato `/api/v1`, throttle por método (o GET do consolidado acima de 50 req/s em hml e prod) e, para consumidor de integração, usage plan com API key. Cliente final não leva API key. `api_key_required` nasce falso em toda rota e só liga na rota que o tfvars marcar. A cota do usage plan é o teto daquele consumidor, distinto do rate-based por IP do WAF.

O reply de erro do serviço é RFC 9457 (`application/problem+json`). O 401 do authorizer e o 400 do request validator são a resposta do gateway, antes do serviço. O reply de sucesso devolve o recurso e o header `X-Correlation-Id`. Cliente que não recebe resposta por timeout trata o POST como incerto e repete com a

mesma `Idempotency-Key`. Sem essa chave, repetir o POST cria outro lançamento: o canal síncrono não deduplica sozinho.

Não há Request-Reply entre os serviços. Um GET do consolidado de volta ao lançamentos, no meio da consolidação, recolocaria o relatório no caminho crítico da escrita.

2. Publish-Subscribe Channel

O relay publica em `LANCAMENTOS_TOPIC_ARN`. O tópico é o canal. A assinatura que existe hoje entrega numa fila. Uma segunda assinatura (outro contexto, um data lake) ganha outra fila e outra role, sem o publicador saber o nome do consumidor. É o motivo de SNS na frente de SQS, e não uma fila em que o lançamentos escreve direto: o publicador conhece o tópico, não os interessados.

O corpo, com raw message delivery, é o JSON da linguagem publicada. O tópico não transforma payload.

3. Point-to-Point e Competing Consumers

A fila `CONSOLIDADO_QUEUE_URL` é point-to-point: uma mensagem é processada por um consumidor lógico (o serviço de consolidado). Várias tasks fazem long polling ao mesmo tempo. O SQS entrega cada mensagem a uma task por vez (visibility timeout de 60 s). Isso é Competing Consumers: a escala de drenagem é o número de tasks, não um consumidor único com thread pool interno.

O bulkhead do Resilience4j (ADR 0023) limita quantas mensagens uma task processa em paralelo, para o pool JDBC não virar o concorrente real. Sem esse teto, Competing Consumers “funcionando bem” esgota `max_connections` no pico de catch-up.

4. Transactional Outbox

O caso de uso grava o lançamento, a linha de idempotência e a linha de `outbox_eventos` na mesma transação. O 201 sai quando esse commit volta. O relay, noutro agendamento, lê com `SELECT … FOR UPDATE SKIP LOCKED` e publica. Dual-write (commit e `Publish` em sequência sem outbox) perde o evento quando o processo morre no meio, ou segura o 201 até o SNS responder. Os dois resultados violam o desenho.

O relay não é um terceiro serviço. É um agendador dentro do processo de lançamentos. Duas tasks não levam a mesma linha por causa do `SKIP LOCKED`.

Colunas de contexto, gravadas no POST e relidas pelo relay: `trace_parent`, `trace_state`, `correlation_id`. O detalhe de propagação está no documento 07.

5. Idempotent Receiver

Entrega at-least-once produz duplicata. O consumidor insere `eventId` em `eventos_processados` na mesma transação do `UPSERT` do saldo. Chave duplicada: a transação termina sem somar de novo e a mensagem é apagada. Reprocessar a DLQ é seguro por causa disso.

O POST tem o mesmo padrão no canal síncrono, com outra chave: `Idempotency-Key` mais a impressão digital do corpo. Chave igual e corpo diferente responde 409. São dois receivers idempotentes, com chaves que não se misturam. `eventId` não substitui `Idempotency-Key`, e o contrário também não.

6. Dead Letter Channel

Depois de `maxReceiveCount` 5, a mensagem vai para `consolidado-lancamentos-dlq`. A fila principal segue. Sem esse canal, uma mensagem inválida volta para sempre, queima receive e esconde o erro atrás de um backlog que não drena.

A DLQ não é reprocessada por código. O runbook está no documento 07: corrigir a causa, redrive, deixar o receiver idempotente absorver a repetição. Versão de evento desconhecida vai para a DLQ de propósito. Alargar o consumidor "para não falhar" engoliria uma quebra de contrato.

7. Message Expiration e retenção

A fila principal retém a mensagem por **14 dias**. Esse é o prazo em que o consolidado pode ficar fora sem perder evento. Não é um TTL de negócio do lançamento: o fato já está no Postgres de lançamentos. Passados 14 dias sem consumidor, a mensagem expira e o saldo daquele evento só volta por reconciliação, não pela fila. O alarme de idade existe para ninguém descobrir a expiração no décimo quinto dia.

Visibility de 60 s é o prazo de "esta tentativa ainda está com um consumidor". Processar leva milissegundos. Sessenta segundos cobrem um GC ruim sem devolver a mensagem cedo, o que duplicaria trabalho (o receiver aguenta a duplicata, mas o alarme de lag mente).

A outbox tem outra retenção, operacional: linha com `publicado_em` preenchido é apagada depois de 7 dias. Linha não publicada não expira por job. O documento de desempenho fixa essa limpeza.

8. Published Language e Anticorruption Layer

A linguagem publicada é o JSON `LancamentoRegistrado` versão 1: `eventId`, `eventType`, `eventVersion`, `occurredAt`, `lancamentoId`, `comercianteId`, `tipo`, `valor` (string decimal), `moeda`, `dataLancamento`, `lancamentoEstornadoId`. Sem descrição e sem categoria. Isso é minimização, não economia de bytes: a descrição pode carregar dado pessoal, e o consolidado não precisa dela para somar.

O consumidor não desserializa esse JSON direto num agregado. O `TradutorEventoLancamento` é a camada anticorrupção: traduz o JSON para o comando interno `AplicarLancamento`. Campo desconhecido é ignorado. `eventVersion` diferente de 1 falha a mensagem, que cai na DLQ depois das tentativas. Assim o modelo de saldo não importa tipo do produtor, e o produtor não importa tabela do consumidor.

9. Event-Carried State Transfer, não Event Notification

Há duas formas de contar o fato ao consolidado.

Event Notification. A mensagem leva só `eventId` (e talvez `lancamentoId`). O consumidor chama o lançamentos para buscar o valor. O relatório passa a depender da API de escrita estar no ar e do contrato HTTP. O RNF crítico deixa de valer no caminho inverso: o consolidado, ao voltar de uma queda, não drena a fila se o

lançamentos também estiver degradado. Além disso cria o acoplamento que a outbox existe para evitar.

Event-Carried State Transfer do agregado inteiro. A mensagem leva descrição, categoria, tudo. O consumidor vira uma segunda cópia do lançamento, com dado pessoal na fila, no disco do SQS e em qualquer assinante futuro. A fila deixa de ser um fato contábil mínimo e passa a ser um extrato.

O que foi escolhido. Transferência do **recorte que a projeção precisa para somar**, e nada mais. O consolidado aplica o saldo sem callback. Um estorno viaja como outro `LancamentoRegistrado`, com tipo oposto e `lancamentoEstornadoId` preenchido: o consumidor não tem regra especial de estorno. Se um dia o relatório precisar da descrição, a decisão se reabre com consciência de que isso copia dado pessoal — não se "acrescenta o campo porque já estava no banco".

10. Correlation Identifier

Três identificadores, com papéis que não se substituem:

Id	Quem cria	Para que serve
`correlationId`	A borda HTTP, a partir de `X-Correlation-Id` ou um UUID novo	Achar a jornada nos logs, inclusive depois da fila
`traceId` / `traceparent`	O tracer W3C no POST	Achar os spans no Tempo ou no X-Ray
`eventId`	O caso de uso, ao gravar a outbox	Idempotência de negócio. Não é id de trace

O `correlationId` vai no MDC, na coluna `correlation_id` da outbox e no atributo de mensagem `correlationId`. O consumidor reabre o MDC com ele. Um suporte que recebe o id do comerciante (o header volta na resposta) filtra log dos dois serviços sem precisar do trace.

11. Message Envelope e atributos

Raw message delivery está ligado: o corpo no SQS é o JSON da linguagem publicada, não o envelope SNS (`Message`, `MessageId`, `TopicArn` em volta). Um envelope em volta quebraria o consumidor e a ACL, que esperam o JSON da seção 6 do blueprint.

Metadado que não é fato de negócio vai em **message attribute**, tipo `String`:

Atributo	Conteúdo
`eventType`	`LancamentoRegistrado`
`traceparent`	W3C, gravado na outbox no POST
`tracestate`	Quando houver
`correlationId`	O mesmo UUID da requisição

Atributo permite filtro de assinatura no futuro (`eventType`) sem parsear o corpo, e tira o contexto de trace de dentro do contrato versionado. Colocar `traceparent` dentro do JSON obrigaria todo consumidor, inclusive um de BI, a conhecer telemetria. Atributo deixa o JSON estável.

O relay publica com esses atributos relendo a linha. O consumidor pede `MessageAttributeNames=All` e `MessageSystemAttributeNames=All`. Sem o `All`, o SQS devolve o corpo e omite o contexto, e o trace quebra no meio mesmo com a coluna preenchida.

12. Retry com backoff e jitter

Dois retries, em camadas diferentes.

Publicação. Resilience4j no cliente SNS do relay: Retry com backoff exponencial e jitter, TimeLimiter, e CircuitBreaker. O jitter existe para duas tasks não martelarem o SNS no mesmo milissegundo quando ele volta. Quando o circuito abre, o relay adia o lote: as linhas ficam na outbox com `proxima_tentativa_em` empurrado, até 60 s. Nada se perde, e o POST não participa.

Consumo. Falha transitória de banco no consumidor: Retry com backoff, dentro do visibility. Falha de domínio (versão desconhecida, JSON inválido) não retenta no aplicativo: devolve a falha, o SQS conta o receive, e a DLQ decide. Retentar 4xx de negócio só adia a DLQ.

O cliente do caixa, do lado de fora, não retenta 4xx. 409 de idempotência não melhora na segunda tentativa com outro corpo. 401 não melhora sem token novo.

13. Circuit Breaker

Não há circuito entre lançamentos e consolidado, porque não há chamada entre eles. Os circuitos que existem:

Onde	O que abre	O que acontece aberto
Relay, cliente SNS	Falha ou lentidão do publish	O lote fica na outbox. O 201 já foi embora.
Consolidado, cliente Redis	Falha ou lentidão do cache	A leitura cai no Postgres. O saldo continua correto, mais lento.
Deploy ECS	Task nova sem readiness	Rollback. É outro mecanismo, com o mesmo nome. ADR 0020.

O estado do circuito entra em `/actuator/health` sem derrubar readiness. Redis aberto não pode tirar a task do ALB: isso transformaria uma degradação de latência em indisponibilidade.

14. Garantia de entrega e ordenação

A garantia é **at-least-once**. O SNS pode entregar duas vezes. O relay pode publicar e morrer antes de gravar `publicado_em`, e publicar de novo. O SQS Standard pode entregar fora da ordem em que o SNS recebeu.

Por que não FIFO. A aplicação do saldo é uma soma. Soma é comutativa: crédito de 10 e débito de 4 no mesmo dia produzem o mesmo saldo em qualquer ordem. A duplicata é resolvida por `eventId`, não por ordenação. FIFO cobraria mais, imporia limite de throughput por message group e exigiria escolher a chave do grupo (`comercianteId` serializaria o caixa de um comerciante atrás de uma mensagem lenta). Nada disso melhora o saldo. FIFO voltaria a ser argumento se um evento futuro não fosse comutativo (saldo que depende do evento anterior, máquina de estados). Esse evento não existe. O ADR 0005 registra a mesma escolha pelo preço; aqui o motivo é a álgebra do domínio.

O que FIFO não consertaria: a janela entre o commit do lançamento e o publish. Essa janela é a consistência eventual, com meta p95 de 5 s, e é aceita.

15. Versionamento de contrato

API HTTP: prefixo `/api/v1`. Campo novo na resposta é aditivo; cliente ignora o que não conhece. Quebra (tirar campo, mudar tipo de `valor`) é `/api/v2` com os dois prefixos no ar até o último cliente antigo sair. O ALB roteia os dois prefixos para o mesmo target group enquanto a convivência durar.

Evento: `eventVersion`. Campo novo entra na versão 1 se for aditivo e o consumidor antigo puder ignorá-lo. Mudança que o consumidor precisaria entender (outro significado de `tipo`, valor em centavos inteiros) é versão 2, publicada em paralelo durante a convivência, com a ACL aceitando as duas e a versão desconhecida indo para a DLQ. Não se reescreve o histórico: evento já publicado permanece na versão em que nasceu.

16. Contract testing

O produtor e o consumidor moram em repositórios Maven separados. Um teste que só sobe os dois no Compose pega divergência tarde. O contrato a travar é o JSON da linguagem publicada mais os atributos obrigatórios (`eventType`, e a presença de `correlationId` / `traceparent` quando o publish sai do relay).

Pact (consumer-driven) cabe: o consolidado publica a expectativa do JSON v1 e dos atributos; o lançamentos verifica essa expectativa no CI, sem subir o outro processo. Até o Pact entrar no pipeline (documento 14), o mínimo que já segura o contrato é o teste de integração do produtor, que serializa o evento, e o teste da ACL, que recusa versão desconhecida. Os dois usam o mesmo fixture. Fixture copiado à mão nos dois projetos é o risco: uma mudança num lado só passa no teste local e quebra na fila. O fixture deve ter uma origem só, ou o Pact substitui a cópia.

17. Critérios de segurança para consumir

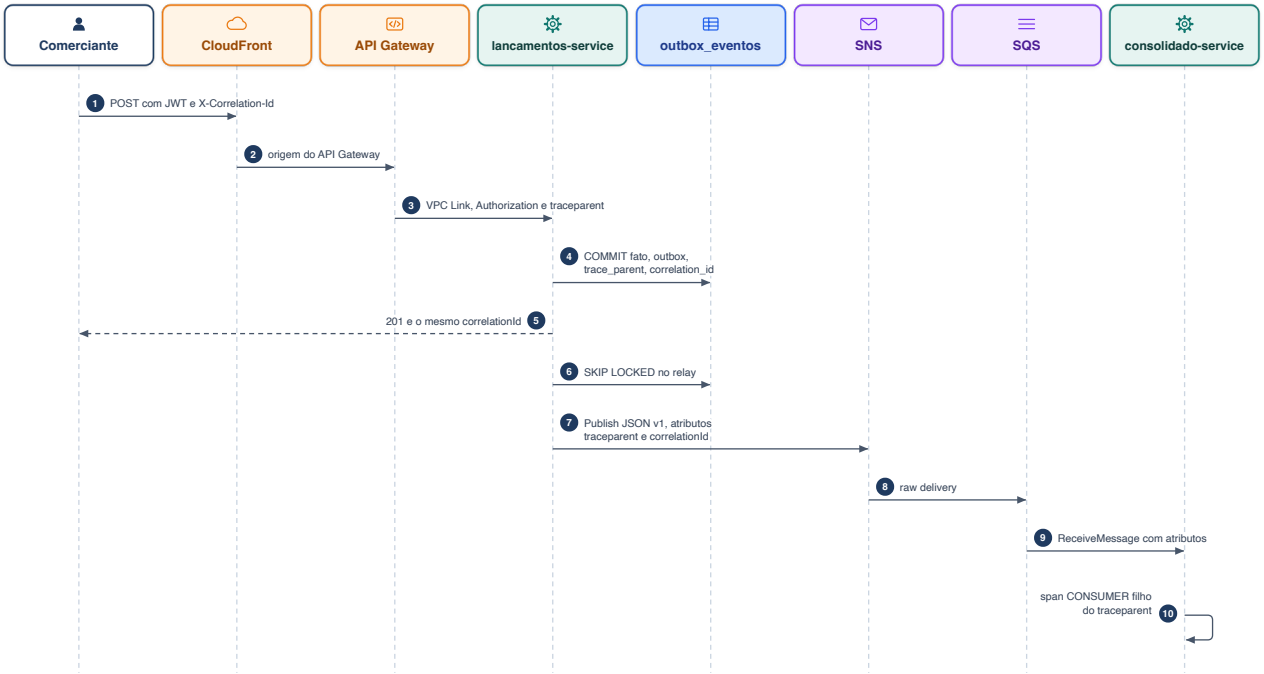
O detalhe de STRIDE e OWASP está no documento 06. O mínimo para um integrador novo:

Controle	HTTP	Mensagem
Identidade	Bearer JWT do issuer combinado. API key só no consumidor de integração cadastrado no usage plan; o cliente final não leva chave.	IAM. Não existe endpoint “injeta evento”.

Controle	HTTP	Mensagem
Autorização	Escopo no authorizer do gateway e de novo no serviço. Leitura não recebe write.	`sns:Publish` só na role de lançamentos, no ARN do tópico. `sqs:ReceiveMessage` e `DeleteMessage` só na role de consolidado, no ARN da fila. Segundo consumidor ganha fila e role novas.
Política de recurso	O ALB não é a autorização.	Policy do tópico e da fila recusa principal que não seja a role nomeada. `Resource` é o ARN, não `*`.
Tenant	`comercianteId` no corpo é ignorado. O claim do token manda.	Só o produtor preenche `comercianteId`. O consumidor não "corrige" tenant.
KMS	Segredo de banco na CMK do serviço.	SSE-KMS na fila, no tópico e na DLQ. A role de consumo tem `kms:Decrypt` nessa chave, não na chave do outro serviço.
TLS	TLS 1.2+ no CloudFront. JDBC com `sslmode=require`. Redis com TLS na AWS.	API da AWS em HTTPS, pelo VPC endpoint em hml e prod.
Validação	Bean validation na borda HTTP e invariante de novo no domínio.	ACL valida versão, tipo fechado, valor decimal positivo, UUID. JSON inválido não é aplicado pela metade.
Segredo no payload	Token não vai para log nem query string.	Senha, descrição e token não entram no evento nem no atributo.

Assinatura HMAC do corpo não está ligada. A policy da fila, o raw delivery dentro da conta e a rede fechada são o controle. HMAC passa a ser necessário se um segundo produtor puder publicar no mesmo tópico ou se a fila deixar de ser privada. Até lá, seria mais uma chave para girar, sem um agressor no caminho que a chave impedisse.

Correlação ponta a ponta — do POST ao consumidor



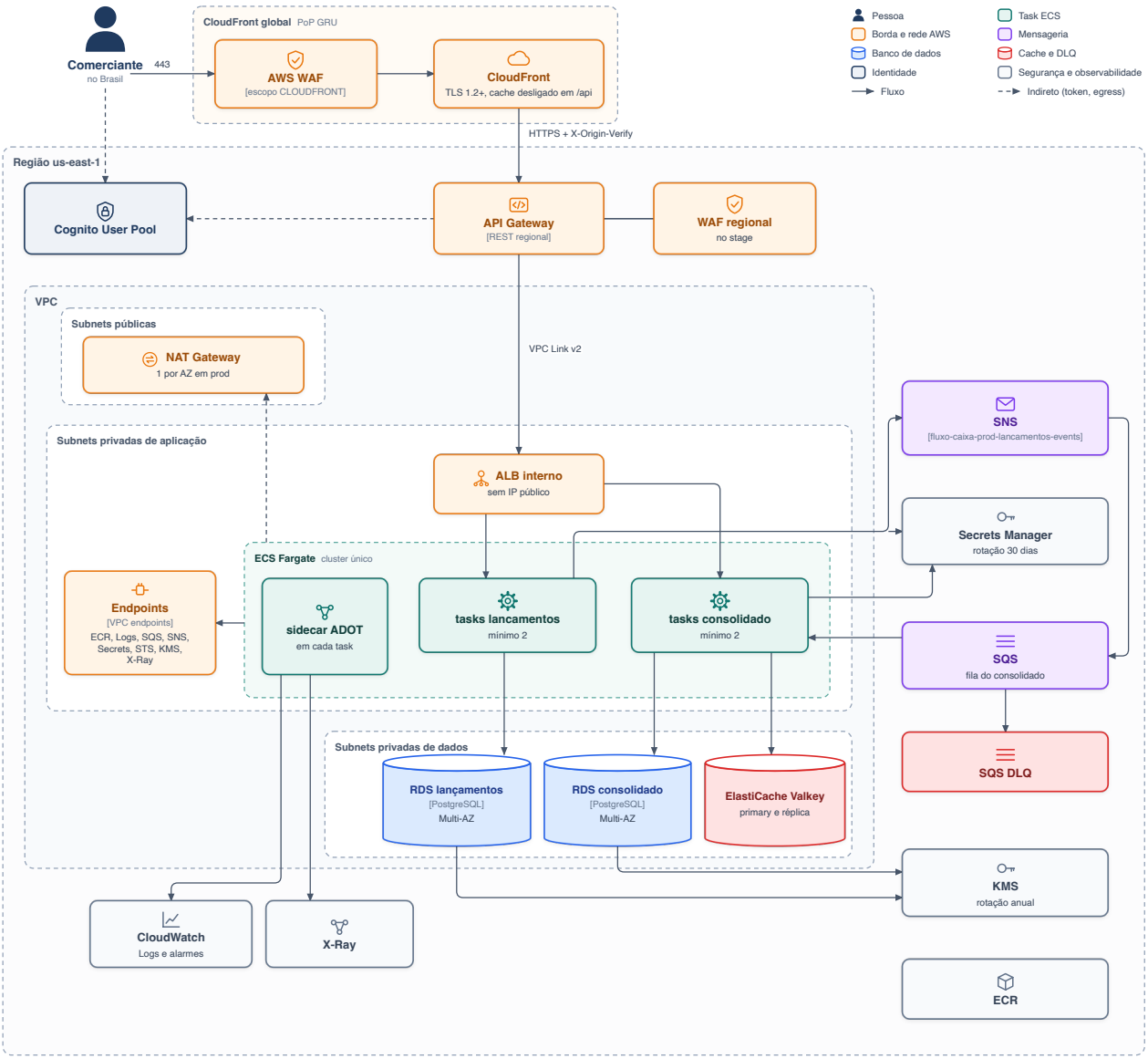
FONTE: DOCUMENTOS-BRUTOS/03-ARQUITETURA/DEPLOYMENT-AWS.MD

Deployment na AWS

Região `us-east-1` (ADR 0021). Três ambientes, `dev`, `hml` e `prod`, com o dimensionamento da seção 15 do blueprint. A topologia abaixo é a de **produção** com API Gateway ligado (ADR 0031): VPC em três zonas, NAT por zona, endpoints de interface, CloudFront com WAF, REST API regional, VPC Link v2, ALB interno, ECS Fargate, dois RDS Multi-AZ, ElastiCache com réplica, SNS/SQS/DLQ, Cognito, Secrets Manager com rotação e ADOT. Hml usa duas zonas, um NAT e endpoints. Dev usa um NAT, sem endpoint, e Fargate Spot. O dia a dia do desenvolvedor é o Docker local, com custo de nuvem zero.

1. Diagrama

Deployment na AWS — produção



S3 entra como gateway endpoint (sem custo horário) para o pull de camadas via ECR. A lista de interface é: ECR api, ECR dkr, CloudWatch Logs, SQS, SNS, Secrets Manager, STS, KMS e X-Ray. KMS e X-Ray não estão na lista mínima da seção 12 do blueprint; existem para a CMK e o sidecar ADOT não saírem por NAT. O agente de integração confere o Terraform.

CloudFront é global. O desenho o coloca ao lado da região porque o Web ACL de escopo `CLOUDFRONT` e o certificado ACM moram em `us-east-1`, e o PoP que o comerciante usa é GRU.

2. Rede

Camada	Quem mora	Regra
Borda	CloudFront, WAF CLOUDFRONT, API Gateway, WAF regional	Única URL pública é o CloudFront. `/api/*` sem cache. O `execute-api` existe e o WAF regional o fecha.
Pública da VPC	NAT	Sem rota de entrada. O ALB não está aqui.
Aplicação	ALB interno, VPC Link, tasks, endpoints	O security group do ALB aceita o prefix list do CloudFront e o security group do VPC Link. Task sem IP público. Egress para RDS, Redis, endpoints e NAT.
Dados	RDS, ElastiCache	5432 só do security group do serviço dono do banco. 6379 só do security group do consolidado.

Três AZs em prod permitem perder uma zona e ainda agendar task nas outras duas. O mínimo de duas tasks usa `spread` por AZ. NAT por AZ evita que a queda de uma zona leve o egress das outras. O preço está no custo: três NAT em `us-east-1` são USD 99/mês; os 27 endpoints são USD 197/mês.

Dev não cria endpoint: o tráfego AWS sai pelo único NAT. Hml cria os endpoints em duas zonas, para o ensaio da rede privada não acontecer pela primeira vez em prod.

3. Borda

ADR 0022 e ADR 0031.

- `PriceClass_All`, HTTP/2 e HTTP/3, TLS `TLSv1.2_2021`, ACM em `us-east-1`.
- WAF de escopo CLOUDFRONT: Common, Known Bad Inputs, SQLi, IP Reputation, rate-based por IP. AnonymousIpList em count no dev. Log do WAF com corpo e `Authorization` redigidos.
- Com `enable_api_gateway = true`, `/api/v1/lancamentos*` e `/api/v1/consolidados*` usam a origem HTTPS do API Gateway (`origin_path` `/v1`, header `X-Origin-Verify`, origin request policy `AllViewerExceptHostHeader`). O comportamento default continua 404 na função, sem chamar origem.
- REST API regional, stage `v1`, authorizer Cognito por escopo, VPC Link v2 no ALB. WAF regional no stage exige o header. Rotação do valor em 30 dias (90 em dev), mantendo o anterior.
- ALB interno, listener HTTP 80. O TLS da internet termina no CloudFront; o trecho VPC Link → ALB é HTTP na rede da AWS. Regras `/api/v1/lancamentos*` e `/api/v1/consolidados*`, mais o prefixo `/v1` reescrito quando o gateway está ligado. Health check: `GET /actuator/health/readiness`, direto no tar-

get, sem passar pelo listener. O access log vai para S3 com SSE-S3: o ELB não aceita CMK nesse bucket.

- O limite rate-based fica **acima** do pico legítimo de um IP. 50 req/s é o pico do serviço, não de um cliente. A ordem de partida é 30.000 requests por IP a cada 5 minutos. O throttle do gateway no GET do consolidado fica acima desses 50 em hml e prod.
- Shield Standard acompanha o CloudFront. Shield Advanced fica de fora.
- `/actuator` e o Swagger não são rota do gateway nem do ALB.
- `enable_api_gateway = false` tira o gateway e devolve a VPC Origin do ADR 0022.

4. Computação

Serviço ECS	Task em prod	Mínimo	Máximo	Capacidade
`lancamentos`	0,5 vCPU, 1 GiB, ARM64, sidecar ADOT incluso, on-demand	2	10	CPU 60% e request por target
`consolidado`	1 vCPU, 2 GiB, ARM64, sidecar incluso, on-demand	2	10	CPU 60%, request por target e backlog da fila

O blueprint (seção 15) fixa o máximo de prod em 10 para os dois. Dev: 0,5 vCPU / 1 GiB, mínimo 1, máximo 2, Fargate Spot, nos dois serviços. Hml: lançamentos 0,5/1 e consolidado 1/2, mínimo 1, máximo 3, on-demand.

Deployment circuit breaker com rollback (ADR 0020).

A task de 1 vCPU do consolidado é maior do que o pico de 50 req/s exige (ρ de cerca de 17% com duas tasks). O tamanho existe para caber o sidecar e o JIT. O lançamentos fica em 0,5 vCPU porque o caminho de escrita é mais curto.

A raiz do container é somente leitura. O `/tmp` é um volume do Fargate e nasce como root. O entrypoint da imagem faz `chmod 1777` e desce para o usuário `app`; sem isso o Tomcat não cria o diretório temporário e a task morre no start.

5. Dados

Recurso	Produção	Papel na falha
RDS `lancamentos`	PostgreSQL 17, `db.t4g.medium`, Multi-AZ, gp3 50 GB, KMS, PITR, senha gerenciada	Standby noutra AZ. Falha do consolidado não o alcança.
RDS `consolidado`	Igual, instância separada. Réplica de leitura parametrizável, default 0	Restaura sem restaurar a trilha de lançamentos.
RDS Proxy	Parametrizável, default desligado	Não substitui o wrapper de segredo. Documento 12.

Recurso	Produção	Papel na falha
ElastiCache	Valkey, `cache.t4g.small`, uma réplica, failover, TLS, sem AUTH token	Perda do primário promove a réplica. Perda do cache degrada para o RDS.
Secrets Manager	Um segredo por RDS, rotação de 30 dias em prod e hml, 90 em dev	O wrapper rebusca. A senha não está na imagem nem no Git.

RPO ≤ 5 min vem do PITR. RTO ≤ 1 h é o tempo de restaurar dentro da região, ensaiado em game day. Não é failover de região.

`DB_READER_URL`, quando a réplica existir, manda só transação `readOnly` para o reader. O consumidor da projeção permanece no primário (ADR 0030).

6. Mensageria e identidade

- SNS, SQS e DLQ com SSE-KMS. Policy: a role de lançamentos publica; a role de consolidado consome. A policy do tópico não inclui `sns:Unsubscribe`: o SNS rejeita essa action. Prefixo `fluxo-caixa-prod-` (e `dev` / `hml` nos outros ambientes). O serviço lê ARN e URL, não o nome.
- Cognito User Pool no plano Essentials, resource server `fluxo-caixa`, escopos `lancamentos:write`, `lancamentos:read`, `consolidado:read`. Prefixo de autoridade `fluxo-caixa/`.
- Pre Token Generation Lambda (v2, plano Essentials) insere `comerciante_id` e os três escopos do resource server no access token. O `InitiateAuth` sozinho não traz escopo customizado; sem o acréscimo o API Gateway responde 403. Sem o claim o serviço responde 401. O domínio hospedado é `fluxo-caixa-<env>-<account_id>`.
- JWKS buscado pelo serviço. Não há PEM na imagem.

7. Observabilidade na implantação

Sidecar ADOT: traces para X-Ray, métricas para CloudWatch. Log JSON com `traceId`, `spanId` e `correlationId`. O relay republica o `traceparent` guardado na outbox. Detalhe e as queries no documento 07.

Alarmes (5xx, p99, profundidade e idade da fila, DLQ, outbox, circuito aberto, `RotationFailed`) publicam num tópico SNS de operação. Container Insights desligado em dev, ligado em hml, enhanced em prod. Retenção de log: 7, 14 e 30 dias.

8. Escalabilidade

Escala horizontal até 10 tasks por serviço em prod. O consolidado também escala pelo backlog da fila. O próximo gargalo, se o pico deixar de ser 50 req/s, é o `SUM` do acumulado e o primário do Valkey, não o CloudFront. A resposta está no documento 12: janela SQL, depois réplica, bem depois outro read store. Nada disso é necessário para o requisito atual.

9. Disaster recovery

Cenário	Comportamento	RTO esperado
Task morta	ECS substitui. ALB tira do target group no readiness.	minutos
AZ morta	Task na outra AZ segue. RDS Multi-AZ promove. ElastiCache promove a réplica. NAT das outras AZs segue.	minutos para dado gerenciado
Mensagem poison	DLQ, serviço continua	operação humana para aquele evento
Corrupção lógica	PITR, no máximo 5 min atrás	até 1 h
Rotação de senha falha	Alarme `RotationFailed`. Wrapper rebusca. Runbook do ADR 0024.	minutos se o runbook rodar
Região `us-east-1` indisponível	Sem active-passive. O RTO de região não é o RTO de 1 h.	evolução, com decisão jurídica se o secundário sair dos Estados Unidos

Restauração é para uma instância nova. Não se restaura por cima da instância que ainda recebe escrita.

10. Ambientes

	dev	hml	prod
AZs	2	2	3
NAT	1	1	3
Endpoint de interface	não	2 AZs	3 AZs
Fargate	Spot	on-demand	on-demand
RDS	micro, Single-AZ, 20 GB	small, Single-AZ, 20 GB	medium, Multi-AZ, 50 GB
Backup	1 dia, sem deletion protection	7 dias	14 dias, PITR, deletion protection

State Terraform em S3, um prefixo por ambiente, lock por lockfile, em `us-east-1` (ADR 0029).

11. O que o diagrama não inclui de propósito

Conta separada por ambiente, Transit Gateway, Shield Advanced, multi-região e EKS. São camadas de uma organização com vários produtos ou com o gatilho do documento 13. Para dois serviços, aumentam o piso sem mudar o RNF. A comparação com `sa-east-1` está no documento de custos: a mesma produção sairia 1,92 vez mais cara, e a região vigente é a Virgínia com a salvaguarda do art. 33, não por desconhecimento do preço de São Paulo.

PARTE 04

Decisões arquiteturais (ADRs)

Registro das 31 decisões, com contexto, alternativas e consequências

FONTE: DOCUMENTOS-BRUTOS/04-ADR/README.MD

Decisões de arquitetura

Formato MADR. Data de todas: **2026-11-10**. O contrato (nomes, portas, JSON) permanece no blueprint. Estes textos registram a alternativa e o preço.

Decisão substituída não é apagada. O status aponta o ADR vigente.

ADR	Status	Decisão
0001	Aceito	Registrar decisão neste formato, no repositório
0002	Aceito	Dois microsserviços, um por bounded context
0003	Aceito	CQRS e integração por evento, sem chamada síncrona entre os serviços
0004	Aceito	Outbox transacional com relay, em vez de dual-write ou CDC
0005	Aceito	SNS + SQS Standard + DLQ
0006	Aceito	PostgreSQL 17, instância RDS por serviço em produção
0007	Substituído pelo 0023	Java 25 e Spring Boot 4, sem patch pinado
0008	Aceito	Portas e adaptadores, vigiados por ArchUnit
0009	Aceito, ver 0026	ECS Fargate (ARM), mínimo de duas tasks
0010	Aceito, ver 0029	Terraform, provider AWS 6.x
0011	Aceito	OIDC com Cognito e Keycloak; JWT validado no serviço
0012	Aceito	Cache-aside com Redis/Valkey só no consolidado
0013	Aceito, ver 0025 e 0027	OpenTelemetry, ADOT, CloudWatch e X-Ray
0014	Aceito	`Idempotency-Key` no POST e consumidor idempotente por `eventId`
0015	Aceito	Lançamento imutável; correção só por estorno
0016	Aceito	`BigDecimal` / `NUMERIC(19,2)` ; string no evento
0017	Aceito	`/api/v1` e problemas RFC 9457
0018	Substituído pelo 0022	ALB público + WAF regional
0019	Substituído pelo 0021	Região `sa-east-1` por residência do dado
0020	Aceito, ver 0028	Rolling com circuit breaker; blue/green como evolução
0021	Aceito	Região `us-east-1` , transferência com salvaguarda
0022	Aceito, ver 0031	CloudFront, WAF, VPC Origin e ALB interno
0023	Aceito	Boot 4.0.9 e Resilience4j 2.4.0
0024	Aceito	Rotação de RDS, KMS, ACM, JWKS e OIDC

ADR	Status	Decisão
0025	Aceito	W3C e correlation id através da outbox e da fila
0026	Aceito	ECS agora; EKS acima de 8–10 serviços
0027	Aceito	CloudWatch e X-Ray agora; SaaS ou self-hosted depois
0028	Aceito	GitOps com ArgoCD quando houver EKS
0029	Aceito	`dev`, `hml`, `prod`, state em S3
0030	Aceito	Réplica desligada até gatilho; projeção já é o read model
0031	Aceito	REST API regional atrás do CloudFront, VPC Link v2 no ALB

Leitura mínima para o RNF crítico: 0002, 0003, 0004, 0005 e 0014. Para a borda: 0018 (histórico), 0022 e 0031.

Registrar decisões de arquitetura em ADR

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O desafio pede justificativa das escolhas, e o sistema vai ser lido por quem não participou da discussão. Sem um lugar estável para o “por quê”, a decisão migra para o comentário do pull request e se perde. O blueprint fixa o contrato; ele não substitui o registro da alternativa rejeitada.

Drivers

- A banca e o time futuro precisam ver opções, não só o resultado.
- A decisão tem de ser curta o bastante para ser lida, e estável o bastante para ser citada.
- O formato precisa caber em Markdown no repositório, sem ferramenta extra.

Opções consideradas

1. Só o README e o blueprint

- Positivo: um arquivo só, sem processo.
- Negativo: o README vira depósito. Trade-off antigo some na próxima edição. Não dá para apontar “a decisão X” numa revisão.

2. Wiki ou página fora do Git

- Positivo: fácil de comentar.
- Negativo: diverge do código. O desafio exige a documentação no repositório. A página externa não viaja com o commit que mudou a fila.

3. ADR no formato MADR, versionado em `documentos-brutos/04-adr`

- Positivo: contexto, drivers, opções com prós e contras, decisão e consequências no mesmo texto. O diff do Git é o histórico. Dá para marcar uma decisão como substituída sem apagar o passado.
- Negativo: mais um documento para atualizar quando a decisão muda. ADR esquecido vira mentira educada.

Decisão

Opção 3. Cada decisão relevante desta entrega tem um ADR numerado, status **Aceito** e data **2026-11-10**. Decisão nova não reescreve a antiga: acrescenta um ADR que a substitui e aponta o número anterior.

O blueprint continua sendo a fonte do contrato (nomes, portas, JSON). O ADR é a fonte do porquê. Se os dois discordarem no contrato, vale o blueprint e o ADR é corrigido.

Consequências

Positivas

- Trade-off fica explícito (SNS contra Kafka, ALB contra API Gateway, região contra custo).
- Dá para revisar uma decisão sem reler a arquitetura inteira.

Negativas

- Trinta arquivos para manter alinhados ao código. Uma decisão implementada diferente do ADR é defeito de documentação, tratado como defeito. Decisão substituída permanece no repositório com status Superseded.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0002-MICROSSERVICOS-POR-BOUNDED-CONTEXT.MD

Microsserviços por bounded context

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O enunciado pede dois serviços — lançamentos e consolidado diário — e exige que o primeiro continue disponível se o segundo cair. Há três desenhos capazes de expressar essa separação, com custos operacionais muito diferentes para um time pequeno e um pico de 50 req/s.

Drivers

- Falha do consolidado não pode derrubar o POST de lançamento (processo, banco e deploy).
- O volume não justifica uma plataforma de dezenas de serviços.
- Dois modelos de dados com ciclos de mudança diferentes: trilha imutável e projeção recomputável.
- Operação que caiba em ECS, dois pipelines e um time só.

Opções consideradas

1. Monolito modular

Dois módulos, um processo, um banco ou dois schemas no mesmo Postgres.

- Positivo: uma transação, um deploy, depuração simples, custo de uma instância. A fronteira hexagonal cabe dentro do monolito.
- Negativo: o processo é um só. Um crash, um pool esgotado ou um deploy ruim leva os dois módulos juntos. Isso falha o RNF crítico mesmo com o código “separado”. Isolar de verdade exigiria dois processos — e aí o monolito já deixou de ser a opção.

2. Funções serverless (Lambda por caso de uso)

- Positivo: escala a zero no dev, sem gerir task.
- Negativo: conexão com RDS em rajada (o pico de 50 req/s abre conexões mais rápido do que o pool de um processo quente), cold start na frente do p95 de 100 ms, e VPC attachment para falar com o banco. O relay de outbox pede um processo com relógio, não uma invocação por linha. Virtual threads e o modelo de programação Spring que escolhemos vivem melhor num processo longo. Ver ADR 0007 e ADR 0009.

3. Dois microsserviços, um por bounded context

`lancamentos-service` e `consolidado-service`, cada um com banco, task definition e pipeline.

- Positivo: a queda do consumidor não compartilha processo nem `max_connections` com a escrita. O deploy de um relatório não reinicia o caixa. O mapa de contextos em domínio cabe inteiro nessa fronteira — não há terceiro serviço "porque microsserviço".
- Negativo: dois deploys, dois RDS em produção, consistência eventual, rastreio distribuído obrigatório. Latência do saldo deixa de ser a latência do POST.

Decisão

Opção 3. A fronteira para no par pedido pelo negócio. Não se extrai "estorno", "categoria" ou "comerciante" para outro serviço. Identidade fica no Cognito/Keycloak, que já é um serviço genérico.

Consequências

Positivas

- O teste de resiliência consegue derrubar só o consolidado e ainda receber 201.
- O schema da projeção muda sem migrar a tabela de auditoria.

Negativas

- Custo fixo de duas instâncias RDS e de dois serviços no ALB. Aceito e quantificado em custos.
- Todo bug de saldo agora tem dois logs. Sem o correlation id do ADR 0013, a opção 3 fica pior que o monolito para operar.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0003-CQRS-E-INTEGRACAO-ASSINCRONA.MD

CQRS e integração assíncrona orientada a eventos

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O comerciante escreve fatos (lançamento) e lê um agregado derivado (saldo do dia e acumulado). Dá para servir as duas coisas da mesma tabela, com o serviço de leitura chamando o de escrita, ou separar o modelo de leitura e empurrar o fato por evento.

Drivers

- O POST não pode depender do consolidado estar de pé.
- O pico de 50 req/s é de leitura do saldo, não de varredura do extrato.
- Estorno retroativo altera o dia de competência, não "cria um ajuste hoje".
- Atraso aceitável: p95 < 5 s (blueprint, meta de consolidação).

Opções consideradas

1. Chamada síncrona REST de lançamentos para o consolidado, no meio do POST

- Positivo: o 201 só sai quando o saldo já bate. Um modelo mental.
- Negativo: o RNF crítico fica impossível. Timeout, circuit breaker ou retry no meio do POST ou atrasam o caixa ou mentem sobre o saldo. Acoplamento de disponibilidade e de deploy.

2. Banco compartilhado, consolidado lendo a tabela de lançamentos

- Positivo: sem evento, sem lag, `SUM` sempre fresco.
- Negativo: o consolidado depende do banco de lançamentos. Carga de leitura de pico e um relatório pesado competem com o INSERT do caixa. Não é database-per-service. Um lock ou um disco cheio no relatório afeta a escrita.

3. CQRS com evento `LancamentoRegistrado`

A escrita grava o fato e publica o evento. A leitura mantém `saldos_diarios` e responde o GET sem consultar o outro serviço. A soma é comutativa; ordem não é requisito.

- Positivo: disponibilidade desacoplada, esquema de leitura feito para o GET, espaço para cache, novos consumidores sem mudar o POST.

- Negativo: janela em que o GET ainda não vê o lançamento. O cliente precisa entender consistência eventual. Operação ganha fila, DLQ e lag.

Decisão

Opção 3. O contrato publicado é o JSON versão 1 do blueprint, no tópico `fluxo-caixa-lancamentos-events`. Não há API interna síncrona entre os dois serviços.

O GET de lançamento (extrato) continua no serviço de escrita: é leitura do próprio agregado, não do read model de saldo. CQRS aqui é entre contextos, não uma proibição de ler o que se acabou de escrever no mesmo serviço.

Consequências

Positivas

- O dimensionamento do consolidado (cache, pool, tasks) não entra no SLO do POST.
- Estorno não precisa de endpoint no consolidado: o mesmo evento, com tipo oposto, corrige o dia.

Negativas

- Tela que gravar e reler o saldo no mesmo clique pode ver o valor antigo por até alguns segundos. O contrato da API deixa isso explícito; não se bloqueia o POST até a projeção.
- Teste de ponta a ponta precisa esperar a consolidação ou consultar a projeção com retry. O smoke faz isso com prazo, não com sleep cego infinito.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0004-TRANSACTIONAL-OUTBOX.MD

Transactional outbox

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O lançamento e o evento de integração precisam concordar. Se o banco confirma e o SNS não recebe, o saldo nunca vê o fato. Se o SNS recebe e o banco desfaz, o saldo soma um fato que não existe. Os dois sistemas não participam da mesma transação.

Drivers

- Zero perda de evento depois de um 201.
- POST com p95 < 200 ms, portanto o publish não pode ser obrigatório dentro do tempo de resposta.
- Duas réplicas do serviço publicando ao mesmo tempo, sem evento duplicado por corrida do relay.
- Volume da premissa: 20 escritas/s no pico, muito abaixo do que justificaria uma plataforma de CDC.

Opções consideradas

1. Dual-write

`COMMIT` e, em seguida, `Publish`.

- Positivo: pouco código.
- Negativo: o processo pode morrer no meio. Retry do publish depois do commit exige exatamente a caixa de saída que esta opção finge não precisar. Publicar antes do commit entrega evento de transação abortada.

2. CDC com Debezium lendo o WAL do Postgres, publicando no SNS ou no Kafka

- Positivo: o domínio não conhece outbox. O log é a fonte. Bom quando há muitos agregados e vários consumidores de banco legado.
- Negativo: um conector para operar, em geral em Kafka Connect ou MSK Connect, com o custo e a complexidade que o ADR 0005 já recusou para a mensageria. Para uma tabela e um evento, é mecanismo demais. Também empurra o formato do evento para o formato da linha, a menos que um SMT o traduza — a tradução volta a ser código nosso.

3. Outbox na mesma transação e relay no processo

Tabela `outbox_eventos(id, aggregate_id, event_type, payload JSONB, criado_em, publicado_em, tentativas, proxima_tentativa_em, trace_parent, trace_state, correlation_id)`. O caso de uso insere o lançamento e a linha, com `proxima_tentativa_em` igual ao instante da escrita, e grava o contexto de

trace da requisição (ADR 0025). Um relay agenda a cada 500 ms, lê lote de 100 com `SELECT … FOR UPDATE SKIP LOCKED` das linhas ainda não publicadas cuja próxima tentativa já chegou, publica e marca `publicado_em`. Falha de publish incrementa `tentativas` e empurra `proxima_tentativa_em` com backoff exponencial até 60 s. O cliente SNS do relay ainda tem CircuitBreaker, Retry com jitter e TimeLimiter (ADR 0023). O POST continua sem chamar o SNS.

- Positivo: o 201 só sai se o evento está durável. Falha do SNS atrasa a publicação e não desfaz o lançamento. `SKIP LOCKED` permite duas tasks. O payload é o JSON do contrato, não um dump da tabela.
- Negativo: polling (atraso médio de uma fração do intervalo de 500 ms, compatível com o p95 de 5 s de ponta a ponta). Tabela que cresce se o publish parar — precisa de alarme de idade e de limpeza das linhas já publicadas. Relógio dentro da API, que compete um pouco de CPU com o POST (irrelevante a 20 escritas/s).

Decisão

Opção 3. O relay vive no `lancamentos-service`, não num terceiro deploy. Extrair um worker só se a métrica mostrar que o agendador atrapalha a API.

Publicação bem-sucedida atualiza `publicado_em` numa transação curta depois do ack do SNS. Se o processo cair depois do publish e antes do update, o evento é republicado. O consumidor idempotente (ADR 0014) torna essa duplicata inofensiva.

Consequências

Positivas

- O RNF de durabilidade do lançamento e o de independência do consolidado ficam no mesmo mecanismo.
- O teste de integração sobe Postgres e LocalStack e prova o ciclo sem AWS de verdade.

Negativas

- Relógio de 500 ms e lote de 100 são parâmetros operacionais. Mudá-los mexe no lag e na carga do banco. Ficam em configuração, com o alarme de outbox pendente do documento 07.
- A limpeza de linhas publicadas é tarefa de manutenção. Sem ela, a tabela cresce para sempre. Retenção sugerida: 7 dias depois de `publicado_em`, para auditoria curta do payload.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0005-SNS-SQS-MENSAGERIA.MD

SNS e SQS como mensageria

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O outbox precisa de um transporte entre o serviço de lançamentos e o de consolidado, com retenção se o consumidor cair, redrive para mensagem inválida e espaço para um segundo consumidor no futuro. A premissa de volume é 20 eventos/s no pico de escrita e cerca de 2 milhões de eventos por mês se a média for ~0,8 evt/s. O requisito de leitura (50 req/s) não passa pela fila.

Drivers

- Retenção de muitos dias (o blueprint fixa 14 na fila).
- DLQ com `maxReceiveCount` 5 e visibility timeout de 60 s.
- Fan-out: um tópico, uma fila hoje, outra fila amanhã sem o produtor mudar.
- Custo e operação compatíveis com dois serviços, não com um cluster.
- LocalStack no ambiente local, SDK v2 na aplicação.
- Ordem total não é necessária: a soma é comutativa (domínio).

Opções consideradas

1. Amazon MSK (Kafka)

- Positivo: log retido, replay por offset, vários consumidores independentes, ecossistema de stream.
- Negativo: o menor cluster Multi-AZ são três brokers mais armazenamento, na casa de **centenas de USD por mês** mesmo em `us-east-1` (e mais caro em `sa-east-1`), duas ordens de grandeza acima da conta de SNS/SQS neste volume. Operação de versão, disco e partição. Local fica mais pesado que um LocalStack. Kafka se paga quando houver replay longo, processamento de stream ou throughput sustentado na casa de milhares de eventos/s — cenário do roadmap, não o de agora.

2. Amazon MQ (ActiveMQ ou RabbitMQ)

- Positivo: fila clássica, roteamento flexível, protocolo que muitos ERPs já falam.
- Negativo: broker para patchar, dimensionar e colocar em Multi-AZ (ativo/espera custa instância dobrada). Não é o encaixe natural do restante da conta, que já é API AWS. Mais caro e mais frágil que SQS para um único consumidor idempotente.

3. Amazon EventBridge

- Positivo: roteamento por regra, schema registry, baixo custo na casa de USD 1 por milhão de eventos.

- Negativo: o buffer de consumidor, a concorrência e a DLQ que queremos são do SQS. EventBridge na frente do SQS acrescenta um hop e uma conta sem uma regra de roteamento que o SNS já não faça com assinatura. Útil no dia em que dezenas de tipos de evento precisarem de filtros diferentes. Hoje existe um tipo.

4. SNS + SQS Standard + DLQ

Tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ `consolidado-lancamentos-dlq`. Assinatura com raw message delivery. Fila Standard, não FIFO.

- Positivo: serviço gerenciado, retenção de 14 dias, long polling, competing consumers nas tasks do consolidado, custo de poucos dólares no volume da premissa (a linha está em custos, abaixo de USD 10/mês). Segundo consumidor = segunda fila na mesma assinatura. LocalStack cobre o fluxo.
- Negativo: sem ordem e sem replay por offset. Mensagem consumida e apagada não volta, salvo retenção ainda não expirada ou redrive da DLQ. Payload máximo 256 KB — o evento v1 está muito abaixo. Duplicata é normal e tem de ser absorvida pelo consumidor.

FIFO foi descartado dentro desta opção. FIFO dá ordem por message group e deduplicação de 5 minutos, com teto de throughput por grupo. A deduplicação curta não substitui a tabela `eventos_processados` (o redrive acontece dias depois). A ordem não é requisito. Standard é o encaixe.

Decisão

Opção 4, fila Standard, criptografia KMS, policy mínima por task role. Cliente AWS SDK for Java v2, sem Spring Cloud AWS, para não acoplar o ciclo de vida do framework ao binding da fila. O listener é código nosso no adaptador, pequeno o bastante para testar com LocalStack.

Consequências

Positivas

- O custo variável de mensageria é irrelevante perto de NAT, endpoint e RDS. A decisão não precisa ser revista por dinheiro até o volume subir ordens de grandeza.
- O teste de resiliência enche a fila com o consumidor parado e depois observa o catch-up.

Negativas

- Sem log de eventos consultável por offset. Reprocessar história inteira exige reler a outbox ou o banco de lançamentos e republicar — procedimento de operação, não feature do ônibus. Se um dia o reprocessamento histórico for rotina, a opção 1 volta à mesa.
- Visibilidade e alarme passam a ser parte do desenho. Fila sem alarme de idade é evento perdido na prática, mesmo com retenção de 14 dias.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0006-POSTGRESQL-DATABASE-PER-SERVICE.MD

PostgreSQL no RDS, um banco por serviço

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Os dois serviços persistem estado relacional com integridade (lançamento imutável, unicidade de estorno, saldo por UPSERT atômico). É preciso escolher o motor e o grau de isolamento: schema, database ou instância.

Drivers

- Transação local forte na escrita e na aplicação do evento.
- `NUMERIC` para dinheiro, índice único parcial para estorno, `SKIP LOCKED` para o relay, JSONB para o payload da outbox.
- Falha de capacidade do consolidado (conexão, disco, IOPS) não pode impedir o INSERT de lançamento.
- RPO ≤ 5 min e recuperação pontual.
- Time que já opera SQL. Volume que cabe numa instância small (ver capacidade).

Opções consideradas

1. Um RDS, dois databases

- Positivo: uma instância, uma janela de backup, metade do custo de compute Multi-AZ.
- Negativo: CPU, disco e `max_connections` são compartilhados. O isolamento que o RNF pede no processo se desfaz no banco. Serve para o Docker local e, se o orçamento apertar, para um dev AWS. Não serve para produção.

2. DynamoDB por serviço

- Positivo: escala horizontal gerenciada, sem conexão persistente.
- Negativo: o relay com `SKIP LOCKED`, a unicidade condicional do estorno e o `SUM` do acumulado viram desenho de chave e GSI. Transação de até 100 itens não é o problema; a perda é o modelo relacional que expressa as invariantes com pouco código. O time pagaria complexidade de modelagem num volume que não precisa dela.

3. PostgreSQL 17, instância RDS por serviço em produção

Dois bancos: `lancamentos` e `consolidado`. Em produção, duas instâncias Multi-AZ, criptografadas com KMS, senha no Secrets Manager, Flyway em cada serviço. Classe inicial `db.t4g.small` (2 GiB, ~225 conexões), porque o cálculo de Little dá menos de uma conexão ativa no pico e no máximo 60 reservadas no teto

de tasks. gp3 50 GB.

- Positivo: SQL, integridade, PITR, failover Multi-AZ, motor maduro para `NUMERIC(19,2)`. Instância separada realiza o database-per-service de verdade. Small é suficiente e custa cerca de metade de medium nesta região (números no documento 08).
- Negativo: duas contas de RDS, o maior item variável junto com a rede. Multi-AZ dobra o preço da hora da instância e o GB do gp3 já vem precificado em dobro na tabela (`SAE1-RDS:Multi-AZ-GP3-Storage`). Operação de versão do Postgres em dois lugares.

Decisão

Opção 3 em produção. No Docker local, um único Postgres cria os dois databases e os usuários `lancamentos` e `consolidado` — o isolamento de credencial existe, o de hardware não, e isso é aceitável na máquina do desenvolvedor. Dev AWS, se existir, usa duas instâncias `db.t4g.micro` Single-AZ para manter a fronteira de segredo sem pagar Multi-AZ.

Não há usuário de banco cruzado. Não há foreign key entre serviços. Não há dblink.

Consequências

Positivas

- A invariante “um estorno” é índice único, não só `if` no código.
- Dá para restaurar o consolidado de um PITR sem rebobinar a trilha de lançamentos, e o contrário.

Negativas

- Relatório que precise de extrato e saldo no mesmo request faz duas chamadas HTTP. Não faz join. É o custo do isolamento e é o que a API já expressa.
- Subir de classe (small → medium) é a resposta a CPU credit ou memória, não a um pressentimento. O gatilho é métrica, documentado no runbook.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0007-JAVA-25-SPRING-BOOT-4.MD

Java 25 e Spring Boot 4

- **Status:** Substituído (Superseded) pelo ADR 0023
- **Data:** 2026-11-10

O texto abaixo escolheu Java 25 e a linha Boot 4 sem pinar o patch e sem Resilience4j. A decisão vigente é Boot 4.0.9 e Resilience4j 2.4.0. A frase deste arquivo que diz que a versão de patch não está pinada não vale mais.

Contexto e problema

O serviço é I/O-bound: JDBC, Redis, JWKS e SNS no meio de poucas dezenas de requisições simultâneas (Little dá 2,8 em voo no pico de leitura). A linguagem precisa expressar o domínio sem ruído, rodar em container enxuto e ter suporte longo.

Drivers

- LTS. Java 25, publicado em setembro de 2025, é a LTS seguinte à 21.
- Virtual threads para o bloqueio de JDBC e HTTP sem um pool de plataforma dimensionado no chute.
- Ecossistema que o time opera: Spring, Flyway, Testcontainers, ArchUnit.
- Imagem de container reproduzível. JDK de distribuição com licença que não cobra por núcleo.

Opções consideradas

1. Java 21 LTS e Spring Boot 3.x

- Positivo: curva conhecida, mais tempo de mercado, menos surpresa de biblioteca.
- Negativo: abre mão da LTS atual e do alinhamento com Spring Boot 4, que é a linha definida no blueprint. Manter 21 “por segurança” em projeto novo em 2026 é dívida de largada.

2. Kotlin ou outra JVM language

- Positivo: null-safety na linguagem.
- Negativo: segundo dialeto para um domínio que cabe em Java claro. Não há requisito que Kotlin resolva e Java 25 não resolva.

3. Java 25 e Spring Boot 4.x na última estável do Maven Central

Runtime Eclipse Temurin 25 (licença de uso sem custo por núcleo; ver custos). Um Maven wrapper por serviço. Virtual threads ligadas no servidor web e nos clientes bloqueantes.

- Positivo: LTS, virtual threads maduras, um processo aguenta o pico de 50 req/s folgado porque o limite deixa de ser thread de plataforma. Spring Boot 4 é a linha que acompanha essa geração de Jakarta e do

ecossistema.

- Negativo: biblioteca que ainda não subiu para a linha 4 quebra o build. A versão de patch não está pinada no blueprint de propósito ("última estável"); o código deve pinar a que resolver o build e registrar no `pom`. Este ADR não inventa número de patch.

Decisão

Opção 3. Dois projetos Maven independentes, pacotes `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`. Imagem com Temurin 25, usuário não-root, JAR em camadas.

Virtual threads não autorizam pool JDBC infinito. O Hikari continua limitado (máximo 10 por task), como calculado na capacidade. Thread barata com conexão cara só muda o lugar do acidente.

Consequências

Positivas

- O modelo de concorrência combina com o perfil medido: poucas requisições em voo, cada uma esperando rede.
- Um wrapper Maven tira a exigência de Maven instalado na máquina, desde que o JDK 25 exista.

Negativas

- Se uma dependência crítica não publicar artefato compatível com Spring Boot 4, a mitigação é subir o patch ou, em último caso, registrar um ADR novo — não misturar Boot 3 num serviço e Boot 4 no outro.
- JDK comercial da Oracle fica fora da imagem. Entrar com ele criaria custo de licença que a estimativa trata como zero.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0008-ARQUITETURA-HEXAGONAL.MD

Arquitetura hexagonal

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O domínio de lançamento (dinheiro, estorno, imutabilidade) é a parte que a banca e o negócio vão questionar. Esse código não pode depender de anotação de framework, de cliente AWS nem de tabela. Ao mesmo tempo, a aplicação precisa de HTTP, JPA, SNS e Redis de verdade.

Drivers

- Teste de invariante sem subir Spring.
- Troca de LocalStack por AWS sem editar regra de estorno.
- Regra de dependência que o build verifica, não uma convenção de pasta que alguém fura na pressa.
- Pacotes já nomeados no blueprint.

Opções consideradas

1. Camadas técnicas clássicas (`controller` , `service` , `repository`) com domínio anêmico

- Positivo: familiar, menos arquivos.
- Negativo: a invariante de estorno acaba no service que já importa o SDK, e o teste vira `@SpringBootTest` para qualquer regra. Em seis meses o “service” conhece fila, token e SQL.

2. Pacotes por feature, sem restrição de compilação

- Positivo: navegação fácil.
- Negativo: a fronteira é social. Não impede `domain` de importar `software.amazon.awssdk` .

3. Portas e adaptadores, com ArchUnit

```
domain/          entidades, value objects, eventos, exceções — sem Spring
application/     casos de uso e portas de saída
adapters/in      web e, no consolidado, messaging
adapters/out     persistence, messaging, cache
config/
```

ArchUnit falha o build se `domain` depender de Spring ou de `adapters` , e se `application` depender de `adapters` .

- Positivo: a regra de estorno mora em `Lancamento` e roda em teste de unidade. O JSON do evento mora no adaptador de mensagem (ACL). O mesmo caso de uso serve ao teste de contrato e ao controller.
- Negativo: mais interfaces e mapeamentos DTO ↔ domínio ↔ entidade JPA. Para dois agregados pequenos, parte dessa estrutura parece cerimônia. É o preço de manter a regra legível quando o adaptador AWS crescer.

Decisão

Opção 3, exatamente nos pacotes do blueprint. O controller não é lugar de regra: extrai o `comerciante_id` do JWT, monta o comando e traduz falha de domínio em problema RFC 9457.

Não se cria porta de saída “por precaução” para um barramento genérico. As portas que existem são as que têm adaptador: repositório, outbox, publish, cache, fila.

Consequências

Positivas

- O nível 4 do C4 é legível porque o domínio não está emaranhado no filtro de segurança.
- Teste de arquitetura pega o atalho antes do code review.

Negativas

- ArchUnit mal escrito vira ruído (falso positivo em `config`). O teste cobre os três pacotes de regra e deixa `config` como composição, que por natureza conhece todo mundo.
- Mapear `Dinheiro` ↔ `NUMERIC` ↔ string do evento em três bordas é repetição consciente. Compartilhar um módulo de domínio entre os serviços recriaria um kernel compartilhado que o context map recusou.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0009-ECS-FARGATE.MD

ECS Fargate

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Os dois processos Java precisam rodar em `us-east-1` (ADR 0021), em pelo menos duas zonas, com deploy repetível, escala horizontal e sem uma frota de máquina para patchar. O pico de CPU é baixo (ρ ≈ 17% com duas tasks no consolidado). O ADR 0026 mantém esta decisão e registra o EKS como evolução.

Drivers

- Mínimo de duas tasks em AZs distintas por serviço.
- Imagem imutável vinda do ECR, scan no push.
- Rolling com circuit breaker e rollback.
- Sidecar ADOT no mesmo task.
- Custo proporcional a task pequena, arquitetura ARM.
- Tempo de plataforma que sobre para o domínio, não para o control plane.

Opções consideradas

1. EKS

- Positivo: padrão de mercado, HPA, ecossistema, portabilidade de manifesto.
- Negativo: control plane pago o mês inteiro (na casa de USD 70 só de API do cluster, mais nós ou Fargate profile). Para quatro tasks, o Kubernetes é o sistema maior da conta. A portabilidade não é requisito: o alvo declarado é AWS.

2. Lambda

- Positivo: sem task ociosa.
- Negativo: já recusado no ADR 0002 por conexão com RDS, cold start e relay de outbox. Não se reabre aqui.

3. EC2 com AMI e autoscaling group

- Positivo: preço de compute menor que Fargate no uso constante, controle do SO.
- Negativo: patch de AMI, capacidade ociosa do nó, user data, e um ASG que ainda precisa de load balancer. Com ρ de 17%, o desconto do EC2 não paga a operação. Reservar instância para dois serviços pequenos é otimização prematura; Savings Plan de Fargate, se o uso estabilizar, vem antes.

4. ECS Fargate, graviton (ARM)

Cluster único, um service por microsserviço. Em produção o lançamentos sobe com 0,5 vCPU e 1 GiB (mínimo 2, máximo 6) e o consolidado com 1 vCPU e 2 GiB (mínimo 2, máximo 10). Aplicação e sidecar ADOT dividem esse tamanho. Escala por target tracking de CPU (60%) e de `ALBRequestCountPerTarget` nos dois. O consolidado soma target tracking do backlog da fila (`mensagens visíveis / tasks em execução` , alvo de 20). Deployment circuit breaker com rollback. Container Insights. Dev usa tasks menores e uma task por serviço. Os números estão em `infra/terraform/envs/*/terraform.tfvars.example` .

- Positivo: sem nó para administrar, task spread nas AZs, encaixe direto com ALB target group, IAM por task role, imagem OCI padrão. ARM no Fargate de `us-east-1` custa USD 0,03238 por vCPU-hora e USD 0,00356 por GB-hora (Price List, vigência 2026-10-01), abaixo da hora x86 da mesma família. Em `sa-east-1` os mesmos SKUs são USD 0,05570 e USD 0,00612.
- Negativo: menos controle de kernel e de daemon. Sidecar come parte da CPU contratada — por isso o consolidado, que segura o pico de 50 req/s, fica em 1 vCPU e não em 0,25. O lançamentos, com 20 escritas/s, cabe em 0,5 vCPU. Cold start de task nova é da ordem de dezenas de segundos; o mínimo de 2 tasks em prod existe para não depender de scale-from-zero no pico. Fargate não fica de graça parado. O exemplo de dev não desliga a task sozinho.

Decisão

Opção 4. Um cluster, dois services, dois target groups. Não há service mesh. A task não recebe IP público; egress por NAT ou por VPC endpoint, como no deployment.

Consequências

Positivas

- O N−1 do RNF (uma task morta e o pico de 50 req/s segue) é configuração de desired count e de spread, não um script.
- A mesma imagem sobe no Docker local e no ECR.

Negativas

- Debug de produção é log, trace e task replacement, não SSH. Quem precisar de shell usa ECS Exec pontual, com IAM, não uma chave na AMI.
- Teto de 6 tasks no lançamentos e de 10 no consolidado é guarda-corpo de custo. Subir o teto sem olhar RDS e Redis só multiplica pool de conexão.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0010-TERRAFORM-IAC.MD

Terraform como IaC

- **Status:** Aceito. O ADR 0029 amplia os ambientes para `dev`, `hml` e `prod` e fixa o state em S3. A escolha do Terraform permanece.
- **Data:** 2026-11-10

Contexto e problema

VPC, ALB, WAF, ECS, dois RDS, ElastiCache, SNS, SQS, Cognito, alarmes e IAM precisam ser recriáveis em `dev` e `prod`, revisáveis em pull request e destruíveis. A região e os nomes de fila já estão fixados no blueprint.

Drivers

- Estado declarativo e plan visível antes do apply.
- Módulos reutilizáveis entre ambientes, com tamanho diferente (NAT único no dev, NAT por AZ na prod).
- Ferramenta que o repositório expressa em HCL, sem gerar JSON ilegível à mão.
- Sem custo de licença para o porte deste sistema. A licença da ferramenta em si está tratada no documento 08: Terraform é BSL, uso gratuito neste contexto; o runtime não embute a CLI.

Opções consideradas

1. AWS CDK

- Positivo: tipos, loops e testes na linguagem do time (TypeScript ou Java), construct de alto nível.
- Negativo: a unidade de review deixa de ser a infraestrutura e passa a ser o programa que a gera. Diff de CloudFormation sintetizado é mais difícil de ler do que diff de HCL. Mais uma cadeia de build para a infra.

2. CloudFormation puro

- Positivo: nativo, sem estado externo além do stack, drift detection da AWS.
- Negativo: verbosidade, módulos fracos comparados ao que os dois ambientes precisam, e templates grandes de VPC/ECS que a comunidade já resolveu em módulo Terraform. Rollbacks de stack têm arestas conhecidas com recurso que não volta atrás (RDS).

3. Terraform (provider AWS 6.x), módulos e ambientes `dev` e `prod`

Estado remoto (S3 + lock), `make tf-validate` no CI, um módulo por preocupação (rede, borda, serviço ECS, dados, mensageria, identidade, observabilidade). Nomes de fila e tópico iguais aos do blueprint, para a aplicação e a infra não divergirem.

- Positivo: plan legível, ecossistema de módulos, o mesmo fluxo nos dois ambientes, provider 6.x alinhado ao blueprint. Validate e fmt cabem no pipeline sem aplicar nada.

- Negativo: estado é um artefato crítico (criptografado, lock, sem commit do `tfstate`). Provider 6.x muda argumento em relação à linha 5; o código não copia exemplo antigo de blog. HCL não impede um `apply` de prod a partir da máquina errada — isso é política de CI, não de sintaxe.

Decisão

Opção 3. Apply de produção só pelo pipeline, com o ambiente `prod` protegido. Apply manual fica restrito a `dev`. O README de `infra/terraform` é o passo a passo; este ADR só registra a escolha da ferramenta.

OpenTofu permanece como saída se a licença BSL se tornar impeditiva para a organização. O HCL escrito para este porte é portável o bastante para essa troca não ser uma reescrita.

Consequências

Positivas

- A revisão da infra é um diff, não um clique no console.
- Desviar o console sem refletir no código aparece como drift a corrigir, não como “ajuste que ficou”.

Negativas

- Dois lugares definem nome de fila: Terraform e configuração da aplicação. O blueprint é a lista canônica; os dois lados copiam dela. Um teste de fumaça na AWS de dev que publica e consome pega divergência que o `fmt` não pega.
- Recurso com dado (RDS) não entra em destroy casual. O ambiente `prod` não tem destroy no alvo do dia a dia do Makefile.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0011-OAUTH2-OIDC-COGNITO.MD

OAuth2/OIDC com Cognito e JWT validado no serviço

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Cada requisição pertence a um comerciante e a um escopo. O identificador do comerciante não pode vir do corpo nem da URL. O ambiente local precisa emitir token sem conta AWS. O ambiente AWS precisa de um provedor gerenciado.

Drivers

- Access token JWT, validado por assinatura e issuer.
- Escopos `lancamentos:write`, `lancamentos:read`, `consolidado:read`.
- Claim `comerciante_id`. Na AWS o prefixo de escopo é `fluxo-caixa/`; no local é vazio. A aplicação lê `app.security.scope-prefix` e `app.security.comerciante-claim`.
- Mesmo código de resource server nos dois ambientes.
- Sem sessão de servidor. A API é stateless.

Opções consideradas

1. API key estática por comerciante

- Positivo: trivial de testar.
- Negativo: sem expiração decente, sem escopo, sem rotação fácil, e o tenant volta a ser um header que o cliente escolhe. Não distingue leitura de escrita.

2. Authorizer só na borda (API Gateway ou ALB authenticate-oidc), serviço confia na rede

- Positivo: o serviço não fala com JWKS.
- Negativo: qualquer caminho que alcance a task (SG folgado, outro serviço no cluster) é chamada anônima com poder de escrita. O comportamento local (sem ALB) divergiria do comportamento AWS. A borda escolhida no ADR 0018 não substitui autorização de aplicação.

3. Cognito na AWS, Keycloak no Docker, resource server em cada serviço

User Pool Essentials com resource server `fluxo-caixa` e os três escopos. Pre Token Generation Lambda v2 coloca `comerciante_id` no access token e, via `TOKEN_SCOPES`, os escopos `fluxo-caixa/lancamentos:write`, `fluxo-caixa/lancamentos:read` e `fluxo-caixa/consolidado:read`. O pool não tem outro papel além de comerciante, então todo token recebe os três. O domínio hospedado (`fluxo-`

`caixa-<env>-<account_id>`) deixa o authorization code disponível. No local, realm `fluxo-caixa`, client público `fluxo-caixa-app` com direct access grants, usuário `comerciante1` / `comerciante1` e o mesmo claim.

O serviço valida o JWT (Spring Security Resource Server). Dentro do Compose, `JWT_ISSUER_URI` permanece `http://localhost:8080/realms/fluxo-caixa` para coincidir com o `iss` do token obtido no host, e `JWT_JWK_SET_URI` aponta para `http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs`, porque de dentro da rede o `localhost` do issuer não é o Keycloak.

- Positivo: padrão OIDC, escopo de verdade, tenant fora do alcance do cliente, paridade de código entre local e AWS, Keycloak sem custo de licença para o laboratório.
- Negativo: a Lambda de pre-token é um pedaço de infra a testar — se ela falhar, o token sai sem claim e o serviço responde 401, o que é o comportamento seguro e também um incidente de login. Cognito tem modelo de preço por MAU (dentro da franquia neste porte; ver custos). O arranjo de issuer versus JWK no Compose é fácil de "corrigir" errado: unificar os dois URLs quebra ou a validação do `iss` ou a busca da chave.

Decisão

Opção 3. Sem claim, 401. Escopo ausente, 403. `comerciante_id` de outro formato que não UUID, 401. Não existe comerciante default.

Autorização de objeto é o filtro por esse claim em toda query. Não se cria papel de administrador nesta versão.

Consequências

Positivas

- O teste local (`make token`) exercita o mesmo filtro que a produção.
- Vazar um token não entrega outro comerciante. O estrago fica no tenant e na validade do token.

Negativas

- Direct access grant no client local é senha no POST do token. O app client da AWS também aceita `USER_PASSWORD_AUTH` para o laboratório: a Lambda completa os escopos que o `InitiateAuth` não emite. O authorization code com PKCE continua no domínio hospedado para um canal interativo.
- Relógio das tasks precisa estar são. Skew grande demais rejeita token bom. NTP da plataforma Fargate cuida disso; o sintoma, se aparecer, é 401 com token recém-emitido.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0012-CACHE-REDIS-ELASTICACHE.MD

Cache Redis/ElastiCache no consolidado

- **Status:** Aceito. O fallback quando o Redis falha ou abre o circuito está no ADR 0023.
- **Data:** 2026-11-10

Contexto e problema

O SLO de leitura é p95 < 100 ms no GET do saldo, com 50 req/s. O Postgres aguenta a taxa mesmo sem cache (a conta de conexões ativas no miss total é ~2), mas não garante o percentil quando o `SUM` do acumulado e a validação do token disputam a mesma latência. O serviço de lançamentos não tem este SLO de leitura derivada e não ganha cache.

Drivers

- Acelerar o hit sem tornar o Redis fonte da verdade.
- Invalidar dia, acumulado e relatório de período quando um evento de qualquer data daquele comerciante chega (estorno retroativo muda o acumulado dos dias seguintes).
- Perder o cache não pode perder saldo nem derrubar a leitura.
- Produção com failover. Local com um container Redis 7.

Opções consideradas

1. Sem cache

- Positivo: um componente a menos, leitura sempre tão fresca quanto a projeção.
- Negativo: o p95 de 100 ms fica inteiro nas costas do RDS e do cálculo do acumulado. Possível neste volume, folgado demais para tratar como arquitetura — o primeiro relatório de período de 366 dias no meio do fechamento come o percentil.

2. Cache embutido na task (Caffeine)

- Positivo: sem rede extra, latência mínima.
- Negativo: duas tasks, duas memórias. Invalidar a geração no evento só limpa a task que consumiu a mensagem. A outra serve saldo velho até o TTL. Para um read model que acabou de mudar, isso é o bug que o comerciante vê.

3. Redis no local e Valkey no ElastiCache, cache-aside com geração

Chaves `consolidado:versao:{comercianteId}`, `consolidado:dia:{comercianteId}:{versao}:{data}` e `consolidado:relatorio:{comercianteId}:{versao}:{inicio}:{fim}`. Cada evento aplicado faz `INCR` na versão daquele comerciante depois do commit. TTL de 30 s nas entradas (`app.cache.ttl`). ElastiCache Val-

key 8.2, nó `cache.t4g.small` com réplica e failover, TLS (`REDIS_SSL=true`) e sem senha. No Docker local o cache é Redis 7 sem TLS. A aplicação fala protocolo Redis; Valkey é a escolha de produção por compatibilidade e por preço de tabela inferior ao Redis equivalente na AWS.

- Positivo: as duas tasks veem a mesma geração. Estorno antigo não exige apagar um conjunto desconhecido de chaves. Queda do cache: o caso de uso lê o Postgres. Queda do primário: a réplica assume.
- Negativo: mais um serviço gerenciado (cerca de USD 37/mês os dois nós small em `us-east-1`, custos; em `sa-east-1` a mesma dupla ficaria perto de USD 71). Invalidação por versão deixa lixo até o TTL (aceito). Se o `INCR` falhar depois do commit do saldo, a versão antiga continua válida até o TTL — janela de leitura obsoleta de no máximo 30 s. Não se finge que o cache é linearizável com o banco.

Decisão

Opção 3, somente no `consolidado-service`. Variáveis `REDIS_HOST`, `REDIS_PORT`, `REDIS_SSL`. Timeout curto no cliente e fallback para o banco quando o Redis não responde. Não se cacheia lançamento: o GET por id já é uma chave primária e não é o pico dos 50 req/s.

Consequências

Positivas

- O SLO de 100 ms tem um caminho de hit que não abre transação.
- O teste de carga consegue um cenário com cache quente e outro com Redis parado, e os dois têm critério de sucesso diferente (latência versus só disponibilidade).

Negativas

- Mais um security group. Não há `REDIS_PASSWORD`: o Terraform liga TLS e só o SG do consolidado alcança a porta 6379.
- Resposta cacheada de dia zerado existe. Um bug na versão serviria zero por até 30 s depois do primeiro lançamento do dia. O `INCR` depois do commit do evento é o que impede isso; o teste de integração cobre “evento aplicado, GET seguinte vê o valor”, com a versão nova.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0013-OBSERVABILIDADE-OTEL.MD

Observabilidade com OpenTelemetry, ADOT, CloudWatch e X-Ray

- **Status:** Aceito. A propagação do contexto pela fila está no ADR 0025. A comparação com SaaS e com stack self-hosted está no ADR 0027.
- **Data:** 2026-11-10

Contexto e problema

Um 201 no lançamento e um saldo errado dois segundos depois não cabem no mesmo log se os dois serviços, a fila e o banco não compartilharem um identificador. O pico, a DLQ e a outbox são sintomas operacionais, não só técnicos: fila crescendo é caixa sem relatório.

Drivers

- Os três pilares: log, métrica e trace, correlacionados.
- Funcionar igual no Docker (Grafana LGTM) e na AWS (CloudWatch e X-Ray).
- Métrica de negócio mínima: lançamento registrado, consolidação aplicada, lag.
- Sem agente proprietário que prenda a telemetria a um fornecedor só.

Opções consideradas

1. Log em texto e métrica de CPU do console

- Positivo: zero esforço.
- Negativo: não acha o evento. Não alarma DLQ. Não distingue erro de lançamentos de erro de consolidado. Reprova o diferencial de observabilidade do desafio e, na prática, reprova a operação do RNF.

2. Agente de APM proprietário

- Positivo: dashboard pronto, profiling.
- Negativo: licença, agente fechado, outro formato. A estimativa de custo trata licença de software como zero; um APM comercial quebraria essa premissa sem necessidade neste porte.

3. OpenTelemetry (OTLP) na aplicação, ADOT na AWS, `grafana/otel-lgtm` no local

Micrometer + OTel. Log JSON estruturado com `trace_id` e `span_id`. Actuator: liveness, readiness, Prometheus. Na AWS, sidecar ADOT exporta trace para X-Ray e métrica para CloudWatch. No local, `OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318` e Grafana em `localhost:3000`.

- Positivo: um SDK, dois destinos. O teste local mostra o trace do POST até o consumidor. Alarmes de 5xx, p99, profundidade da fila, idade da mensagem, DLQ e outbox pendente têm para onde apontar. Sem

licença.

- Negativo: sidecar custa CPU dentro da task (o consolidado reserva 1 vCPU / 2 GiB e o lançamentos 0,5 vCPU / 1 GiB, sidecar incluso). Cardinalidade de label mal escolhida (por exemplo `lancamentoId` como dimensão de métrica) explode a conta do CloudWatch. X-Ray tem preço por trace; amostragem é obrigatória, não opcional. O dashboard de produção está no módulo Terraform de observabilidade. O endpoint de interface do X-Ray existe para o ADOT não precisar de egress aberto.

Decisão

Opção 3. Amostragem de cabeça. Requisição pela borda: decide o API Gateway, com a regra padrão do X-Ray (a primeira requisição de cada segundo e 5% das demais), e os serviços seguem a flag `Sampled` do pai. Trace sem pai: 1.0 em dev, 0.1 em hml e prod, por `MANAGEMENT_TRACING_SAMPLING_PROBABILITY`. Erro não tem retenção garantida: isso pede tail sampling no ADOT, que fica como evolução. O correlation id e o log cobrem o erro não amostrado. Log em JSON, nível INFO, sem payload de lançamento completo no log (descrição pode ser dado pessoal; o id basta).

Métricas de domínio, não só RED: contador de lançamentos por tipo, contador de eventos aplicados, gauge de outbox não publicada, lag de consolidação (instante do evento versus instante da aplicação).

Consequências

Positivas

- O runbook de “saldo atrasado” começa pelo lag e pela idade da mensagem, não por uma opinião.
- Trocar o backend de observabilidade no futuro não troca a instrumentação.

Negativas

- ADOT sidecar e a aplicação precisam concordar na propagação de contexto W3C até o atributo da mensagem SNS/SQS. Se o relay não copiar o `traceparent` para a mensagem, o trace quebra no meio e o desenho perde metade do valor. Isso é critério de aceite da instrumentação, não enfeite.
- Custo de log cresce com DEBUG esquecido em produção. O perfil `aws` não habilita DEBUG de framework.

Idempotência na escrita e no consumo

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Há duas duplicatas de naturezas diferentes. O cliente do caixa pode repetir o POST porque a resposta se perdeu (rede, timeout do app). O SQS entrega a mesma mensagem mais de uma vez, e o relay pode republicar se cair depois do `Publish` e antes de marcar a outbox. As duas, se tratadas com "INSERT sempre", criam dinheiro que não existiu.

Drivers

- Repetir o POST com a mesma intenção devolve o mesmo lançamento.
- Reentrega do evento não altera o saldo duas vezes.
- A chave do cliente não vaza entre comerciantes.
- Retry com backoff só é seguro se o efeito for idempotente.

Opções consideradas

1. Confiar em "o cliente não repete" e em FIFO com deduplicação de 5 minutos

- Positivo: menos tabela.
- Negativo: timeout no celular repete sim. A janela de deduplicação do FIFO não cobre redrive da DLQ no dia seguinte nem republicação da outbox. E o ADR 0005 não adotou FIFO.

2. Idempotência só no consumidor

- Positivo: protege o saldo, que é o agregado derivado.
- Negativo: o extrato fica com dois lançamentos iguais para uma venda. O caixa, que é a fonte da verdade, estaria errado. O consolidado estaria "certo" em cima de um fato duplicado.

3. Chave no POST e `eventId` no consumo

HTTP. Header opcional `Idempotency-Key` (UUID), único por `(comerciante_id, chave)`, no POST de lançamento e no POST de estorno. A coluna `idempotencia.impressao_digital` guarda o SHA-256 do corpo canônico (no registro: tipo, valor com duas casas, data, descrição e categoria; no estorno: id do lançamento e motivo). A resposta não é armazenada: a repetição relê o lançamento pelo `lancamento_id`.

Situação	Resultado
Primeira vez	201 e persiste

Situação	Resultado
Mesma chave, mesma impressão	200 e o mesmo lançamento
Mesma chave, impressão diferente	409

Sem header, cada POST é um fato novo — o blueprint deixa a chave opcional. Cliente de PDV deve enviá-la; cliente de teste pode omitir.

Consumo. `eventos_processados(event_id primary key)` na mesma transação do `UPSERT` do saldo. Conflito de chave: a mensagem é descartada da fila e o saldo não muda.

- Positivo: as duas pontas do dinheiro ficam protegidas. Retry do relay, do SQS e do cliente deixam de ser incidente.
- Negativo: a obrigação de definir o que é “o mesmo corpo”. A impressão cobre o corpo canônico, não o header de autorização. A linha não expira nesta entrega: a mesma chave continua válida enquanto o registro existir. Limpeza é operação, não um TTL de 24 h no código.

Decisão

Opção 3. A idempotência HTTP é por comerciante. Um tenant não bloqueia a chave de outro. O `eventId` é gerado na escrita (UUID) e vai no payload; o consumidor não deduplica por `lancamentoId` apenas, porque o contrato manda deduplicar por `eventId` — na prática os dois são 1:1 hoje, e usar o campo do contrato evita uma segunda regra.

Consequências

Positivas

- O teste de carga pode repetir request sem inventar lançamento.
- Redrive da DLQ é seguro.

Negativas

- 200 na repetição e 201 na criação exigem que o cliente trate os dois como sucesso. Está no RF-04. Cliente que só aceita 201 vai marcar erro numa retentativa que deu certo.
- Hash instável (campo com espaço, escala `150.0` versus `150.00`) gera 409 legítimo e surpreende. A normalização do valor para duas casas acontece antes do hash.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0015-LANCAMENTOS-IMUTAVEIS-ESTORNO.MD

Lançamentos imutáveis, correção por estorno

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Caixa editável apaga a história. Um `UPDATE` do valor ou um `DELETE` do lançamento deixa o saldo sem explicação e a auditoria dependente de trigger que alguém pode desligar. O negócio precisa corrigir erro de digitação sem reescrever o fato original.

Drivers

- Trilha reconstruível só com inserts.
- Estorno único. Estorno de estorno proibido — senão a história vira uma corrente ambígua.
- A correção atinge a data de competência original, porque é lá que o saldo do dia ficou errado.
- O consolidado não deve aprender uma segunda linguagem para “desfazer”.

Opções consideradas

1. Update e delete lógicos (`cancelado = true`)

- Positivo: uma linha só, listagem simples.
- Negativo: o fato muda. O evento de integração precisa de um tipo “cancelado” ou o consumidor não fica sabendo. Dois caminhos (criar e cancelar) para um domínio que cabe em “sempre nasce um fato”.

2. Lançamento de ajuste em data de hoje, original intocado sem vínculo

- Positivo: append-only.
- Negativo: o dia original continua errado para sempre. O acumulado até hoje fecha, o relatório do dia do erro não. O comerciante olha o dia, não o acumulado, quando confere o caixa.

3. Novo lançamento de tipo oposto, mesma data, mesmo valor, vínculo `lancamentoEstornadoId`

O original permanece. Índice único parcial impede o segundo estorno. Lançamento que já tem `lancamentoEstornadoId` preenchido não pode ser alvo. O evento publicado é o mesmo `LancamentoRegistrado`, e o consolidado soma o tipo oposto no dia.

O blueprint não tem campo `motivo` no agregado e o POST de estorno não envia `descricao`, embora aceite `{ "motivo": "..." }`. Interpretação desta decisão: a descrição do novo lançamento é o motivo, limitado a 255 caracteres, ou `Estorno do lançamento {id}` se o motivo vier vazio. O motivo não entra no evento v1.

- Positivo: a tabela é a auditoria. O saldo do dia original fecha. Um consumidor só. A proibição de estorno duplo é restrição de banco, então duas requisições paralelas não furam o `if`.
- Negativo: a listagem do dia mostra o original e o estorno, e o saldo "parece" ter mais movimento. É o comportamento honesto; esconder o par na UI é problema de apresentação, fora desta API. Corrigir um estorno feito por engano exige um lançamento novo deliberado, não um meta-estorno — o operador precisa entender isso.

Decisão

Opção 3. Não há endpoint de update nem de delete. Não há evento `LancamentoEstornado`.

Consequências

Positivas

- PITR e backup carregam a história completa sem um log paralelo obrigatório.
- O RF-18 (estorno corrige o dia original) cai direto do modelo, sem job noturno.

Negativas

- Lançamento na competência errada e valor certo se corrige com estorno mais um lançamento novo no dia certo. São dois fatos. O sistema não oferece "mover de data" porque mover seria update.
- A descrição sintética do estorno é uma interpretação documentada aqui, não um campo do blueprint. Se a implementação gravar outra frase, o contrato de API ainda se sustenta; o que não pode ocorrer é persistir o motivo só em log.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0016-REPRESENTACAO-MONETARIA.MD

Representação monetária

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Saldo errado por centavo destrói a confiança no consolidado mais rápido do que uma indisponibilidade. O erro clássico é `double`: 0,10 não existe exatamente em binário. O valor atravessa JSON de request, coluna SQL, evento e cache.

Drivers

- Duas casas decimais, meio para cima no sentido bancário usual de escala, sem surpresa de `0.30000000000000004`.
- Moeda única BRL nesta versão.
- Valor sempre positivo no lançamento; o sinal contábil é o tipo (crédito ou débito), não o número.
- Contrato do evento já fixado como string `"150.00"`.

Opções consideradas

1. `double` / `float` de ponta a ponta

- Positivo: menos código.
- Negativo: inaceitável para dinheiro. Rejeitado sem meio-termo.

2. Inteiro de centavos (`15000` = R$ 150,00)

- Positivo: aritmética exata, fácil de somar no evento.
- Negativo: toda borda precisa lembrar a unidade. Um cliente que manda `150.00` pensando em reais grava R$ 1,50 ou R$ 15.000 se alguém converter duas vezes. O blueprint mostra o request em reais (`150.00`) e o evento em string decimal, não em centavos.

3. `BigDecimal` no domínio, `NUMERIC(19,2)` no Postgres, string de duas casas no evento

Value object `Dinheiro` com escala 2, `> 0` e moeda `BRL`. A API aceita número JSON no request, como no exemplo do blueprint (`"valor": 150.00`), e o desserializador usa `BigDecimal`, nunca `double`. A resposta devolve string de duas casas para o cliente não reintroduzir binário ao reexibir. O evento usa string, `toPlainString`, sempre com duas casas. A soma do saldo usa `BigDecimal` e a coluna `NUMERIC`.

- Positivo: o tipo carrega a unidade. O evento é estável entre linguagens (JavaScript que lesse número perderia precisão em valores grandes; string não). `NUMERIC(19,2)` cabe muito além do caixa de um comerciante e rejeita escala ilegal no banco.

- Negativo: o request é número e o evento é string. Essa assimetria está no blueprint e é uma aresta real: um cliente JavaScript que faça `JSON.parse` e reenvie o número pode ter alterado a escala antes de chegar aqui. O serviço normaliza para duas casas ao construir `Dinheiro`. Valor com três casas (`10.125`) é 400, não arredondamento silencioso — arredondar esconderia erro de integração.

Decisão

Opção 3. Proibido `double` em campo de valor, inclusive em teste. Comparação de dinheiro por `compareTo` na mesma escala, não por igualdade de objeto com escala diferente (`10.0` versus `10.00` são o mesmo valor depois da normalização).

Consequências

Positivas

- A soma de muitos lançamentos de centavos fecha com o saldo do dia.
- O contrato do evento não depende de biblioteca decimal do consumidor: é texto.

Negativas

- Três representações (número no POST, numeric no banco, string no evento e na resposta) pedem teste de contrato em cada borda. Sem esse teste, a decisão vira intenção.
- Multi-moeda não cabe neste value object sem trocar a invariante de “sempre BRL”. Quando vier, vem com ADR novo, não com um `if` de moeda espalhado.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0017-API-REST-VERSIONADA-RFC9457.MD

API REST versionada e problemas RFC 9457

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

O canal do comerciante (e qualquer integração) precisa de um contrato estável para escrever lançamento e ler saldo. Erro hoje, se vier como `{ "message": "erro" }` com HTTP 200 ou com 500 genérico, obriga o cliente a ler texto. A evolução não pode quebrar quem já manda o JSON v1.

Drivers

- Prefixo de versão visível na URL.
- Erro com tipo, título, status e detalhe, `Content-Type: application/problem+json`.
- Documentação gerada do código (springdoc em `/v3/api-docs` e `/swagger-ui.html`).
- O mesmo estilo nos dois serviços.
- Evento com versão própria, independente da versão HTTP.

Opções consideradas

1. Sem versão na URL, evolução “compatível sempre”

- Positivo: URL curta.
- Negativo: a primeira quebra não tem para onde mandar o cliente antigo. Campo opcional resolve adição, não remoção nem mudança de semântica de estorno.

2. Versionamento só por header (`Accept: application/vnd.fluxocaixa.v1+json`)

- Positivo: URL estável, versão rica.
- Negativo: invisível no access log padrão e mais trabalhoso no curl e no WAF por path. Para dois recursos, é cerimônia.

3. `/api/v1` e RFC 9457

Recursos como no blueprint. Erros com `type` relativo (`/problems/valor-invalido` , `/problems/estorno-invalido` , `/problems/idempotencia-conflito` , `/problems/recurso-nao-encontrado`), `title` , `status` , `detail` , `instance` . Corpo de sucesso em `application/json` . OpenAPI via springdoc.

Códigos usados de forma estável:

Situação	HTTP
Token ausente ou inválido, claim ausente	401
Escopo insuficiente	403
Payload inválido, período maior que 366 dias	400
Recurso de outro tenant ou inexistente	404
Estorno repetido, estorno de estorno, mesma `Idempotency-Key` com outro corpo	409
Falha inesperada	500, sem stack no corpo

- Positivo: o cliente trata 409 de forma diferente de 400. O path no ALB casa com a versão. O Swagger é a referência viva do que o código faz.
- Negativo: `type` relativo exige que o cliente não presuma um host fixo. Quebrar o v1 exige manter v1 e v2 juntos por um tempo — custo real de código, e é exatamente o que a versão compra. Problema genérico demais (`about:blank` para tudo) desperdiça o RFC; por isso os `type` acima são fechados e poucos.

Decisão

Opção 3. A versão do evento (`eventVersion`) não segue a versão da URL. Dá para evoluir o JSON do SNS sem abrir `/api/v2` , e o contrário também.

Paginação da lista: `page` a partir de 0, `size` limitado a 50. Resposta com `content` , `page` , `size` , `totalElements` , `totalPages` .

Consequências

Positivas

- O teste de contrato e o smoke assertam status e `content-type` , não uma frase em português.
- WAF e métrica RED separam os dois serviços pelo path, que já carrega o recurso.

Negativas

- Mensagem em `detail` pode mudar sem ser quebra; `type` e `status` não podem. Isso precisa estar dito no OpenAPI para o consumidor não parsear `detail` .
- Dois Swaggers (8081 e 8082). Um portal unificado seria o BFF que não adotamos. Até lá, são duas UIs.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0018-BORDA-ALB-WAF.MD

Borda com ALB e AWS WAF

- **Status:** Substituído (Superseded) pelo ADR 0022
- **Data:** 2026-11-10

O texto abaixo é o registro histórico do ALB público com WAF regional. A borda seguinte é o ADR 0022; o ADR 0031 acrescenta a REST API regional na frente do ALB interno.

Contexto e problema

Os dois serviços precisam de um ponto HTTPS único, com TLS, roteamento por path, health check independente e uma camada que barre abuso antes da task. A autorização de negócio continua no serviço (ADR 0011).

Drivers

- Dois target groups, readiness em `/actuator/health/readiness`.
- TLS com ACM.
- Proteção contra volume abusivo e payload obviamente malicioso, sem cortar o pico legítimo de 50 req/s.
- Mesma aplicação no local, sem a borda AWS — logo a borda não pode ser o único lugar que valida JWT.
- Custo fixo previsível.

Opções consideradas

1. Amazon API Gateway HTTP API na frente, VPC Link até as tasks

- Positivo: throttling por rota, JWT authorizer, plano de uso, se um dia houver API key de parceiro.
- Negativo: hop a mais e cobrança por requisição. O VPC Link não é de graça. O authorizer na borda, se for o único controle, contradiz o ADR 0011. Não temos portal de desenvolvedor nem transformação de payload. Para dois paths estáveis, o Gateway compra recurso que não usamos e cobra por request que o ALB já cobre com LCU.

2. CloudFront + WAF na frente do ALB

- Positivo: borda anycast, cache de GET, WAF no edge.
- Negativo: o GET de saldo é por tenant e não é anônimo; cache na CDN é fácil de errar (vazamento entre comerciantes) e o ganho é pequeno com o Redis já na região. Mais um componente. Pode entrar depois, na frente, sem mudar o serviço, se a latência de clientes distantes de São Paulo doer.

3. ALB público com ACM e AWS WAF

Regras de path:

- `/api/v1/lancamentos*` → lançamentos
- `/api/v1/consolidados*` → consolidado

WAF: managed rules Common, Known Bad Inputs e Amazon IP Reputation, mais uma regra rate-based calibrada acima do pico legítimo de um único cliente. O pico de 50 req/s é do serviço; a regra por IP não pode usar 50 como teto de bloqueio sem medir o fechamento real.

- Positivo: encaixe nativo com ECS, health check por target group (o consolidado doente não tira o lançamento do balanceador), TLS, WAF no mesmo recurso, custo dominado pela hora do ALB (USD 0,034/h em `sa-east-1`) e não pelo request. O serviço local ignora a borda e continua correto porque valida o JWT sozinho.
- Negativo: o ALB não versiona API, não transforma body e não emite API key. Rate-based é instrumento grosso (por IP, janela de 5 minutos). Cliente atrás de NAT compartilhado pode somar vários comerciantes no mesmo IP. Por isso o limite começa folgado e o abuso fino, se aparecer, volta a ser argumento para a opção 1.

Decisão

Opção 3. Actuator que não seja liveness/readiness não é publicado no listener público. HTTP 80 redireciona para 443 ou fica fechado; não há API em claro.

Consequências

Positivas

- Falha de readiness do consolidado não remove as tasks de lançamentos. Isso é o RNF crítico na borda, não só na fila.
- WAF managed rule pega uma classe de abuso sem código na aplicação.

Negativas

- Dois lugares podem dizer 401: nenhum hoje na borda, só o serviço. Se no futuro um authorizer for ligado no ALB, os dois precisam concordar no issuer e nos escopos, senão o diagnóstico vira “funciona no local e não na AWS”.
- Regra rate-based mal calibrada é a forma mais fácil de violar o teto de 5% de perda com tráfego legítimo. Mudança nessa regra exige olhar o k6 e o access log, não só o console.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0019-REGIAO-SA-EAST-1-LGPD.MD

Região sa-east-1 e LGPD

- **Status:** Substituído (Superseded) pelo ADR 0021
- **Data:** 2026-11-10

O texto abaixo é o registro histórico da decisão de residência no Brasil. A decisão vigente é `us-east-1`, com salvaguarda de transferência internacional. Não usar este ADR como fonte da região atual. Os USD 641 e USD 1.214 citados no corpo são da topologia anterior (RDS `db.t4g.small`, ALB público, sem ambiente de hml). A conta vigente está na revisão atual do documento 08.

Contexto e problema

Lançamento de caixa identifica um comerciante e descreve a atividade econômica dele. Pode conter dado pessoal na descrição livre (nome de cliente escrito pelo operador, por exemplo). A região de computação e de armazenamento define se esse dado fica no Brasil e quanto a arquitetura custa.

Drivers

- Residência do dado no Brasil como padrão, não como projeto futuro.
- Minimização: o evento de integração não leva descrição nem categoria.
- Custo comparado com `us-east-1`, explícito, para a escolha não ser só discurso.
- Um comerciante, um claim, sem base legal "inventada" no código: a base da operação do caixa é a execução do serviço pedido pelo próprio comerciante, titular da conta.

Opções consideradas

1. `us-east-1`, mais barato

A mesma topologia custa cerca de **USD 641/mês** contra cerca de **USD 1.214/mês** em São Paulo (premissas e tabela no documento 08), relação de 1,89 vez.

- Positivo: quase metade do custo fixo de NAT, endpoint, RDS e Fargate.
- Negativo: dado de caixa fora do Brasil. Transferência internacional, base legal adicional e um contrato de transferência que este desenho não precisa assumir. Latência maior para um operador no Brasil, secundária perto da questão legal.

2. Multi-região active-passive desde o início

- Positivo: queda de `sa-east-1` deixa de ser o fim do RTO.
- Negativo: replica dado pessoal para outro lugar, dobra custo e abre um desenho de conflito que o volume não pede. Está no roadmap como evolução, com o secundário também no Brasil se a região existir, ou com decisão jurídica explícita se for fora.

3. Tudo em `sa-east-1`

VPC, RDS, ElastiCache, SNS, SQS, Cognito, logs, traces, backups e ECR na região de São Paulo. Sem réplica cross-region. Descrição de lançamento não atravessa o evento. Log de aplicação não imprime descrição nem token.

- Positivo: o dado fica onde o titular está. O desenho é simples. A escolha de custo é consciente: pagamos o prêmio da região para não tratar LGPD como fase 2.
- Negativo: a região é mais cara e tem menos alguns lançamentos de serviço no dia do anúncio global (nada do que usamos está ausente: Fargate, RDS PostgreSQL, ElastiCache, Cognito, WAF). Uma indisponibilidade regional longa não tem failover. O RTO de 1 h do blueprint é intra-região.

Decisão

Opção 3. Backup do RDS na mesma região. Não se copia snapshot para `us-east-1` "por segurança" sem uma análise de transferência — cópia esquecida anula a decisão.

Direitos do titular (acesso, eliminação) não são um endpoint nesta versão: eliminação de lançamento contradiz a trilha imutável. O caminho é processo operacional sobre o comerciante (encerrar a conta, reter o que a obrigação comercial exigir, apagar o que for só cadastro no Cognito). Detalhe no documento de segurança. Não se promete `DELETE` de lançamento para "atender a LGPD" destruindo a contabilidade.

Consequências

Positivas

- A estimativa de custo já nasce na região certa. Ninguém descobre o prêmio de São Paulo na fatura.
- O evento mínimo reduz réplica de dado pessoal no consolidado, no cache e na fila.

Negativas

- Descrição continua no banco de lançamentos e pode conter pessoal. Isso é dado a proteger (criptografia, acesso, minimização de log), não dado a replicar.
- Se a organização já padronizou `us-east-1` para tudo, esta decisão é uma exceção a registrar na governança, não um serviço escondido em outra conta sem dono.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0020-ESTRATEGIA-DE-DEPLOY.MD

Estratégia de deploy

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Há dois serviços com SLOs diferentes. Um deploy do consolidado não pode reiniciar o lançamento, e um deploy ruim do lançamento não pode deixar as duas tasks novas no ar recebendo tráfego enquanto as velhas já morreram. O volume não justifica, no primeiro dia, a maquinaria de blue/green com teste sintético no target group espelho.

Drivers

- Rolling com saúde verificada antes de seguir para a próxima task.
- Rollback automático se a task nova não fica pronta.
- Deploy independente por serviço.
- Migração de banco compatível com duas versões da aplicação ao mesmo tempo (a task antiga e a nova convivem durante o rolling).
- Caminho de evolução para blue/green quando o custo de um rollback lento for maior que o custo da ferramenta.

Opções consideradas

1. Recreate (sobe o novo, derruba o velho)

- Positivo: simples, sem duas versões convivendo.
- Negativo: furo de disponibilidade em todo deploy. Incompatível com 99,9% se o deploy for rotina. Também impede expandir o mínimo de duas tasks com segurança.

2. Blue/green com CodeDeploy desde a primeira versão

- Positivo: troca de target group, teste na versão verde, retorno instantâneo para a azul.
- Negativo: duas frotas durante a janela, mais código de pipeline, e o ganho só aparece quando o rollback de rolling (que já existe) for lento demais ou quando quisermos desviar 5% do tráfego. Para o pico atual, é a evolução, não o mínimo.

3. Rolling do ECS com deployment circuit breaker e rollback

`minimumHealthyPercent` 100 e `maximumPercent` 200: a task nova sobe e passa no readiness antes de a antiga sair. Circuit breaker do ECS reverte a deployment se as tasks novas falham. Cada service tem o seu. Flyway só aplica mudança compatível com a versão anterior (coluna nova nullable, sem renomear no mesmo

passo). O passo destrutivo espera todas as tasks estarem na versão nova.

- Positivo: nativo do ECS, sem CodeDeploy para operar no dia 1, rollback sozinho, zero downtime no caso feliz, e o consolidado pode ser implantado sem tocar no service de lançamentos.
- Negativo: o rollback de rolling não é instantâneo: ele inicia outra deployment. Bug lógico que passa no readiness (a task sobe e responde 200 no health, mas calcula saldo errado) não é pego pelo circuit breaker. Isso se pega com teste antes do deploy e com alarme depois, não com o mecanismo de rollout.

Decisão

Opção 3 agora. A opção 2 fica como evolução explícita: quando houver suite sintética pós-deploy (smoke contra o target group verde) e a necessidade de desvio gradual, troca-se o controller de deployment para CodeDeploy blue/green sem mudar a imagem nem o contrato. O gancho é o alarme de 5xx nos cinco minutos seguintes ao deploy, que já existe no documento 07 — hoje ele chama gente; no blue/green ele pode travar a troca.

Migração incompatível não anda no mesmo release que o código que a exige. Dois releases: expande, depois contrai.

Consequências

Positivas

- Deploy vira rotina, não janela. O error budget não é gasto em release.
- O circuit breaker cobre imagem que não sobe, porta errada e readiness que não fica verde.

Negativas

- Readiness que sempre retorna 200 anula a decisão. O readiness do lançamentos verifica o banco; o do consolidado verifica o banco e, de forma degradada e explícita, não transforma Redis fora em “task morta” — senão uma falha de cache derrubaria todas as tasks em sequência. Redis fora é degradação, task segue pronta, métrica de erro do cache dispara.
- Duas versões no ar durante minutos exigem disciplina de Flyway. Um `DROP COLUMN` no mesmo release do código que para de usá-la quebra a task antiga no meio do rolling.

GitOps com ArgoCD, e blue/green com análise, ficam para o dia do EKS (ADR 0028). Não mudam o rolling deste ADR enquanto o scheduler for ECS.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0021-REGIAO-US-EAST-1.MD

Região us-east-1, com transferência internacional

- **Status:** Aceito
- **Data:** 2026-11-10
- **Substitui:** ADR 0019

Contexto e problema

O ADR 0019 fixou `sa-east-1` para o dado de caixa nascer no Brasil. A revisão 2 do blueprint muda a borda para CloudFront com WAF de escopo `CLOUDFRONT` e certificado ACM, e passa a operar três ambientes. WAF e ACM dessa borda são recursos de `us-east-1` mesmo que o banco fique em São Paulo. Manter a computação em `sa-east-1` e a borda em `us-east-1` parte a conta em duas regiões sem resolver residência de verdade: o corpo do POST, que pode trazer descrição com dado pessoal, é inspecionado na borda.

A pergunta deixa de ser “onde é mais barato?” e passa a ser “residência estrita ainda é o requisito, e o que ela custa agora que a borda já toca a Virgínia?”.

Drivers

- Custo comparado com preço de lista, não com ordem de grandeza.
- WAF de escopo CloudFront e o certificado do domínio alternativo do CloudFront exigem `us-east-1`.
- Serviço novo da AWS aparece primeiro na Virgínia. VPC Origin e o ciclo do Advanced JDBC Wrapper são exemplos desta leva, não uma promessa genérica.
- Latência São Paulo → Virgínia na casa de 110–130 ms de RTT, e o que o CloudFront de fato reduz.
- LGPD art. 33: transferência internacional é permitida com salvaguarda, e é proibida se o controlador não conseguir demonstrá-la. Residência não é o único modo de cumprir a lei.
- Um caminho de volta explícito, se um contrato exigir dado só no Brasil.

Preço unitário (Price List, consulta em 2026-11-10)

Item	us-east-1	sa-east-1
Fargate ARM, vCPU-hora	0,03238	0,05570
Fargate ARM, GB-hora	0,00356	0,00612
RDS `db.t4g.medium` Multi-AZ, hora	0,129	0,275
RDS `db.t4g.micro` Single-AZ, hora	0,016	0,034
gp3 Multi-AZ, GB-mês	0,230	0,438

Item	us-east-1	sa-east-1
Valkey `cache.t4g.small`, hora	0,0256	0,0488
NAT, hora e por GB	0,045	0,093
VPC endpoint, hora por AZ	0,010	0,021
ALB, hora	0,0225	0,034
CloudFront, GB, edge América do Sul, primeiros 10 TB	0,110	0,110
CloudFront, GB, edge Estados Unidos, primeiros 10 TB	0,085	—
CloudFront HTTPS América do Sul, 10 mil requests	0,0220	0,0220
EKS control plane, hora	0,10	0,10

A produção inteira, mesmas quantidades, fica em **USD 778/mês** na Virgínia e **USD 1.467/mês** em São Paulo (1,89 vez). Os três ambientes 24×7 diferem em **USD 1.098/mês**, cerca de **USD 13,2 mil/ano**. A conta está no documento 08. CloudFront não entra nessa diferença: o comerciante no Brasil é tarifado pelo edge de São Paulo nos dois desenhos. O API Gateway desta revisão usa o mesmo unitário de request nas duas colunas, então também não entra na diferença.

Não há linha de transferência inter-região no total, porque o desenho não replica dado entre regiões. O custo que a decisão compara é o de morar numa região, não o de copiar bytes para a outra.

Opções consideradas

1. Continuar em `sa-east-1` (ADR 0019)

- Positivo: o disco do RDS, o backup e a fila nascem no Brasil. O texto do ADR 0019 continua válido como descrição desse mundo.
- Negativo: paga-se o prêmio de 1,9 vez. O WAF de CloudFront e o ACM do certificado continuam em `us-east-1`. O corpo do POST atravessa a borda mesmo assim. A "residência" fica incompleta no exato ponto em que a descrição, se o operador escrever dado pessoal, está no request.

2. Multi-região active-passive desde já

- Positivo: queda de uma região deixa de encerrar o RTO.
- Negativo: dobra o piso, replica dado pessoal e exige um único escritor. Continua evolução, não ponto de partida. O RTO de 1 h desta entrega é intra-região.

3. Tudo em `us-east-1`, com salvaguarda de transferência

VPC, RDS, ElastiCache, SNS, SQS, Cognito, logs, traces, backups, ECR, KMS e o ALB interno na Virgínia. CloudFront global, com WAF e ACM em `us-east-1` e PoP em GRU para o comerciante. Sem cópia para `sa-east-1`.

- Positivo: um lugar só. O prêmio de São Paulo sai da fatura. WAF, ACM e dado ficam sob a mesma decisão de transferência, em vez de uma residência no papel e uma borda fora dela. Serviço novo não espera

a região de São Paulo habilitar.

- Negativo: dado de caixa fora do Brasil. O RTT de backbone entra no tempo que o comerciante percebe. Uma indisponibilidade de `us-east-1` não tem failover. A salvaguarda do art. 33 precisa existir de verdade (contrato, minimização, criptografia), não como frase neste ADR.

Decisão

Opção 3. O ADR 0019 fica substituído. Backup e snapshot permanecem em `us-east-1`. Não se copia snapshot para `sa-east-1` "para ter uma cópia no Brasil": essa cópia reabre a transferência no sentido inverso e finge uma residência que o sistema não opera.

Latência

A ordem de grandeza do RTT entre São Paulo e a Virgínia, no backbone, é **110–130 ms**. Não é SLA da AWS; é a faixa usada para a decisão. O CloudFront mitiga o que é handshake, não o que é física:

- TLS 1.2+ termina no edge de GRU. O handshake (dois RTT ou mais) não vai até a Virgínia.
- HTTP/2 e HTTP/3 com conexão persistente: o request seguinte, na conexão já quente, paga **um** RTT de backbone mais o processamento.
- O que não some: esse um RTT. Um POST cujo processamento no ALB leva 80 ms chega ao cliente no Brasil perto de 200 ms com a conexão quente, e bem acima disso na primeira conexão se alguém medir incluindo o TLS até a Virgínia sem edge.

O SLO de p95 < 200 ms do POST continua sendo o tempo **no ALB**, dentro da região, como no documento 05. Redefinir esse SLO como tempo no navegador em São Paulo, com o processamento atual, não fecharia. O caixa aceita a diferença. Um pregão não aceitaria; este produto não é um pregão.

LGPD

O art. 33 da Lei 13.709/2018 permite transferir dado pessoal para fora do Brasil quando o controlador oferece e comprova garantias de cumprimento dos princípios e dos direitos do titular. O mecanismo desta entrega:

- Cláusulas-padrão contratuais e o Data Processing Addendum da AWS, aceitos na conta. AWS é o operador de infraestrutura; o controlador do caixa é o comerciante, e esta solução é operadora perante ele.
- Minimização que já estava no ADR 0019 e continua: o evento não leva descrição nem categoria; o log não leva descrição nem token.
- Criptografia em trânsito e em repouso, CMK, acesso por IAM.
- Registro desta transferência neste ADR e no documento 06.

Dado envolvido: identificador do comerciante, descrição livre (pode conter pessoal por uso, não por contrato), logs de acesso, backups. Não há CPF no contrato.

Se a residência passar a ser exigência (cláusula contratual, norma setorial, ou leitura da ANPD que esta salvaguarda não satisfaça):

1. A computação e o armazenamento voltam para `sa-east-1`. O delta é o da tabela do documento 08: cerca de **USD 690/mês** a mais na produção, cerca de **USD 1.098/mês** nos três ambientes.

2. O WAF de escopo CloudFront continua vendo o corpo do POST. Residência estrita também questiona essa inspeção. A alternativa é WAF regional no ALB em São Paulo e abrir mão do desenho de VPC Origin, de volta à família do ADR 0018, com a borda pública.
3. Snapshot esquecido em `us-east-1` anula a volta. O runbook de saída apaga backup, log e segredo na Virgínia depois da migração, com CloudTrail dessa exclusão.
4. Esta volta é um ADR novo. Não se “reativa” o 0019 por configuração.

Direitos do titular continuam como no ADR 0019: não há `DELETE` de lançamento. Encerrar a conta apaga o cadastro no Cognito e retém a trilha pelo prazo comercial. A transferência internacional não cria um direito de apagar a contabilidade.

Consequências

Positivas

- A estimativa de custo nasce na região em que a fatura vai chegar.
- Borda, segredo e banco obedecem à mesma decisão de lugar.
- O control plane do CloudFront deixa de ser uma exceção escondida numa conta que se dizia “só São Paulo”.

Negativas

- O tempo percebido no Brasil inclui o RTT de backbone. Quem prometer 200 ms no PDV em São Paulo estará prometendo o SLO errado.
- A salvaguarda é contratual e operacional. Sem o DPA aceito na conta e sem a minimização no log, este ADR vira texto.
- Indisponibilidade regional de `us-east-1` continua fora do RTO de 1 h.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0022-CLOUDFRONT-VPC-ORIGIN.MD

Borda CloudFront, WAF e VPC Origin

- **Status:** Aceito, ver ADR 0031
- **Data:** 2026-11-10
- **Atualiza:** ADR 0018, que fica substituído

O texto abaixo é a borda CloudFront + WAF + VPC Origin + ALB interno. O ADR 0031 coloca, por padrão, uma REST API regional entre o CloudFront e o ALB. Com `enable_api_gateway = false`, vale este desenho sem alteração.

Contexto e problema

O ADR 0018 escolheu ALB público com WAF regional. A revisão 2 quer uma única porta de entrada, TLS perto do comerciante no Brasil, o ALB sem IP público, e um lugar onde um front estático futuro caiba sem abrir outra borda. A autorização continua no serviço.

Drivers

- Dois target groups e readiness em `/actuator/health/readiness`. Falha do consolidado não tira o lançamento do balanceador.
- TLS 1.2+ com ACM. O certificado de domínio alternativo do CloudFront mora em `us-east-1`.
- WAF antes da task, sem cortar o pico legítimo de 50 req/s.
- O binário local valida JWT sozinho. A borda não pode ser o único authorizer.
- ALB sem rota a partir da internet.
- Custo fixo previsível. Neste volume a borda inteira fica perto de USD 47/mês em `us-east-1` (custos).

Opções consideradas

1. ALB público e WAF regional (ADR 0018)

- Positivo: um hop a menos, encaixe direto com o target group, WAF na mesma região do ALB.
- Negativo: o ALB tem endereço alcançável. Quem descobre o DNS do balanceador fala com ele sem passar no WAF, a menos que o security group seja tão estreito que na prática já é uma allowlist — e a allowlist de “a internet inteira” não é allowlist. TLS termina na Virgínia (ou em São Paulo, no desenho antigo), então o handshake paga o RTT inteiro. Não há onde pendurar um front estático depois.

2. API Gateway HTTP API, VPC Link até as tasks

- Positivo: throttle por rota, authorizer JWT, plano de uso no dia em que existir parceiro com API key.
- Negativo: cobrança por request, hop a mais, e um authorizer na borda que, se for o único controle, contradiz o ADR 0011. Não há portal de desenvolvedor nem transformação de payload. Para dois paths está-

veis, o Gateway compra recurso que esta API não usa. Continua sendo a opção se um dia houver API key de parceiro e cota por consumidor. Não é o dia 1.

3. CloudFront com WAF de escopo CLOUDFRONT, VPC Origin e ALB interno

Distribuição com `PriceClass_All`, HTTP/2 e HTTP/3, política TLS `TLSv1.2_2021`, certificado ACM em `us-east-1`. WAF: AWSManagedRulesCommonRuleSet, KnownBadInputs, SQLiRuleSet, AmazonIpReputationList, rate-based por IP; AnonymousIpList em modo count no dev. Origem: VPC Origin para o ALB interno, nas subnets privadas. O security group do ALB aceita só o prefix list `com.amazonaws.global.cloudfront.origin-facing`. Sem IP público no ALB.

Comportamento de `/api/*`: `CachingDisabled` e origin request policy `AllViewerExceptHostHeader`.

- Positivo: a internet vê o CloudFront. O ALB não tem rota pública. Shield Standard vem com o CloudFront, sem linha de preço. TLS termina em GRU. A mesma distribuição recebe, depois, um behavior para S3 com Origin Access Control, sem segundo domínio de borda. O WAF de escopo CloudFront é o que a região `us-east-1` já exige, alinhado ao ADR 0021.
- Negativo: mais um componente. O request de API não cacheia, então o CloudFront não reduz carga no ALB; reduz handshake e superfície. O corpo do POST é inspecionado na borda (ver LGPD no ADR 0021). Rate-based continua instrumento grosso, por IP, janela de 5 minutos. Cliente atrás de NAT compartilhado soma vários comerciantes no mesmo IP. O limite começa folgado (a ordem de 30.000 requests por IP a cada 5 minutos, herdada do ADR 0018) e se calibra com access log, não com o número 50 do pico do serviço.

Decisão

Opção 3. O ADR 0018 fica substituído.

Cache desligado em `/api/*` porque a resposta é do comerciante autenticado. Cachear isso na CDN erra de dois jeitos: serve saldo de um tenant a outro se a chave de cache não incluir o token (e incluir o token na chave é um vazamento do token no edge), ou não ganha hit nenhum porque cada token é uma chave. O Redis, dentro da VPC, já é o cache do saldo. A CDN não é o segundo cache dessa API.

Headers de resposta, via Response Headers Policy: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`. Não se cacheia, então HSTS é o header que importa de verdade: empurra o cliente para HTTPS nas visitas seguintes.

`/actuator` que não seja liveness/readiness não é regra do ALB. Prometheus fica fora do listener. HTTP na distribuição redireciona para HTTPS. Não há API em claro.

Shield Standard fica ligado pelo fato de a entrada ser CloudFront. Shield Advanced não entra: o piso mensal dele é outra ordem de grandeza, sem um evento de negócio que o peça.

Consequências

Positivas

- Varrer o intervalo público do ALB deixa de ser um caminho. O prefix list é a allowlist.

- Falha de readiness do consolidado continua local ao target group dele. O RNF crítico sobrevive à borda nova.
- O serviço local não muda: valida o JWT com o mesmo código.

Negativas

- Dois lugares podem dizer 403: o WAF e o serviço. 403 de WAF não é 403 de escopo. O access log da distribuição e o log do serviço precisam ser distinguíveis no runbook, senão o erro de budget vira caça ao fantasma.
- Regra rate-based apertada demais é a forma mais fácil de violar o teto de 5% de perda com tráfego legítimo.
- Log de WAF com o corpo do POST guarda descrição. O log redige corpo e o header `Authorization`. Sem essa redação, a minimização do ADR 0021 fura na borda.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0023-RESILIENCE4J-E-VERSOES.MD

Java 25, Spring Boot 4.0.9 e Resilience4j 2.4.0

- **Status:** Aceito
- **Data:** 2026-11-10
- **Atualiza:** ADR 0007, que fica substituído

Contexto e problema

O ADR 0007 escolheu Java 25 e Spring Boot 4 sem pinar o patch, e registrou que biblioteca incompatível com a linha 4 exigiria outro ADR. Na prática o código chegou a andar em Boot 4.1.x, e o starter `resilience4j-spring-boot4` disponível não entrava nesse classpath: ele compila contra `spring-boot-autoconfigure` 4.0.0. O relay ficou só com `proxima_tentativa_em`, sem jitter e sem circuito. O blueprint, na revisão 2, fixa o patch e exige o Resilience4j.

Drivers

- Um patch só, nos dois serviços. Boot 4.0 num e 4.1 no outro não é opção.
- O starter oficial, não um auto-config copiado para o repositório.
- Circuito e retry onde há chamada externa (SNS, Redis), sem colocar essa chamada no caminho do POST.
- Suporte do Boot explícito, com gatilho de saída no dia em que o Resilience4j declarar 4.1.

Matriz

Versões alvo. O agente de integração confere o que o `pom` resolveu (`mvn dependency:tree`). O que esta decisão trava é a linha, não um patch transitivo que o BOM ainda pode mover dentro da linha.

Peça	Linha
Java	25 (LTS)
Spring Boot	**4.0.9**, última 4.0.x desta decisão
Spring Framework	7.0, trazido pelo BOM do Boot 4.0
Resilience4j	**2.4.0**, artefato `resilience4j-spring-boot4`, mais `spring-boot-starter-aop` (AspectJ), como o Boot 4.0 exige para o starter
Jackson	3, trazido pelo Boot 4.0 (`tools.jackson`)
Hibernate	7, no serviço que usa JPA (lançamentos)
Testcontainers	2, artefatos `testcontainers-junit-jupiter` e `testcontainers-postgresql`
AWS SDK for Java	v2 (pin observado no pom em 2026-11-10: 2.57.12; o patch pode ter sido ajustado pelo agente de serviço)

Peça	Linha
AWS Advanced JDBC Wrapper	linha compatível com o driver PostgreSQL usado pelo Boot 4.0; o plugin `awsSecretsManager` é o requisito, a versão exata é a do pom

O único release do `resilience4j-spring-boot4` na 2.4.0 compila contra autoconfigure 4.0.0 / `resilience4j-spring6`. Boot 4.1 não tem suporte declarado. Subir o parent para 4.1 com esse starter no classpath é build quebrado ou, pior, auto-config que não dispara. Copiar o auto-config para dentro do serviço seria um fork. Os dois foram recusados.

Onde cada padrão entra

Padrão	Onde	Para quê
CircuitBreaker + Retry (backoff exponencial com jitter) + TimeLimiter	Relay, cliente SNS	Falha do publish não vira loop. Circuito aberto adia o lote; o evento fica na outbox.
CircuitBreaker + timeout, fallback no Postgres	Consolidado, cliente Redis	Cache lento ou morto não segura o GET nem derruba a readiness.
Bulkhead (semaphore)	Consumidor SQS	Limita o paralelismo da task para o pool JDBC continuar sendo o teto de propósito, não de acidente.
Retry com backoff	Consumidor, falha transitória de banco	Dentro do visibility. Não retenta erro de domínio.
RateLimiter	Consultas do consolidado, **opcional** e desligado	Só se um cliente legítimo, atrás de um IP que o WAF não separa, passar a ameaçar o p99. Ligado cedo demais, vira a causa da perda de 5%.

Métricas do Resilience4j saem no Micrometer e no `/actuator/prometheus`. O estado do circuito aparece em `/actuator/health` sem mudar a readiness para `DOWN`. Não existe circuito entre os dois serviços: não existe chamada entre eles.

O backoff da coluna `proxima_tentativa_em` continua. O Resilience4j não substitui a outbox; ele impede que o relay, com o SNS doente, publique em rajada no instante em que o SNS volta.

Ciclo de suporte e gatilho de upgrade

A política publicada pela Spring para o Boot dá à minor uma janela de suporte open source e, depois, suporte comercial. A página de suporte do projeto é a fonte da data: esta decisão não inventa o dia do GA. Se a janela open source da 4.0 estiver no fim, isso **não** autoriza pular para 4.1 enquanto o starter não declarar compatibilidade. Ficar na última patch 4.0.9, com o CVE acompanhado, é preferível a um classpath que o Resilience4j não suporta.

Gatilho. O primeiro artefato `resilience4j-spring-boot4` cuja POM compila contra `spring-boot-autoconfigure` 4.1.x. No dia em que isso existir:

1. Os dois poms sobem juntos, na mesma pull request.
2. Rodam ArchUnit, Testcontainers e o teste de contrato do evento.

3. Não se mantém um serviço na 4.0 e o outro na 4.1.

Até lá, dependabot que proponha Boot 4.1 é recusado com ponteiro para este ADR.

Opções consideradas

1. Boot 4.1 sem Resilience4j

Era o estado que o ADR 0007 acabou descrevendo na implementação. O relay só com timestamp. Sem jitter, duas tasks acordam juntas. Sem circuito, um SNS lento ocupa o agendador até o timeout do cliente HTTP.

2. Boot 4.1 com auto-config forkado

- Positivo: “temos circuito” e o parent novo.
- Negativo: o fork não recebe o patch do Resilience4j. O dia em que o starter oficial sair para 4.1, o fork briga com ele.

3. Boot 4.0.9 e Resilience4j 2.4.0

- Positivo: o starter oficial. Os dois serviços na mesma patch. O circuito fica onde há I/O externo.
- Negativo: a linha 4.0 envelhece enquanto a 4.1 segue. Aceito, com o gatilho acima. Virtual threads e o teto do Hikari continuam como no ADR 0007: thread barata não autoriza pool infinito.

Decisão

Opção 3. Java 25, Temurin, imagem não-root, um Maven wrapper por serviço. Pacotes `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`.

Consequências

Positivas

- O POST continua sem SNS no caminho, e o relay ganha um limite que o timestamp sozinho não expressa.
- A matriz cabe numa página. Surpresa de versão vira diff de pom, não virada de sprint.

Negativas

- Ficar na 4.0 é dívida com data. O gatilho precisa de um dono (a revisão mensal do release note do Resilience4j), senão a dívida vence em silêncio.
- RateLimiter opcional é uma tentação. O default é desligado.

Rotação de segredos

- **Status:** Aceito
- **Data:** 2026-11-10

Contexto e problema

Senha de banco em variável de ambiente, gravada na task definition, não gira. Quando gira à mão, o pool JDBC segue com a senha velha até alguém reiniciar a task, e o intervalo entre as duas ações é uma janela de 500. O blueprint exige rotação automática no RDS, releitura pelo Advanced JDBC Wrapper, e o mesmo tratamento para KMS, ACM, JWKS e a credencial de CI.

Drivers

- Segredo nenhum no Git e nenhum na imagem.
- Rotação que a aplicação sobrevive sem deploy.
- CI sem access key de longa duração.
- Alarme quando a rotação falha, não quando o cliente já está tomando 500.
- Lista fechada do que é segredo, com dono e frequência. O que não está na lista não é “segredo informal”.

Opções consideradas

1. Senha no Secrets Manager, lida só na subida da task

- Positivo: sai do Git. A task definition aponta o ARN.
- Negativo: rotação deixa a task com a senha velha na memória do pool. O conserto é redeploy, e quem gira o segredo precisa lembrar do redeploy. Esqueceu, o RDS aceita a senha nova e a task usa a antiga.

2. RDS IAM authentication

- Positivo: token curto, sem senha de longa duração.
- Negativo: o token dura 15 minutos e o pool precisa de um plugin que o renove. É outro mecanismo, válido, e não é o que o blueprint padronizou. Fica como alternativa se o wrapper não se sustentar. Não é a decisão.

3. Senha gerenciada pelo RDS, rotação no Secrets Manager, wrapper que rebusca

`manage_master_user_password` com rotação automática. O contrato da task é `DB_URL=jdbc:postgresql://...`. Com `DB_SECRET_ARN` o serviço normaliza para um único `jdbc:aws-wrapper:postgresql://...` e liga `failover,awsSecretsManager`. No endpoint do RDS Proxy (hostname

com `.proxy-`) o `failover` sai e fica só `awsSecretsManager`: o plugin iria na instância e furaria o proxy. Ausente o segredo (local e teste), o driver PostgreSQL comum usa `DB_USERNAME` e `DB_PASSWORD`, mesmo que a URL tenha vindo com o prefixo do wrapper.

Hikari `maxLifetime` de 15 minutos, menor que o intervalo de rotação. A role da task tem `secretsmanager:GetSecretValue` só no ARN do próprio segredo e `kms:Decrypt` na CMK dele.

- Positivo: a rotação não pede deploy. Falha de autenticação faz o wrapper ler o segredo de novo. O failover de Multi-AZ usa o mesmo wrapper, em vez de um pool cego ao DNS novo.
- Negativo: mais uma dependência no classpath. Local não exerce esse caminho; o teste que importa é de integração com o wrapper apontando para um segredo de mentira, mais o ensaio de rotação em dev. Sem esse ensaio, o plugin é configuração não verificada.

Decisão

Opção 3. Frequência: **30 dias** em hml e prod, **90 dias** em dev. Dev gira menos porque o ambiente é descartável e a rotação semanal só acrescenta ruído; 90 dias ainda impede senha eterna.

O restante gira pelo mecanismo do serviço, não por um cron nosso:

Segredo ou credencial	Dono	Mecanismo	Frequência
Senha do RDS de lançamentos	RDS + Secrets Manager	Rotação gerenciada, wrapper rebusca	30 dias (prod, hml); 90 (dev)
Senha do RDS de consolidado	idem	idem	idem
CMK (RDS, fila, log, segredo)	KMS	Rotação automática da CMK	anual
Certificado ACM do CloudFront	ACM	Renovação gerenciada, certificado em `us-east-1`	gerenciada; alarme se faltar menos de 14 dias e a renovação não tiver ocorrido
Chave de assinatura do JWT (Cognito)	Cognito	JWKS buscado pelo serviço, cache curto	quando o Cognito girar; o serviço não guarda PEM
Credencial de CI	GitHub OIDC → role IAM	Sem access key. Trust policy com o repositório e o environment	a role não tem segredo para girar
Token do Redis	não existe nesta versão	TLS sem AUTH	—
Senha do Grafana local	imagem `otel-lgtm`	`admin` / `admin`, só no Compose	nunca em produção, porque esse Grafana não vai para a AWS
Usuário do Keycloak local	realm de laboratório	senha de laboratório no Compose	não tem correspondente na AWS

AUTH token do ElastiCache fica desligado. Ligar cria o terceiro segredo (USD 0,40/mês) e um segundo wrapper de credencial no caminho do GET. Vale quando o security group deixar de ser a fronteira suficiente — por exemplo, subnet de dados compartilhada com outra aplicação. Até lá, TLS mais security group. A decisão de

ligar é consciente, registrada aqui para não parecer esquecimento.

CI: GitHub Actions assume a role com `id-token`. Não há `AWS_ACCESS_KEY_ID` em secret do repositório. Environment `prod` exige aprovação. A role de CI de aplicação não tem `rds:DeleteDBInstance`. A role de Terraform é outra, também via OIDC, presa ao workflow de infra.

Alarme: EventBridge em `RotationFailed` do Secrets Manager, no tópico operacional. O alarme dispara na falha da rotação, não no 500 do dia seguinte.

Runbook de rotação manual ou emergencial

Quando a rotação automática falhar, ou quando uma senha tiver vazado:

1. Confirmar o alarme e o segredo (qual ARN, qual serviço). Não girar os dois "por garantia" sem olhar o log: o segundo giro no meio do primeiro confunde o wrapper.
2. `aws secretsmanager rotate-secret` no ARN, ou a rotação imediata no console, se a lambda de rotação estiver saudável.
3. Se a lambda estiver quebrada: colocar a senha nova com `PutSecretValue`, atualizar o usuário no RDS para a mesma senha, e só então observar as tasks. O wrapper rebusca quando a autenticação falha. Se o pool não soltar a conexão velha, `maxLifetime` de 15 minutos limita a espera; forçar novo deployment do serviço ECS corta essa espera.
4. Verificar conexão nova no Performance Insights ou em `pg_stat_activity` (usuário da aplicação, timestamp recente) e o 5xx do serviço.
5. Se a senha vazou para log ou ticket, tratar como incidente: girar, invalidar o que foi exposto, e corrigir o log. Girar sem corrigir o log repete o vazamento no próximo stack trace.

KMS com rotação anual não exige esse runbook: a chave antiga continua decriptando o ciphertext já gravado. Não se desabilita a CMK "para forçar". Desabilitar a CMK do RDS é indisponibilidade, não rotação.

Consequências

Positivas

- Vazamento da senha de hoje tem prazo. O prazo é 30 dias no máximo, e minutos se o runbook rodar.
- O mesmo binário sobe local sem o wrapper, porque `DB_SECRET_ARN` vazio cai no driver comum.

Negativas

- O caminho de produção do JDBC não é o caminho do Testcontainers. O ensaio de rotação em dev é obrigatório, uma vez por trimestre, junto do game day.
- Dois mecanismos (wrapper e, se um dia ligarmos, Proxy) que ambos falam de conexão. Proxy não substitui o wrapper para segredo, e o wrapper não multiplexa conexão. Ligar os dois sem essa frase vira configuração duplicada.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0025-RASTREABILIDADE-CONTEXTO.MD

Rastreabilidade e propagação de contexto

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0013

Contexto e problema

O ADR 0013 escolheu OpenTelemetry, ADOT, CloudWatch e X-Ray. O trace do POST morria no commit: o relay roda noutra thread, o SNS não carrega o contexto W3C sozinho, e o consumidor abria um trace novo. O lag de consolidação virava "dois sistemas" no Grafana. Suporte com um id na mão não achava a linha do outro serviço.

Drivers

- Um trace só, do POST ao `UPSERT` do saldo, também no X-Ray.
- Um identificador que o cliente possa guardar e devolver, sem ser o trace id (o cliente não tem obrigação de falar W3C).
- O contrato JSON do evento estável. Telemetria não entra no corpo.
- Log JSON em que o campo exista de verdade, com exemplo, não só o nome da chave num diagrama.

Opções consideradas

1. Só `eventId` no log

- Positivo: zero propagação. O id de negócio já existe.
- Negativo: o `eventId` nasce no commit. O tempo de JWT, a ida ao banco e a recusa 409 não têm `eventId`. O cliente não recebe esse id antes do 201. Não dá para correlacionar o 401.

2. `traceparent` dentro do JSON do evento

- Positivo: um campo só, o consumidor que já parseia o corpo vê o contexto.
- Negativo: toda versão do contrato e todo consumidor de BI passam a carregar telemetria. O JSON deixa de ser só fato contábil. A raw delivery continua, mas o payload inchou com dado que não é linguagem publicada.

3. W3C no HTTP, coluna na outbox, atributo na mensagem

`traceparent` e `tracestate` entram e saem no HTTP pelo Micrometer Tracing com a ponte OpenTelemetry. `X-Correlation-Id` é aceito se for UUID; se vier ausente ou inválido, o serviço gera outro, devolve no response e guarda no MDC como `correlationId`.

No commit, a linha da outbox grava `trace_parent`, `trace_state` e `correlation_id`. O relay, noutra thread, abre um span `PRODUCER` filho desse pai e publica os atributos `traceparent`, `tracestate` (se houver), `correlationId` e `eventType`, todos `String`. O corpo não muda.

O consumidor pede os atributos (`MessageAttributeNames=All`), abre um span `CONSUMER` filho do produtor, preenche o MDC e limpa no fim. Log JSON leva `traceId`, `spanId`, `correlationId` e, quando existirem, `comercianteId`, `lancamentoId`, `eventId`.

- Positivo: o contrato da seção 6 do blueprint permanece. O trace atravessa a fronteira assíncrona. O cliente guarda um UUID, não um header de trace.
- Negativo: esquecer `MessageAttributeNames=All` quebra a propagação em silêncio (o corpo chega, o contexto não). Isso é teste de contrato, não convenção oral. O formato do trace no X-Ray não é o hex de 32 caracteres do log; o documento 07 diz como navegar.

Decisão

Opção 3. Amostragem segue a do ADR 0013 (cabeça no ingress: regra do X-Ray no API Gateway; trace sem pai a 10% em hml e prod). O filho herda a decisão de amostragem do pai: um POST amostrado produz o consumidor amostrado, e um POST não amostrado não acende o consumidor. Amostrar os dois lados de forma independente faria o trace “sumir no meio” justamente nos pedidos que se queria ver.

`comercianteId` é atributo de span e campo de log. Não é label de métrica.

Consequências

Positivas

- Um id devolvido no 201 alcança o log do consolidado.
- Dá para distinguir falha de publish (span produtor sem filho consumidor) de falha de projeção (filho consumidor com erro).

Negativas

- Três ids na mesa (`correlationId`, `traceId`, `eventId`). O documento de integração fixa o papel de cada um. Usar um no lugar do outro é bug de suporte.
- A coluna nova na outbox é obrigatória no insert do POST. Relay que publica sem reler a coluna recria o trace quebrado que este ADR existe para acabar.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0026-ECS-AGORA-EKS-EVOLUCAO.MD

ECS agora, EKS como evolução

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0009, que permanece aceito

Contexto e problema

O ADR 0009 colocou os dois serviços em ECS Fargate. A pergunta volta porque a comparação com Kubernetes é o que uma banca espera ver respondida com número, não com preferência. EKS tem control plane, ecossistema e uma stack que o ECS não pede. Também tem um piso.

Drivers

- Dois serviços, um time, pico que cabe em quatro tasks.
- Custo de control plane explícito: USD 0,10/hora, **USD 73/mês** por cluster, nas duas regiões (Price List, 2026-11-10).
- Operação que o time consegue fazer sem um time de plataforma.
- Porta de saída no dia em que o número de serviços ou de times mudar o desenho.

Comparação

Eixo	ECS Fargate	EKS
Control plane	incluso no preço da task	USD 73/mês por cluster, mesmo vazio
Onde o processo roda	task, sem nó	nó (ou Fargate profile, que devolve o preço de Fargate e tira o bin packing)
Produção deste projeto	USD 778/mês na conta base	USD 950/mês com 3 × `m7g.large`, mesma rede e mesmos dados (custos)
Operação	service, task definition, target group	nó, NodePool, add-on, versão do control plane, upgrade com extended support se atrasar
Curva	o time que já escreve o Terraform de ECS opera o deploy	pede quem já operou Kubernetes, ou tempo para aprender no sistema de caixa
Ecossistema	o da AWS (autoscaling de service, circuit breaker de deploy)	o da CNCF, listado no documento 13
Portabilidade	imagem OCI, que também roda no EKS	manifesto Kubernetes, que não roda no ECS
Bin packing	a task reserva 0,5 ou 1 vCPU o mês inteiro, ociosa ou não	vários processos no mesmo nó, com o desperdício só da folga do nó

Eixo	ECS Fargate	EKS
Autoscaling	CPU, request do ALB, backlog da fila, nativo	HPA, e KEDA para a fila; Karpenter para o nó
Service mesh	não há, e não falta: os serviços não se chamam	Istio ou Linkerd quando houver chamada síncrona interna
GitOps	o pipeline faz push do novo task definition	ArgoCD puxa o manifesto (ADR 0028)

A densidade só paga o control plane quando há o que empacotar. Quatro tasks pequenas em três nós `m7g.large` deixam nó sobrando e ainda pagam USD 73. O bin packing começa a favorecer o EKS quando dezenas de processos dividem o nó. A conta de 30 serviços está no documento 13.

Opções consideradas

1. EKS agora

- Positivo: o manifesto nasce no formato da evolução. Não há migração depois.
- Negativo: USD 73 mais três nós (USD 179) no lugar de USD 87 de Fargate, mais a stack se alguém "já que tem cluster" instalar Istio e Prometheus. Dois serviços não usam o que o cluster oferece. O risco é operar a plataforma em vez do caixa.

2. ECS agora, EKS quando o gatilho disparar

- Positivo: a fatura e a operação cabem no time. A imagem OCI não se joga fora na migração: o que muda é o scheduler e o deploy.
- Negativo: o dia da migração existe. Task definition não vira Deployment sozinha. O documento 13 é o mapa dessa migração, para ela não ser inventada na pressa.

3. ECS em todo o horizonte

- Positivo: nunca se paga control plane.
- Negativo: trava a resposta. Com muitos times, o ECS sem uma plataforma interna vira um Terraform por time e um pipeline por serviço, que é uma plataforma pior, só que sem o nome.

Decisão

Opção 2. ECS Fargate ARM, como no ADR 0009, para os dois serviços.

EKS passa a valer quando a maioria destes for verdadeira:

- mais de **8 a 10** serviços deployáveis, ou mais de um time publicando com calendário próprio;
- necessidade de plataforma interna (template de serviço, política de admissão, GitOps) que o time aceite operar;
- bin packing ou escala por fila (KEDA) que o autoscaling do ECS já não expresse sem contorção.

Um desses sozinho não basta. Dez serviços de um time só ainda podem viver em ECS. Dois serviços com vontade de Istio não justificam o cluster.

Até o gatilho, não se cria “um EKS pequeno para ir aprendendo” em prod. Aprender é cluster efêmero, destruído, fora da conta do caixa.

Consequências

Positivas

- A decisão de hoje tem preço (USD 778 contra USD 950) e um número de serviços que a reabre.
- A stack open source fica especificada antes de alguém instalar o chart da moda. Especificada não é instalada.

Negativas

- Quem comparar só o ecossistema, sem a fatura e sem o tamanho do time, vai achar a decisão tímida. O documento 13 existe para essa conversa ter os dois lados.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0027-OBSERVABILIDADE-OSS-VS-SAAS.MD

Observabilidade: gerenciada agora, SaaS ou self-hosted depois

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0013

Contexto e problema

O sinal é OpenTelemetry nos dois desenhos (local e AWS). O destino é que muda o custo e o plantão. CloudWatch e X-Ray já estão no caminho do ADOT. Grafana Cloud, Datadog, New Relic, Dynatrace e uma stack self-hosted no EKS resolvem o mesmo problema com outra fatura e outra pessoa de plantão. O volume de hoje é pequeno; a escolha errada ou é um piso de SaaS ocioso ou é um Prometheus que ninguém atualiza.

Drivers

- Um destino só para log, métrica e trace, com correlação (ADR 0025).
- Custo no volume real (4 tasks, ~20 GB de log/mês, ~2.000 séries) e no volume de 30 serviços.
- Custo de gente na conta. Premissa de planejamento: USD 8.000/mês carregados por engenheiro sênior.
- Não operar Loki sem o time que já opera cluster.

Opções consideradas

1. ADOT, CloudWatch e X-Ray

É o ADR 0013. Cerca de USD 45/mês na estimativa de produção.

- Positivo: sem contrato novo, sem agente extra na imagem, alarme no mesmo SNS operacional. O trace já tem caminho para o X-Ray.
- Negativo: PromQL não é a língua do CloudWatch. Métrica customizada a USD 0,30 por métrica/mês pune cardinalidade. Log acima de ~100 GB/mês, a USD 0,50/GB de ingestão em `us-east-1`, deixa de ser “incluso no ruído”.

2. SaaS (Grafana Cloud, Datadog, New Relic, Dynatrace)

No volume deste projeto, a lista de 2026-11-10 fica entre cerca de USD 19 (Grafana Cloud Pro, dentro da franquia) e cerca de USD 52 (Datadog no SKU serverless). Números e premissas no documento 13 e no documento 08.

- Positivo: alguém atualiza o armazenamento. Correlação de log e trace é o produto. Grafana Cloud é o salto menor, porque o local já é Grafana.

- Negativo: dado de telemetria (com `comercianteId`) sai para outro operador. É outra transferência, com o DPA daquele fornecedor, não coberto pelo DPA da AWS. O contrato precede o agente.

3. Stack self-hosted (Prometheus ou Mimir, Loki, Tempo, Grafana)

Cerca de USD 136/mês de nó e disco no cenário do documento 08, mais fração de pessoa.

- Positivo: o dado de telemetria fica na conta. A língua é a mesma do laboratório.
- Negativo: USD 2.000 de um quarto de engenheiro superam USD 136 de nó e superam o SaaS. Sem o plantão, a stack cai e o caixa fica cego — pior do que um CloudWatch sem PromQL.

Decisão

Opção 1, agora.

Grafana Cloud é a primeira alternativa no dia em que o time quiser PromQL ou o log passar de ~100 GB/mês, **e** o DPA do fornecedor estiver aceito. Datadog, New Relic e Dynatrace entram na mesma conversa quando o valor pedido for APM com perfil ou análise de causa, não quando o valor pedido for "ver o trace": isso o X-Ray e o Tempo local já fazem. A comparação de 30 serviços está no documento 13 para essa conversa não usar o preço de 4 tasks.

Self-hosted entra junto com o EKS, quando o ADR 0026 disparar e já existir quem atualize o cluster. Instalar a stack antes do time é adotá-la duas vezes: uma agora, outra quando o cluster existir.

Consequências

Positivas

- Um destino, já instrumentado. O dinheiro de observabilidade neste volume é ruído perto de NAT e RDS.
- A troca futura não mexe no código de negócio: o SDK é OpenTelemetry. Muda o exporter.

Negativas

- Investigação rica (PromQL, exemplars) fica mais pobre no CloudWatch do que no Grafana. Aceito no tamanho atual.
- A estimativa de USD 45 é ordem de grandeza. A primeira fatura calibra. Se vier o dobro, a conversa de Grafana Cloud antecipa, sem esperar os 30 serviços.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0028-GITOPS-ARGOCD-EVOLUCAO.MD

GitOps com ArgoCD, como evolução

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0020

Contexto e problema

O deploy de hoje é push: o GitHub Actions assume uma role e chama `UpdateService` no ECS, com a imagem que o CI já construiu. Funciona com dois serviços. Deixa de funcionar como modelo quando o que se implanta é um conjunto de manifestos (quando o EKS existir) ou quando o que está rodando diverge do que está no Git e ninguém vê.

Drivers

- Build once, já decidido: a mesma imagem promove dev → hml → prod.
- O Git como fonte do que deveria estar rodando.
- Drift visível.
- Não operar ArgoCD antes de existir o cluster que ele gerencia.

Opções consideradas

1. Push pelo Actions, para sempre

- Positivo: o pipeline atual. Um log, uma role, um `UpdateService`.
- Negativo: o estado desejado fica repartido entre o Terraform (a task definition) e o workflow (a tag da imagem). Os dois podem divergir, e o próprio comentário do workflow de CD já avisa: o próximo `terraform apply` devolve a tag antiga se ela não tiver sido alinhada. Isso é drift com nome.

2. ArgoCD agora, em ECS

- Positivo: um só modelo mental.
- Negativo: ArgoCD não é o controlador nativo de service ECS. Forçar GitOps em ECS antes do EKS cria um controlador terceiro para um recurso que o Terraform e o Actions já cobrem. Dois donos do service.

3. Push no ECS agora; ArgoCD quando houver EKS

Repositório de manifests (ou uma pasta por ambiente), ArgoCD em modo pull, app-of-apps, ApplicationSet por serviço. A imagem promovida é um commit de tag nesse repositório, não um `kubectl` na workstation. Drift detection ligado: o que mudou no cluster fora do Git aparece como diff, não como “ajuste”.

- Positivo: o cluster não aceita o que o Git não diz. O mesmo mecanismo serve aos add-ons da stack (Karpenter, External Secrets, cert-manager), senão a plataforma vira clique.

- Negativo: mais um componente de alta disponibilidade. ArgoCD fora do ar não tira o caixa do ar (o que já está implantado continua), mas trava o deploy. Precisa de dono.

Decisão

Opção 3. Enquanto o scheduler for ECS, o deploy continua o do ADR 0020: rolling, circuit breaker, a tag imutável publicada pelo CI, promoção dev → hml → prod com aprovação no environment. O alinhamento da tag com o Terraform é dever do pipeline, não um lembrete no comentário do YAML.

Quando o ADR 0026 disparar, o GitOps entra no mesmo passo do cluster, não "na fase 2 do EKS". Cluster sem ArgoCD e com apply manual é o modo mais rápido de a stack open source divergir do documento.

Pull, não push: o cluster busca o Git. A role de CI não precisa de privilégio de `kubectl apply` em prod. A role do ArgoCD precisa, e fica no cluster, com a trilha de quem aprovou o commit.

Consequências

Positivas

- A evolução tem repositório, direção do fluxo e o que fazer com drift, antes de a ferramenta existir.
- Build once não muda com a ferramenta. Muda o ato de implantar.

Negativas

- Até lá, o drift Terraform versus tag do ECS é um risco real. O documento de CI/CD trata esse ponto no pipeline atual, para não esperar o ArgoCD resolver um problema que o ECS já tem.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0029-TRES-AMBIENTES-STATE-REMOTO.MD

Três ambientes e state remoto do Terraform

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0010, que permanece aceito

Contexto e problema

O ADR 0010 escolheu Terraform com ambientes `dev` e `prod` e state remoto. A revisão 2 acrescenta `hml`, com tamanho próprio (seção 15 do blueprint): hml ensaia a produção sem pagar três NAT e sem Multi-AZ. Dois ambientes obrigavam o ensaio a acontecer em dev (barato demais, rede diferente) ou em prod (tarde demais).

Drivers

- Um state por ambiente, lock, sem `tfstate` no Git.
- Promoção de imagem dev → hml → prod, com a infra de cada um já existente. Promover infra "junto com a imagem" no mesmo apply é como o dev de um experimento derruba a rede de prod.
- Hml parecido com prod no caminho da aplicação (endpoints, Container Insights, on-demand) e menor na rede e no banco.
- Apply de prod só pelo pipeline, com aprovação.

Opções consideradas

1. Dev e prod, hml "quando precisar"

- Positivo: uma conta a menos de NAT e de RDS.
- Negativo: o primeiro ensaio de migração Flyway com duas versões da aplicação acontece em prod. O custo de hml (USD 359/mês em `us-east-1` se ficar ligado o mês inteiro) é o preço de não ensaiar em prod.

2. Três contas AWS, uma por ambiente

- Positivo: explosão de raio. Uma credencial de dev não vê prod.
- Negativo: para dois serviços, o custo de operar três contas (trail, usuário, limite) compete com o benefício. A separação que importa neste porte é state, role e approval, não a conta. Conta separada continua evolução se o produto ganhar outro sistema na mesma organização.

3. Três ambientes, três states, na mesma conta, em S3

Prefixo por ambiente no bucket de state, lock nativo do S3 (`use_lockfile`), criptografia com KMS, versionamento do bucket. Um módulo, três `tfvars` . Nomes de fila e tópico com o prefixo do ambiente, como o blueprint já exige (`fluxo-caixa-dev-` , `fluxo-caixa-hml-` , `fluxo-caixa-prod-`). A aplicação continua lendo ARN e URL, não o nome.

- Positivo: `plan` de hml não é `plan` de prod. O lock impede dois applies no mesmo state. O versionamento do bucket é o rollback do state, não do RDS.
- Negativo: a mesma conta significa que uma role mal escrita enxerga os dois RDS. A fronteira é IAM, e IAM errado não tem a rede de segurança da conta separada. A role de dev não tem permissão no state de prod. Isso é política, testada no pipeline, não só escrita.

Decisão

Opção 3. Backend S3 em `us-east-1` , junto da infra. Não há state em `sa-east-1` .

Dimensionamento é o da seção 15 do blueprint, repetido nos tfvars, não copiado para este ADR para não virar uma terceira fonte. O que este ADR fixa é a existência dos três e a regra de cada um:

Ambiente	Para quê	O que não é
dev	Integração na AWS, Spot, Single-AZ, sem endpoint	Produção pequena
hml	Ensaio de migração, de rotação de segredo e de deploy, com endpoints e sem Multi-AZ	Cópia fiel da fatura de prod
prod	Tráfego do comerciante, Multi-AZ, deletion protection, três NAT	Lugar de experimentar tfvars

Destroy de prod não é alvo do dia a dia. Destroy de dev é permitido e é o jeito certo de parar a fatura, melhor do que “desligar task” e deixar NAT.

O pipeline de Terraform (`fmt` , `validate` , `tflint` , Checkov, e o que o documento 14 acrescenta) roda nos três. Apply automático, se existir, para em dev. Hml e prod exigem aprovação do environment.

Consequências

Positivas

- O ensaio de Flyway com rolling tem um lugar que não é o caixa do comerciante.
- A fatura de dev (USD 147) e a de hml (USD 359) estão na estimativa antes de alguém criar “rapidinho”.

Negativas

- Três ambientes ligados 24×7 custam USD 1.216/mês antes de qualquer otimização de horário. Hml desligado à noite economiza task, não endpoint. A decisão de manter hml ligado é operacional e se revê na primeira fatura, com o número na mesa.

- State remoto é agora um componente crítico. Perder o bucket sem versionamento é perder a capacidade de mudar a infra com segurança. Versionamento e a CMK fazem parte do ambiente, não do “depois”.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0030-REPLICA-LEITURA-ESTRATEGIA-DE-DADOS.MD

Réplica de leitura e estratégia de dados

- **Status:** Aceito
- **Data:** 2026-11-10
- **Complementa:** ADR 0006 e ADR 0003

Contexto e problema

O consolidado lê um saldo que ele mesmo projeta. Com o cache quente, o Postgres quase não participa do GET. Com o cache frio, a leitura e a escrita da projeção dividem o mesmo primário. O blueprint deixa uma réplica parametrizável, desligada, e uma variável `DB_READER_URL` para o dia em que a leitura for o problema. Sem um gatilho, a variável vira "ligada em prod porque produção tem réplica".

Drivers

- O read model já é outro banco (o do consolidado). Réplica não é o primeiro passo de CQRS; CQRS já aconteceu.
- Estorno retroativo muda o passado. Estratégia que grava saldo acumulado em todas as linhas futuras paga esse estorno com contenção.
- Custo de uma réplica `db.t4g.medium`: USD 53/mês em `us-east-1`. Barato comparado ao RDS primário, caro comparado a um índice que faltava.
- Lag visível. Réplica atrasada mente o caixa.

Opções consideradas

1. Réplica ligada desde o dia 1

- Positivo: o caminho de código (`readOnly` → reader) nasce exercitado.
- Negativo: USD 53/mês para um `SUM` de 1.800 linhas que o primário faz em milissegundos. E um modo de falha novo (lag) no caminho do número que o comerciante lê.

2. Sem caminho de réplica no código, até precisar

- Positivo: menos um data source.
- Negativo: o dia em que o primário enche, a mudança é de código e de infra no mesmo incidente. A variável opcional custa poucas classes e zero dólar enquanto o URL está vazio.

3. Código pronto, réplica desligada, gatilho numérico

`DB_READER_URL` vazio: todo mundo no primário. Preenchido: `@Transactional(readOnly = true)` vai ao reader, com `LazyConnectionDataSourceProxy` para a conexão nascer na hora em que a transação sabe para qual lado vai. Escrita da projeção, outbox e lançamento ficam no primário sempre.

A réplica só é criada quando o documento 12 disparar: crédito de CPU da instância T caindo por causa de leitura, com hit ratio de cache ainda alto; ou p99 de cache miss acima de 300 ms com o tempo no Postgres; ou conexões de leitura empurrando o primário para 70% de `max_connections`.

Lag acima de 5 s: alarme, e a leitura volta ao primário.

Estratégia de dado em volta disso, na ordem em que se tenta:

1. Índice e uma sentença só para o acumulado (janela), agora.
2. Réplica de leitura do consolidado, no gatilho.
3. Snapshot mensal do acumulado, se a janela passar de 300 ms no p99 com história grande.
4. Aurora com reader autoscaling, se a quantidade de readers tiver de variar e o I/O justificar I/O-Optimized.
5. DynamoDB, OpenSearch ou Athena só quando a pergunta deixar de ser a chave `(comerciante, data)`. DynamoDB não é o destino natural do relatório com estorno retroativo.

- Positivo: o incidente de leitura tem um procedimento que não é “aumenta a instância”. O código não muda naquele dia; muda o tfvar e a variável.
- Negativo: código de routing pouco exercitado até o gatilho. O teste de integração com dois Postgres (Testcontainers) cobre o routing uma vez. Sem esse teste, a opção 3 é a opção 2 com uma classe morta.

Decisão

Opção 3. Default de reader: **zero**. RDS Proxy: **desligado**, pelo motivo do documento 12 (o wrapper já cobre segredo; na instância, também o failover; o Proxy cobre multiplexação, que ainda não falta).

Aurora não é o default. DynamoDB não substitui a projeção enquanto o produto tiver relatório de período e estorno em data antiga.

Limitação da AWS sobre read replica

A documentação do RDS com Secrets Manager diz que RDS for PostgreSQL com senha mestre gerenciada pelo Secrets Manager **não aceita read replica**. O módulo `rds-postgres` mantém `manage_master_user_password = true` e recusa `read_replica_count > 0` numa `validation` da variável e numa `precondition` da instância. `DB_READER_URL` não é injetada enquanto o output `reader_endpoint` for nulo.

Quando o gatilho da opção 3 disparar, a ordem compatível com a senha gerenciada é:

1. **RDS Multi-AZ DB Cluster** (recomendação). Um escritor e dois standbys legíveis, reader endpoint no `DB_READER_URL`. Não é a read replica proibida: é outro formato de implantação, e a senha continua no Secrets Manager. O custo deixa de ser os USD 53 de uma réplica e passa a ser o de três instâncias. O plugin do wrapper na instância/cluster segue `failover,awsSecretsManager`.
2. **Aurora PostgreSQL** com readers e autoscaling, se o número de readers tiver de variar no dia e o I/O justificar. Também convive com senha gerenciada.
3. **Credencial da réplica fora do Secrets Manager**, com rotação por Lambda. Aí a read replica clássica volta a existir, e o módulo teria de aceitar `manage_master_user_password = false`. É a saída que abandona a rotação nativa do RDS. Não é a recomendação.

Se `enable_rds_proxy` estiver ligado, o host de `DB_URL` contém `.proxy-`. Os serviços não usam o plugin `failover` contra esse host: o plugin abriria conexão na instância e furaria o proxy. Fica só `awsSecretsManager`. O proxy já promove o escritor.

Consequências

Positivas

- USD 53 ficam de fora da fatura até a evidência.
- O read model continua um só. Não se cria um terceiro banco “por simetria com CQRS”.

Negativas

- Alguém vai ligar a réplica porque “produção pede”. O parâmetro default 0 e este ADR são a resposta. Ligar sem o gatilho é mudança de decisão, não tuning.
- O teste com dois bancos precisa existir no consolidado. Se não existir, o agente de integração anota a dívida no lugar do teste, não aqui.

FONTE: DOCUMENTOS-BRUTOS/04-ADR/0031-API-GATEWAY-REST-REGIONAL.MD

API Gateway REST regional atrás do CloudFront

- **Status:** Aceito
- **Data:** 2026-11-10
- **Atualiza:** ADR 0022. Não substitui o CloudFront. O ADR 0018 continua substituído pelo 0022; este ADR é o passo seguinte da mesma borda.
- **Implementa:** seção 20 do blueprint

Contexto e problema

O ADR 0022 deixou o CloudFront como única porta pública e o ALB interno, sem IP público. O JWT era validado só na task. Isso cumpre o "o mesmo binário funciona no laboratório", e também deixa três controles para depois: rejeitar token inválido antes de gastar task, cota por consumidor de integração, e um contrato de borda que não muda quando o runtime atrás do ALB mudar.

O desafio pede critério de segurança para consumo de integração. Cliente final e PDV não são o mesmo consumidor. O primeiro autentica com o usuário do Cognito. O segundo precisa de teto próprio, identificável, sem abrir uma segunda API.

Drivers

- Token inválido não deve abrir conexão até o ECS. Authorizer na borda economiza vCPU e reduz a superfície que um token ruim enxerga.
- Throttle e quota por consumidor, não só rate-based por IP. NAT de shopping soma comerciantes no mesmo IP; o WAF não distingue o PDV do caixa.
- Validação do corpo antes da VPC. `tipo`, `valor` e data fora do contrato morrem com 400 no gateway.
- mTLS para parceiro B2B fica disponível no domínio customizado da REST API, sem virar requisito do comerciante no app.
- WAF regional no stage, porque o hostname `execute-api` continua alcançável. CloudFront não é a única URL que a AWS publica.
- Um catálogo. O stage `v1` é a versão. O export OpenAPI sai da própria API.
- Canary de stage e access log por rota, com `requestId` e o trace do X-Ray.
- Trocar ECS por EKS ou Lambda não muda o path público `/api/v1`.

O flag `enable_api_gateway = false` devolve o desenho do ADR 0022. O default dos três ambientes é `true`.

Opções consideradas

1. Sem gateway: CloudFront → VPC Origin → ALB

É o ADR 0022, e continua no código quando o flag está desligado.

- Positivo: um hop a menos. Sem cobrança por request. O laboratório e a AWS validam o JWT no mesmo lugar.
- Negativo: token inválido paga task, ALB e pool JDBC. Não há usage plan. Não há validação de JSON na borda. Não há canary de contrato. O único teto fino é o WAF por IP.

2. HTTP API (API Gateway v2)

- Positivo: cerca de USD 1,00 por milhão de requests em `us-east-1`, contra USD 3,50 da REST. JWT authorizer nativo, VPC Link v2 para o ALB, menos recurso para declarar. É a escolha certa quando não há usage plan, não há validação de body no gateway, não há canary e o volume passa de dezenas de milhões por mês — aí a diferença de USD 2,50 por milhão deixa de ser troco.
- Negativo, para este contrato: usage plan e API key são da REST API. Request validator com JSON Schema é da REST API. Canary de stage é da REST API. Cache de stage, se um dia for ligado com chave que inclua o token, é da REST API. mTLS de domínio customizado para parceiro B2B é da REST API. WAF a HTTP API passou a aceitar em parte do portfólio, mas o conjunto que o desafio pede (cota por consumidor + validação + canary + WAF no stage) não cabe na HTTP API sem completar o que falta na aplicação.

3. REST API regional

Região da API igual à do ALB (`us-east-1`). Não é edge-optimized: o anycast já é o CloudFront. Endpoint configuration `REGIONAL`.

- Positivo: usage plan, API key opcional por rota, request validator, modelo JSON Schema, canary, cache desligável, WAF no stage, mTLS no domínio customizado, access log com os campos da seção 20, authorizer Cognito com `authorization_scopes`.
- Negativo: USD 3,50 por milhão. Mais recurso no Terraform. O stage entra no path da integração privada (`/v1/api/v1/...`). O hostname do ALB interno não cabe em certificado público, então o hop até o ALB continua HTTP dentro da VPC, como já era no ADR 0022.

4. ALB com Lambda authorizer

- Positivo: um hop a menos que o gateway. O authorizer vê o JWT antes da task.
- Negativo: Lambda no caminho crítico, cold start, e nada de usage plan, validator ou canary. Outro runtime para operar. O ADR 0022 já recusou complicar o ALB.

5. Kong ou Apigee self-hosted

- Positivo: portal, transformação e política ricas.
- Negativo: cluster para dois paths estáveis. O custo fixo e a operação engolem a diferença de USD 12,50/mês entre REST e HTTP API em cinco milhões de requests. Entra na conversa se houver dezenas de APIs de produto, não neste caixa.

Decisão

Opção 3, na frente do que o ADR 0022 já construiu.

```
Cliente → CloudFront (+ WAF CLOUDFRONT)
       → API Gateway REST regional (+ WAF REGIONAL)
       → VPC Link v2 → ALB interno → ECS
```

Provider AWS 6.72 confirma o caminho. `aws_api_gateway_integration` tem `integration_target`. O `connection_id` é um `aws_apigatewayv2_vpc_link` (VPC Link v2: subnets e security group). O alvo é o ARN do ALB. Não há NLB. A documentação da AWS de novembro de 2025 descreve essa integração; o VPC Link v1 (`aws_api_gateway_vpc_link`) continua existindo e só aceita NLB. Não foi usado.

O que cada peça faz:

- **Authorizer Cognito** em todo método, com escopo `fluxo-caixa/lancamentos:write`, `fluxo-caixa/lancamentos:read` ou `fluxo-caixa/consolidado:read`. Token inválido ou escopo errado responde 401/403 no gateway. A task **continua** validando o mesmo JWT e lendo `comerciante_id`. Defesa em profundidade: o gateway não é o resource server, é a primeira peneira. O laboratório, sem gateway, não muda de código.
- **Throttle** de stage e de método. Em hml e prod o GET do consolidado fica acima de 50 req/s (80 e 100). Em dev fica em 20, de propósito.
- **Usage plan + API key** só para o mapa `api_consumers`. Vazio por default. `api_key_required` é por rota e nasce `false`: o comerciante não carrega API key.
- **Validator** de corpo e parâmetros no POST de lançamento. O POST de estorno valida só parâmetros: o corpo é opcional, como a seção 7. O model `EstornarLancamento` documenta `motivo` (até 255, opcional), e o serviço aplica esse limite quando o JSON vem. Ligar a validação de corpo nesse método rejeitaria a ausência.
- **WAF regional** com Common, Known Bad Inputs, SQLi e IP Reputation, mais a regra que bloqueia o request cujo header `X-Origin-Verify` não é o valor `current` nem o `previous`. O CloudFront grava o valor `current` num custom header da origem `api-gateway` e usa `AllViewerExceptHostHeader`: o `Host` tem de ser o do `execute-api`, senão a REST API responde 403; o header customizado da origem substitui qualquer valor que o cliente tenha mandado com o mesmo nome.
- **Rotação** do segredo no Secrets Manager, CMK própria, Lambda no protocolo `createSecret` / `setSecret` / `testSecret` / `finishSecret`. A Lambda gera o valor novo, grava `previous` com o current antigo, atualiza o custom header do CloudFront (distribuição achada pela tag `Name`) e os dois byte matches da regra. O valor anterior continua válido até a rotação seguinte, que é a janela em que a distribuição ainda propaga. `rotate_immediately = false` para o apply não girar o header. Dev 90 dias, hml e prod 30, no mesmo ritmo da senha do RDS.
- **Stage** `v1`: access log JSON, X-Ray, métrica detalhada, cache desligado, canary parametrizável. O canary aponta para o mesmo deployment enquanto não houver um segundo deployment; `percent_traffic` 0 deixa o bloco pronto sem desviar tráfego.
- **Path do stage.** A integração privada inclui o nome do stage no path que o ALB vê (`/v1/api/v1/...`). O listener reescreve `^/v1(/.*)$` para `$1` só quando `enable_api_gateway` está ligado. Sem o prefixo, a regra não casa e o path do ADR 0022 segue inteiro. O timeout de integração é 10 s. A URI leva o path e os path params; os headers `Authorization`, `traceparent`, `tracestate`, `X-Correlation-Id` e

`Idempotency-Key` são mapeados. `X-Origin-Verify` é limpo antes do ALB para não cair no log da aplicação.

mTLS de parceiro não é criado nesta entrega. Não há domínio de parceiro nem truststore. A REST API é o lugar onde isso entra, com domínio customizado e S3 truststore, sem redesenhar o ECS.

Consequências

- Hop a mais. A ordem de grandeza publicada pela AWS para o gateway regional fica em 10–30 ms. O SLO de latência deste sistema continua medido no serviço (p95 do POST < 200 ms, p99 < 500 ms; GET de consolidado p95 < 100 ms com cache). O comerciante percebe o gateway em cima do RTT já discutido no ADR 0021. Não se aperta o SLO do processo para "absorver" o gateway: os dois números são séries diferentes. O alarme de `Latency` p99 do gateway está em 1 s; o de `IntegrationLatency` p99, em 800 ms.
- Custo variável. Com a premissa já usada no CloudFront (0,2 / 1 / 5 milhões de requests por mês), a REST API custa USD 0,70 / 3,50 / 17,50. A HTTP API, na mesma premissa, custaria USD 0,20 / 1,00 / 5,00. A diferença em produção é USD 12,50/mês. O WAF regional acrescenta cerca de USD 10 de piso mais USD 0,60 por milhão. A conta fechada está no documento de custos.
- O bootstrap do header é `secret_string_wo` com senha efêmera. O resource da versão não guarda o valor. CloudFront e o WAF leem `AWSCURRENT` via data source, então o apply depois da rotação acompanha o par vigente em vez de reverter ao sorteio inicial. O header continua no state da distribuição, o par no state do data source e nas strings da regra. `secret_string_wo_version` permanece 1; incrementá-lo regrava o segredo e deve ser uma decisão explícita, não um efeito colateral do plan.
- `ListDistributions` da Lambda é `Resource: "*"`. A API da AWS não aceita ARN nesse action. O update da distribuição está limitado a `distribution/*` da conta, e o código recusa qualquer distribuição cuja tag `Name` não seja a do ambiente.
- Cliente browser em outra origem continua sem CORS, como já era no ALB. Não foi pedido.
- OpenAPI do gateway e OpenAPI do springdoc podem divergir se alguém alterar o DTO e esquecer o model. O model é a peneira da borda; o Bean Validation continua sendo a regra do domínio.

Custos que esta decisão assume

Preço de lista em `us-east-1`, o mesmo já usado no documento de custos para WAF (USD 5 por Web ACL, USD 1 por regra, USD 0,60 por milhão) e o preço que a seção 20 fixa para a API (REST USD 3,50/milhão, HTTP USD 1,00/milhão). VPC Link v2 não entra com linha horária: o custo que se evita é o NLB que a integração antiga exigia. Lambda de rotação: uma invocação por período, abaixo de um centavo. Segredo extra: USD 0,40/mês. CMK extra: USD 1/mês.

PARTE 05

Requisitos não funcionais

SLI, SLO, error budget, escala, resiliência e plano de carga

FONTE: DOCUMENTOS-BRUTOS/05-REQUISITOS-NAO-FUNCIONAIS.MD

05 — Requisitos não funcionais

Este documento operacionaliza os RNF do contexto e as metas do blueprint. Número sem teste é intenção; a coluna de evidência diz onde a intenção vira verificação.

1. SLI, SLO e error budget

Janela: 30 dias corridos (43.200 minutos). O SLI de disponibilidade e de latência do processo continua medido no serviço, dentro de `us-east-1`, e no próprio processo no local. O API Gateway acrescenta cerca de 10–30 ms nesse caminho (ADR 0031). Esse acréscimo não entra no SLO do processo: entra no tempo que o comerciante percebe, junto com o RTT entre o edge de São Paulo e a Virgínia (cerca de 110–130 ms numa conexão já quente, ADR 0021). Bloqueio do WAF e 401 do authorizer são séries à parte, não 5xx do serviço. Se um contrato futuro medir o SLO na borda, o orçamento de latência do processo encolhe esses 10–30 ms; hoje os dois relógios ficam separados de propósito.

Serviço	SLI	SLO	Error budget
Lançamentos	Proporção de minutos em que o POST e o GET respondem sem 5xx e sem timeout, com o consolidado saudável ou não	99,9%	43,2 min/mês
Consolidado	O mesmo para o GET de saldo e de período	99,5%	216 min/mês
Lançamentos	Latência do POST	p95 < 200 ms, p99 < 500 ms	estoura se o percentil passar por 1 h contínua no pico
Consolidado	Latência do GET diário	p95 < 100 ms com hit de cache; p99 < 300 ms	idem
Consolidado	Taxa de erro durante pico ≥ 50 req/s	≤ 5% (teto do enunciado); ≤ 1% (meta interna)	2,5 erros/s no teto; 0,5 erro/s na meta
Consolidação	`aplicadoEm − occurredAt`	p95 < 5 s em regime estável	não se aplica ao catch-up depois de indisponibilidade longa
Dados	RPO / RTO intra-região	≤ 5 min / ≤ 1 h	PITR e game day

Política de queima:

- Queima rápida: 2% do budget mensal em 1 hora (lançamentos ≈ 0,9 min de erro naquela hora; consolidado ≈ 4,3 min). Alarme page.
- Queima lenta: 5% do budget em 6 horas. Alarme ticket.
- Budget esgotado no mês: congela deploy que não seja correção de confiabilidade. O congelamento é regra de time, não de ferramenta; o ADR 0020 deixa o rollback técnico independente disso.

Latência e disponibilidade são SLOs separados. Um serviço rápido que fica fora estoura budget de disponibilidade. Um serviço no ar com p95 de 800 ms estoura o de latência e ainda pode estar “disponível”.

2. Escalabilidade

Escala horizontal primeiro.

Alavanca	Como está hoje	Limite em que muda o desenho
Tasks ECS	Prod: mínimo 2, máximo 10 nos dois serviços. Lançamentos 0,5 vCPU / 1 GiB. Consolidado 1 vCPU / 2 GiB. CPU, request do ALB e, no consolidado, backlog da fila	Acima do teto, rever pool JDBC (10 no lançamentos, 20 no consolidado, por task) contra `max_connections` do `db.t4g.medium`
Cache	Versão por comerciante, TTL 30 s, Redis local e Valkey na AWS	Hit ratio < 80% no pico: ou o padrão de acesso não é o da premissa P6, ou a versão está sendo incrementada à toa
Banco	`db.t4g.medium` Multi-AZ em prod, gp3 50 GB, 3.000 IOPS inclusos. Réplica desligada até o gatilho	Leitura do `SUM` : réplica, não instância maior. Ver 12
Fila	Standard, competing consumers = número de tasks	Se um comerciante sozinho gerar mais do que uma task drena, particionar por `comercianteId` (fila FIFO com message group, ou várias filas). Não é o caso a 20 evt/s
Particionamento de dado	Não há. Um banco cabe	Shard por `comerciante_id` só se um único Postgres não guardar o histórico. Com uma linha por lançamento e dezenas de milhões de linhas, ainda é cedo

Não há scale-to-zero em produção. O mínimo de duas tasks é o que torna o N−1 verdadeiro no pico de 50 req/s (ρ sobe de 17% para 33% com uma task, ainda abaixo do joelho). Scale-from-zero economizaria a task ociosa e gastaria o SLO no primeiro minuto do fechamento.

Auto scaling reage em minutos. Por isso o desired count do fechamento conhecido (dia de pico de calendário) pode ser elevado por schedule antes do evento, em vez de esperar a CPU. Isso é ajuste de operação, não de código.

3. Resiliência

Mecanismo	Onde	Efeito
Duas AZs, tasks espalhadas	ECS	Perda de uma zona não zera o serviço
RDS Multi-AZ	Cada banco	Promoção do standby sem mudar a string de conexão além do failover do endpoint
ElastiCache com réplica	Consolidado	Promoção do nó. Se o cluster inteiro some, o GET cai no Postgres

Mecanismo	Onde	Efeito
Fila com 14 dias	SQS	Consolidado pode ficar fora uma semana sem perder evento
Outbox	Lançamentos	SNS fora não desfaz o 201
Backoff do relay	`proxima_tentativa_em` até 60 s, mais Resilience4j no cliente SNS: retry com jitter, time limiter e circuit breaker (ADR 0023)	O POST não chama o SNS. Circuito aberto adia o lote; o evento fica na outbox
Timeout curto	Redis e chamadas AWS do relay	Thread virtual não fica presa; o POST não chama esses clientes
DLQ após 5 tentativas	SQS	Poison message sai da frente
Visibility 60 s	SQS	Processamento normal é de dezenas de milissegundos; 60 s cobre um GC ruim sem devolver a mensagem cedo demais
Bulkhead de pool	Hikari máximo 10	Stampede não abre 200 conexões
Degradação	Redis fora ou circuito aberto	Leitura segue no Postgres, mais lenta. Readiness continua verde (ADR 0020, ADR 0023)
Deploy circuit breaker	ECS	Task nova doente não substitui as saudáveis até o fim
Backoff do cliente do caixa	Fora do nosso processo	429/503 com `Retry-After` quando viermos a introduzir limite de aplicação; hoje o limite grosso é o WAF

O que não está coberto: perda da região `us-east-1`. O RTO de 1 h não é multi-região. Dizer o contrário no slide de venda seria falso; a evolução está no documento 10.

Game day mínimo, uma vez por trimestre, no dev AWS ou com falha injetada no local:

1. Parar o `consolidado-service` e registrar lançamento (espera-se 201).
2. Encher a fila, religar, medir tempo de catch-up.
3. Failover do RDS (reboot with failover) durante POST.
4. Redis inacessível durante GET (espera-se 200 mais lento, não 500 em massa).
5. Mensagem inválida na fila (espera-se DLQ em 5 recebimentos, fila principal seguindo).

O script `make resiliencia` cobre o item 1 e a recuperação da fila no ambiente local. Os itens de RDS e ElastiCache são ensaio de nuvem.

4. Desempenho

Metas e o caminho que as sustenta:

Meta	Caminho quente	O que não pode entrar nesse caminho
POST p95 < 200 ms	Validar JWT (JWKS em cache), um commit com dois ou três inserts	Publish SNS, chamada ao consolidado, `SUM` de saldo
GET saldo p95 < 100 ms	JWT em cache + GET Redis	`SUM` no hit; no miss o p95 da meta de cache não se aplica e vale o p99 < 300 ms
50 req/s, erro < 1%	2 tasks, ρ ≈ 17%, pool folgado	Rate limit do WAF abaixo do tráfego real; pool de 1 conexão

Premissas de tempo usadas na conta (a confirmar no k6, não tratadas como medida): S_hit 40 ms, S_miss 120 ms, hit ratio 80%, S médio 56 ms, L = 2,8. Se o k6 mostrar S_hit de 80 ms, L dobra e continua pequeno; a decisão de duas tasks não muda. Se mostrar que uma task não sustenta 150 req/s, o mínimo de HA continua 2 e o μ da planilha é que se corrige.

5. Plano de teste de carga (k6)

Script em `tests/load`, alvo `make load`. Ambiente: Docker local com os dois serviços e, quando houver, o dev AWS. Não se aponta k6 para produção.

Cenários:

Cenário	Carga	Critério de sucesso
Leitura nominal	50 req/s de GET diário por 10 min, cache quente, token válido	`http_req_failed` < 1%; p95 < 100 ms
Leitura no teto do enunciado	O mesmo, limiar de falha relaxado só neste cenário	erro < 5%; zero crescimento de DLQ
N–1	Uma task do consolidado, 50 req/s por 5 min	erro < 1%; p99 < 300 ms
Escrita	20 req/s de POST com `Idempotency-Key` única por iteração, por 5 min	p95 < 200 ms; erro < 1%; linhas de outbox não publicadas voltam a zero em < 10 s depois do fim
Escrita com consolidado parado	20 req/s de POST, consumer down	p95 do POST < 200 ms; erro do POST < 1%; fila cresce; ao religar, fila drena
Cache frio	GET com Redis flush no início	erro < 1%; p99 < 300 ms; conexões do Hikari abaixo de 10 por task

O relatório do k6 fica como evidência do RNF-05. A corrida de 1 minuto no Docker local, com os números medidos, está em testes. Ela não substitui os 10 minutos nem o N–1 com duas tasks.

6. Matriz de rastreabilidade

RNF	Mecanismo	Evidência
RNF-01 disponibilidade de lançamentos 99,9%	Service ECS separado, ALB target group próprio, outbox	Alarme de 5xx; `make resiliencia` com consolidado parado e POST 201

RNF	Mecanismo	Evidência
RNF-02 consolidado 99,5%	Multi-AZ, 2 tasks, readiness	Alarme de 5xx do target group do consolidado
RNF-03 latência do POST	Commit local, sem SNS no caminho	k6 cenário de escrita; histograma HTTP
RNF-04 latência do GET	Cache-aside	k6 local de 1 min: p95 6,19 ms e p99 9,48 ms no GET, um container (11)
RNF-05 50 req/s e perda	2 tasks, ρ ≈ 17%, meta interna 1%	Mesma corrida: 50 GET/s, `http_req_failed` 0,00%. A topologia de duas tasks continua por revisar na AWS
RNF-06 lag < 5 s	Relay 500 ms, fila, consumidor concorrente	Métrica de lag; smoke que espera o saldo
RNF-07 durabilidade	Transação única lançamento+outbox, RDS PITR	Teste de integração com falha injetada depois do commit e antes do publish
RNF-08 retenção 14 dias	Atributo da fila	`make tf-validate` e revisão do módulo SQS; teste de atributo no LocalStack
RNF-09 RPO/RTO	PITR 5 min, runbook de restore	Game day de restore em dev, cronometrado
RNF-10 isolamento	Sem cliente HTTP entre os serviços	ArchUnit ou teste que falha se o módulo de lançamentos depender do pacote do consolidado; mais `make resiliencia`
RNF-11 segurança	OIDC, escopos, WAF, KMS	06; testes 401/403/404 cruzado
RNF-12 observabilidade	OTel, alarmes de fila e DLQ	Trace do POST até o consumidor no Grafana local; alarme com ação no Terraform
RNF-13 manutenibilidade	Hexagonal, ArchUnit, `/api/v1`, `eventVersion`	Build com ArchUnit; contrato do evento em teste
RNF-14 custo	Fargate ARM, RDS medium em prod, fila gerenciada, região `us-east-1`, REST API	08: cerca de USD 778/mês em prod, USD 147 em dev e USD 359 em hml, 24×7
RNF-15 LGPD	Transferência internacional com salvaguarda, evento sem descrição	ADR 0021; revisão do payload v1
RNF-16 escala	Auto scaling. Teto 10 nos dois serviços. Backlog da fila por target tracking	Revisão do módulo ECS. Teste de política em dev (subir CPU artificial) ainda não executado

Itens cuja evidência depende de infra que outro fluxo ainda está criando ficam com o teste nomeado. Quando o teste existir, a matriz não muda de mecanismo — muda de “especificado” para “executado”. O estado desta entrega é especificado e amarrado ao comando; a execução contínua é do CI descrito em testes.

PARTE 06

Segurança

STRIDE, OAuth2, multi-tenant, OWASP API, criptografia, LGPD e rotação de segredos

FONTE: DOCUMENTOS-BRUTOS/06-SEGURANCA.MD

06 — Segurança

Critério de consumo e de integração, não só de login. O lançamento é fato financeiro: autenticar sem autorizar o tenant, ou integrar a fila sem idempotência e sem política, abre buraco que o TLS não fecha.

1. Ameaças (STRIDE)

Ameaça	Superfície neste sistema	O que reduz
Spoofing	Token forjado; cliente que manda `comerciante_id` no JSON; chamada direta ao execute-api	JWKS no gateway e de novo no serviço, issuer fixo, claim lido só do token, WAF regional exige `X-Origin-Verify`
Tampering	Evento alterado na fila; lançamento alterado no banco; corpo JSON fora do contrato	TLS, SSE-KMS na fila, sem endpoint de update, security group do RDS fechado, request validator no gateway
Repudiation	Comerciante nega um débito	Trilha append-only, `criadoEm`, `eventId`, log com o id do lançamento (sem a descrição)
Information disclosure	Saldo de outro comerciante; descrição em log; backup aberto	404 cruzado, filtro em toda query, log minimizado, RDS criptografado, backup na mesma região
Denial of service	Pico falso, payload grande, poison message, token inválido em massa	WAF rate-based, throttle e usage plan no gateway, `size` máximo 50, período máximo 366 dias, pool JDBC limitado, DLQ
Elevation of privilege	Token de leitura usado para POST; task role que publica e consome	Escopo no authorizer e de novo no serviço, IAM separado por serviço, sem papel admin na API

Não há superfície de upload de arquivo, de webhook de saída definido pelo cliente nem de SQL concatenado. SSRF não tem por onde entrar enquanto a aplicação não buscar URL vinda do request. Se um dia buscar, o item entra nesta tabela antes do código.

2. Autenticação e autorização

- OAuth2/OIDC. Access token JWT. O API Gateway valida assinatura e escopo na borda. Os dois serviços validam de novo (ADR 0011). As duas camadas não são redundância acidental: o gateway não roda no laboratório, e um token que passasse por um bug do authorizer ainda encontra o resource server.
- Escopos: `lancamentos:write`, `lancamentos:read`, `consolidado:read`. Autoridade Spring `SCOPE_` + prefixo. Prefixo vazio no Keycloak, `fluxo-caixa/` no Cognito.
- Multi-tenant: claim `comerciante_id`. Toda query tem esse predicado. 404 para id alheio, para não confirmar existência.
- O consolidado não confia no `comercianteId` do evento sem ser o campo que o produtor colocou. A fila não é pública: só o task role de lançamentos publica, só o de consolidado consome. Um evento com `comercianteId` arbitrário só nasce se o serviço de escrita for comprometido — aí o problema já é outro, e a trilha ainda mostra o `lancamentoId`.

- Client local `fluxo-caixa-app` com direct access grant é de laboratório. Produção não replica senha de usuário no curl do README.

3. Critérios para consumir e integrar

Quem for integrar (PDV, BFF futuro, segundo consumidor da fila) atende a esta lista. Ela é o diferencial de "critério de segurança para consumo de serviços" do desafio.

Critério	Integração HTTP	Integração por evento
Autenticação	Bearer JWT do nosso issuer. API key só para consumidor de integração no usage plan (`api_consumers`). Cliente final não carrega chave: `api_key_required` nasce falso.	IAM da fila. Sem endpoint HTTP de "injeta evento".
Autorização	Escopo no authorizer (`fluxo-caixa/lancamentos:write` , `lancamentos:read` , `consolidado:read`) e de novo no serviço. Leitura não recebe write.	Task role só da fila que lhe cabe. Segundo consumidor ganha fila própria e role própria, não a senha da fila do consolidado.
Tenant	Não enviar `comercianteId` no corpo. O servidor descarta.	O produtor é o único autorizado a preencher `comercianteId` . Consumidor não "corrige" tenant.
Transporte	TLS 1.2+ no CloudFront (ACM em `us-east-1`). Entre task e RDS/Redis, TLS (Redis `REDIS_SSL=true` na AWS).	TLS do endpoint AWS + SSE-KMS.
Rede	A URL pública é o CloudFront. O `execute-api` responde, e o WAF regional só deixa passar `X-Origin-Verify` atual ou anterior. O ALB não tem IP público.	Security group: task não fala com a internet exceto NAT residual; SQS/SNS via endpoint.
Contrato	`/api/v1` , problema RFC 9457, ignorar campo desconhecido.	`eventVersion` . Versão desconhecida vai para DLQ, não é aplicada pela metade.
Idempotência	`Idempotency-Key` no POST de quem cobra ou vende de verdade.	Deduplicar por `eventId` .
Abuso	Throttle do stage e do método, usage plan do consumidor, 401/403 do authorizer, 409 e 429. Não fazer retry cego de 4xx.	Visibility timeout respeitado; não apagar mensagem antes do commit.
Segredo	Token não vai para log, query string nem `Referer` .	Segredo de banco não vai na imagem, no evento nem no Terraform em claro.
mTLS serviço a serviço	Não há chamada entre os dois serviços. Quando houver, o critério passa a ser mTLS ou IAM auth (VPC Lattice / mesh), não uma rede "confiável" aberta.	A fila já é o controle: sem mTLS adicional no SQS, a policy IAM é o mecanismo equivalente.

Assinatura do evento (HMAC do payload) não está ligada. A combinação de raw message delivery com policy da fila e rede fechada é o controle atual. Assinatura do payload entra se um segundo produtor puder publicar no mesmo tópico ou se a fila deixar de ser privada. Até lá, assinatura seria chave para distribuir e girar sem um agressor no caminho. A decisão fica registrada para não parecer esquecimento.

Rate limit: WAF na borda, grosso, por IP, e throttle do API Gateway por método. O GET do consolidado fica acima de 50 req/s em hml e prod. Usage plan dá cota ao consumidor de integração. Limite fino por comerciante, se necessário, é filtro na aplicação com contador no Redis — não está no caminho crítico desta versão

porque o pico legítimo ainda é pequeno e um limite apertado viola o RNF de perda. O WAF não substitui validação de entrada. O request validator do gateway aplica o JSON Schema do POST; o Bean Validation e o domínio repetem a regra.

Validação de entrada: tipo fechado, valor com no máximo duas casas e positivo, descrição 1..255, categoria até 60, data ISO, período ≤ 366 dias, `size` ≤ 50, UUID na chave de idempotência. O model do gateway é a primeira peneira. Bean validation na task, invariante de novo no domínio. O que passa das duas bordas e fura o domínio é bug, não "defesa em profundidade opcional".

4. OWASP API Security Top 10 (2023)

Item	Como aparece aqui	Controle
API1 BOLA	GET de lançamento por id	Id sempre com `comerciante_id` do token. 404 se não casar. Teste com dois usuários.
API2 Broken authentication	JWT	Issuer, audiência, expiração, algoritmo assimétrico, no authorizer e no serviço. Segredo simétrico não é aceito.
API3 Broken object property	Mass assignment de tenant ou de `id`	O model do gateway recusa propriedade desconhecida. O DTO de criação não tem `comercianteId` nem `id`.
API4 Unrestricted resource consumption	Lista e período	`size` ≤ 50, período ≤ 366, timeout de banco, pool limitado, WAF, throttle do gateway e usage plan.
API5 BFLA	Escopo	`consolidado:read` não registra lançamento. O authorizer e o serviço respondem 403.
API6 Sensitive business flows	POST de crédito repetido	Idempotency-Key. Não há fluxo anônimo de criação de conta nesta API.
API7 SSRF	Não há URL no contrato	Não buscar URL do cliente. Se surgir webhook, allowlist e bloqueio de IP interno.
API8 Security misconfiguration	Actuator, CORS, stack trace, bypass do CloudFront	Health não é rota do gateway. CORS fechado até existir front conhecido. Problem Details sem stack. Debug desligado em `aws`. Header `X-Origin-Verify` no WAF regional.
API9 Improper inventory	Serviço esquecido	Dois serviços, um contrato `/api/v1` no gateway, path único no ALB interno, uma distribuição CloudFront. Sem ambiente "demo" com senha na internet.
API10 Unsafe consumption of APIs	Consumo do próprio evento	ACL, versão, DLQ, idempotência. Não se chama API de terceiro com o payload do comerciante nesta versão.

5. Criptografia

Dado	Em trânsito	Em repouso
API pública	TLS no CloudFront (ACM em `us-east-1`), TLS 1.2+. O API Gateway regional também é HTTPS. O trecho VPC Link → ALB interno é HTTP na VPC. O ALB não tem IP público	—

Dado	Em trânsito	Em repouso
JDBC	`sslmode=require` na AWS	RDS criptografado com KMS
Redis	TLS na AWS (`REDIS_SSL=true`); sem TLS no Docker local	ElastiCache at-rest encryption
SNS, SQS, DLQ	HTTPS da API AWS	SSE-KMS
Segredos	SDK sobre TLS	Secrets Manager, KMS
Backup RDS	Canal da AWS	Mesma chave, `us-east-1` . Sem cópia para `sa-east-1`
Imagem ECR	HTTPS	Criptografia do ECR
Access log do ALB	Entrega do ELB na VPC	SSE-S3. O ELB não grava nesse bucket com CMK
Disco local do desenvolvedor	—	Fora do escopo; não se copia base de produção para o notebook

Nove CMKs em cada ambiente (storage e secret de cada RDS, ElastiCache, mensageria, logs, ECR e o header `X-Origin-Verify`), com rotação anual. O access log do ALB é a exceção: SSE-S3, porque o ELB recusa o bucket com CMK.

6. Segredos e IAM

- Senha de banco gerada pelo Secrets Manager, injetada na task, nunca em variável no repositório. O exemplo `lancamentos/lancamentos` do blueprint é só o Compose local.
- Task role de lançamentos: `sns:Publish` no tópico nomeado, `secretsmanager:GetSecretValue` no segredo dela, escrita de log no log group dela. Sem `sqs:ReceiveMessage` .
- Task role do consolidado: `sqs:ReceiveMessage` , `DeleteMessage` , `GetQueueAttributes` na fila e, para o redrive operado por gente e não pela task, a role de operação é outra. `GetSecretValue` só do segredo dela.
- Role de CI: push no ECR e update do service. Sem `rds:DeleteDBInstance` no pipeline de aplicação.
- Role humana de prod: leitura de log e métrica. Apply de Terraform em prod por pipeline, não por access key no laptop.
- Lambda de pre-token: permissão só de execução e log. Sem acesso a RDS.

Least privilege aqui é lista curta de ARN, não `Resource: "*"` .

Rotação

O mecanismo, o runbook e o alarme `RotationFailed` estão no ADR 0024. Resumo operacional:

Segredo ou credencial	Dono	Mecanismo	Frequência
Senha do RDS de lançamentos	RDS + Secrets Manager	`manage_master_user_password`, Advanced JDBC Wrapper rebusca via `DB_SECRET_ARN`	30 dias em prod e hml; 90 em dev
Senha do RDS de consolidado	idem	idem	idem
CMK	KMS	Rotação automática	anual
Certificado do CloudFront	ACM em `us-east-1`	Renovação gerenciada	alarme se faltarem menos de 14 dias
Chave de assinatura JWT	Cognito	JWKS dinâmico, cache curto no serviço	quando o Cognito girar
`X-Origin-Verify`	Secrets Manager + Lambda	Protocolo create/set/test/finish. O Terraform lê `AWSCURRENT` (`current` e `previous`) e reaplica esse par no CloudFront e no WAF. O bootstrap é write-only e não regrava o segredo. O valor anterior continua aceito até a rotação seguinte	30 dias em prod e hml; 90 em dev
Credencial de CI	GitHub OIDC → role IAM	Sem access key de longa duração	a role não tem segredo
AUTH token do Redis	—	Desligado. TLS e security group	—

Hikari `maxLifetime` de 15 minutos, menor que o intervalo de rotação. A role da task tem `GetSecretValue` só no ARN do próprio segredo e `kms:Decrypt` na CMK dele. Local e teste não definem `DB_SECRET_ARN` e usam o driver PostgreSQL comum.

O header `X-Origin-Verify` não volta ao valor de bootstrap no apply seguinte. O resource da versão usa `secret_string_wo`: o sorteio efêmero não fica nesse resource, e `secret_string_wo_version` só reenvia o segredo se for incrementado. CloudFront e a Web ACL leem o estágio `AWSCURRENT`. O que permanece no state é o par lido pelo data source, o custom header da origem e as strings da regra. A distribuição não tem como guardar o header fora do state. Não há `ignore_changes` nesses dois recursos: rota nova e regra nova continuam no plan.

O POST de estorno aceita corpo ausente. O validator desse método não exige JSON. Se o cliente manda `motivo`, o serviço aplica `@Size(max = 255)`. O POST de lançamento continua validando o corpo no gateway.

Runbook curto, quando a rotação automática falhar ou a senha vazar: girar um segredo por vez; o wrapper rebusca na falha de autenticação; se o pool segurar a conexão velha, o `maxLifetime` limita a espera e um deployment novo do ECS corta. Confirmar sessão nova em `pg_stat_activity` e o fim do 5xx. Não desabilitar a CMK do RDS para “forçar” rotação: isso é indisponibilidade.

7. Supply chain

Controle	Onde
Dependabot ou equivalente no GitHub	Maven e Actions, semanal
Build reproduzível	Maven wrapper commitado, versão de plugin pinada
Imagem	Dockerfile com Temurin oficial, usuário não-root, sem segredo em `ENV`
Scan de imagem	ECR scan on push, imagem imutável (tag por digest no deploy, não `latest` em prod)
SBOM	CycloneDX gerado no build Maven (`cyclonedx-maven-plugin`) e anexado ao artefato do CI
SAST	No pipeline, sobre o diff (a ferramenta exata pode ser CodeQL no Actions, que não tem licença extra para repositório público)
DAST	Contra o dev ou o local, com token de teste, fora do pico de produção. ZAP baseline cabe como evolução; o mínimo desta entrega é o teste de autorização automatizado
Ação de CI	SHA pinado nas actions de terceiros, não `@v1` flutuante, quando o workflow for escrito

LocalStack, k6 e Grafana não entram na imagem de produção. A licença AGPL deles não se mistura ao binário do serviço porque eles não são distribuídos dentro dele. Isso está reiterado em custos para não virar "tudo é open source" sem olhar a licença.

8. LGPD e auditoria

Dado pessoal potencial: descrição livre do lançamento, identificador do comerciante, logs de acesso. Não há CPF no contrato. Se o operador escrever CPF na descrição, o campo vira dado pessoal por uso, e a mitigação é orientação de produto mais minimização — o sistema não tem como adivinhar.

Princípio	Tratamento
Finalidade	Operar o caixa do próprio comerciante.
Minimização	Evento sem descrição e sem categoria. Log sem descrição e sem token.
Transferência	`us-east-1` , com cláusulas-padrão e DPA da AWS (ADR 0021). O corpo do POST é inspecionado no WAF; o log do WAF redige corpo e `Authorization` .
Acesso	O comerciante lê os próprios lançamentos pela API. Exportação em massa é evolução.
Eliminação	Lançamento não se apaga. Cadastro no Cognito pode ser removido ao encerrar a conta, preservando a trilha pelo prazo comercial aplicável. Não há `DELETE /lancamentos/{id}` .
Segurança	Seções anteriores.

Auditoria:

- CloudTrail na conta, em todas as regiões usadas, log de gestão imutável (bucket com versionamento e sem delete para o dia a dia).
- A trilha de negócio é a tabela `lancamento`, append-only, com estorno vinculado.
- `eventos_processados` diz qual evento já virou saldo.
- Não se usa a descrição como log de auditoria paralelo.

9. O que a segurança não promete

Não há pentest contratado nesta entrega. Não há WAF com Bot Control pago. Não há mTLS entre serviços porque não há chamada entre eles. Não há detecção de fraude em lançamento (valor atípico, horário): isso é regra de negócio futura, com falso positivo que o comerciante teria de confirmar. Registrar o limite do controle evita que a lista acima seja lida como certificação.

PARTE 07

Observabilidade e monitoramento

Trace W3C ponta a ponta, logs, métricas RED/USE, alarmes e runbooks

FONTE: DOCUMENTOS-BRUTOS/07-OBSERVABILIDADE.MD

07 — Observabilidade

Instrumentação OpenTelemetry na aplicação. Destino local: `grafana/otel-lgtm` (Loki, Tempo, Mimir, Grafana). Destino AWS: sidecar ADOT para CloudWatch e X-Ray (ADR 0013). A propagação através da fila é o ADR 0025. A escolha de não operar Loki em produção, por enquanto, é o ADR 0027.

Log JSON. Sem descrição de lançamento, sem valor em INFO, sem token, sem `Authorization`.

1. Contexto no HTTP

Dois mecanismos, os dois obrigatórios.

W3C Trace Context e X-Ray. `traceparent` / `tracestate` e `X-Amzn-Trace-Id`, na entrada e na saída, via Micrometer Tracing com a ponte OpenTelemetry. O Boot 4.0 só conhece W3C e B3 em `management.tracing.propagation.type`; um bean `TextMapPropagator` acrescenta o propagador X-Ray da AWS sem tirar o W3C. O `OTEL_PROPAGATORS` do processo Java não entra nessa decisão. Um `traceparent` válido é `00-{trace-id de 32 hex}-{parent-id de 16 hex}-{flags}`. Flags `01` significa amostrado. O header do API Gateway `Root=1-{8 hex}-{24 hex}` vira o mesmo trace id de 32 hex. Se os dois headers chegam, o X-Ray vence. Se não chegou nenhum, o serviço abre um trace novo. A flag `Sampled` do pai remoto manda; `management.tracing.sampling.probability` só decide trace sem pai (1.0 no local e no dev, 0.1 em hml e prod).

`X-Correlation-Id`. Se o request trouxer um UUID, esse é o `correlationId`. Se vier ausente, vazio ou com outro formato, o serviço gera um UUID. A resposta devolve o mesmo header, inclusive no 4xx e no 5xx. O MDC leva o campo `correlationId`. O cliente de caixa guarda esse valor. Ele não precisa saber W3C.

Os dois não se substituem. O trace id é do tracer. O correlation id é da jornada, estável para o suporte, e atravessa a fila mesmo quando a amostragem decide não guardar o span.

2. Campos obrigatórios do log

Uma linha JSON, campos no topo (structured logging do Boot):

Campo	Quando
`traceId`	sempre, no fio de uma requisição ou de uma mensagem
`spanId`	sempre, junto do trace
`correlationId`	sempre, nesses mesmos fios
`service.name`	sempre
`comercianteId`	depois que o tenant é conhecido (token válido, ou evento já traduzido)
`lancamentoId`	no POST que gravou, no estorno, e no consumo do evento daquele lançamento
`eventId`	na outbox e no consumo

`traceId` e `spanId` no MDC são os da W3C (32 e 16 hex), não o formato `1-...` do console do X-Ray.

O mesmo trace no POST e no consumo

Medido em 2026-11-10 no Compose, com `X-Correlation-Id: c4d5e6f7-8091-4a2b-8c3d-4e5f60718293`. O `traceId` comum é `5c31753c7c9eebe969f69289ba632f8f`. As linhas abaixo são as que os containers imprimiram, sem token.

Requisição, no `lancamentos-service`:


```
{"@timestamp":"2026-11-
        10T23:24:20.970Z","level":"INFO","logger":"br.com.fluxocaixa.lancamentos.config.LogDeR
        equisicao","message":"Requisição POST /api/v1/lancamentos
        status=201","service":"lancamentos-
        service","traceId":"5c31753c7c9eebe969f69289ba632f8f","spanId":"e079d3d23bf8d0c8","cor
        relationId":"c4d5e6f7-8091-4a2b-8c3d-4e5f60718293","comercianteId":"11111111-1111-
        1111-1111-111111111111","lancamentoId":"01a1c7c6-dc4a-70dd-aecd-03828f5ed8ef"}
```


Publicação do outbox, no mesmo serviço, outro `spanId`:


```
{"@timestamp":"2026-11-
        10T23:24:21.284Z","level":"INFO","logger":"br.com.fluxocaixa.lancamentos.adapters.out.
        messaging.ProcessadorDeOutbox","message":"Evento de outbox
        publicado","service":"lancamentos-
        service","traceId":"5c31753c7c9eebe969f69289ba632f8f","spanId":"c4f0ee7f6b7abe4d","cor
        relationId":"c4d5e6f7-8091-4a2b-8c3d-4e5f60718293","comercianteId":"11111111-1111-
        1111-1111-111111111111","lancamentoId":"01a1c7c6-dc4a-70dd-aecd-
        03828f5ed8ef","eventId":"01a1c7c6-dc5a-706e-a94f-4135c79b9329"}
```


Consumo, no `consolidado-service`, o mesmo `traceId`, o mesmo `correlationId` e o mesmo `eventId`:


```
{"@timestamp":"2026-11-10T23:24:21.273992585Z","@version":"1","message":"Evento de lançamento
        consolidado","logger_name":"br.com.fluxocaixa.consolidado.adapters.in.messaging.Proces
        sadorEventoLancamento","thread_name":"virtual-
        134","level":"INFO","level_value":20000,"traceId":"5c31753c7c9eebe969f69289ba632f8f","
        spanId":"d35330fe0bfdc05f","eventId":"01a1c7c6-dc5a-706e-a94f-
        4135c79b9329","correlationId":"c4d5e6f7-8091-4a2b-8c3d-
        4e5f60718293","comercianteId":"11111111-1111-1111-1111-111111111111"}
```

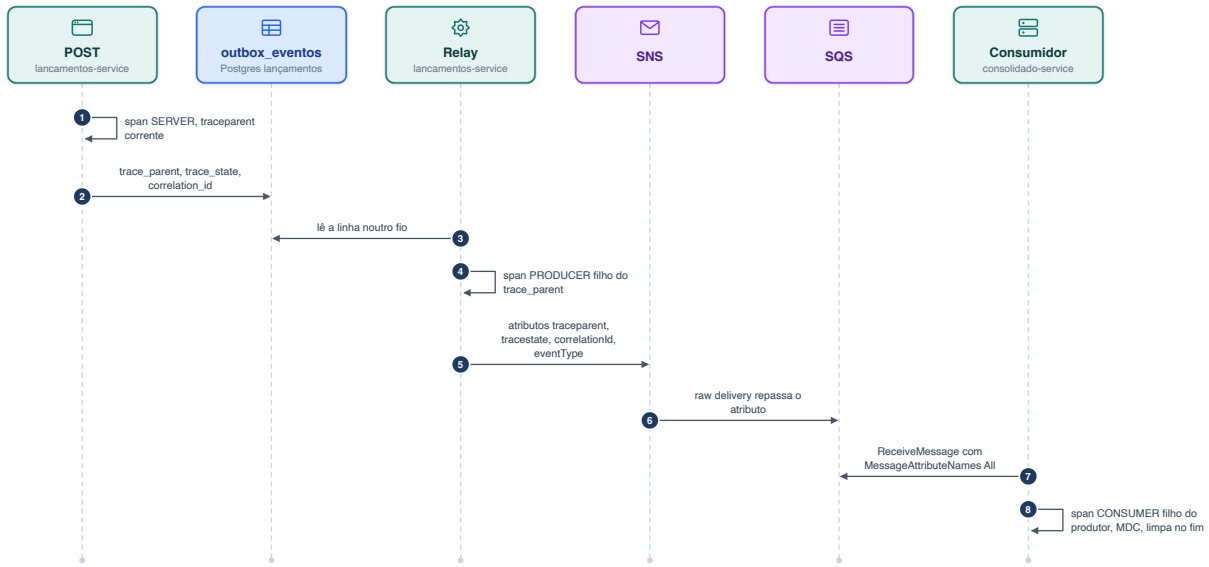

O Tempo do `otel-lgtm` devolveu esse trace em `http://localhost:3000/api/datasources/proxy/uid/tempo/api/traces/5c31753c7c9eebe969f69289ba632f8f` (a porta 3200 do Tempo não está publicada no host; o Grafana faz o proxy). No mesmo trace aparecem os dois serviços, com `service.name` igual ao `OTEL_SERVICE_NAME` do Compose (`lancamentos` e `consolidado`):

- `lancamentos`: `http post /api/v1/lancamentos` (SERVER) e `publicar-lancamento` (PRODUCER)
- `consolidado`: `consolidado.consumir.lancamento` (CONSUMER)

O perfil `local` e o `aws` do lançamentos exportam OTLP de trace para `${OTEL_EXPORTER_OTLP_ENDPOINT}/v1/traces`, no mesmo formato que o consolidado. Sem esse endpoint, o log tinha `traceId` e o Tempo só recebia o consumidor.

3. Da outbox até o consumidor

Trace da outbox até o consumidor



O relay não herda o MDC da thread do POST. A thread do agendador não tem esse contexto. A fonte é a coluna. Publicar “o trace de agora” abriria um trace órfão, filho de ninguém, e o exemplo da seção 2 não fecharia. A task agendada não abre span: execução vazia, a cada meio segundo, não vira trace. O span `PRODUCER` só nasce quando há evento para publicar, já com o contexto restaurado da coluna. `/actuator/health` e as observações `spring.security.*` ficam fora do tracing. Sem o span HTTP, autorização e filtro viravam trace raiz a cada health check.

Atributos SNS, tipo `String`, repassados pelo raw delivery: `eventType`, `traceparent`, `tracestate` quando existir, `correlationId` e `X-Amzn-Trace-Id`. O JSON do evento não ganha esses campos.

Spans: kind `PRODUCER` no publish, kind `CONSUMER` no receive. Nome estável (`lancamento publish`, `lancamento process`) para o Tempo e o X-Ray agruparem. Atributos de span: `eventId`, `lancamentoId`, `comercianteId`, `messaging.system=aws_sqs` no consumidor e `aws_sns` no produtor. Sem o valor do lançamento e sem a descrição.

Amostragem: a decisão é a do pai. POST amostrado implica produtor e consumidor amostrados. Decisão independente nos dois processos faz o trace sumir no meio.

No X-Ray o trace único é POST → relay → SNS → SQS → consolidação, desde que o exportador receba o contexto que o consumidor extraiu. Sem `MessageAttributeNames=All`, o X-Ray mostra dois traces e o log mostra dois `traceId`.

4. Achar o trace a partir do log, e o contrário

Grafana local (Tempo e Loki)

O Compose sobe o `otel-lgtm` na porta 3000. Serviço exporta OTLP para `http://localhost:4318` no perfil `local`.

1. Explore → Tempo. Colar o `traceId` de 32 hex. O trace mostra o span HTTP, o span do produtor e o span do consumidor.
2. Explore → Loki. A linha do POST e a do consumo:

```
{service_name=~"lancamentos-service|consolidado-service|lancamentos|consolidado"} | json |
traceId="5c31753c7c9eebe969f69289ba632f8f"
```

Pelo id que o cliente guardou:

```
{service_name=~"lancamentos-service|consolidado-service|lancamentos|consolidado"} | json |
correlationId="c4d5e6f7-8091-4a2b-8c3d-4e5f60718293"
```

3. Derived field no Loki, do `traceId` para o Tempo, para o clique na linha abrir o trace. Sem o derived field, a correlação existe e o clique não. Vale a configuração uma vez no datasource local e, se um dia o Grafana de produção existir, de novo lá.

O `traceId` do Tempo local é o mesmo hex do log. Não há conversão.

CloudWatch Logs Insights e X-Ray

O log do container chega como uma linha em `@message`. O Insights não promove o campo JSON sozinho em todo log group. O `parse` é o que funciona:

```
fields @timestamp, @message
| parse @message '"traceId":"*"' as traceId
| parse @message '"correlationId":"*"' as correlationId
| parse @message '"eventId":"*"' as eventId
| filter traceId = "4bf92f3577b34da6a3ce929d0e0e4736"
| sort @timestamp asc
```

Pelo correlation id, a mesma query com `filter correlationId = "7c9e6679-7425-40de-944b-e07fc1f90ae7"`. Os dois log groups (lançamentos e consolidado) entram no mesmo Insights, senão a segunda linha fica no outro grupo e o operador conclui que o consumidor não logou.

O console do X-Ray mostra o id no formato `1-{tempo em hex}-{24 hex}`. Não é o `traceId` de 32 caracteres do log. O exportador ADOT faz a conversão. Para achar o segmento a partir do log:

- filtrar no X-Ray pela annotation `correlation_id` (o sidecar deve promovê-la a annotation, não só a atributo de span, porque o filtro do console enxerga annotation);
- ou usar o id convertido que o exportador grava. Não se pede ao suporte para recalcular o id na mão.

O caminho do dia a dia em produção é: `requestId` do access log do API Gateway → `xrayTraceId` no mesmo JSON → `correlationId` (header e campo do log) → linha do serviço no Insights → annotation no X-Ray. O access log do stage traz `requestId`, `extendedRequestId`, `xrayTraceId`, `ip`, `status`, `latency`, `integrationLatency`, `apiKeyId`, `traceparent` e `correlationId`. O X-Ray do stage está ligado. O `traceparent` que o CloudFront repassa segue no mapeamento da integração, então o span do serviço continua filho do contexto que o cliente mandou. O caminho local continua o mesmo hex no Loki e no Tempo.

5. Métricas

Métrica	Lançamentos	Consolidado
Rate	req/s por rota e status	req/s por rota e status
Errors	5xx / total; 4xx separado (4xx não queima budget de disponibilidade)	idem
Duration	histograma do POST, do GET por id e da lista	histograma do GET diário e do período

403 do WAF não aparece no histograma do serviço. Aparece no log do WAF. Os dois entram no painel de borda, separados, para um bloqueio não parecer queda da task.

USE

Recurso	Utilização	Saturação	Erro
Task ECS	CPU e memória	throttling de CPU	task stopped
RDS	CPU, IOPS de gp3, credit balance	conexões / `max_connections`	failover, erro de autenticação depois de rotação
Redis	CPU, memória, hit ratio	evictions	timeout, circuito aberto
SQS	mensagens visíveis, idade da mais antiga	a idade é a saturação	DLQ

Resilience4j

Estado do circuito (fechado, aberto, meio-aberto) do cliente SNS e do cliente Redis, taxa de chamada recusada, latência do bulkhead do consumidor. Saem no Prometheus. O health mostra o estado e não muda a readiness: circuito do Redis aberto com fallback no Postgres é degradação, não task doente.

Caixa

Métrica	Significado
`lancamentos_registrados_total{tipo}`	Fatos aceitos. Parado na hora em que deveria vender é incidente de negócio.
`lancamentos_estornados_total`	Volume de correção.
`consolidacao_eventos_aplicados_total`	Consumo efetivo.
`consolidacao_lag_seconds`	`aplicadoEm − occurredAt`. SLO p95 < 5 s em regime.
`outbox_pendente`	Linhas com `publicado_em` nulo.
`outbox_idade_segundos`	Idade da mais antiga não publicada.
`sqs_idade_mensagem_segundos`	`ApproximateAgeOfOldestMessage`.
`sqs_mensagens_visiveis`	Backlog.

Métrica	Significado
`sqs_dlq_visiveis`	Qualquer valor > 0 é alarme.
`cache_hit_ratio`	A premissa de 80%.

`eventId` e `comercianteId` são atributo de span, não dimensão de métrica.

6. Dashboards

Um por serviço e um de ponta a ponta.

Lançamentos. RED do POST, Hikari em uso contra o máximo, `outbox_pendente`, idade da outbox, taxa de 409, estado do circuito do SNS.

Consolidado. RED do GET, hit ratio, Hikari, lag, idade da mensagem, DLQ, estado do circuito do Redis, ocupação do bulkhead.

Ponta a ponta. Taxa de POST, taxa de evento aplicado, lag p95, e o painel "consolidado fora, lançamentos ok".

Borda. 4xx e 5xx da distribuição CloudFront, bloqueio do WAF por regra, e o widget do API Gateway no dashboard `operacao`: Count, 4XX, 5XX, Latency p99 e IntegrationLatency p99. Sem este painel, o SLO medido no serviço e o tempo que o comerciante vê ficam na mesma curva e a conversa de RTT (ADR 0021) não tem dado.

No local, o Grafana em `http://localhost:3000` já recebe OTLP. O explore da seção 4 é a verificação, mesmo antes do JSON do dashboard existir na infra.

7. Alarmes

Limiares iniciais. Todos no tópico SNS operacional. O alarme não corrige sozinho.

Alarme	Condição	Por que esse número
5xx lançamentos	> 1% por 5 min, pelo menos 20 req	Acima de uma falha isolada, abaixo de queimar o budget em silêncio
5xx consolidado	> 1% por 5 min	A meta interna. 5% é o teto do enunciado, não o alarme
p99 POST	> 500 ms por 10 min, medido no serviço	Igual ao SLO server-side
p99 GET consolidado	> 300 ms por 10 min	Igual ao SLO
Readiness	target unhealthy	O ECS já substitui; o alarme é para o humano
Outbox pendente	> 100 linhas ou idade > 30 s por 5 min	Relay de 500 ms não justifica 30 s
Idade da mensagem SQS	> 30 s por 5 min em regime	Acima do SLO de 5 s, com folga de um retry
DLQ > 0	qualquer mensagem	Poison message é anormal

Alarme	Condição	Por que esse número
API Gateway 5XX	soma > 5 em 1 min, 2 de 3 períodos	Falha de integração ou 5xx do serviço visto pelo gateway
API Gateway 4XX	soma > 100 em 1 min, 2 de 3 períodos	Volume anormal de token inválido ou contrato recusado. Não é 5xx
API Gateway Latency p99	> 1 s	Métrica em milissegundos. Acima do SLO do processo mais o hop
API Gateway Integration-Latency p99	> 800 ms	O backend passou de 800 ms. O timeout da integração é 10 s
RDS conexões	> 70% de `max_connections`	Antes de recusar conexão. Disparar com o teto de tasks ainda longe indica leak
RDS CPU credit	balance em queda por 30 min	Gatilho de classe, e um dos gatilhos de réplica do documento 12
Redis evictions	> 0 sustentado	Memória curta
Circuito Redis aberto	aberto por 5 min	Fallback no Postgres acontecendo de verdade
Circuito SNS aberto	aberto por 5 min	Outbox vai acumular; o POST segue
`RotationFailed`	qualquer evento	A senha não girou. O 500 de autenticação é o alarme atrasado. ADR 0024
Certificado ACM	menos de 14 dias e renovação não concluída	A renovação gerenciada falhou em silêncio

Alarme de “fila com mensagem” não existe. Fila com mensagem é o sistema funcionando. O sinal é a idade.

8. Runbooks

Saldo atrasado, POST normal

1. Ver `consolidacao_lag_seconds` e a idade da mensagem mais antiga.
2. Buscar o `correlationId` do POST no Insights, nos dois log groups. Se a linha do consumidor não existe e a outbox está velha, o relay ou o SNS parou. Ver o estado do circuito do SNS.
3. Se a outbox está em dia e a idade da fila sobe, o consumidor parou ou está lento. Ver readiness e erro de banco.
4. Se há DLQ, ir para o runbook de DLQ. Não reprocessar no escuro.
5. Dizer ao comerciante: o lançamento está durável; o relatório atrasou. Não pedir para lançar de novo.

Consolidado fora

1. Confirmar que o target group de lançamentos segue saudável. Se não segue, não é o incidente isolado.
2. Deixar a fila acumular. Não desligar o relay: a outbox esvazia para a fila, que retém 14 dias.
3. Corrigir a causa.

4. Ao voltar, observar a drenagem. Premissa: cerca de 100 evt/s por task. 72 mil mensagens (1 h a 20 evt/s) levam cerca de 6 minutos com duas tasks.
5. O SLO de lag de 5 s não vale durante o catch-up. O alarme de idade fica silenciado só nessa janela.

Mensagem na DLQ

1. Ler o corpo e o `eventType`. O v1 não tem descrição; não inventar log dela.
2. Bug corrigido e `eventId` ainda válido: redrive para a fila principal. O consumidor idempotente torna o redrive seguro.
3. Versão desconhecida: parar o produtor novo. Não alargar o consumidor para engolir o contrato.
4. Lixo: descartar com registro do `eventId` e da razão.

POST lento

1. No trace, separar JWT, commit e tempo total. O RTT até a Virgínia não está nesse trace: ele está na frente, entre o edge e o ALB. Se o cliente reclama de 300 ms e o span do servidor mostra 80 ms, a diferença é o backbone, não o RDS.
2. Se o commit domina, olhar lock, crédito de CPU do RDS e o índice parcial da outbox.
3. Se o JWT domina, o JWKS não está em cache ou o Cognito está inalcançável.

Redis fora

1. O GET deve continuar 200, mais lento. Se a readiness caiu, a probe está errada (Redis não pode matar a task). Corrigir a probe antes de escalar, senão o ECS reinicia em círculo.
2. Failover do ElastiCache ou recriação do nó. A projeção no Postgres é a fonte.

Rotação falhou

1. O alarme `RotationFailed` é o começo, não o 5xx.
2. Seguir o runbook do ADR 0024: um segredo por vez, wrapper rebusca, `maxLifetime` de 15 minutos limita a conexão velha, deployment novo corta a espera.
3. Confirmar conexão nova em `pg_stat_activity` e o fim do 5xx.

Erro de budget

1. Queima rápida: parar deploy que não seja correção.
2. Olhar o WAF (bloqueio súbito de IP legítimo) antes de culpar a task. 403 de WAF não é 5xx, e o cliente vive os dois como “sistema fora”.

9. Ambiente local

Porta	Uso
3000	Grafana (`admin` / `admin` quando a imagem pedir)
4317	OTLP gRPC
4318	OTLP HTTP, valor de `OTEL_EXPORTER_OTLP_ENDPOINT`

O que validar com `make up` e um POST:

1. A linha do POST tem `traceId`, `spanId` e `correlationId`.
2. A resposta traz `X-Correlation-Id`.
3. Segundos depois, a linha do consumidor tem o mesmo `traceId` e o mesmo `correlationId`.
4. O Tempo mostra os três spans (HTTP, produtor, consumidor) num trace só.
5. A métrica de lançamento registrado incrementa.

Sem o item 3, o dashboard de produção nasce cego. O passo está no README da raiz.

10. O que fica de fora de propósito

Profiling contínuo, RUM (não há frontend), log de SQL em INFO, métrica por comerciante, e um Grafana self-hosted em produção antes do time de plataforma. Contagem por comerciante, se o negócio pedir, é consulta no banco, não label de métrica.

PARTE 08

Estimativa de custos

Infraestrutura e licenças de dev, hml e prod em us-east-1, comparativos de região, EKS e SaaS

FONTE: DOCUMENTOS-BRUTOS/08-CUSTOS.MD

08 — Custos e licenças

Estimativa de **10 de novembro de 2026** para os três ambientes do blueprint, seção 15, na topologia do deployment. Não é fatura. A coluna de `us-east-1` é o orçamento. A coluna de `sa-east-1` repete o mesmo desenho só para mostrar o prêmio que o ADR 0021 deixou de pagar.

Câmbio: USD 1 = BRL 5,70. Premissa de trabalho em 2026-11-10, não cotação oficial. Trocar o câmbio muda o real e não muda a decisão em dólar.

Horas/mês: 730.

Fonte dos unitários. AWS Price List API consultada em 2026-11-10. Vigência publicada: Fargate em 2026-10-01, RDS e gp3 em 2026-10-01 (publicação 2026-10-29), ElastiCache em 2026-10-01 (publicação 2026-10-28), CloudFront em 2026-10-01 (publicação 2026-10-19), NAT e ALB em 2026-11-01 (publicação 2026-11-10), VPC endpoint em 2026-11-01 (publicação 2026-11-05), EKS em 2026-10-01 (publicação 2026-10-27), `m7g.large` em 2026-11-01. WAF, Secrets Manager, KMS, mensageria, CloudWatch e transferência residual estão marcados como estimativa de lista. Validar o total no AWS Pricing Calculator antes de aprovar orçamento.

1. O que a conta ensina antes dos números

O pico de 50 req/s não domina o custo. Em produção, 27 endpoints de interface (9 serviços × 3 zonas) custam mais do que as quatro tasks Fargate, e dois RDS `db.t4g.medium` Multi-AZ custam mais do que a borda inteira. CloudFront, neste volume, é uma linha pequena: o ganho dele é TLS no edge e WAF, não desconto de request.

A mesma produção em São Paulo sai **1,89 vez** a de Virgínia (USD 1.467,93 / USD 778,35). O grosso está em NAT, endpoint, RDS e Fargate. CloudFront cobra pelo edge que atende o comerciante (América do Sul), não pela região da origem: mudar o ALB para `us-east-1` não aumenta essa linha. O API Gateway desta conta usa o unitário de request de `us-east-1` nas duas colunas.

Réplica de leitura e RDS Proxy ficam **fora** do total. O blueprint deixa os dois desligados. Ligá-los soma as linhas da seção 7.

Ambiente diário do desenvolvedor: Docker local, **USD 0**. A linha “dev” é a conta AWS se o ambiente ficar ligado o mês inteiro.

2. Premissas de quantidade

Premissa	dev	hml	prod
AZs	2	2	3
NAT	1	1	3
Endpoints de interface	0 (o tráfego AWS sai pelo NAT)	9 × 2 AZs	9 × 3 AZs

Premissa	dev	hml	prod
Tasks no ar o mês inteiro	1 + 1, Fargate Spot orçado a 50% do on-demand	1 + 1, on-demand	2 + 2, on-demand
Tamanho	ambos 0,5 vCPU / 1 GiB	lançamentos 0,5/1; consolidado 1/2	lançamentos 2 × 0,5/1; consolidado 2 × 1/2
RDS	2 × `db.t4g.micro` Single-AZ, 20 GB gp3	2 × `db.t4g.small` Single-AZ, 20 GB gp3	2 × `db.t4g.medium` Multi-AZ, 50 GB gp3
ElastiCache Valkey	1 × `cache.t4g.micro`	1 × `cache.t4g.small`	2 × `cache.t4g.small`
ALB	interno, 0,1 LCU	interno, 0,25 LCU	interno, 1 LCU
CloudFront	0,2 milhão de requests, 1 GB	1 milhão, 4 GB	5 milhões, 10 GB
WAF	1 Web ACL, 6 regras, requests do dev	1 Web ACL, 5 regras	1 Web ACL, 5 regras, 5 milhões
NAT, dados	40 GB (sem endpoint: pull de imagem, log, API)	10 GB residual	20 GB residual
Endpoint, dados	0	30 GB	50 GB
Segredos	2 (um RDS por serviço)	2	2
CMK	9	9	9
Réplica / Proxy	não	não	não, no total-base

Os nove endpoints são `ecr.api`, `ecr.dkr`, `logs`, `sqs`, `sns`, `secretsmanager`, `sts`, `kms` e `xray`. A lista mínima do blueprint (seção 12) não cita KMS nem X-Ray. Os dois entram para a CMK e o sidecar ADOT não voltarem pelo NAT. S3 é gateway, sem hora. O agente de integração confere se o Terraform criou os nove.

Fargate Spot não tem SKU on-demand na Price List. A AWS publica desconto de até 70%. O orçamento de dev usa **50%** do on-demand para não subestimar o mês em que o desconto realizado for menor. Hml e prod ficam on-demand: hml ensaia a produção, e interrupção de Spot sujaria o ensaio.

3. Preço unitário

Item	us-east-1	sa-east-1	sa / us
Fargate ARM, vCPU-hora	0,03238	0,05570	1,72
Fargate ARM, GB-hora	0,00356	0,00612	1,72
RDS `db.t4g.micro` Single-AZ, hora	0,016	0,034	2,13
RDS `db.t4g.small` Single-AZ, hora	0,032	0,069	2,16
RDS `db.t4g.medium` Single-AZ, hora	0,065	0,137	2,11
RDS `db.t4g.medium` Multi-AZ, hora	0,129	0,275	2,13

Item	us-east-1	sa-east-1	sa / us
gp3 Single-AZ, GB-mês	0,115	0,219	1,90
gp3 Multi-AZ, GB-mês	0,230	0,438	1,90
Valkey `cache.t4g.micro`, hora	0,0128	0,0240	1,88
Valkey `cache.t4g.small`, hora	0,0256	0,0488	1,91
ALB, hora	0,0225	0,0340	1,51
ALB, LCU-hora	0,008	0,011	1,38
NAT, hora e por GB	0,045	0,093	2,07
VPC endpoint, hora por AZ	0,010	0,021	2,10
VPC endpoint, GB (primeiro PB)	0,01	0,01	1,00
CloudFront, GB, edge América do Sul, primeiros 10 TB	0,110	0,110	igual
CloudFront, GB, edge Estados Unidos, primeiros 10 TB	0,085	—	o edge americano é mais barato e mais longe do comerciante
CloudFront HTTPS, América do Sul, por 10 mil (GET e POST encaminhado)	0,0220	0,0220	igual
EKS, control plane, hora	0,10	0,10	1,00
EC2 `m7g.large` Linux, hora	0,0816	—	usado só no cenário EKS

O GB-mês do gp3 Multi-AZ já é o dobro do Single-AZ. Não se multiplica de novo pelo standby.

CloudFront não tem preço “da região da conta”. O comerciante no Brasil, com `PriceClass_All`, cai no edge de São Paulo e paga a tarifa da América do Sul qualquer que seja a origem. `PriceClass_100` empurraria o TLS até um edge americano: no volume de produção a diferença é cerca de USD 6 por mês (10 GB × 0,025 de delta no GB, mais o request mais barato do edge americano) e devolve o handshake para um RTT de 110–130 ms. Não compensa. A conta usa `PriceClass_All`.

WAF de escopo CloudFront (lista pública, igual nas duas regiões): USD 5 por Web ACL/mês, USD 1 por regra/mês, USD 0,60 por milhão de requests. Managed rules usadas aqui não acrescentam a tarifa de Bot Control. Shield Standard não tem linha.

Secrets Manager: USD 0,40 por segredo/mês. Três segredos (duas senhas de RDS e o header `X-Origin-Verify`) = USD 1,20. As chamadas `GetSecretValue` deste volume ficam abaixo de um centavo (USD 0,05 por 10 mil). A Lambda de rotação do header roda uma vez por período e fica abaixo de um centavo.

KMS: USD 1 por CMK/mês. O módulo cria as mesmas nove nos três ambientes (storage e secret de cada RDS, ElastiCache, mensageria, logs, ECR e `origin-verify`), então a linha é USD 9. O access log do ALB usa SSE-S3: o ELB não grava nesse bucket com CMK, e não existe chave `alb-logs`. As chamadas deste volume ficam abaixo de USD 1 e estão absorvidas na linha residual, não somadas de novo.

4. Três ambientes em us-east-1

Cada linha é o produto arredondado a centavo. O total é a soma das linhas arredondadas.

Linha	dev	hml	prod
Fargate ARM	14,42	43,25	86,51
ALB interno	17,01	17,89	22,27
WAF CloudFront	11,12	10,60	13,00
CloudFront	0,55	2,64	12,10
API Gateway REST	0,70	3,50	17,50
WAF regional	10,12	10,60	13,00
NAT	34,65	33,30	99,45
VPC endpoints	0,00	131,40	197,10
Dados dos endpoints	0,00	0,30	0,50
RDS compute	23,36	46,72	188,34
RDS storage	4,60	4,60	23,00
ElastiCache	9,34	18,69	37,38
Mensageria (estimativa)	0,50	1,50	4,00
Observabilidade (estimativa)	8,00	20,00	45,00
Secrets Manager	1,20	1,20	1,20
KMS	9,00	9,00	9,00
ECR	0,50	0,50	1,00
Transferência residual (estimativa)	2,00	3,00	8,00
Total	**147,07**	**358,69**	**778,35**

Dev USD 147/mês (R$ 838). **Hml USD 359/mês** (R$ 2.046). **Prod USD 778/mês** (R$ 4.435).

Os três ligados 24×7: **USD 1.284,11/mês**, **USD 15.409,32/ano**, cerca de **R$ 87.833/ano**.

Fórmulas de produção que ancoram o resto:

- Fargate: 2 × (0,5 × 0,03238 + 1 × 0,00356) × 730 + 2 × (1 × 0,03238 + 2 × 0,00356) × 730 = 86,51.
- Endpoints: 27 × 0,010 × 730 = 197,10.
- RDS: 2 × 0,129 × 730 = 188,34. Storage: 2 × 50 × 0,230 = 23,00.
- NAT: 3 × 0,045 × 730 + 20 × 0,045 = 99,45.
- CloudFront: 5.000.000 × 0,0000022 + 10 × 0,110 = 12,10.
- API Gateway REST: 5.000.000 × 3,50 / 1.000.000 = 17,50. A HTTP API, na mesma premissa, seria 5,00. A diferença é 12,50.

- WAF regional: 5 de Web ACL + 5 regras × 1 + 5 × 0,60 = 13,00. As regras são o header `X-Origin-Verify` e quatro managed rules (Common, Known Bad Inputs, SQLi, IP Reputation).

A premissa de volume é a mesma da linha de CloudFront: **0,2 milhão** de requests em dev, **1 milhão** em hml e **5 milhões** em prod. REST API em `us-east-1`: USD 3,50 por milhão. HTTP API: USD 1,00 por milhão. Dev: 0,70 contra 0,20. Hml: 3,50 contra 1,00. O WAF regional usa a mesma lista do WAF de borda (USD 5 por ACL, USD 1 por regra, USD 0,60 por milhão): piso de USD 10 mais o request. Segredo extra do header: USD 0,40, então a linha de Secrets sobe de 0,80 para 1,20. A CMK `origin-verify` já está nas nove da linha de KMS (USD 9), não é um dólar por cima de outra contagem. Lambda de rotação: uma invocação por período, abaixo de um centavo, sem linha. VPC Link v2 não tem hora nesta conta; o NLB que ele evita teria hora e NLCU.

	dev	hml	prod
REST API	0,70	3,50	17,50
HTTP API, se fosse a escolha	0,20	1,00	5,00
Diferença	0,50	2,50	12,50

HTTP API passa a ser a melhor escolha quando não houver usage plan, validator, canary nem mTLS de parceiro, e o volume subir o bastante para os USD 2,50 por milhão pesarem. Não é este contrato. O ADR 0031 registra o trade-off.

Mensageria, observabilidade e transferência residual são ordem de grandeza. Dois milhões de eventos por mês ficam abaixo de USD 10 mesmo com o request regional mais caro; a linha de USD 4 não muda a decisão. Observabilidade de produção junta log com retenção de 30 dias, Container Insights enhanced, X-Ray e alarmes.

X-Ray, lista pública não reconsultada nesta passagem: **USD 5 por milhão de traces gravados**. Um POST é um trace (API Gateway, lançamentos, relay e consolidado compartilham o id). Health e o relay sem evento não abrem trace. Quem decide na borda é o API Gateway, com a regra padrão do X-Ray: a primeira requisição de cada segundo e 5% das demais. Trace sem pai usa 1.0 em dev e 0.1 em hml e prod. Sobre o volume da linha de CloudFront, concentrado em cerca de 10 horas úteis por dia: dev grava praticamente tudo, **0,2 milhão** (USD 1,00); hml fica abaixo de 1 requisição por segundo e também grava quase tudo, cerca de **0,9 milhão** (USD 4,50); prod, com média de 4 a 5 requisições por segundo nas horas úteis, grava cerca de **1,3 milhão** (USD 6,50). Esse valor já cabe na linha de observabilidade (8 / 20 / 45). O total da seção 4 não muda. A mesma lista vale nas duas regiões. Se o log passar de dezenas de GB, a linha cresce a USD 0,50/GB de ingestão em `us-east-1` (preço de lista público, não reconsultado na API nesta passagem) e o gatilho de observabilidade passa a valer.

Backup de RDS dentro de 100% do armazenamento provisionado não foi somado de novo. Performance Insights de 7 dias fica na franquia. Cognito Essentials com 1.000 MAU e a Lambda de pre-token ficam na franquia (USD 0).

5. Os mesmos três ambientes em sa-east-1

Mesmas quantidades, preços da coluna `sa-east-1`. WAF e CloudFront não mudam: o Web ACL de escopo CloudFront é global na tarifa, e o edge do comerciante continua sendo a América do Sul.

Linha	dev	hml	prod
Fargate ARM	24,80	74,39	148,79
ALB interno	25,62	26,83	32,85
WAF CloudFront	11,12	10,60	13,00
CloudFront	0,55	2,64	12,10
API Gateway REST	0,70	3,50	17,50
WAF regional	10,12	10,60	13,00
NAT	71,61	68,82	205,53
VPC endpoints	0,00	275,94	413,91
Dados dos endpoints	0,00	0,30	0,50
RDS compute	49,64	100,74	401,50
RDS storage	8,76	8,76	43,80
ElastiCache	17,52	35,62	71,25
Mensageria (estimativa)	1,00	3,00	8,00
Observabilidade (estimativa)	12,00	28,00	60,00
Secrets Manager	1,20	1,20	1,20
KMS	9,00	9,00	9,00
ECR	0,50	0,50	1,00
Transferência residual (estimativa)	4,00	6,00	15,00
Total	**248,14**	**666,44**	**1.467,93**

O request da REST API nesta coluna usa o unitário de `us-east-1` (USD 3,50 por milhão). Esta passagem não reconsultou o Price List de `sa-east-1` para o API Gateway. WAF, segredo e CMK seguem a mesma lista já usada nas duas regiões. Se o request em São Paulo for mais caro, a linha sobe e o ADR 0021 continua sendo o lugar dessa conta.

Ambiente	us-east-1	sa-east-1	Diferença	sa / us
dev	147,07	248,14	+101,07	1,69 (+69%)
hml	358,69	666,44	+307,75	1,86 (+86%)
prod	778,35	1.467,93	+689,58	1,89 (+89%)
Soma 24×7	1.284,11	2.382,51	+1.098,40	1,86
Ano	15.409,32	28.590,12	+13.180,80	1,86

O ano em São Paulo seria cerca de **R$ 162.964**. A diferença anual de **USD 13.181** (R$ 75.130) é o preço de manter o dado no Brasil com esta topologia. O ADR 0021 registra por que a transferência internacional com salvaguarda foi aceita, e o que se refaz se um contrato exigir residência.

Não há transferência inter-região nesta conta: nada replica de Virgínia para São Paulo. O preço de saída entre regiões não entra no total. O que a decisão compara é o preço de **morar** em cada região.

6. Onde o dinheiro de produção está

Bloco em us-east-1	USD/mês	Parte
Rede fixa (NAT + endpoints)	296,55	38%
RDS compute + storage	211,34	27%
Fargate	86,51	11%
Borda (ALB + WAF CloudFront + CloudFront + REST API + WAF regional)	77,87	10%
ElastiCache	37,38	5%
Observabilidade estimada	45,00	6%
O resto	23,70	3%

A alavanca continua sendo rede e classe de banco. O request do pico cabe na linha da REST API (USD 17,50 em cinco milhões) e não muda essa ordem.

7. Opcionais que o total não inclui

Opcional	Conta em us-east-1	USD/mês	Em sa-east-1
Réplica de leitura do consolidado, `db.t4g.medium` Single-AZ, 50 GB	0,065 × 730 + 50 × 0,115	53,20	110,96
Réplica também no lançamentos	o dobro	106,40	221,92
RDS Proxy nos dois bancos (`db.t4g.medium` = 2 vCPU cada)	4 × 0,015 × 730	43,80	87,60 (4 × 0,030 × 730)
RDS Proxy só no consolidado	2 × 0,015 × 730	21,90	43,80

São Paulo do Proxy foi confirmado na Price List: **USD 0,030 por vCPU-hora** para PostgreSQL. O filtro da API em `us-east-1` não devolveu a linha nesta consulta. A conta usa **USD 0,015**, metade do SKU confirmado e o valor de lista histórico da página do RDS Proxy. Conferir na calculadora antes de ligar `rds_proxy_enabled`.

VPC Origin não tem linha própria conhecida nesta consulta. A cobrança usada é a do CloudFront mais a do ALB. Se a AWS passar a tarifar a origem VPC, a linha entra aqui antes do parâmetro ir a prod.

8. Cenário EKS equivalente (produção, us-east-1)

Troca só a computação. O resto da tabela de produção permanece (RDS, NAT, endpoint, borda, cache).

Linha	USD/mês
Produção ECS, base	778,35
Menos Fargate	–86,51
Control plane EKS, 0,10 × 730	73,00
3 × `m7g.large` on-demand, 3 × 0,0816 × 730	178,70
EBS gp3, 3 × 30 GB × 0,08	7,20
EKS com a mesma observabilidade gerenciada	**950,74**

O EBS gp3 a USD 0,08/GB-mês é preço de lista público de `us-east-1`, não reconsultado na API nesta passagem.

Três nós `m7g.large` (2 vCPU, 8 GiB) cabem as quatro tasks de hoje mais o sidecar e os DaemonSets, um nó por AZ. É o mínimo honesto de N–1. Karpenter não reduz essa conta: com três nós estáveis não há o que consolidar. O critério e a stack estão no documento 13.

Stack open source no lugar dos USD 45 de CloudWatch/X-Ray: mais 2 × `m7g.large` (119,14) + 200 GB gp3 (16,00) + 50 GB S3 (1,15, a USD 0,023/GB de lista). Sobe **USD 136,29** e tira os 45. Total **USD 1.042,03/mês** de infra, antes de gente. O control plane em São Paulo também é USD 0,10/hora; o que encarece o EKS lá é o nó, não o control plane.

Três clusters (dev, hml, prod) seriam 3 × 73 = USD 219 só de control plane, mais nós. O cenário acima é um cluster de produção. Dev e hml continuam em ECS no desenho de custo, que é o que o ADR 0026 decide.

9. Observabilidade SaaS no volume deste projeto

Premissas de produção: 4 tasks (6 GiB de memória de task no total), cerca de 2.000 séries ativas, 20 GB de log/mês, 5 GB de trace/mês, amostragem de 10%. Preços de lista consultados em 2026-11-10, com a fonte em cada linha. O detalhe de 30 serviços está no documento 13.

Opção	USD/mês neste volume	Por quê nessa ordem
CloudWatch + X-Ray + ADOT (a linha já somada)	45	Já está na conta. Sem mais uma plataforma.
Grafana Cloud Pro	19	Taxa de plataforma. 2.000 séries cabem nas 10.000 inclusas; 20 GB de log e 5 GB de trace cabem nos 50 GB inclusos de cada um. Página grafana.com/pricing.
AMP + Amazon Managed Grafana	~34	Ingestão: 2.000 × 86.400 amostras/mês = 172,8 milhões. A USD 0,90 por 10 milhões no primeiro degrau (página pública do AMP, não revalidada na API) = 15,55. Dois editores a USD 9 = 18. Log continua no CloudWatch.

Opção	USD/mês neste volume	Por quê nessa ordem
Datadog, SKU serverless anual	~52	Workload USD 3 × 4 tasks + APM serverless USD 6 × 4, mais 20 GB × 0,10 e 5 milhões de eventos indexados a USD 1,70/milhão (15 dias). Tabela datadoghq.com/pricing/list.
Dynatrace full-stack	~48	USD 58 por 8 GiB/mês na página de preço. 6 GiB de task × (58/8) = 43,50, mais 20 GiB de log a USD 0,20 (rate card).

Nenhuma dessas linhas, sozinha, paga um engenheiro. A premissa de planejamento é custo carregado de **USD 8.000/mês** por engenheiro sênior (não é pesquisa salarial). Um quarto de pessoa (USD 2.000) é maior do que qualquer SaaS desta tabela e maior do que os USD 136 da stack self-hosted. Por isso a decisão de agora é ADOT + CloudWatch + X-Ray (ADR 0027). Grafana Cloud é a primeira troca se o time quiser PromQL sem operar Loki. Self-hosted espera o time de plataforma que também justificaria o EKS.

10. Otimizações, na ordem em que o dinheiro responde

1. Não criar dev “igual produção”. O dev desta conta já não tem endpoint. Ainda assim o NAT (USD 35) pesa mais do que as duas tasks em Spot (USD 14).
2. Desligar task de dev e hml fora do expediente (264 h em vez de 730 h, cerca de 36% das horas) reduz a linha de Fargate para esse fator: dev cai cerca de USD 9, hml cerca de USD 28. NAT, RDS e, no hml, os endpoints continuam. Parar o RDS de dev no fim de semana é o corte seguinte (USD 28 de compute+disco), com o limite da AWS de 7 dias parado.
3. Não ligar reader nem Proxy até o gatilho do documento 12. A réplica clássica de USD 53 nem pode subir: PostgreSQL com senha gerenciada pelo Secrets Manager não aceita read replica. No gatilho, a saída compatível é o Multi-AZ DB Cluster (três instâncias e reader endpoint), não essa réplica. O Proxy dos dois bancos, cerca de USD 44, continua desligado pelo mesmo motivo de conexão.
4. Questionar os nove endpoints no hml. Metade das AZs economizaria USD 66 e deixaria o ensaio de PrivateLink diferente da produção. Só vale se hml deixar de ser o ensaio da rede.
5. Savings Plan de computação de 1 ano, cerca de 20% em USD 87 de Fargate de prod, economiza perto de USD 17. Não é a alavanca. Reserved do RDS medium, historicamente na casa de 30% a 40% no on-demand, em USD 188 seria da ordem de USD 60 a 75 por mês. A célula de RI não foi fechada nesta consulta. Comprometer só depois de duas faturas estáveis: o medium pode voltar a small se o crédito de CPU sobrar.
6. Graviton já está na conta. Voltar para x86 piora a vCPU.
7. Amostragem de trace em 10% e proibição de DEBUG em produção. Log por comerciante como dimensão de métrica estoura CloudWatch (USD 0,30 por métrica customizada/mês no primeiro degrau de lista) e vaza tenant.
8. `PriceClass_All` permanece. A economia de trocar de classe é cerca de USD 6 e piora o handshake de quem está no Brasil.

11. Licenças

Custo de licença de runtime e de biblioteca de aplicação: **zero**.

Componente	Licença	Custo	Observação
Eclipse Temurin 25	GPLv2 com Classpath Exception	0	JDK da imagem. Oracle JDK comercial fica de fora.
Spring Boot 4.0.9, Spring Security, Micrometer	Apache 2.0	0	
Resilience4j 2.4.0	Apache 2.0	0	ADR 0023
AWS SDK v2, Advanced JDBC Wrapper, OpenTelemetry	Apache 2.0	0	
Flyway Community	Apache 2.0	0	Não usar o tier Teams.
PostgreSQL 17	PostgreSQL License	0	O RDS cobra infra, não licença.
Redis 7 (local) / Valkey (ElastiCache)	BSD / BSD	0	ElastiCache cobra o nó.
Keycloak	Apache 2.0	0	Só local.
LocalStack	imagem community	0	Não depender de recurso Pro.
Grafana `otel-lgtm`, k6	AGPL	0	Só dev e teste. Não entram na imagem do serviço.
Testcontainers, ArchUnit, JUnit, springdoc	Apache / EPL conforme o projeto	0	
Terraform CLI	BSL (HashiCorp)	0 para este uso	A CLI não entra no runtime. OpenTofu (MPL) é a saída se a BSL for impeditiva. ADR 0010.
Cognito, WAF, CloudFront, o resto da AWS	serviço	infra, não licença	Já está nas tabelas.

Stack open source de EKS (Karpenter, Argo, Kyverno, Istio, Prometheus, Loki, Tempo, Grafana) também é licença zero de runtime e custo de nó mais gente. Esse custo está na seção 8 e no documento 13, não numa linha de licença escondida.

Se alguém adicionar Oracle JDBC comercial, Debezium com plataforma paga, Datadog como agente obrigatório da imagem, ou LocalStack Pro como dependência do `make up`, a premissa desta seção deixa de valer e a estimativa ganha uma linha nova.

PARTE 09

Arquitetura de transição

Migração do legado com Strangler Fig, do estado atual ao alvo

FONTE: DOCUMENTOS-BRUTOS/09-ARQUITETURA-DE-TRANSICAO.MD

09 — Arquitetura de transição

Os dois serviços nascem no contorno certo. Não há monolito nosso para estrangular. O que existe no comerciante, quase sempre, é uma planilha ou um ERP on-premises com um banco relacional onde o "lançamento" é uma linha editável. A transição é o caminho desse legado até o Fluxo de Caixa, sem um fim de semana em que o caixa para e os números não fecham.

O padrão é **Strangler Fig**: o legado continua operando, o sistema novo assume fatias do fluxo, e o legado só sai quando a fatia nova já é a fonte da verdade.

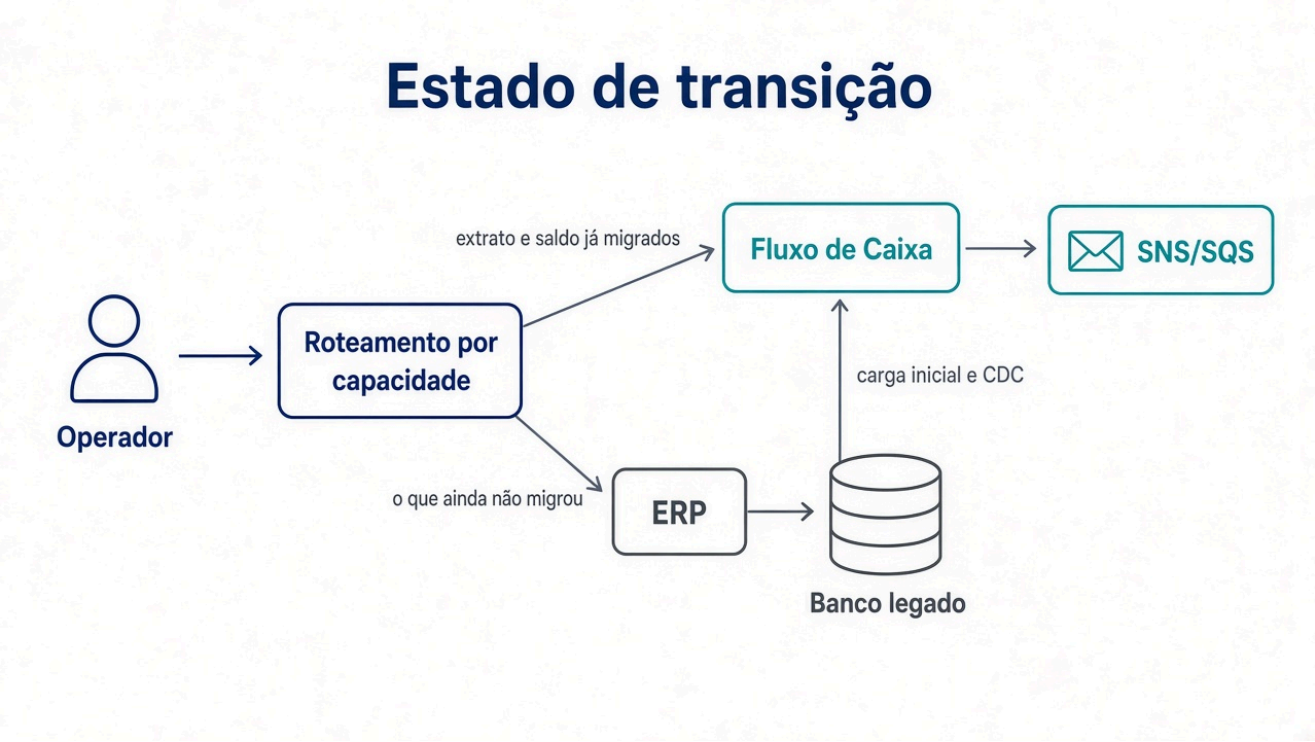
1. Estado atual (legado)



Características que mudam o desenho da migração:

- Lançamento editável e apagável. O sistema novo não aceita isso (ADR 0015).
- Saldo calculado na hora, muitas vezes com fórmula que arredonda em ponto flutuante.
- Um único banco. Relatório e escrita caem juntos — exatamente o oposto do RNF crítico.
- Sem `comerciante_id` de UUID. O "tenant" é a instalação inteira, ou uma coluna `loja`.

2. Estado de transição



O roteador não é um produto novo obrigatório. No início pode ser o próprio processo do operador: “venda do balcão já vai na API; o contas a pagar do ERP continua no ERP”. Um gateway entra só quando as duas UIs precisam parecer uma.

3. Estado alvo



O legado não aparece. Relatório antigo, se ainda for exigido por um contador, lê a API de período — não o banco novo por baixo.

4. Fases

Fase 0 — Inventário

Mapear, na planilha ou no ERP, o que é lançamento de caixa e o que é outra coisa (pedido, nota, boleto). Só o movimento de crédito e débito com data de competência entra. Definir a tradução para `CREDITO` / `DEBITO`, a moeda (se aparecer outra que não BRL, a migração para e o ADR 0016 é reaberto), e o UUID do comerciante que será o claim.

Critério de saída: uma tabela de de-para assinada por quem opera o caixa, com exemplos reais de estorno no legado (muitas vezes um delete, às vezes uma linha negativa).

Fase 1 — Carga inicial

Extrair o histórico que precisa ser consultado (sugestão: exercício corrente mais o anterior) e gravar via API de lançamentos, não via `INSERT` direto no Postgres do serviço. A API aplica dinheiro, tenant e outbox. O consolidado reconstrói o saldo pelo evento, e o `SUM` vira a prova.

Idempotency-Key = chave natural do legado (`sistema`, `tabela`, `id`). Reexecutar a carga não duplica.

Linha negativa ou deletada no legado vira par lançamento + estorno, não um valor negativo — o domínio novo não tem valor negativo.

Critério de saída: para uma amostra de dias, o saldo do consolidado empata com o relatório do legado no centavo. Diferença vai para uma lista de exceção, não para “ajuste manual no banco”.

Fase 2 — Coexistência com CDC

Enquanto o operador ainda lança nos dois mundos, o legado continua mudando. AWS DMS (ou CDC equivalente) lê o banco on-premises e empurra mudança para um adaptador. O adaptador é ACL: traduz linha editável em comando `RegistrarLancamento` ou `EstornarLancamento`. Não escreve na tabela do serviço.

Update no legado vira estorno do fato anterior mais lançamento novo, porque o sistema novo não tem update. Delete vira estorno. A chave de idempotência estável é o que impede o CDC de reaplicar a mesma mudança.

O sentido inverso (novo → legado) só existe se um relatório antigo ainda não migrado precisar ver o movimento novo. Aí o evento `LancamentoRegistrado` ganha um consumidor de sincronização, fila própria, sem reutilizar a fila do consolidado. Se nenhum relatório antigo precisar, não se constrói a volta. Sincronizar nos dois sentidos sem dono é a forma mais rápida de duplicar saldo.

Fase 3 — Roteamento progressivo

Ordem sugerida, da capacidade mais isolada para a mais arriscada:

1. Consulta de saldo e de período no sistema novo, em paralelo ao relatório velho, sem desligar o velho. O operador compara.
2. Extrato do dia (GET de lançamentos) no sistema novo.

3. Escrita de um tipo de movimento (por exemplo venda do balcão) só na API nova. O CDC deixa de reimportar esse tipo, senão duplica.
4. Escrita dos demais tipos.
5. Estorno só na API nova.

Cada passo tem um interruptor (configuração ou feature flag) para voltar o roteamento. Não se apaga o legado no mesmo dia em que se desvia o tráfego.

Fase 4 — Desligamento

Quando, por um ciclo de fechamento inteiro, o saldo novo e o legado empatam e ninguém escreve mais no legado:

1. Parar o CDC.
2. Somente leitura no ERP por um período combinado (30 a 90 dias).
3. Arquivar o banco legado na mesma região da produção (`us-east-1`), com a mesma CMK e a mesma política de retenção. Não se cria uma cópia em `sa-east-1` para “trazer o arquivo de volta” sem uma decisão de residência.
4. Remover o adaptador e a fila de sincronização.

5. Riscos e mitigação

Risco	Mitigação
Dupla escrita (operador lança nos dois)	Uma capacidade, um sistema dono, por fase. Idempotency-Key da carga. Conciliação diária dos totais.
CDC reaplicado	Chave natural do legado na idempotência. Consumidor que não faz `INSERT` cru.
Arredondamento do legado não fecha centavo	Relatório de diferença por dia antes de desligar. Não “arredondar para bater”.
Update do legado sem par no modelo novo	Tradução explícita para estorno + novo lançamento, testada com casos reais da fase 0.
Saldo do novo atrasado e o operador compara no mesmo segundo	A comparação de convivência usa o lag conhecido (p95 < 5 s) e, na carga, espera a fila zerar.
Segredo do banco on-premises na conta AWS	DMS com segredo no Secrets Manager, rede privada (VPN ou Direct Connect), usuário de leitura do legado, não o dono do schema.
Escopo crescendo para nota fiscal e boleto	Fora de escopo. O strangler é do caixa, não da suíte inteira do ERP.

6. Rollback

Rollback de fase é devolver o roteamento, não restaurar banco por cima.

Fase em que o problema aparece	Rollback
Carga inicial	Apagar o ambiente novo ou os lançamentos da carga (ainda não há produção real) e corrigir o de-para. Se a carga já convive com movimento novo, não se apaga: estorna-se o lote, com chave de lote para achar as linhas.
CDC	Desligar a task de replicação. O legado segue como estava. O novo pode ter fatos a mais: estorno do lote afetado, não `DELETE`.
Roteamento de leitura	Apontar a consulta de volta ao ERP. Sem migração de dado.
Roteamento de escrita	Fechar a flag de escrita na API. Movimento que já entrou no novo permanece e segue para o saldo. O legado não reabre para o mesmo movimento sem o de-para, senão o dia fica duplo.
Desligamento	Só é seguro enquanto o legado está em somente leitura e o CDC pode ser religado. Depois do arquivamento, o rollback é restaurar backup do legado, com RTO medido em dia, não em hora. Por isso a fase 4 tem período de somente leitura.

Não há rollback que “desfaça o strangler” reescrevendo o Postgres novo a partir do ERP sem passar pela API. Esse atalho fura outbox, idempotência e trilha.

7. O que não se faz na transição

- Replicação bidirecional permanente como estado final.
- Manter o relatório do ERP lendo o RDS novo com usuário compartilhado.
- Aceitar valor negativo “só na migração” e limpar depois.
- Big bang na sexta-feira de fechamento de mês.

A arquitetura-alvo não muda por causa da transição. O que a transição adiciona — adaptador de ACL, fila extra, DMS — é temporário e tem data de desligamento. Sem essa data, o strangler vira dois sistemas para sempre.

PARTE 10

Evoluções futuras

Roadmap e o que ficou de fora, com o motivo

FONTE: DOCUMENTOS-BRUTOS/10-EVOLUCOES-FUTURAS.MD

10 — Evoluções futuras

Nada desta lista está no caminho do RNF crítico. Cada item diz qual dor o traz e por que não entrou agora. A ordem é a de valor para o comerciante, não a de moda.

1. Fechamento de caixa e período contábil

Dor. Hoje um lançamento retroativo é sempre aceito. O caixa "de ontem" muda depois que o operador já conferiu.

Quando. Assim que existir o rito de fechar o dia. Até lá, travar competência é regra sem dono.

Como. Um contexto novo, Fechamento, que publica "dia D fechado". Lançamentos consulta essa política antes de aceitar competência fechada. Não se coloca a trava dentro do agregado como constante. Reabrir o dia é comando explícito, auditado, não um update.

2. Multi-moeda

Dor. Nenhuma, enquanto a premissa P1 (só BRL) for verdade.

Como. `Dinheiro` ganha moeda de verdade, o saldo diário ganha chave de moeda, e o evento deixa de fixar `BRL`. Conversão é outro contexto (taxa, data da taxa, fonte). Misturar conversão dentro do lançamento esconde o câmbio. Exige ADR que substitui o 0016.

3. Conciliação bancária e Open Finance

Dor. O saldo do caixa interno não é o saldo do banco. O comerciante concilia no olho.

Como. Contexto de Conciliação, consumidor do evento e de um extrato OFX/Open Finance. Casa movimento com lançamento por valor, data e um identificador externo que o lançamento ainda não tem. Não se estende o consolidado para "virar banco". A fila do consolidado não transporta extrato: fila nova, contrato novo.

4. Relatório assíncrono e exportação

Dor. O período de 366 dias cabe no GET. O ano inteiro em CSV, com categoria, não cabe, e não deve segurar a API do pico.

Como. Comando de exportação, arquivo no S3, URL pré-assinada com validade curta, notificação quando pronto. A geração lê réplica, não o primário. Categoria no arquivo exige `categoria` no evento (campo aditivo, `eventVersion` 1 ainda serve se for opcional) ou uma consulta ao serviço de lançamentos. Preferência: campo aditivo no evento, para a exportação não criar chamada síncrona entre os serviços.

5. Papel de contador e vários comerciantes

Dor. Um contador não consegue ler dez lojas sem dez tokens.

Como. O claim deixa de ser um único UUID e passa a ser uma autorização explícita (lista ou um serviço de permissão). O filtro “igual ao claim” vira “pertence ao conjunto autorizado”. Sem isso, qualquer atalho de query string `?comercianteId=` reabre BOLA. É mudança de segurança, não de tela.

6. Multi-região active-passive

Dor. Indisponibilidade longa de `us-east-1`. O RTO atual não cobre a região.

Como. Piloto com um único escritor, Postgres com réplica lógica ou snapshot replicado, fila regional (SNS não atravessa região sozinho). Active-active de lançamento é conflito de saldo. O secundário no Brasil exige a volta de residência descrita no ADR 0021, com o delta de custo do documento 08. O ADR 0021 já recusou multi-região no dia 1.

7. Kafka / MSK

Dor. Nenhuma a 20 evt/s.

Gatilho. Replay histórico vira rotina, ou o throughput sustentado passa a milhares de eventos/s, ou vários times consomem o log com retenção de semanas e processamento próprio. Aí o ADR 0005 se revê. Até lá, segunda fila no mesmo SNS resolve o segundo consumidor por uma fração do preço do cluster.

8. Event sourcing do lançamento

Dor. Nenhuma enquanto o estado couber numa linha e a correção for estorno.

Como. O log passaria a ser a fonte e a tabela, uma projeção. Útil se o agregado ganhar ciclo de vida longo (parcela, agendamento, conciliação dentro do mesmo agregado). Adotar agora duplicaria o que a tabela append-only e a outbox já contam, com a complexidade de snapshot e de evolução de evento de domínio. O evento de integração continua existindo nos dois desenhos.

9. Service mesh ou VPC Lattice

Dor. Nenhuma com dois serviços sem chamada entre si.

Gatilho. Várias chamadas síncronas internas, necessidade de mTLS padronizado e de política de tráfego. Até lá, security group e IAM na fila são o controle, e um mesh seria o sistema mais complexo da conta.

10. BFF e API Gateway

Dor. Nasce quando existir tela que compõe extrato e saldo e esconde os dois paths.

Como. BFF na frente, serviços onde estão. O Gateway, recusado no ADR 0022, volta a fazer sentido com throttling por parceiro e plano de uso. O JWT continua validado no serviço. A distribuição CloudFront já é o lugar do front estático, com S3 e Origin Access Control, sem segundo domínio de borda.

11. FinOps

Dor. A estimativa de custos vai descolar da fatura no primeiro mês, sobretudo NAT, endpoint e log.

Como. Budget com alerta em 50% e 80% do previsto (USD 778 em prod, USD 359 em hml e USD 147 em dev, na conta do documento 08, ambientes 24×7). Tag obrigatória `system=fluxo-caixa` e `env`. Revisão mensal das três linhas que realmente pesam: endpoint, NAT e RDS. Rightsizing do RDS pelo credit balance, não pelo palpite. Infracost no pull request, como no documento 14. Isso é processo, e o Terraform já nasce com a tag.

12. Feature flags

Dor. A transição e o experimento de fechamento precisam de interruptor sem deploy.

Como. Configuração externa simples (AppConfig ou até parâmetro de ambiente no início). Flag não é lugar de esconder regra de dinheiro por cliente. Flag de "aceitar competência fechada" é de comportamento de produto e tem dono.

13. Contract testing (Pact ou equivalente)

Dor. O JSON v1 vai quebrar no silêncio quando alguém renomear campo.

Como. O produtor de lançamentos verifica o contrato contra o consumidor do consolidado no CI. O teste de contrato do evento vale mais que um Pact HTTP entre serviços que não se chamam. Entra assim que os dois repositórios Maven estiverem estáveis. Até lá, um teste de serialização do JSON de exemplo do blueprint já pega o grosso.

14. Réplica de leitura e particionamento

Dor. O `SUM` do acumulado ou o relatório longo começa a aparecer no p99.

Como. Primeiro índice e cache (já estão). Depois réplica de leitura só do consolidado. Shard por comerciante é a última etapa e exige roteamento que hoje não existe. A conta de capacidade mostra que isso está longe do requisito de 50 req/s.

15. O que não entra nem como evolução sem um fato novo

Reescrever em outra linguagem, trocar PostgreSQL por DynamoDB no mesmo volume, adotar EKS para dois services, e event sourcing "para ficar pronto". São mudanças de plataforma sem dor de negócio. O gatilho para reabri-las é um ADR novo com número medido — lag, custo ou limite de throughput — e não uma preferência.

PARTE 11

Estratégia de testes

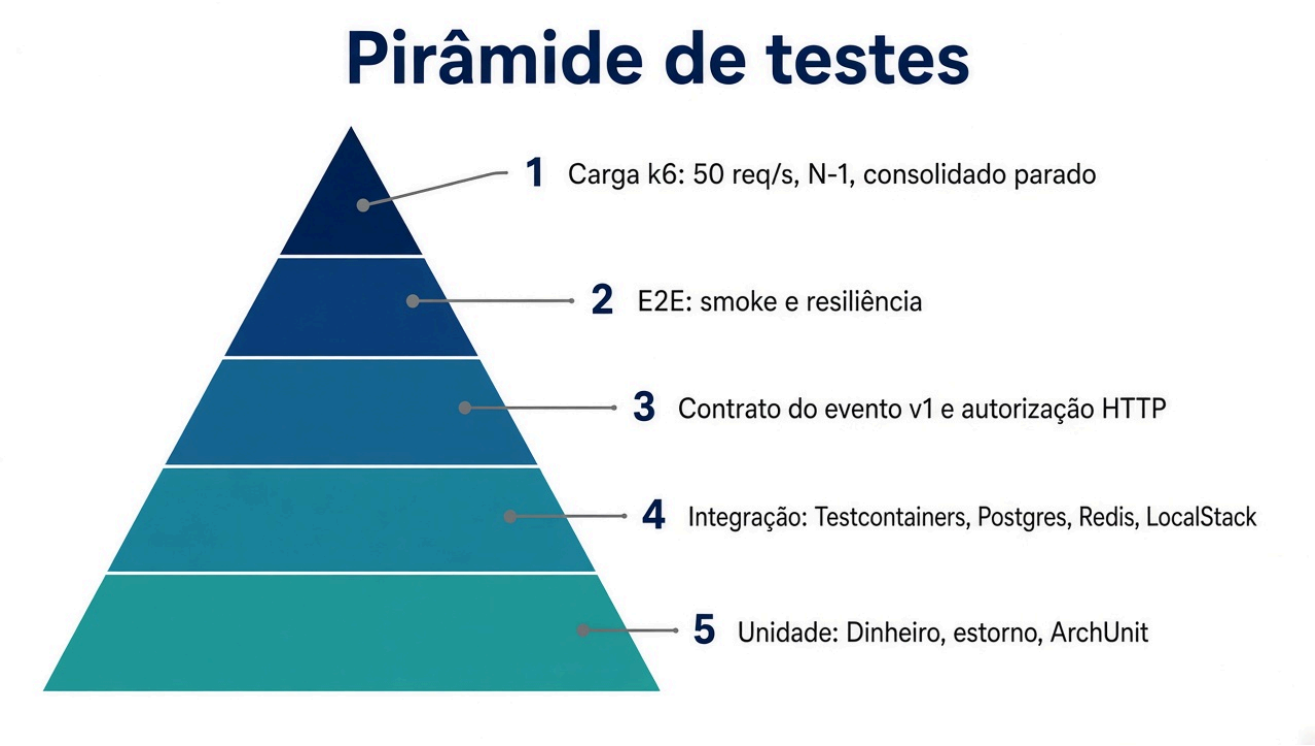
Pirâmide, smoke, resiliência, carga e evidências

FONTE: DOCUMENTOS-BRUTOS/11-TESTES.MD

11 — Testes

A estratégia cobre a regra de dinheiro, o isolamento entre os serviços e o pico de 50 req/s. Teste que só sobe a aplicação e recebe 200 não segura o RNF crítico.

1. Pirâmide



A base é larga de propósito. Estorno duplo e escala de dinheiro não precisam de Docker. A ponta é estreita e cara: k6 não roda a cada commit, roda no candidato a release ou sob demanda.

Camada	Onde	O que prova
Unidade	`domain` e casos de uso com portas falsas	Valor rejeitado, estorno único, estorno de estorno, descrição do estorno, soma comutativa do saldo
ArchUnit	teste no build de cada serviço	`domain` sem Spring e sem `adapters` ; `application` sem `adapters`
Integração	Testcontainers: PostgreSQL 17, Redis, LocalStack	Transação de outbox, relay publica, consumidor aplica uma vez, reentrega não duplica, Flyway sobe limpo
Contrato	Teste de serialização do JSON do blueprint e, na evolução, Pact	`valor` string com duas casas, campo desconhecido ignorado, versão desconhecida não aplicada
Autorização	MockMvc ou equivalente com tokens de dois comerciantes	401, 403, 404 cruzado, claim ignorada se vier no corpo

Camada	Onde	O que prova
E2E smoke	`tests/e2e/smoke.sh` via `make smoke`	Sobe a jornada: token, POST, GET do lançamento, GET do saldo depois do lag
Resiliência	`tests/e2e/resiliencia.sh` via `make resiliencia`	Consolidado parado, POST segue 201, fila acumula, volta e o saldo alcança
Carga	`tests/load` via `make load`	Cenários do documento 05
Segurança estática	SAST no CI (CodeQL ou equivalente) e Dependabot	Dependência e padrão perigoso
Segurança dinâmica	Evolução: ZAP baseline contra o local	Não bloqueia esta entrega; o teste de autorização cobre BOLA e BFLA

Não se mocka o Postgres para provar `SKIP LOCKED` nem índice único de estorno. Isso é teste de integração. Mock de relógio e de porta de publish, no unitário do caso de uso, é o lugar certo.

2. O que cada RNF exige de teste

RNF	Teste mínimo
Independência do consolidado	`make resiliencia`
50 req/s e perda < 1%	k6 de leitura nominal
Idempotência HTTP	Integração: dois POST com a mesma chave; um com a mesma chave e outro corpo (409)
Idempotência do consumidor	Integração: a mesma mensagem duas vezes, um saldo só
Dinheiro	Unidade: `10.125` rejeitado, `0` rejeitado, soma de centavos
Tenant	Dois tokens, o id do outro retorna 404
Outbox	Integração: derrubar o publish, ver a linha com `publicado_em` nulo, restaurar, ver a publicação

3. Como executar

Pré-requisitos e a subida do ambiente estão no README da raiz. Os alvos do Makefile:

Alvo	Função
`make infra-up`	Postgres, Redis, LocalStack, Keycloak, Grafana/OTel
`make up`	Sobe `lancamentos-service` e `consolidado-service`
`make down`	Derruba o ambiente local
`make test`	Testes de build dos dois serviços (unidade, ArchUnit, integração)
`make token`	Imprime um access token do usuário `comerciante1`

Alvo	Função
`make smoke`	Jornada feliz ponta a ponta (`tests/e2e/smoke.sh`)
`make resiliencia`	Consolidado indisponível e recuperação (`tests/e2e/resiliencia.sh`)
`make load`	k6, se instalado (`tests/load`)
`make tf-validate`	`terraform fmt -check` e `validate` em `infra/terraform`

Ordem local típica:

```
make infra-up
make up
make token
make smoke
make resiliencia
make test
```

`make load` exige k6 instalado e o ambiente no ar. Não aponta para produção.

Integração com Testcontainers precisa de Docker, o mesmo daemon do Compose. Se o Docker não estiver acessível, o teste de integração falha de verdade — não se pula em silêncio no CI.

4. Dados de teste

Usuário local: `comerciante1` / `comerciante1`, claim `comerciante_id=11111111-1111-1111-1111-111111111111`.

Não se usa cópia de produção. Carga de teste cria os próprios lançamentos e, no k6, chave de idempotência por iteração para a repetição do runner não parecer “perda” nem “duplicata”.

5. CI

Workflow em `.github/workflows`, em duas velocidades:

1. **Push e pull request, filtrados por caminho.** `.github/workflows/ci.yml` roda `./mvnw -B verify` em `services/lancamentos-service` e `services/consolidado-service` com Java 25 (Temurin) quando muda `services/**`. `.github/workflows/terraform.yml` roda `terraform fmt -recursive -check` e `terraform init -backend=false && terraform validate` em `infra/terraform/envs/dev` e `envs/prod` quando muda `infra/terraform/**`. O `make test` e o `make tf-validate` locais fazem o mesmo. Não sobe o Compose inteiro.
2. **No candidato a release, ou manual.** Compose, `make smoke`, `make resiliencia`. k6 fica manual ou noturno: dez minutos a 50 req/s num runner compartilhado compete com o resto da organização e não precisa ser a cada commit.

Falha de ArchUnit, de teste de estorno ou de contrato do evento quebra o build. Falha de smoke no release bloqueia a imagem. Alarme de produção não substitui isso: quando o alarme toca, o lançamento ruim já passou.

6. Evidência local medida em 2026-11-10

Ambiente: `make up` neste repositório, um container de cada serviço, Postgres 17, Redis 7, LocalStack, Keycloak e `grafana/otel-lgtm`. Não é a topologia de duas tasks da AWS e não substitui o cenário de 10 minutos da seção de carga do documento 05.

`make test`: `lancamentos-service` 68 testes, `consolidado-service` 68 testes, os dois com `BUILD SUCCESS` (unidade, ArchUnit e Testcontainers).

`make smoke`: crédito 150, débito 40 e estorno do débito. Saldo consolidado do dia `2026-11-10` convergiu para `150.00`.

`make resiliencia`: com o consolidado parado, os dois POST responderam 201. Depois de religar, o saldo do dia `2099-01-15` convergiu para 7 (10 de crédito e 3 de débito).

Carga, 1 minuto, k6 local (`DURATION=1m POST_RATE=2 k6 run tests/load/consolidado-pico.js`): 50 GET/s no consolidado e 2 POST/s no lançamentos. 3.121 iterações, 3.122 requisições HTTP (a sobra é o token do setup).

Métrica	Consolidado (GET)	Lançamentos (POST)	Total
Taxa de erro HTTP	0,00%	0,00%	0,00% (0 de 3.122)
p95	5,58 ms	20,37 ms	6,86 ms
p99	6,95 ms	21,94 ms	14,98 ms
Checks	100% `consolidado 200`	100% `lancamento aceito`	3.122/3.122

Os limiares do script passaram (`p95` do consolidado abaixo de 100 ms, `p95` do lançamentos abaixo de 500 ms, erro abaixo de 5%). O trace comum desta rodada está no documento 07: `traceId` `5c31753c7c9eebe969f69289ba632f8f`, com spans de `lancamentos` e `consolidado` no Tempo. `/actuator/prometheus` e o Swagger UI respondem no perfil `local`. No perfil `aws` o Swagger fica desligado.

7. Lacunas conscientes

- Blue/green com teste sintético no target group ainda não existe (ADR 0020).
- Game day de failover do RDS é procedimento de nuvem, não script do repositório, até a conta dev existir.
- DAST completo não está no caminho crítico.
- Pact ainda não está ligado; o teste de serialização do exemplo do blueprint é o substituto inicial, descrito nas evoluções.

PARTE 12

Desempenho e dados

Índices, paginação keyset, réplica de leitura, cache e evolução do armazenamento

FONTE: DOCUMENTOS-BRUTOS/12-DESEMPENHO-E-DADOS.MD

12 — Desempenho e dados

O volume de hoje não pede particionamento, réplica nem outro banco. Este documento fixa o que já vale na primeira versão, o que cada evolução custa, e o número que autoriza a ligá-la. Plano de execução (`EXPLAIN`) das consultas críticas está nos serviços, com o `EXPLAIN (ANALYZE, BUFFERS)` que rodou no banco:

- services/lancamentos-service/docs/performance.md
- services/consolidado-service/docs/performance.md

A decisão abaixo não substitui o plano. O plano é a evidência de que o índice e a janela já bastam neste volume.

Premissas de volume, herdadas do contexto: pico de leitura 50 req/s, pico de escrita 20 lançamentos/s, hit ratio de cache ≥ 80%. Uma escrita média de 1/s (bem abaixo do pico) produz cerca de 31 milhões de linhas por ano. Um comerciante ativo gera, no máximo, uma linha de saldo por dia: 1.800 linhas em cinco anos. Quase toda otimização de leitura de saldo é prematura nesse tamanho. A lista de lançamentos e a outbox são as que podem doer antes.

1. Índices que já entram

Lançamentos. A listagem é `WHERE comerciante_id = ? AND data_lancamento = ? ORDER BY criado_em DESC LIMIT ?` . O índice composto `(comerciante_id, data_lancamento, criado_em DESC)` cobre o filtro e a ordem. Sem a terceira coluna, o plano filtra por comerciante e data e ainda ordena. Com ela, o `LIMIT` para no topo do índice.

O detalhe por id usa a chave primária e **também** `comerciante_id` no predicado. Índice só em `id` sem o predicado de tenant é o bug de BOLA, não uma otimização. Os dois ficam no `WHERE` .

Outbox. Índice parcial `WHERE publicado_em IS NULL` em `(proxima_tentativa_em, id)` . O relay lê dezenas de linhas pendentes, não a história já publicada. Índice na tabela inteira gasta escrita em linha que o `SELECT` do relay nunca mais toca. A coluna `publicado_em` nula é exatamente a fila interna.

Consolidado. A chave primária `(comerciante_id, data)` atende o GET do dia (ponto) e o relatório de período (range em `data` com `comerciante_id` fixo). Não há segundo índice “para o relatório”. `eventos_processados(event_id)` é chave primária: a idempotência é um lookup, não uma varredura.

Índice a mais sem consulta na frente é custo de escrita no caminho do POST. Cada índice novo no lançamentos precisa apontar a query do `EXPLAIN` que ele muda.

2. Paginação keyset

A API de lista é `page` e `size` (offset). Offset 0 com `size` 50, apoiado no índice composto, lê 50 linhas. Offset grande (`page` 10.000, `size` 50) faz o banco contar e descartar 500.000 linhas para devolver 50. Isso não é o uso do caixa diário. É o uso de um extrato que alguém pagina até o fim, ou de um cliente que usa `page` como cursor errado.

Gatilho. p95 da lista acima de 100 ms, ou `EXPLAIN` mostrando linhas removidas por filtro na casa de dezenas de milhares. Abaixo disso, offset fica.

Como, quando o gatilho chegar. Keyset: `WHERE comerciante_id = ? AND data_lancamento = ? AND (criado_em, id) < (:cursorCriadoEm, :cursorId) ORDER BY criado_em DESC, id DESC LIMIT ?`. O cliente manda o cursor da última linha, não um número de página. O índice composto já serve, com `id` no fim para desempate. A resposta ganha `proximaCursor`. A página numerada pode coexistir em `/api/v1` até os clientes migrarem; keyset novo não quebra o contrato velho se for parâmetro opcional.

3. Particionamento por mês

Partição declarativa de `lancamentos` por mês de `data_lancamento` (ou de `criado_em`; os dois só coincidem se a política disser qual é a chave de retenção). O ganho real não é o `SELECT` do dia, que o índice composto já resolve. O ganho é detach de mês antigo e vacuum que não varre a história inteira.

Gatilho, todos ao mesmo tempo não são necessários; basta um.

- A tabela passa de **100 milhões** de linhas (cerca de três anos a 1 escrita/s de média, ou bem antes se o pico de 20/s for sustentado por horas todos os dias).
- Autovacuum não dá conta: idade de transação não congelada subindo, ou o POST espera lock de vacuum no trace.
- A retenção deixar de ser “guardar tudo” e passar a ser “destacar o mês e arquivar no lake”.

Antes disso, partição custa FK, índice único global (a unicidade de `id` precisa incluir a chave de partição, ou o id deixa de ser único sozinho) e Flyway mais difícil, para um `SELECT` que já é indexado. Não se particiona no dia 1.

A outbox **não** se particiona. Ela se limpa.

4. Retenção da outbox

Linha publicada há mais de **7 dias** é apagada por job. Sete dias cobrem uma investigação (“esse evento saiu?”) sem deixar a tabela crescer no ritmo da escrita. A linha não publicada não entra no delete. O índice parcial continua pequeno se o relay estiver saudável. Se não estiver, o alarme de idade (documento 07) dispara aos 30 s, muito antes da retenção importar.

Gatilho para encurtar para 2 dias: a tabela de publicadas, mesmo com o índice parcial, passa a dominar o backup ou o vacuum. **Gatilho para alongar:** um incidente real em que o payload de 7 dias atrás fez falta e o evento no SNS já tinha sido consumido. O fato contábil continua na tabela `lancamento`; a outbox não é a trilha legal.

5. Saldo acumulado

O saldo do dia é uma linha. O saldo acumulado até uma data é a soma dos `saldo` daquele comerciante com `data <= :data`. Para um comerciante, cinco anos são cerca de 1.800 somas de `NUMERIC`. Isso é barato. O relatório de período, se for implementado como 366 consultas correlacionadas, deixa de ser barato por N+1, não pelo volume.

Agora. O período calcula o acumulado em **uma** sentença, com função de janela:

```
SUM(saldo) OVER (PARTITION BY comerciante_id ORDER BY data)
```

no intervalo pedido, mais a soma anterior ao `inicio` buscada uma vez. O `EXPLAIN` dessa sentença é o que o `performance.md` do consolidado deve mostrar. Trezentas e sessenta e seis idas ao banco são defeito, não "simplesidade".

Por que não gravar `saldo_acumulado` na linha, já. Um estorno retroativo muda o saldo de um dia antigo. Todo acumulado posterior daquele comerciante muda junto. Atualizar N linhas futuras na mesma transação do evento acopla a latência do consumidor ao tamanho da história e cria contenção na linha de "hoje" se dois eventos do mesmo comerciante chegarem juntos. A soma é comutativa no dia; o acumulado armazenado **não** é comutativo em relação a um estorno no passado.

Snapshots mensais, quando o gatilho chegar. Guardar, por comerciante e mês fechado, a soma daquele mês. O acumulado até uma data vira "soma dos snapshots dos meses cheios anteriores + soma dos dias do mês corrente". Um estorno em mês fechado recalcula **aquele** snapshot, não os dias um a um. O mês corrente continua sendo a janela.

Gatilho do snapshot. Um destes:

- p99 do relatório de período, com cache frio, acima de **300 ms** por uma semana, com o plano mostrando a janela sobre dezenas de milhares de linhas de um único comerciante (muitos pontos por dia, se o modelo deixar de ser "um saldo por dia");
- ou a história por comerciante acima de **50 mil** linhas de saldo.

Enquanto o modelo for um saldo por dia, 50 mil linhas são mais de um século. O gatilho prático, portanto, é o p99, não a contagem. Se o p99 estourar com 1.800 linhas, o problema é plano ou pool, não falta de snapshot. O `EXPLAIN` distingue os dois.

Materializar o acumulado em coluna, atualizada de forma incremental, só depois do snapshot provar que ainda não basta. Não é o primeiro degrau.

6. Réplica de leitura

Variável `DB_READER_URL`, sempre `jdbc:postgresql://`. Presente: transação `@Transactional(readOnly = true)` vai para o reader, via routing data source e `LazyConnectionDataSourceProxy`. Ausente: tudo vai ao primário, e o código de leitura não muda. O consumidor que faz `UPSERT` **não** é `readOnly`. Mandar a projeção para o reader quebraria o saldo.

O default é zero reader. A AWS não cria **read replica** de RDS for PostgreSQL quando a instância usa senha gerenciada pelo Secrets Manager (limitação). O módulo `rds-postgres` recusa `read_replica_count > 0` com `manage_master_user_password`. O caminho de código fica pronto; o recurso clássico de réplica não sobe nessa combinação.

Três saídas compatíveis, quando o gatilho abaixo disparar:

1. **RDS Multi-AZ DB Cluster** (recomendação). Um escritor e dois standbys legíveis, com reader endpoint. A senha continua gerenciada pelo Secrets Manager: o reader do cluster não é a read replica que a limitação

proíbe. O wrapper usa `failover,awsSecretsManager` no endpoint do cluster. Custa três instâncias, não uma réplica extra de USD 53.

2. **Aurora PostgreSQL** com readers e autoscaling, se a quantidade de readers tiver de variar no dia e o I/O justificar. Mantém senha gerenciada. É o passo da seção 7, não o primeiro.
3. **Senha fora do Secrets Manager**, com rotação por Lambda, e aí a read replica clássica volta a ser aceita pela AWS. Perde a rotação nativa do RDS. Só se as duas opções acima não couberem.

O lançamentos só ganha reader se a lista paginada, e não o POST, for o que satura o primário.

Com RDS Proxy ligado, `DB_URL` aponta para o hostname `*.proxy-*`. O serviço tira o plugin `failover` nesse host: o plugin descobriria a instância e furaria o proxy. O proxy já faz o failover. O wrapper continua só com `awsSecretsManager`.

Gatilho, no consolidado, com evidência de uma semana.

- Credit balance da instância T em queda por causa de **leitura**, com o cache hit ainda acima de 80% (se o hit caiu, o primeiro conserto é o cache, não a réplica); ou
- p99 do GET com cache miss acima de 300 ms e o trace mostrando o tempo no Postgres, não no JWT nem no Redis; ou
- conexões do primário acima de 70% de `max_connections` sustentadas por leituras.

Lag de réplica acima de **5 s** gera alarme e o roteamento de leitura volta ao primário (tirar `DB_READER_URL` ou um disjuntor no routing). Saldo atrasado 5 s na réplica, somado ao atraso da fila, mente o caixa. Prefere-se o primário lento ao número velho sem aviso.

Uma réplica clássica `db.t4g.medium` com 50 GB sairia **USD 53,20/mês** em `us-east-1` (custos), e é exatamente o recurso que a senha gerenciada impede. Não se liga "porque CQRS pede réplica". O read model já é outro banco. O número de USD 53 fica na tabela como o preço que **não** se paga neste desenho.

RDS Proxy, no mesmo tema de conexão e não de leitura: default desligado. O Advanced JDBC Wrapper já refaz a credencial. Na instância, o plugin `failover` cobre o Multi-AZ. No proxy, esse plugin fica desligado. O Proxy entra quando as conexões de aplicação se aproximam de 70% de `max_connections` (dez tasks do consolidado × pool 20 = 200; um `db.t4g.medium` fica na casa de 450 conexões pela fórmula de memória, então 200 ainda é folga) **ou** quando o tempo de reabrir o pool no failover doer no SLO. Custo de Proxy nos dois bancos: cerca de **USD 44/mês** em `us-east-1`, com a ressalva de preço do documento 08. Ligar Proxy só por rotação de senha é pagar duas vezes pelo que o wrapper já faz.

7. Aurora PostgreSQL

Aurora com storage I/O-Optimized e reader autoscaling é a evolução quando o RDS único deixar de ser o formato certo, não um upgrade estético.

I/O-Optimized vale quando a cobrança de I/O passa de cerca de 25% do compute (a regra prática que a própria AWS usa para esse comparação). No gp3 de hoje, 3.000 IOPS e 125 MB/s já vêm no disco base. O volume deste caixa não chega perto. Enquanto o `EXPLAIN` e o Performance Insights mostrarem IOPS dentro do incluso, Aurora I/O-Optimized é um piso de preço mais alto pelo mesmo trabalho.

Reader autoscaling vale quando o gatilho da réplica da seção 6 dispara **e** a quantidade de readers precisa variar ao longo do dia (fechamento com muita leitura, madrugada com nenhuma). Um reader fixo é mais simples e já está na seção 6. Autoscaling de reader é o passo seguinte, com alarme de lag igual.

Aurora Serverless v2 não escala a zero. O mínimo de 0,5 ACU o mês inteiro, neste volume, empata ou perde de `db.t4g.medium` Multi-AZ e troca o modelo operacional. Fica de fora do default. O documento de custos registra a mesma conclusão.

8. Cache em camadas

Valkey (ElastiCache) é o cache compartilhado entre as tasks do consolidado. Cache-aside do saldo do dia, invalidação por geração do comerciante, TTL de 30 s. Perda do cache degrada para o Postgres. O circuito do Resilience4j torna essa degradação explícita.

Caffeine, no processo, na frente do Redis, guardaria o saldo quente daquela task e o JWKS. A 50 req/s com duas tasks, o Redis vê dezenas de operações por segundo. Isso não aparece no CPU de um `cache.t4g.small`.

Gatilho do Caffeine para saldo. O span do Redis no GET passa de **5 ms** no p95, ou o cache passa de **10 mil** operações por segundo. Abaixo disso, um cache local só cria invalidação duplicada (a geração no Redis já existe para isso) e saldo divergente entre tasks por até o TTL local.

JWKS no Caffeine (ou no cache do Spring) já é outra história: a chave de assinatura muda raramente, e cada request que busca o JWKS no Cognito paga rede. Esse cache é local, com TTL de minutos, e não espera o gatilho acima. Falha de refresh não pode derrubar o POST enquanto a chave em memória ainda for a que assinou o token.

9. CQRS já é o banco de leitura

A projeção em `saldos_diarios` **é** o read model. O write model é o lançamento imutável, noutro Postgres, noutro serviço. Não há um terceiro banco "de leitura" hoje, e não falta: a consulta do caixa é a chave `(comerciante, data)` e uma soma.

Um read model novo só se justifica quando a pergunta deixar de caber nessa chave. Os três candidatos abaixo são alternativas para esse dia, não etapas obrigatórias.

DynamoDB no lugar da projeção

Cabe quando o acesso for ponto (`comercianteId` + `data`) a uma taxa em que as conexões do Postgres, e não a CPU da aplicação, forem o teto — na prática, acima de **1.000 req/s** sustentados de GET, com o cache já na frente. On-demand cobra por request e tira o patch do banco. A um preço de lista da ordem de USD 1,25 por milhão de writes e USD 0,25 por milhão de reads eventualmente consistentes (página pública do DynamoDB em `us-east-1`, ordem de grandeza para a conversa), 50 req/s o mês inteiro são cerca de 130 milhões de reads, dezenas de dólares, **em cima** do RDS que a escrita da projeção ainda exige. Não substitui o RDS até o consumidor também sair do SQL.

O relatório de período e o estorno retroativo são o ponto fraco. DynamoDB não faz a janela `SUM() OVER`. O acumulado vira item materializado, e o estorno antigo reescreve muitos itens. É o problema da seção 5, pior, porque não há transação multi-item barata do tamanho de um ano. DynamoDB ganha o GET do dia em esca-

la alta. Perde o relatório e o estorno. Enquanto os dois existirem no produto, o Postgres da projeção continua o lugar certo.

OpenSearch para relatório analítico

Cabe quando o produto pedir texto livre na descrição, faceta por categoria, ou busca que não é "saldo entre duas datas". Um domínio pequeno ligado o mês inteiro é outra fatura fixa (a menor instância de busca fica na casa de dezenas de dólares por mês, acima do CloudFront desta conta). **Gatilho:** um requisito de busca que, no Postgres, seria `LIKE` ou varredura, com p95 acima do SLO, e que não se resolve com o índice composto. Até lá, OpenSearch é um sistema a operar sem uma pergunta.

Data lake (S3 + Athena) para BI

Cabe quando financeiro ou um analista quiser cruzar meses sem encostar no banco da API. Exportação diária da projeção (e, se a governança permitir, do lançamento sem a descrição, ou com a descrição só no bucket de acesso restrito) em Parquet. Athena cobra da ordem de USD 5 por TB varrido. No volume de um ano deste caixa, a varredura é centavos. O lake não serve o GET da API: latência de segundo e sem o SLO de 100 ms. Serve o relatório que ninguém deve rodar no primário.

Gatilho: uma pergunta recorrente que hoje seria um `SELECT` ad hoc no RDS de produção. A primeira ocorrência já justifica o export. Não justifica trocar a API.

10. Pool, timeout e diagnóstico

Parâmetro	Lançamentos	Consolidado	Motivo
Hikari `maximumPoolSize`	10 por task	20 por task	Teto de conexão. Dez tasks no máximo do scaling dão 100 e 200 conexões, cada uma no seu banco.
`minimumIdle`	2	2	Conexão ociosa ocupa `max_connections`. Dois cobrem o silêncio sem abrir dez na primeira request.
`maxLifetime`	15 min	15 min	Menor que o intervalo de rotação (30 dias em prod) e curto o bastante para soltar conexão velha depois de um failover. O wrapper busca a senha nova; o pool não fica com socket de horas.
`connectionTimeout`	2 s	2 s	Falhar rápido quando o banco não aceita mais conexão, em vez de segurar a thread virtual à espera.
`statement_timeout` do usuário da API	2 s	2 s	Uma sentença ruim não segura conexão por dezenas de segundos.
`statement_timeout` do usuário do consumidor	—	5 s	O `UPSERT` é curto; 5 s separam "lento" de "travado" sem matar um plano legítimo no catch-up.

Usuários de banco separados (API, consumidor, migração Flyway) existem para o timeout e o grant não serem globais. Flyway não usa o usuário da API.

`pg_stat_statements` ligado, para achar a sentença que o `EXPLAIN` pontual não mostra de novo. Performance Insights com retenção de 7 dias, que fica na franquia, para ver espera (IO, lock, CPU) no mesmo intervalo do alarme. Os dois são o laço: Insights diz "está esperando lock"; `pg_stat_statements` diz qual query;

o `performance.md` guarda o plano.

O que não se faz: log de SQL em INFO em produção, e métrica de latência com `comerciante_id` como label. Os dois vazam ou explodem cardinalidade, e nenhum substitui o `EXPLAIN`.

PARTE 13

ECS e EKS

Quando o EKS passa a valer, stack open source, Karpenter e custo de observabilidade

FONTE: DOCUMENTOS-BRUTOS/13-ECS-VS-EKS.MD

13 — ECS agora, EKS quando couber

Decisão: ECS Fargate para os dois serviços. EKS é evolução com gatilho, não uma preferência adiada por falta de opinião. O ADR 0026 registra a escolha. Este documento é a comparação que o ADR não cabe, a stack que só existe se o EKS existir, e o preço de observar o sistema de quatro jeitos.

Números de infra repetem o documento 08. Se divergirem, vale o 08 e este texto se corrige.

1. O critério

ECS permanece enquanto forem dois serviços e um time. EKS passa a valer quando a maioria for verdadeira:

- mais de **8 a 10** serviços, ou mais de um time com calendário de deploy próprio;
- uma plataforma interna que alguém aceite operar (template, política, GitOps);
- bin packing ou escala pela fila que o autoscaling do service ECS não expresse sem contorção.

Produção deste projeto em ECS: **USD 778/mês**. O equivalente em EKS, com a mesma rede e os mesmos dados, três nós `m7g.large` e o control plane: **USD 950/mês**. A diferença de USD 172 compra um control plane (USD 73) e nós que ficam folgados para quatro tasks. Não compra capacidade que o Fargate não tenha.

2. Comparação

Eixo	ECS Fargate	EKS
Control plane	incluso	USD 0,10/hora = **USD 73/mês** por cluster, `us-east-1` e `sa-east-1` (Price List, vigência 2026-10-01)
Operação do nó	não há nó	AMI, disco, daemonset, CVE do nó, versão do Kubernetes, extended support se a versão atrasar
Curva	service e task definition	o acima, mais o ecossistema da seção 3
Portabilidade	imagem OCI	manifesto. A imagem continua valendo; o deploy não
Densidade	a task paga 0,5 ou 1 vCPU o mês inteiro	vários pods por nó. Com 4 tasks, o nó é que fica ocioso
Autoscaling da aplicação	target tracking de CPU, request do ALB e backlog da fila	HPA, KEDA na fila, Karpenter no nó
Autoscaling da capacidade	a AWS provisiona o Fargate	Cluster Autoscaler ou Karpenter, ou Managed Node Group com tamanho fixo
Mesh	sem uso: os serviços não se chamam	Istio ou Linkerd quando houver chamada síncrona interna
GitOps	push do Actions (ADR 0028)	ArgoCD em pull
Custo deste porte	USD 87 de Fargate dentro dos USD 778	USD 73 + USD 179 de nó + USD 7 de disco, no lugar dos USD 87

Eixo	ECS Fargate	EKS
Falha de zona	task na outra AZ, sem o nó para drenar	o nó da zona sai, o pod reagenda, o volume pode não ir junto

O que não entra na tabela como vantagem do EKS neste porte: service mesh, porque não há tráfego leste-oeste; Karpenter, porque três nós estáveis não se consolidam; Argo Rollouts, porque o circuit breaker do ECS já cobre o rollback do rolling.

3. Stack open source, se o EKS existir

Nada desta lista é instalado no ECS. A lista existe para o dia do gatilho não virar um conjunto de charts escolhidos pelo tutorial da semana. Cada linha diz a função e por que está aqui e não o vizinho.

Peça	Função	Por que esta
Karpenter	Sobe e consolida nó	Reage em segundos e empacota por NodePool. Ver seção 5: não entra com três nós fixos.
ArgoCD	GitOps, app-of-apps, ApplicationSet	Um Application por serviço gerado pelo ApplicationSet, e um app-of-apps para a própria plataforma. Pull, com drift. ADR 0028.
Argo Rollouts	Canary e blue/green com análise	O passo além do rolling do ECS, quando houver métrica para promover (erro, lag). Sem a análise, canary é só tráfego partido.
KEDA	Escala o consumidor pela profundidade da fila SQS	O autoscaling do ECS já olha a fila. No EKS, o HPA de CPU não vê mensagem. KEDA vê.
External Secrets Operator	Materializa Secrets Manager em Secret	A rotação continua no Secrets Manager (ADR 0024). O cluster não vira outro cofre.
cert-manager	Certificado dentro do cluster	O certificado público continua no ACM, na frente do CloudFront. cert-manager é para o que o ACM não cobre (webhook interno, mesh).
AWS Load Balancer Controller	ALB ou NLB a partir de Ingress	Sem ele, o Service do tipo LoadBalancer vira um balanceador por serviço. Com a borda no CloudFront, o alvo interno continua um ALB só.
External-DNS	Registro no Route 53 a partir do Ingress	Tira o registro da mão. Sem dono de DNS no time, é mais um reconciliador para vigiar. Entra com o cluster, não antes.
Kyverno ou OPA Gatekeeper	Política de admissão	Imagem só do ECR, sem `:latest` em prod, sem privileged, recurso com request e limit. Kyverno escreve a política em YAML, mais perto do manifesto. Gatekeeper escreve em Rego, mais expressivo e mais distante. Um dos dois, não os dois.
Istio ou Linkerd	mTLS e política de tráfego	Quando existir chamada síncrona entre serviços. Linkerd é o menor dos dois. Istio é o que se escolhe se a política de tráfego for o requisito, não só a identidade. Antes disso, security group e IAM na fila. Mesh em dois serviços que não se chamam é latência e um control plane a mais.

Peça	Função	Por que esta
OpenTelemetry Collector	Recebe OTLP e exporta	O sidecar ADOT de hoje vira Collector no nó (DaemonSet) ou um gateway. O SDK da aplicação não muda.
Prometheus e Mimir, ou AMP	Métrica	Prometheus cabe no volume de 2.000 séries. Mimir (ou AMP) quando a retenção e a quantidade de séries passarem do que um Prometheus único aguenta. Não se opera os dois.
Grafana	Painel	O mesmo desenho do laboratório.
Loki	Log	Em S3, não em disco do nó, para o nó poder morrer.
Tempo	Trace	O equivalente self-hosted do X-Ray e do Tempo do `otel-lgtm`.
Alertmanager	Roteamento de alarme	O SNS continua sendo o destino que acorda gente. Alertmanager decide rota e silêncio; não substitui o tópico.
Velero	Backup de manifesto e, se houver, volume	O dado de negócio está no RDS, com PITR. Velero protege o cluster (manifesto, CRD), não o saldo. Sem ele, reconstruir o cluster é reinstalar a stack de memória.
Trivy Operator	CVE de imagem dentro do cluster	O CI já escaneia a imagem. O operator pega o que foi implantado fora do CI e o que envelheceu depois do scan.
Falco	Comportamento em runtime	Valor real quando há nó compartilhado por times. Com um time e dois serviços, é ruído. Entra com o multi-time, não no primeiro dia do cluster.

4. Custos de observar

Fontes consultadas em **2026-11-10**:

- Grafana Cloud, grafana.com/pricing: plataforma Pro USD 19/mês; métrica USD 6,50 por mil séries ativas no primeiro degrau pago (10.001–100.000), com 10.000 inclusas; log e trace com escrita a partir de USD 0,40/GB, e 50 GB inclusos na plataforma.
- Datadog, datadoghq.com/pricing/list: Infrastructure Pro USD 15/host/mês no anual (USD 18 mês a mês); Container Monitoring USD 1/container/mês no anual; log ingerido USD 0,10/GB; log indexado, 15 dias, USD 1,70 por milhão no anual; métrica customizada USD 5 por 100/mês; Serverless Workload USD 3 e Serverless APM USD 6 por instância ativa/mês no anual.
- Dynatrace, dynatrace.com/pricing: full-stack USD 58 por 8 GiB/mês. Rate card: ingestão de log USD 0,20/GiB.
- AMP: USD 0,90 por 10 milhões de amostras no primeiro degrau, preço de lista público da página do Amazon Managed Service for Prometheus, não revalidado na Price List API nesta passagem. Amazon Managed Grafana: USD 9 por editor/mês, preço de lista público.
- CloudWatch Logs: USD 0,50/GB de ingestão em `us-east-1`, preço de lista público, também não revalidado na API nesta passagem.
- New Relic: a célula exata não foi recortada nesta consulta. A faixa pública recente de ingestão fica entre USD 0,30 e USD 0,55 por GB. A linha abaixo usa USD 0,40 e está marcada como a confirmar antes de contrato. Usuário full-platform, quando a conta cobra um, muda a conta mais do que o GB neste volume.

Gente. USD 8.000/mês carregados por engenheiro sênior, premissa de planejamento, não pesquisa salarial.

Este projeto (produção)

Premissas: 4 tasks, 6 GiB de memória de task, 2.000 séries, 20 GB de log/mês, 5 GB de trace/mês, amostragem de 10%.

Opção	USD/mês	Conta
CloudWatch + X-Ray (já na fatura)	45	estimativa do documento 08
Grafana Cloud Pro	19	plataforma; o uso cabe na franquia
AMP + 2 editores	~34	172,8 milhões de amostras × 0,90 / 10 milhões = 15,55; editores 18. Log segue no CloudWatch
Datadog serverless, anual	~52	4 × 3 + 4 × 6 + 20 × 0,10 + 5 milhões indexados × 1,70
Dynatrace full-stack	~48	6 GiB × (58/8) = 43,50, mais 20 × 0,20
Self-hosted no EKS	136 de nó	2 × `m7g.large` (119,14) + 200 GB gp3 (16) + 50 GB S3 (1,15), e tira os 45 do CloudWatch se substituir de verdade
New Relic, ordem de grandeza	< 20 de ingestão	25 GB × 0,40. O usuário full-platform, se cobrado, domina. A confirmar

Um quarto de engenheiro (USD 2.000) é maior do que todas as linhas. Self-hosted a USD 136 de nó custa USD 2.136 com essa fração de pessoa. Grafana Cloud a USD 19 custa USD 19 se ninguém a mais for contratado para operá-la. Por isso o destino de agora é CloudWatch. Grafana Cloud é a troca quando o PromQL fizer falta e o DPA do fornecedor estiver aceito. Datadog e Dynatrace entram se o pedido for APM com o produto deles, não para “ter trace”.

Trinta serviços

Premissas, de propósito redondas, para a comparação não depender de um serviço que não existe:

- 30 serviços, 60 pods, 12 nós `m7g.xlarge` (4 vCPU, 16 GiB) se o bin packing couber cerca de cinco pods por nó com folga.
- 45.000 séries ativas, scrape de 30 s.
- 30 GB de log por dia = 900 GB/mês. Trace: 50 GB/mês.
- Indexação de log no Datadog: 20 milhões de eventos/mês (erro e amostra), não o log inteiro.
- Métrica customizada faturável no Datadog: 5.000, depois da franquia por host. Emitir as 45.000 séries como custom metric seria outra ordem (centenas de dólares só nessa linha) e é exatamente o que não se faz.

Opção	USD/mês	Leitura
Grafana Cloud	~590	plataforma 19 + 35 mil séries × 6,50/mil = 227,50 + log acima de 50 GB × 0,40 = 340. Se processar e reter incidirem em cima da escrita, o teto do log sobe e o total vai para a casa de USD 700. A página lista os três medidores; a conta usa a escrita como piso.
Datadog, anual, sem APM de host	~610	12 × 15 de infra + 60 × 1 de container + 900 × 0,10 de ingestão + 20 × 1,70 de índice + 50 × 5 de custom metric. APM por host está na mesma tabela e não foi recortado nesta consulta: soma por cima. No Fargate o SKU serverless (USD 6 de APM) não se aplica a nó EC2.

Opção	USD/mês	Leitura
Dynatrace, host inteiro	~1.570	12 × 16 GiB = 192 GiB, 192/8 × 58 = 1.392, mais 900 × 0,20 = 180. Se a cobrança fosse só a memória do pod (60 GiB), cairia para cerca de 435 + 180. A página cobra o host monitorado; a conta usa o host.
New Relic, só ingestão a 0,40	~380	950 GB × 0,40, mais usuário se a conta exigir. A confirmar.
AMP, teto no primeiro degrau	~380	45.000 × 86.400 = 3,89 bilhões de amostras. No primeiro degrau, 388,8 × 0,90 ≈ 350, mais USD 27 de três editores. O degrau seguinte da página histórica é menor; o teto conservador é este. Log não está no AMP: 900 GB no CloudWatch a 0,50/GB = USD 450, e essa é a linha que dói.
Self-hosted	~410 de infra	3 × `m7g.xlarge` (3 × 0,1632 × 730 ≈ 357) + S3 na casa de USD 50 para o log do mês. Mais **0,5 engenheiro (USD 4.000)** se o time ainda não existir.

Com o time de plataforma já pago por causa do EKS, o custo marginal da stack self-hosted é o nó (USD 410), e ela ganha do Dynatrace e compete com o Grafana Cloud. Sem esse time, os USD 4.000 de gente fazem o SaaS ganhar em todas as linhas. A escolha é pelo custo total, não pelo preço do GB.

Situação	Escolha
Dois serviços, sem time de plataforma	CloudWatch e X-Ray
O mesmo time quer PromQL e aceita o DPA	Grafana Cloud
Trinta serviços, time de plataforma já existe por causa do EKS	self-hosted no cluster, log em S3
O contrato pede APM com análise de causa pronta e há orçamento de plataforma	Dynatrace ou Datadog, com a linha de APM de host reconfirmada na tabela antes da assinatura
O critério é ingestão barata e o time aceita o modelo de usuário da New Relic	New Relic, depois de fechar a célula que esta consulta deixou em aberto

5. Karpenter

Quando adota. Cluster com dezenas de pods, tamanho de nó que muda ao longo do dia, mistura de Spot e on-demand, e alguém para olhar disruption. Aí o Karpenter substitui o Cluster Autoscaler com Managed Node Group: provisiona o formato certo em vez de escalar um grupo homogêneo, e devolve nó ocioso.

NodePool deste produto, no dia em que existir.

- Um pool on-demand Graviton (`m7g` , `c7g`) com limite de CPU, para o mínimo de N−1 não depender de Spot.
- Um pool Spot Graviton, com tipos compatíveis com o pod (ARM64, o mesmo da imagem de hoje), para o consumidor e para dev.
- Consolidação ligada no pool Spot e conservadora no on-demand.
- Disruption budget: não esvaziar mais de um nó por vez, e não esvaziar durante a janela de fechamento. Sem o budget, a consolidação reinicia o consumidor no meio do catch-up.

- Pod de lançamentos e de consolidado com `topologySpreadConstraints` por zona, como o `spread` do ECS faz hoje.

Quando não adota. Poucos nós estáveis — o caso de três `m7g.large` o mês inteiro. Managed Node Group com tamanho fixo é mais simples e não fica consolidando à toa. Fargate profile também dispensa Karpenter, e dispensa junto o bin packing: volta o preço de Fargate, que é o que o ECS já cobra sem o control plane. Karpenter em cima de Fargate profile não tem nó para provisionar.

Spot no consumidor aceita interrupção porque a mensagem volta quando a visibility expira e o receiver é idempotente. Spot no único pod de lançamentos de um ambiente que se diz de produção não aceita: o SLO de 99,9% não combina com reclamação de capacidade. Por isso o pool on-demand existe, pequeno, e o Spot não é o único pool.

PARTE 14

CI/CD e qualidade

Pipeline, ferramentas de qualidade e segurança e GitOps

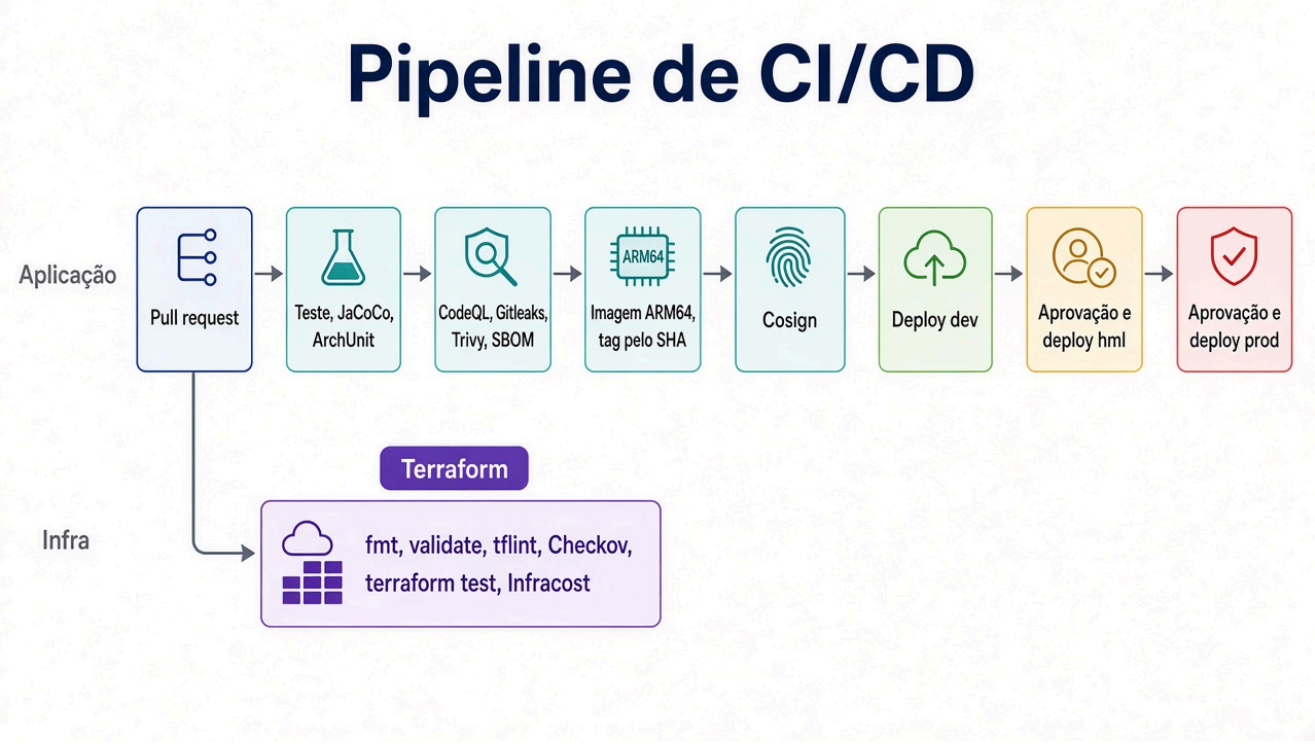
FONTE: DOCUMENTOS-BRUTOS/14-CICD-E-QUALIDADE.MD

14 — CI/CD e qualidade

Contrato do pipeline desta entrega: build uma vez, a mesma imagem promove dev → hml → prod, e o que entra em prod passou por teste, análise e uma aprovação humana. O scheduler de hoje é ECS (ADR 0020). GitOps entra com o EKS (ADR 0028).

Conferência com o YAML em 2026-11-10. O CI (`.github/workflows/ci.yml`) roda `./mvnw verify` com JaCoCo, Trivy na imagem ARM64 (`ignore-unfixed` , HIGH/CRITICAL com correção quebram), SBOM CycloneDX, CodeQL, Gitleaks e dependency-review. SonarCloud entra só quando existem `SONAR_TOKEN` e `vars.SONAR_ORGANIZATION` ; sem isso o job segue verde. O workflow de Terraform faz `fmt` , `validate` em dev, hml, prod e bootstrap, tflint, Checkov, Trivy config, `terraform test` e Infracost. O CD (`.github/workflows/cd.yml`) só começa se `vars.AWS_CD_ENABLED` for `true` . Região `us-east-1` . Push na `main` publica em dev; hml e prod saem por `workflow_dispatch` , com a mesma imagem copiada por digest e assinada com Cosign. Dependabot cobre Maven, Docker, Actions e Terraform (`.github/dependabot.yml`). A comparação de preço com a região antiga está no documento 08.

1. O pipeline



Build once: o digest publicado em dev é o digest de hml e o de prod. Reconstruir em cada ambiente muda o binário que foi testado. A promoção é troca de tag apontando para o mesmo digest, com aprovação do environment do GitHub em hml e em prod. Dev pode seguir o merge sem aprovação, se o environment existir sem reviewer.

A role de cada ambiente é assumida por OIDC. Não há access key no secret do repositório (ADR 0024). A role de aplicação faz push no ECR daquele ambiente e `UpdateService`. Não faz `DeleteDBInstance`. A role de Terraform é outra.

Região dos workflows: `us-east-1`.

2. O que o pipeline de aplicação faz

Passo	Para quê
`./mvnw verify`	Teste, ArchUnit, integração com Testcontainers. Os dois serviços, em matriz.
JaCoCo	Relatório no artefato do CI. O número não é vaidade: regressão de cobertura no pacote de domínio e no de aplicação é o sinal. Meta de percentual único para o repositório inteiro empurra teste para getter.
Sonar-Cloud	Quality gate no PR: bug, vulnerabilidade, duplicação, e o coverage que o JaCoCo entregou. A ferramenta hospedada evita um SonarQube para operar.
CodeQL	SAST no código e no workflow. Gratuito no repositório público; no privado, cabe na licença do GitHub que o projeto já paga ou não — confirmar antes de tratar como de graça.
Gitleaks	Segredo no diff. Roda antes do build longo.
depen-dency-review	Dependência nova no PR, com o alerta de vulnerabilidade conhecida.
Trivy fs e Trivy imagem	O filesystem pega o que a imagem ainda não tem (Dockerfile, workflow). A imagem ARM64 é o que vai para o ECR. `CRITICAL` e `HIGH` com correção disponível quebram o build. `ignore-unfixed` evita bloquear o que ninguém pode corrigir hoje, e o achado continua visível.
SBOM CycloneDX	Gerado no Maven e anexado ao artefato. Sem SBOM, o Trivy da semana que vem não sabe o que estava em prod.
Cosign	Assina o digest. O deploy recusa imagem sem assinatura verificável. A chave vem de OIDC (keyless, Fulcio) para não criar outro segredo de longa duração.

Actions de terceiro ficam pinadas por SHA. Tag flutuante (`@v4`) é o vetor de supply chain que o resto do pipeline existe para reduzir.

3. O que o pipeline de infra faz

Passo	Para quê
`terraform fmt -check`	Diff legível.
`validate` nos três ambientes, `init -backend=false`	Sintaxe e tipo, sem state.
tflint	Erro de provedor que o validate não vê.

Passo	Para quê
Checkov	Política. Soft-fail é aceitável no primeiro mês, com a lista do que falha virando issue. Soft-fail eterno é não ter a ferramenta.
`terraform test`	Módulo com contrato (nome de fila com prefixo do ambiente, RDS não público, ALB interno). Sem teste, o módulo só quebra no apply.
Infracost	Comentário no PR com o delta de USD/mês. Não bloqueia sozinho: bloqueia se o delta de prod passar de um teto combinado (proposta: USD 50/mês de aumento sem aprovação explícita). O preço de lista do Infracost não substitui o documento 08; ele pega o diff que o documento ainda não viu.

Apply de prod não roda no pull request. Roda no pipeline, no environment `prod`, com aprovação, a partir do merge.

4. O que ainda pode entrar, e por quê

Agrupado pelo motivo. “Porque a banca conhece o nome” não é motivo.

Qualidade

Ferramenta	Motivo para entrar	Quando não
SonarQube self-hosted	O quality gate do SonarCloud já cobre. Self-hosted só se o código não puder sair da rede.	Enquanto SonarCloud for aceitável no DPA. Operar SonarQube para dois serviços é plataforma sem plataforma.
PIT, mutation testing	O teste do domínio (dinheiro, estorno, idempotência) é o lugar em que cobertura mente: asserção fraca passa no JaCoCo. PIT no módulo de domínio, não no repositório inteiro, porque o tempo de mutação no adapter de AWS não devolve o mesmo sinal.	No PR de todo mundo, todo dia, se o tempo passar de poucos minutos. Aí vai para o agendamento noturno.
ArchUnit	Já está no `verify`. Fica. É o que impede o domínio de importar Spring.	—
Pact	O produtor e o consumidor não compilam juntos. O contrato do evento quebra em silêncio até o Compose. Padrões de integração.	Enquanto o fixture compartilhado for um só e o teste da ACL recusar versão desconhecida, o Pact pode esperar o segundo consumidor. No segundo consumidor, deixa de poder.
k6 no pipeline	Sobe o Compose, roda o cenário curto de escrita e o de leitura, falha se o erro passar de 1%. O cenário de 10 minutos não cabe no PR; cabe no agendamento. Documento 05.	Apontado para hml em todo PR, queima tempo e a fatura, e mistura regressão de código com ruído de ambiente.

Segurança

Ferramenta	Motivo	Quando não
OWASP ZAP baseline	DAST no dev ou no Compose, com token de teste. Pega header ausente e rota que não devia existir. Não substitui o teste de BOLA, que é de domínio.	Em prod, e sem um usuário de teste com dado fictício.

Ferramenta	Motivo	Quando não
Semgrep	Regra própria (proibir `comercianteId` vindo do body, proibir log da descrição) mais rápida de escrever do que uma query do CodeQL.	Duplicar o CodeQL sem uma regra que ele não tenha.
Snyk	SCA com o mesmo papel do dependency-review e do Trivy. Um deles, com dono. Dois SCA sem dono geram o mesmo CVE em dois painéis.	Se dependency-review + Trivy já quebram o build no CVE corrigível.
Dependabot ou Renovate	O patch sobe sozinho. Renovate agrupa melhor o BOM do Boot. Dependabot já é o caminho citado no documento 06. O que importa é a regra: Boot 4.1 é recusado enquanto o ADR 0023 mandar.	Bot sem essa regra vira o vetor que furar a matriz de versão.
Prowler ou Security Hub	Postura da conta (trail, bucket público, SG aberto), não do código. Security Hub agrega o que a conta já tem. Prowler é o relatório que se leva para a conversa.	No PR de aplicação. É agendamento da conta.
Cosign + admissão	A assinatura sem alguém que a confira é arquivo. No ECS, o deploy verifica antes do `UpdateService`. No EKS, Kyverno recusa a imagem sem o atestado.	Política de admissão no ECS: não há o hook. A verificação fica no pipeline.

Entrega

Ferramenta	ECS hoje	EKS depois
ArgoCD	não	sim, no mesmo passo do cluster. Repositório de manifests, pull, app-of-apps, drift.
Argo Rollouts	não; o circuit breaker do ECS faz o rollback do rolling	canary com análise de 5xx e de lag, quando o rolling ficar caro
Feature flag (OpenFeature com Unleash, ou AppConfig)	quando a transição do legado precisar desviar um tipo de movimento sem deploy (documento 09)	o mesmo. Flag não é motivo para EKS.
CodeDeploy blue/green no ECS	evolução do ADR 0020, se o rollback do rolling ficar lento e ainda não houver EKS	deixa de ser o caminho; o Rollout assume

Flag sem dono de expiração vira ramo eterno. A flag da transição nasce com data para morrer.

FinOps

Ferramenta	Motivo
Infracost	Delta no PR, já na seção 3.
Kubecost ou OpenCost	Só com nó. No Fargate a conta é a do CUR da AWS, e o Infracost mais a fatura bastam. OpenCost entra com o EKS para ver qual namespace gasta o nó. Kubecost é a distribuição com mais produto em cima; OpenCost é o projeto. Um dos dois.

DORA

Quatro números, tirados do próprio GitHub e do deploy, sem plataforma:

- frequência de deploy (quantos deploys de prod por semana);

- lead time (do merge ao `UpdateService` saudável);
- taxa de falha (deploy que o circuit breaker reverteu, ou hotfix em menos de um dia);
- tempo de restauração (do alarme ao rollout estável).

Com dois serviços, uma planilha mensal chega. Comprar ferramenta de DORA antes de ter doze deploys é medir o vazio. O número serve para a conversa de “estamos com medo de deployar”, não para um painel.

5. ECS e EKS, lado a lado

	ECS, agora	EKS, no gatilho
Artefato	imagem assinada no ECR	a mesma imagem, mais o manifesto no Git
Promo-ção	tag do digest, aprovação, `UpdateService`	commit da tag no repositório de manifests, ArgoCD sincroniza
Rollback	circuit breaker do service, ou deploy da tag anterior	Rollout aborta, ou revert do commit
Política	Checkov no Terraform, Trivy no CI	mais Kyverno no cluster
Segredo	task lê o ARN	External Secrets lê o mesmo ARN
Drift	Terraform plan agendado	ArgoCD, contínuo

O que não se faz na migração: manter o Actions dando `kubectl apply` e o ArgoCD ao mesmo tempo. Dois escritores. O ADR 0028 escolhe o pull.

PARTE 15

Blueprint técnico

Contrato compartilhado: nomes, portas, evento, recursos AWS e metas de SLO

FONTE: DOCUMENTOS-BRUTOS/BLUEPRINT-TECNICO.MD

Blueprint Técnico de Referência — Fluxo de Caixa

> Fonte única de verdade para nomes, contratos e convenções compartilhadas entre serviços, infraestrutura e documentação. Qualquer divergência deve ser resolvida a favor deste documento.

1. Visão geral

Solução para um comerciante controlar o fluxo de caixa diário (lançamentos de débito e crédito) e consultar o saldo diário consolidado.

- **Estilo**: microsserviços por *bounded context*, CQRS entre escrita (Lançamentos) e leitura (Consolidado), integração assíncrona orientada a eventos.
- **Requisito crítico (RNF)**: o serviço de Lançamentos **não pode ficar indisponível** se o Consolidado cair. Em pico, o Consolidado recebe **50 req/s** com no máximo **5% de perda**.
- **Como o RNF é atendido**: Lançamentos não chama o Consolidado de forma síncrona. Ele grava o lançamento e o evento na mesma transação (Transactional Outbox) e um *relay* publica no SNS. O Consolidado consome via SQS (com DLQ), é idempotente e serve leituras a partir de uma projeção materializada + cache Redis. Se o Consolidado cair, as mensagens ficam retidas no SQS (retenção de 14 dias) e são processadas quando ele voltar.

2. Stack

Item	Escolha
Linguagem	Java 25
Framework	Spring Boot **4.0.9** (última 4.0.x) + **Resilience4j 2.4.0** (`resilience4j-spring-boot4` , compilado contra Boot 4.0.0 / `resilience4j-spring6`). Boot 4.1.x fica bloqueado até o Resilience4j publicar suporte — ver seção 18
Build	Maven com wrapper (`mvnw`), um projeto Maven independente por serviço
Banco	PostgreSQL 17 (Amazon RDS em produção), um banco por serviço, migrações Flyway
Mensageria	Amazon SNS (tópico) + Amazon SQS (fila + DLQ). Localmente: LocalStack. Cliente: AWS SDK for Java v2 (sem Spring Cloud AWS)
Cache	Redis 7 / Valkey (Amazon ElastiCache em produção), apenas no Consolidado
Segurança	OAuth2/OIDC, JWT validado pelos serviços (Spring Security Resource Server). AWS: Amazon Cognito. Local: Keycloak
Observabilidade	Micrometer + OpenTelemetry (OTLP), logs JSON estruturados, Actuator. AWS: ADOT/CloudWatch/X-Ray. Local: `grafana/otel-lgtm`

Item	Escolha
Execução AWS	ECS Fargate atrás de **CloudFront (com AWS WAF) → VPC Origin → ALB interno**, região `us-east-1` **(N. Virginia)**
IaC	Terraform (provider AWS 6.x), modular, ambientes `dev`, `hml`, `prod`, state remoto em S3
CI/CD	GitHub Actions (build once, promote dev → hml → prod)

3. Layout do repositório

```
fluxo-caixa/
├── README.md
├── Makefile
├── documentos-brutos/            # documentação bruta (fontes Markdown)
│   ├── blueprint-tecnico.md      # este arquivo
│   ├── 04-adr/                   # Architecture Decision Records (MADR)
│   └── ...
├── services/
│   ├── lancamentos-service/      # projeto Maven independente
│   └── consolidado-service/      # projeto Maven independente
├── infra/
│   ├── terraform/                # módulos + ambientes (dev, prod)
│   └── local/                    # docker-compose, init LocalStack, realm Keycloak
├── tests/
│   ├── load/                     # scripts k6
│   └── e2e/                      # smoke tests ponta a ponta (scripts)
└── .github/workflows/
```

4. Bounded contexts

Contexto	Tipo	Serviço	Responsabilidade
Lançamentos (Gestão de Caixa)	Core	`lancamentos-service`	Registrar créditos e débitos, estornos, consultar lançamentos. Fonte da verdade.
Consolidação Diária	Core/Supporting	`consolidado-service`	Manter a projeção do saldo diário por comerciante e servir relatórios.
Identidade e Acesso	Generic	Cognito / Keycloak	Autenticação, emissão de tokens, escopos.

Relação: Lançamentos é *upstream* (Published Language: eventos JSON versionados); Consolidado é *downstream* conformista com uma camada anticorrupção no adaptador de mensageria.

5. Modelo de domínio

Lançamentos

- **Agregado** `Lancamento` (imutável após criado):
  - `id` (UUID v7 ou v4), `comercianteId` (UUID), `tipo` (`CREDITO` | `DEBITO`), `valor` (Value Object `Dinheiro`: `BigDecimal` escala 2, `> 0`, moeda `BRL`), `dataLancamento` (`LocalDate`, data de com-

petência), `descricao` (1..255), `categoria` (opcional, até 60), `lancamentoEstornadoId` (UUID, nulo se não for estorno), `criadoEm` (`Instant`).

- **Invariantes**: valor positivo com no máximo 2 casas; descrição obrigatória; um lançamento só pode ser estornado uma vez; estorno de um estorno é proibido; estorno gera um novo lançamento de tipo oposto com o mesmo valor e mesma `dataLancamento` do original (o saldo do dia original é corrigido).
- Não existe update/delete de lançamento — correção é feita via estorno (trilha de auditoria).

Consolidado

- **Agregado** `SaldoDiario` chave (`comercianteId` , `data`): `totalCreditos` , `totalDebitos` , `saldo` (= créditos – débitos), `quantidadeLancamentos` , `versao` (optimistic locking), `atualizadoEm` .
- **Saldo acumulado**: calculado na consulta como soma dos `saldo` de todos os dias `<= data` do comerciante.
- Aplicação de evento é comutativa (soma), portanto a ordem de chegada não importa; a idempotência é garantida pela tabela `eventos_processados(event_id PK)` gravada na mesma transação da atualização.

6. Contrato do evento (Published Language)

Publicado no tópico SNS, entregue no SQS com **raw message delivery = true** (o corpo da mensagem SQS é exatamente o JSON abaixo). Atributo de mensagem SNS/SQS `eventType` = `LancamentoRegistrado` .

```
{
  "eventId": "0b8f6c1e-6d1a-4b8e-9a43-0f1b7e2f9c11",
  "eventType": "LancamentoRegistrado",
  "eventVersion": 1,
  "occurredAt": "2026-11-10T18:00:00.123Z",
  "lancamentoId": "5f0e1c7a-1d2b-4c3d-8e9f-0a1b2c3d4e5f",
  "comercianteId": "11111111-1111-1111-1111-111111111111",
  "tipo": "CREDITO",
  "valor": "150.00",
  "moeda": "BRL",
  "dataLancamento": "2026-11-10",
  "lancamentoEstornadoId": null
}
```

- `valor` é **string decimal** com 2 casas (evita perda de precisão).
- Um estorno é publicado como `LancamentoRegistrado` com tipo oposto e `lancamentoEstornadoId` pre-enchido — o Consolidado não precisa de lógica especial.
- Entrega *at-least-once*; consumidor deve ser idempotente por `eventId` .
- Evoluções: campos novos são aditivos; quebra de contrato exige `eventVersion` 2 e período de convivência.

7. APIs REST

Base path `/api/v1` . Erros em **RFC 9457 Problem Details** (`application/problem+json`). OpenAPI via springdoc (`/v3/api-docs` , `/swagger-ui.html`). O `comercianteId` **nunca** vem do corpo/URL: é extraído do JWT (claim configurável `app.security.comerciante-claim` , padrão `comerciante_id`).

lancamentos-service (porta 8081)

Método	Caminho	Escopo	Descrição
POST	`/api/v1/lancamentos`	`lancamentos:write`	Registra lançamento. Header opcional `Idempotency-Key` (UUID): a mesma chave e o mesmo corpo devolvem o mesmo recurso (201 na primeira, 200 na repetição); a mesma chave com corpo diferente responde **409**.
GET	`/api/v1/lancamentos/{id}`	`lancamentos:read`	Detalhe
GET	`/api/v1/lancamentos?data=YYYY-MM-DD&page=0&size=50`	`lancamentos:read`	Lista paginada por data
POST	`/api/v1/lancamentos/{id}/estornos`	`lancamentos:write`	Estorna lançamento (body opcional `{ "motivo": "..." }`). O header `Idempotency-Key` vale também aqui: a impressão digital é o id estornado mais o motivo.

Request POST: `{ "tipo": "CREDITO", "valor": 150.00, "dataLancamento": "2026-11-10", "descricao": "Venda balcão", "categoria": "VENDAS" }`

- `valor` na requisição é número JSON (desserializado para `BigDecimal`); nas respostas e no evento é string decimal com 2 casas.
- Idempotência: primeira criação retorna **201 Created**; repetição com a mesma `Idempotency-Key` e a mesma impressão digital retorna **200 OK** com o mesmo recurso. A coluna `idempotencia.impressao_digital` guarda o SHA-256 do corpo canônico. Corpo diferente com a mesma chave responde **409**.
- Lista paginada: `{ "conteudo": [...], "pagina": 0, "tamanho": 50, "totalElementos": 1, "totalPaginas": 1 }`.
- Estorno: a `descricao` do lançamento de estorno é o `motivo` informado ou, se ausente, `Estorno do lançamento {id}`.

consolidado-service (porta 8082)

Método	Caminho	Escopo	Descrição
GET	`/api/v1/consolidados/diario/{data}`	`consolidado:read`	Saldo do dia: `{ data, totalCreditos, totalDebitos, saldo, saldoAcumulado, quantidadeLancamentos, atualizadoEm }`. Dia sem movimento retorna 200 com zeros.
GET	`/api/v1/consolidados/diario?inicio=YYYY-MM-DD&fim=YYYY-MM-DD`	`consolidado:read`	Relatório do período (máx. 366 dias): `{ inicio, fim, dias: [ <mesmo formato do saldo do dia> ] }`

Escopos: autoridades `SCOPE_<prefixo><escopo>`. Propriedade `app.security.scope-prefix` (padrão vazio; na AWS/Cognito = `fluxo-caixa/`).

8. Arquitetura interna dos serviços (Hexagonal / Ports & Adapters)

Pacotes base: `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`.

```
domain/          # entidades, value objects, eventos de domínio, exceções — sem Spring
application/     # casos de uso (ports in), ports out (interfaces)
adapters/
  in/web/        # controllers REST, DTOs, exception handler
  in/messaging/  # (consolidado) listener SQS
  out/persistence/  # JPA/JDBC
  out/messaging/    # (lancamentos) publicador SNS + outbox relay
  out/cache/        # (consolidado) Redis
config/
```

Regras verificadas por ArchUnit: `domain` não depende de Spring nem de `adapters`; `application` não depende de `adapters`.

9. Recursos de mensageria

No LocalStack os nomes são fixos. Na AWS o Terraform prefixa o ambiente (`fluxo-caixa-dev-` / `fluxo-caixa-prod-`). Os serviços não leem esses nomes: publicam em `LANCAMENTOS_TOPIC_ARN` e consomem `CONSOLIDADO_QUEUE_URL`.

Recurso	Nome no local
Tópico SNS	`fluxo-caixa-lancamentos-events`
Fila SQS	`consolidado-lancamentos-queue` (visibility 60s, retenção 14d, long polling 20s)
DLQ	`consolidado-lancamentos-dlq` (maxReceiveCount = 5)

Outbox (lancamentos): tabela `outbox_eventos(id UUID PK, aggregate_id, event_type, payload JSONB, criado_em, publicado_em NULL, tentativas INT, proxima_tentativa_em)`. O relay, a cada 500 ms, lê um lote de 100 com `SELECT ... FOR UPDATE SKIP LOCKED` onde `publicado_em` é nulo e `proxima_tentativa_em` já passou. Falha de publish grava a próxima tentativa com backoff exponencial até 60 s.

10. Variáveis de ambiente (contrato serviço ↔ infra)

Comuns: | Variável | Exemplo local | |---|---|| `SERVER_PORT` | 8081 / 8082 || `DB_URL` | `jdbc:postgresql://localhost:5432/lancamentos` ou `/consolidado` || `DB_USERNAME` / `DB_PASSWORD` | `lancamentos` / `lancamentos`, `consolidado` / `consolidado` || `AWS_REGION` | `us-east-1` || `AWS_ENDPOINT_URL` | `http://localhost:4566` no local. Na AWS a variável não é definida. Ausência ou string em branco faz o SDK usar o endpoint regional padrão; não se passa string vazia para `endpointOverride` || `JWT_ISSUER_URI` | `http://localhost:8080/realms/fluxo-caixa` || `APP_SECURITY_SCOPE_PREFIX` | vazio local; `fluxo-caixa/` na AWS || `OTEL_EXPORTER_OTLP_ENDPOINT` | `http://localhost:4318` || `SPRING_PROFILES_ACTIVE` | `local` / `aws` |

Específicas: | Serviço | Variável | Exemplo local | |---|---|---| | lancamentos | `LANCAMENTOS_TOPIC_ARN` | `arn:aws:sns:us-east-1:000000000000:fluxo-caixa-lancamentos-events` | | consolidado | `CONSOLIDADO_QUEUE_URL` | `http://localhost:4566/000000000000/consolidado-lancamentos-queue` | | consolidado | `REDIS_HOST` / `REDIS_PORT` / `REDIS_SSL` | `localhost` / `6379` / `false` |

Health checks: `/actuator/health/liveness` e `/actuator/health/readiness` (ALB usa readiness). Métricas: `/actuator/prometheus`.

11. Ambiente local (docker-compose em `infra/local`)

Container	Porta	Observação
postgres	5432	cria bancos/usuários `lancamentos` e `consolidado`
redis	6379	
localstack	4566	cria tópico, fila, DLQ e subscription com raw delivery
keycloak	8080	realm `fluxo-caixa`, client `fluxo-caixa-app` (public, direct access grants), client scopes `lancamentos:write`, `lancamentos:read`, `consolidado:read` como default; usuário `comerciante1` / `comerciante1` com atributo `comerciante_id=11111111-1111-1111-1111-111111111111` mapeado como claim no access token
otel-lgtm	3000 (Grafana), 4317, 4318	
lancamentos-service	8081	build a partir de `services/lancamentos-service/Dockerfile`
consolidado-service	8082	build a partir de `services/consolidado-service/Dockerfile`

Dentro da rede do compose, `JWT_ISSUER_URI` deve permanecer `http://localhost:8080/realms/fluxo-caixa` para bater com o `iss` do token obtido pelo host; usar `spring.security.oauth2.resourceserver.jwt.jwk-set-uri` apontando para `http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs` (variável `JWT_JWK_SET_URI`) e validar o issuer separadamente.

12. AWS (alvo)

> **Revisão 2 (2026-11-10):** região passou para `us-east-1`, borda passou a ser CloudFront + WAF com ALB **interno** via VPC Origin, e há três ambientes. Onde esta seção divergir das seções 14–19, valem as seções 14–19.

- VPC em 3 AZs (subnets públicas, privadas de aplicação e privadas de dados), NAT Gateway (1 em dev, 1 por AZ em prod), VPC endpoints (ECR api/dkr, S3, Logs, SQS, SNS, Secrets Manager, STS).
- CloudFront + WAF (escopo CLOUDFRONT) → VPC Origin → ALB **interno** (ver seção 14). Roteamento por caminho: `/api/v1/lancamentos*` → lancamentos, `/api/v1/consolidados*` → consolidado.
- ECS Fargate: cluster único, um serviço por microsserviço, mínimo 2 tasks em AZs distintas, auto scaling (CPU e `ALBRequestCountPerTarget`), deployment circuit breaker com rollback, Container Insights.
- RDS PostgreSQL **separado por serviço** em prod (isolamento de falha), Multi-AZ, criptografado com KMS, senha gerenciada pelo Secrets Manager.
- ElastiCache Redis/Valkey com réplica e failover automático (consolidado).
- SNS + SQS + DLQ criptografados (KMS) com políticas restritas. A policy do tópico não leva `sns:Unsubscribe`.
- Access log do ALB em S3 com SSE-S3. O ELB não aceita CMK nesse bucket. Nove CMKs por ambiente: storage e secret de cada RDS, ElastiCache, mensageria, logs, ECR e `origin-verify`.
- Cognito User Pool (Essentials) + domínio hospedado + Resource Server `fluxo-caixa` (escopos acima) + Pre Token Generation Lambda (v2) para incluir `comerciante_id` e os mesmos escopos no access token.
- Observabilidade: CloudWatch Logs, ADOT collector como sidecar (traces → X-Ray, métricas → CloudWatch/AMP), dashboards e alarmes (5xx, latência p99, profundidade da fila, idade da mensagem mais antiga, mensagens na DLQ) com notificação via SNS.
- ECR com scan on push e imagens imutáveis.

13. Metas de NFR (SLOs)

Métrica	Meta
Disponibilidade Lançamentos	99,9% mensal, independente do Consolidado
Disponibilidade Consolidado	99,5% mensal
Latência POST lançamento	p95 < 200 ms, p99 < 500 ms
Latência GET consolidado	p95 < 100 ms (cache), p99 < 300 ms
Throughput Consolidado	≥ 50 req/s sustentados, erro ≤ 5% (meta interna ≤ 1%)
Atraso de consolidação (eventual consistency)	p95 < 5 s entre registro e reflexo no saldo
RPO / RTO	RPO ≤ 5 min (PITR RDS), RTO ≤ 1 h

14. Borda: CloudFront + WAF + VPC Origin (Revisão 2)

- **CloudFront** é a única entrada pública. Distribuição com `PriceClass_All` (necessário para usar os edges da América do Sul, onde estão os comerciantes), HTTP/2 e HTTP/3, TLS 1.2+ (`TLSv1.2_2021`), certificado ACM em `us-east-1`, domínio customizado parametrizável.
- **AWS WAF** com escopo `CLOUDFRONT` (exige recursos em `us-east-1`): AWSManagedRulesCommonRuleSet, KnownBadInputs, SQLiRuleSet, AmazonIpReputationList, AnonymousIpList (count em dev) e rate-based rule por IP. Logs do WAF em CloudWatch Logs/S3.

- **Origem**: CloudFront **VPC Origin** apontando para **ALB interno** (subnets privadas). O ALB não tem IP público; seu security group aceita apenas o prefix list gerenciado `com.amazonaws.global.cloudfront.origin-facing`.
- **Cache e rotas**: só `/api/v1/lancamentos*` e `/api/v1/consolidados*` vão ao ALB, com `CachingDisabled` + origin request policy `AllViewerExceptHostHeader` (respostas são por comerciante e autenticadas — não cachear na borda). O comportamento default do CloudFront devolve 404 numa função de viewer-request e não chama a origem. O listener do ALB repete a mesma fronteira: as duas regras encaminham, e a ação padrão é 404 fixo. `/actuator` e o Swagger não são rota pública. O health check do target group fala com a task direto, sem passar pelo listener. Ganho do CloudFront aqui é TLS terminado no edge de São Paulo (GRU), conexões persistentes pela backbone AWS até `us-east-1`, absorção de DDoS (Shield Standard), WAF na borda e ponto único para futuros front-ends estáticos (S3 + OAC).
- **HTTP até o ALB**: o TLS da internet termina no CloudFront. O trecho CloudFront → VPC Origin → ALB interno é HTTP na rede da AWS. O hostname do ALB interno não cabe num certificado ACM público, e o security group só aceita o prefix list `com.amazonaws.global.cloudfront.origin-facing`. Não há API em claro na internet.
- Headers de segurança via Response Headers Policy (HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy).

15. Ambientes e dimensionamento (us-east-1)

Item	dev	hml	prod
AZs	2	2	3
NAT Gateway	1	1	1 por AZ (3)
VPC interface endpoints	não (sai pelo NAT)	sim, 2 AZs	sim, 3 AZs
lancamentos (Fargate ARM64)	0,5 vCPU / 1 GB, min 1 / max 2, Fargate Spot	0,5 vCPU / 1 GB, min 1 / max 3	0,5 vCPU / 1 GB, min 2 / max 10, on-demand
consolidado (Fargate ARM64)	0,5 vCPU / 1 GB, min 1 / max 2, Fargate Spot	1 vCPU / 2 GB, min 1 / max 3	1 vCPU / 2 GB, min 2 / max 10, on-demand
RDS Postgre-SQL 17 (1 por serviço)	`db.t4g.micro`, single-AZ, 20 GB gp3	`db.t4g.small`, single-AZ, 20 GB gp3	`db.t4g.medium`, Multi-AZ, 50 GB gp3. Read replica clássica fica em 0: a AWS não cria réplica quando a senha é gerenciada pelo Secrets Manager
RDS Proxy	não	não	parametrizável (padrão desligado)
ElastiCache Valkey	`cache.t4g.micro`, 1 nó	`cache.t4g.small`, 1 nó	`cache.t4g.small`, primário + réplica, Multi-AZ
CloudFront + WAF	sim	sim	sim
Retenção de logs	7 dias	14 dias	30 dias

Item	dev	hml	prod
Container Insights	desligado	ligado	ligado (enhanced)
Deletion protection / backups	não / 1 dia	não / 7 dias	sim / 14 dias + PITR

Valores são **defaults** nos `*.tfvars` de cada ambiente; tudo é parametrizável.

16. Rastreabilidade ponta a ponta (traces nos logs)

- Propagação **W3C Trace Context** (`traceparent` , `tracestate`) em HTTP (entrada e saída) via Micrometer Tracing + OpenTelemetry bridge.
- Header `X-Correlation-Id` : aceito na requisição (UUID; se ausente/inválido é gerado), devolvido na resposta, colocado no MDC como `correlationId` .
- **Logs JSON** (structured logging do Boot) sempre com: `traceId` , `spanId` , `correlationId` e, quando aplicável, `comercianteId` , `lancamentoId` , `eventId` .
- **Outbox**: no momento do POST, o `traceparent` e o `correlationId` correntes são gravados na linha de `outbox_eventos` (colunas `trace_parent` , `trace_state` , `correlation_id`). O relay, que roda em outra thread, publica no SNS reconstituindo esse contexto (span `PRODUCER` filho do span do POST), para que o trace não se quebre na fronteira assíncrona.
- **Message attributes SNS** (repassados ao SQS pela raw delivery): `eventType` , `traceparent` , `tracestate` (se houver), `correlationId` — todos `String` . O payload JSON do evento (seção 6) não muda.
- **Consumidor SQS**: `ReceiveMessage` com `MessageAttributeNames=All` e `MessageSystemAttributeNames=All` ; extrai o contexto e abre span `CONSUMER` filho do span do produtor; popula MDC (`traceId` , `spanId` , `correlationId` , `eventId` , `comercianteId`) e limpa ao final. Na AWS o X-Ray recebe o trace único POST → relay → SNS → SQS → consolidação.

17. Rotação de segredos

- **RDS**: senha gerenciada pelo Secrets Manager (`manage_master_user_password`) com **rotação automática** configurável (padrão 30 dias em prod/hml, 90 em dev) e chave KMS própria.
- **Contrato de `DB_URL`** : a task recebe sempre `jdbc:postgresql://<host>:5432/<db>` . `DB_READER_URL` , quando existir, usa o mesmo esquema. O serviço aceita também `jdbc:aws-wrapper:postgresql://` e normaliza para um único prefixo: com `DB_SECRET_ARN` no perfil `aws` , a URL efetiva é `jdbc:aws-wrapper:postgresql://...` (prefixo repetido é removido, não duplicado); sem segredo, o prefixo do wrapper é retirado e o driver PostgreSQL usa `DB_USERNAME` / `DB_PASSWORD` .
- **Plugins**: instância Multi-AZ usa `failover,awsSecretsManager` . Se o host for endpoint de RDS Proxy (o nome contém `.proxy-`), o plugin `failover` fica de fora e só `awsSecretsManager` permanece. O failover do wrapper descobre a instância e conecta nela, furando o proxy e o security group da task. O proxy já faz o failover.
- **Read replica clássica**: a AWS não cria read replica de RDS for PostgreSQL quando `manage_master_user_password` está ligado. O módulo recusa `read_replica_count > 0` nessa combi-

nação. O caminho de código `DB_READER_URL` continua pronto. No gatilho de leitura, a saída compatível com senha gerenciada é o RDS Multi-AZ DB Cluster (reader endpoint) ou o Aurora PostgreSQL; a terceira saída é senha fora do Secrets Manager, rotacionada por Lambda. Ver seção 19 e o ADR 0030.

- Hikari `max-lifetime` < intervalo de rotação. A task role recebe `secretsmanager:GetSecretValue` apenas no ARN do próprio segredo + `kms:Decrypt`.
- **Demais**: KMS com rotação anual automática; certificados ACM renovados automaticamente; chaves de assinatura JWT rotacionadas pelo Cognito (serviços buscam JWKS dinamicamente); CI/CD sem chaves de longa duração (GitHub OIDC → IAM role); EventBridge notifica falhas de rotação (`RotationFailed`) no tópico de alertas.

18. Resiliência em código (Resilience4j)

- Versões: Java 25, Spring Boot 4.0.9, Resilience4j 2.4.0 (`resilience4j-spring-boot4` + `spring-boot-starter-aop` / `aspectj` conforme exigido pelo Boot 4.0). Compatibilidade documentada em `documentos-brutos/` e nos READMEs dos serviços, com matriz de versões e gatilho de upgrade para Boot 4.1.
- **lancamentos**: CircuitBreaker + Retry (backoff exponencial com jitter) + TimeLimiter de **1 s** na publicação SNS do relay; quando o circuito abre, o relay adia o lote (eventos permanecem no outbox, nada se perde) — o POST nunca depende do SNS.
- **consolidado**: CircuitBreaker + TimeLimiter de **300 ms** no Redis, com fallback para o banco; Bulkhead de **8** chamadas no consumo SQS; Retry com backoff nas falhas transitórias de banco no consumidor; RateLimiter das consultas **desligado** por padrão.
- **Cliente HTTP do AWS SDK**: os dois serviços usam `url-connection-client` e excluem `apache5-client`. O AWS SDK 2.57.12 traz `httpcore5` 5.3.6 (CVE-2026-54399 e CVE-2026-54428). Tirar o cliente Apache elimina a biblioteca, em vez de piná-la. O long polling do SQS segura o socket por 20 s; o socket timeout do consolidado é 25 s, e o timeout da tentativa da API fica em 32 s. O SNS do lançamentos não faz long poll: socket e tentativa ficam em 2 s, acima do time limiter de 1 s.
- Métricas do Resilience4j expostas no Micrometer/Prometheus; estados do circuito aparecem em `/actuator/health` (sem derrubar readiness).

19. Desempenho de banco e consultas

- **lancamentos**: índice composto `(comerciante_id, data_lancamento, criado_em DESC)` cobrindo a listagem paginada; paginação keyset opcional; particionamento declarativo por mês em `lancamentos` como evolução documentada; `outbox_eventos` com índice parcial `WHERE publicado_em IS NULL` e limpeza periódica (retenção) dos publicados.
- **Leitura/escrita separadas**: variável opcional `DB_READER_URL` no esquema `jdbc:postgresql://`; quando presente, transações `@Transactional(readOnly = true)` são roteadas para o reader (`LazyConnectionDataSourceProxy` + routing data source); ausente, tudo vai ao primário. Read replica clássica de PostgreSQL não convive com senha gerenciada pelo Secrets Manager. O reader compatível, quando o gatilho disparar, é o endpoint de leitura do Multi-AZ DB Cluster ou de um Aurora. Planos medidos: lancamentos e consolidado.
- **consolidado**: PK `(comerciante_id, data)` atende consulta por dia e por período (range scan); saldo acumulado via soma até a data — documentar e, se o volume crescer, materializar `saldo_acumulado` incremental ou snapshots mensais.

- Hikari dimensionado por task, `pg_stat_statements` e Performance Insights habilitados, `statement_timeout` por usuário, `EXPLAIN ANALYZE` das consultas críticas documentado.

20. API Gateway (Revisão 3)

A composição Terraform implementa este desenho quando `enable_api_gateway = true` (padrão em dev, hml e prod). `false` volta ao caminho da seção 14: CloudFront → VPC Origin → ALB interno.

```
Cliente → CloudFront (+ WAF global) → API Gateway REST API regional (+ WAF regional)
        → VPC Link v2 → ALB interno → ECS Fargate
```

O provider AWS 6.72 aceita a integração privada da REST API com VPC Link v2 direto no ALB (`aws_apigatewayv2_vpc_link` + `integration_target` em `aws_api_gateway_integration`). Não há NLB. O VPC Link v1 (`aws_api_gateway_vpc_link`) só aponta para NLB e não foi usado.

A integração privada inclui o nome do stage no path enviado ao backend (`/v1` + `/api/v1/...`). O listener do ALB, com o flag ligado, reescreve `^/v1(/.*)$` para o path do serviço. Sem o prefixo, o rewrite não casa e o path da seção 14 segue inteiro. O hop API Gateway → ALB é HTTP na VPC: o hostname interno do ALB não cabe num certificado ACM público. Timeout de integração: 10 s.

- **Tipo**: REST API **regional** (não edge-optimized — a borda já é o CloudFront). Escolhido sobre HTTP API por suportar **usage plans + API keys por consumidor**, **WAF associado**, **request validation**, **mTLS em domínio customizado**, **canary de stage** e cache opcional. Trade-off de custo (REST ~USD 3,50/milhão vs HTTP ~USD 1,00/milhão) documentado no ADR 0031.
- **Autorização na borda**: Cognito User Pool authorizer validando o JWT e os escopos por rota (`lancamentos:write`, `lancamentos:read`, `consolidado:read`, com o prefixo do resource server). Os serviços **continuam** validando o JWT (defesa em profundidade, zero trust) e extraindo `comerciante_id`.
- **Rotas expostas**: somente `/api/v1/lancamentos`, `/api/v1/lancamentos/{id}`, `/api/v1/lancamentos/{id}/estornos` e `/api/v1/consolidados/diario`, `/api/v1/consolidados/diario/{data}`, com os métodos da seção 7. Nada de `/actuator`, `/v3/api-docs` ou `/swagger-ui` passa pelo gateway.
- **Proteção contra bypass do CloudFront**: CloudFront envia header `X-Origin-Verify` com valor guardado no Secrets Manager (com **rotação**: Lambda grava o novo valor, atualiza o origin custom header do CloudFront e a regra do WAF regional aceitando atual + anterior durante a janela). O WAF regional do API Gateway bloqueia requisições sem o header correto. A origem usa `AllViewerExceptHostHeader` para o `Host` ser o do `execute-api`.
- **Throttling**: limites por stage e por método (ex.: POST lançamentos e GET consolidado com burst/rate próprios, dimensionados acima de 50 req/s de pico em hml e prod; menor em dev) e **usage plans** por consumidor de integração (API key + quota), além do rate-based rule do WAF de borda.
- **Integração privada**: VPC Link v2 para o ALB interno. Timeout de integração 10 s. Path params e os headers `Authorization`, `traceparent`, `tracestate`, `X-Correlation-Id` e `Idempotency-Key` seguem para o serviço. `X-Origin-Verify` é removido nesse hop.
- **Observabilidade**: access logs JSON no CloudWatch (com `requestId`, `extendedRequestId`, `xrayTraceId`, `ip`, `status`, `latency`, `integrationLatency`, `apiKeyId`, além de `traceparent` e

`correlationId`), X-Ray tracing ativo no stage, métricas e alarmes (5XX, 4XX, Latency p99, IntegrationLatency).

- **Alternativa parametrizável**: `enable_api_gateway = false` mantém CloudFront → VPC Origin → ALB interno (seção 14).

PARTE 16

Serviços e ambiente local

Guia dos microsserviços, matriz de compatibilidade, desempenho de consultas e laboratório local

FONTE: SERVICES/LANCAMENTOS-SERVICE/README.MD

lancamentos-service

Serviço de escrita do fluxo de caixa. Registra créditos e débitos, estorna lançamentos e publica o evento `LancamentoRegistrado` sem depender do serviço de consolidado.

Responsabilidade

- Fonte da verdade dos lançamentos de um comerciante.
- Não altera nem apaga lançamento: correção é um novo lançamento de estorno (tipo oposto, mesmo valor e mesma data de competência).
- Grava o lançamento e o evento na mesma transação (transactional outbox) e um relay publica no tópico SNS `fluxo-caixa-lancamentos-events`.
- Se o consolidado estiver fora, este serviço continua aceitando lançamentos. As mensagens ficam no outbox e depois no SQS.

Arquitetura interna

Hexagonal, pacote base `br.com.fluxocaixa.lancamentos`:

```
domain/                 agregado, value objects e evento — sem Spring
application/            casos de uso e ports
adapters/in/web/        API REST, Problem Details e claim do JWT
adapters/out/persistence/  JPA + Flyway
adapters/out/messaging/    publicador SNS e relay da outbox
config/                 segurança, AWS, OpenAPI e observabilidade
```

O `comercianteId` sai da claim JWT `comerciante_id` (configurável). Ele não vai no corpo nem na URL.

Endpoints

Porta local `8081`. Base `/api/v1`. Erros em `application/problem+json` (RFC 9457).

Método	Caminho	Escopo
POST	`/api/v1/lancamentos`	`lancamentos:write`
GET	`/api/v1/lancamentos/{id}`	`lancamentos:read`
GET	`/api/v1/lancamentos?data=YYYY-MM-DD&page=0&size=50`	`lancamentos:read`
POST	`/api/v1/lancamentos/{id}/estornos`	`lancamentos:write`

OpenAPI: `/v3/api-docs` e `/swagger-ui.html`. No perfil `local` o Swagger é público. No perfil `aws` o springdoc fica desligado e essas rotas exigem autenticação se alguém as religar. Igual ao consolidado.

Probes: `/actuator/health/liveness` e `/actuator/health/readiness`. Métricas: `/actuator/prometheus`.

O header `Idempotency-Key` (UUID) é opcional no registro e no estorno. A primeira chamada responde `201`; a repetição da mesma chave e do mesmo conteúdo responde `200` com o mesmo recurso. A mesma chave com outro conteúdo responde `409`.

O valor sai como string decimal com duas casas (`"150.00"`), no mesmo formato do evento.

```
curl -sS -D- -X POST http://localhost:8081/api/v1/lancamentos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: $(uuidgen)" \
  -d '{"tipo":"CREDITO","valor":150.00,"dataLancamento":"2026-11-10","descricao":"Venda
      balcão","categoria":"VENDAS"}'

curl -sS "http://localhost:8081/api/v1/lancamentos?data=2026-11-10&page=0&size=50" \
  -H "Authorization: Bearer $TOKEN"

curl -sS -X POST http://localhost:8081/api/v1/lancamentos/$ID/estornos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"motivo":"Cliente devolveu"}'
```

Lista paginada:

```
{
  "conteudo": [],
  "pagina": 0,
  "tamanho": 50,
  "totalElementos": 0,
  "totalPaginas": 0
}
```

Variáveis de ambiente

Variável	Uso
`SERVER_PORT`	Porta HTTP. Padrão `8081`.
`DB_URL`	JDBC do PostgreSQL do serviço. No perfil `aws` com segredo, pode ser `jdbc:postgresql://...`; o serviço prefixa `jdbc:aws-wrapper:`.
`DB_USERNAME` / `DB_PASSWORD`	Credenciais do banco. Obrigatórias sem `DB_SECRET_ARN`.
`DB_SECRET_ARN`	ARN do segredo do RDS. Só ativa o AWS JDBC Wrapper no perfil `aws`.
`DB_READER_URL`	JDBC opcional da réplica. Ausente = um único DataSource.
`OUTBOX_RETENCAO`	Quanto tempo um evento publicado permanece na outbox. Padrão `7d`.
`OUTBOX_ADIAMENTO_CIRCUIT O`	Quanto o relay espera com o circuito aberto. Padrão `30s`.
`OUTBOX_LIMPEZA_CRON`	Cron UTC da limpeza. Padrão `0 15 4 * * *`.

Variável	Uso
`AWS_REGION`	Região. Padrão `us-east-1`.
`AWS_ENDPOINT_URL`	Endpoint do LocalStack. Vazio na AWS.
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	Usadas só quando `AWS_ENDPOINT_URL` está definido. Padrão `test` / `test`. Na AWS a task role entra pela cadeia padrão do SDK.
`LANCAMENTOS_TOPIC_ARN`	ARN do tópico `fluxo-caixa-lancamentos-events`.
`JWT_ISSUER_URI`	Issuer esperado do access token.
`JWT_JWK_SET_URI`	Opcional. Busca as chaves nesse URI e continua validando o issuer.
`APP_SECURITY_SCOPE_PREFIX`	Prefixo do escopo. Vazio no local; `fluxo-caixa/` na AWS.
`APP_SECURITY_COMERCIANTE_CLAIM`	Claim do comerciante. Padrão `comerciante_id`.
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP HTTP (traces e métricas). Ex.: `http://localhost:4318`.
`SPRING_PROFILES_ACTIVE`	`local` ou `aws`.

Profiles:

- `local`: detalhe do health liberado para diagnóstico.
- `aws`: `server.forward-headers-strategy=framework` para o ALB.

Threads virtuais e shutdown gracioso ficam ligados em qualquer profile.

Rastreabilidade

- Propagação W3C (`traceparent` / `tracestate`) e `X-Amzn-Trace-Id` na entrada HTTP. O Boot não lê `OTEL_PROPAGATORS`; o bean compõe os dois. A flag `Sampled` do pai manda.
- Header `X-Correlation-Id`: UUID. Se vier ausente ou inválido, o serviço gera outro, devolve no response e grava no MDC como `correlationId`.
- Log JSON de console com `traceId`, `spanId`, `correlationId` e, quando a operação tem, `comercianteId`, `lancamentoId` e `eventId`.
- O POST e o estorno gravam `trace_parent`, `trace_state` e `correlation_id` na `outbox_eventos`.
- O relay não abre span quando a outbox está vazia. Com evento, abre um span `PRODUCER` filho do contexto gravado e publica os message attributes `eventType`, `traceparent`, `tracestate` (se houver), `correlationId` e `X-Amzn-Trace-Id`. O JSON do evento não muda.

Resiliência da publicação SNS

O POST não chama o SNS. O relay publica com Resilience4j, instância `publicacaoSns`:

- Circuit breaker (janela de 10, mínimo 5 chamadas, 50% de falha, 30 s aberto).

- Retry com backoff exponencial e jitter (3 tentativas, 50 ms a 400 ms).
- Time limiter de 1 s, com timeout do cliente AWS em 2 s. O rate limiter não é usado neste serviço.
- Com o circuito aberto o lote fica na outbox (`proxima_tentativa_em` adiantada por `OUTBOX_ADIAMENTO_CIRCUITO`, padrão 30 s). Nada é descartado.
- Métricas no Micrometer (`resilience4j.circuitbreaker.*`, `resilience4j.retry.*`, `resilience4j.timelimiter.*`).
- O estado do circuito aparece em `/actuator/health` (`CIRCUIT_OPEN` / `CIRCUIT_HALF_OPEN`). O readiness inclui só `readinessState` e `db`, então o circuito aberto não tira a task do ALB.

Eventos já publicados saem da outbox no cron `OUTBOX_LIMPEZA_CRON` (padrão 04:15 UTC) depois de `OUTBOX_RETENCAO` (padrão 7 dias).

Cliente HTTP da AWS

Os dois serviços usam o `url-connection-client` do AWS SDK e excluem o `apache5-client`. O SDK 2.57.12 traz `httpcore5` 5.3.6, afetado por CVE-2026-54399 e CVE-2026-54428. Tirar o cliente Apache tira a biblioteca do classpath. Um override para `httpcore5` 5.4.4 corrigiria o CVE e deixaria a dependência, sujeita ao próximo BOM. A publicação SNS não faz long polling: socket timeout e tentativa ficam em 2 s. O consolidado, que faz long polling de 20 s, sobe o socket para 25 s. O mesmo cliente JDK serve aos dois; o timeout acompanha a chamada.

Banco, segredo e réplica

Sem `DB_SECRET_ARN`, o pool usa o driver PostgreSQL e `DB_USERNAME` / `DB_PASSWORD`. O Flyway usa esse mesmo pool de escrita.

O contrato com o Terraform é `DB_URL=jdbc:postgresql://...`. O serviço também aceita `jdbc:aws-wrapper:postgresql://` e não duplica o prefixo. No perfil `aws`, se `DB_SECRET_ARN` estiver definido, a URL efetiva é `jdbc:aws-wrapper:postgresql://...`. Na instância os plugins são `failover,awsSecretsManager`. Se o host contiver `.proxy-` (RDS Proxy), fica só `awsSecretsManager`: o failover do wrapper iria na instância e furaria o proxy. Sem segredo, o prefixo do wrapper é removido e o driver PostgreSQL usa `DB_USERNAME` / `DB_PASSWORD`. A rebusca do segredo acontece quando a autenticação falha depois da rotação. Hikari `max-lifetime` é 15 min, abaixo da rotação mínima de 30 dias. Cada conexão nova faz `SET statement_timeout = '30s'`.

`DB_READER_URL` é opcional. Quando existe, `@Transactional(readOnly = true)` vai para a réplica (`LazyConnectionDataSourceProxy` + roteamento). Sem a variável, leitura e escrita usam o primário. O Flyway continua no primário nos dois modos.

A listagem usa o índice `(comerciante_id, data_lancamento, criado_em DESC)`. O relay usa o índice parcial `outbox_eventos (proxima_tentativa_em, criado_em) WHERE publicado_em IS NULL`. Planos em `docs/performance.md`. `page` vai de 0 a 1000 e `size` de 1 a 100. Corpo de `/api/` acima de 8 KB responde 413. Campo desconhecido no JSON responde 400.

Como rodar localmente

Suba Postgres, LocalStack e Keycloak pelo compose em `infra/local` (outro módulo) e então:

```
export SERVER_PORT=8081
export DB_URL=jdbc:postgresql://localhost:5432/lancamentos
export DB_USERNAME=lancamentos
export DB_PASSWORD=lancamentos
export AWS_REGION=us-east-1
export AWS_ENDPOINT_URL=http://localhost:4566
export LANCAMENTOS_TOPIC_ARN=arn:aws:sns:us-east-1:000000000000:fluxo-caixa-lancamentos-events
export JWT_ISSUER_URI=http://localhost:8080/realms/fluxo-caixa
export JWT_JWK_SET_URI=http://localhost:8080/realms/fluxo-caixa/protocol/openid-connect/certs
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
export SPRING_PROFILES_ACTIVE=local
./mvnw spring-boot:run
```

Dentro do compose, `JWT_JWK_SET_URI` aponta para `http://keycloak:8080/...` e `JWT_ISSUER_URI` permanece o issuer do token obtido no host.

Imagem:

```
docker build -t lancamentos-service:local .
```

Testes

Java 25 e Docker (Testcontainers sobe PostgreSQL 17 e LocalStack).

```
./mvnw verify
```

O `verify` roda testes de domínio, casos de uso, API (MockMvc + JWT), ArchUnit, roteamento de leitura, Resilience4j (retry, timeout e abertura do circuito) e o fluxo POST → outbox → SNS → SQS, incluindo `traceparent` e `correlationId`. O JaCoCo exige pelo menos 80% de linhas em `domain` e `application`.

Matriz de compatibilidade

Versões resolvidas pelo BOM do Spring Boot 4.0.9, salvo onde há versão explícita no `pom.xml`.

Componente	Versão
Java	25
Spring Boot	4.0.9
Spring Framework	7.0.10
Resilience4j	2.4.0 (`resilience4j-spring-boot4` , `resilience4j-spring6`)
AspectJ	1.9.25.1 (`spring-boot-starter-aspectj`)
Jackson	3.1.7

Componente	Versão
Hibernate	7.2.25.Final
Flyway	11.14.1
Testcontainers	2.0.5
AWS SDK for Java v2	2.57.12
AWS Advanced JDBC Wrapper	4.6.0
springdoc-openapi	3.0.3
Micrometer / tracing	1.16.8 / 1.6.8
Driver PostgreSQL	42.7.14

Quando voltar ao Boot 4.1. O Resilience4j 2.4.0 declara `spring-boot-autoconfigure` 4.0.0. Boot 4.1 muda pacotes (health, starters) e o starter 2.4.0 não sobe nele. O gatilho é uma release do Resilience4j cujo `resilience4j-spring-boot4` dependa de `spring-boot-autoconfigure` 4.1.x (ou um artefato novo equivalente). Aí sobe junto o springdoc da linha 3.1+/3.2, que acompanha o Boot 4.1, e o BOM passa a escolher Framework, Jackson, Hibernate e Testcontainers. Até lá este serviço permanece em 4.0.9.

Colunas além do desenho mínimo

- `idempotencia.impressao_digital`: distingue repetição legítima de reuso da mesma chave com outro corpo.
- `outbox_eventos.proxima_tentativa_em`: agenda o backoff do relay entre tentativas.
- `outbox_eventos.trace_parent`, `trace_state`, `correlation_id`: contexto W3C e correlação capturados no POST.

FONTE: SERVICES/LANCAMENTOS-SERVICE/DOCS/PERFORMANCE.MD

Desempenho das consultas — lancamentos-service

Medido com `EXPLAIN (ANALYZE, BUFFERS)` no PostgreSQL 17 do Testcontainers, depois de `ANALYZE`. Volume da amostra: 4.000 lançamentos de um comerciante em 10 datas (cerca de 400 no dia consultado) e, no outbox, 1.500 pendentes fora da janela mais 30 dentro da janela do relay.

O `ORDER BY criado_em ASC` da listagem percorre o índice `(comerciante_id, data_lancamento, criado_em DESC)` de trás para frente. A contagem da página usa o mesmo índice. O relay e a limpeza usam os índices parciais.

Paginação por offset fica limitada a `page <= 1000` e `size <= 100`. Keyset (`criado_em`, `id`) e particionamento mensal de `lancamentos` ficam como evolução se o offset passar a dominar o plano.

Listagem

```
Limit  (cost=0.28..18.05 rows=50 width=95) (actual time=0.016..0.037 rows=50 loops=1)
  Buffers: shared hit=10
  ->  Index Scan Backward using idx_lancamentos_comerciante_data_criado on lancamentos
(cost=0.28..142.41 rows=400 width=95) (actual time=0.015..0.031 rows=50 loops=1)
        Index Cond: ((comerciante_id = '...'::uuid) AND (data_lancamento = '2026-11-
10'::date))
        Buffers: shared hit=10
Planning Time: 0.560 ms
Execution Time: 0.055 ms
```

Contagem da página

```
Aggregate  (cost=93.38..93.39 rows=1 width=8) (actual time=0.305..0.306 rows=1 loops=1)
  Buffers: shared hit=66
  ->  Bitmap Heap Scan on lancamentos  (cost=24.38..92.38 rows=400 width=0) (actual
time=0.050..0.259 rows=400 loops=1)
        Recheck Cond: ((comerciante_id = '...'::uuid) AND (data_lancamento = '2026-11-
10'::date))
        Heap Blocks: exact=62
        Buffers: shared hit=66
        ->  Bitmap Index Scan on idx_lancamentos_comerciante_data_criado  (cost=0.00..24.28
rows=400 width=0) (actual time=0.038..0.038 rows=400 loops=1)
              Buffers: shared hit=4
Planning Time: 0.065 ms
Execution Time: 0.344 ms
```

Relay da outbox

`SELECT ... FOR UPDATE SKIP LOCKED` com faixa em `proxima_tentativa_em` (a carga de teste fica no futuro para o relay real, que usa `now()`, não publicar esses registros). 30 linhas entram no índice parcial; as outras 1.500 ficam de fora do predicado.

```
Limit  (cost=33.30..33.67 rows=30 width=207) (actual time=0.050..0.072 rows=30 loops=1)
  Buffers: shared hit=33
  ->  LockRows  (cost=33.30..33.67 rows=30 width=207) (actual time=0.050..0.069 rows=30
loops=1)
        ->  Sort  (cost=33.30..33.37 rows=30 width=207) (actual time=0.041..0.043 rows=30
loops=1)
              Sort Key: criado_em
              Sort Method: quicksort  Memory: 28kB
              ->  Bitmap Heap Scan on outbox_eventos  (cost=4.58..32.56 rows=30 width=207)
(actual time=0.017..0.026 rows=30 loops=1)
                    Recheck Cond: ((proxima_tentativa_em >= ...) AND (proxima_tentativa_em <=
...) AND (publicado_em IS NULL))
                    Heap Blocks: exact=1
                    ->  Bitmap Index Scan on idx_outbox_eventos_pendentes  (cost=0.00..4.58
rows=30 width=0) (actual time=0.007..0.007 rows=30 loops=1)
Planning Time: 0.419 ms
Execution Time: 0.104 ms
```

O sort de 30 linhas cabe em memória. Com a janela do relay (`proxima_tentativa_em <= now()`) o mesmo índice parcial atende o filtro.

Limpeza dos publicados

```
Delete on outbox_eventos  (cost=0.13..8.15 rows=0 width=0) (actual time=0.041..0.042 rows=0
loops=1)
  ->  Index Scan using idx_outbox_eventos_publicados on outbox_eventos  (cost=0.13..8.15
rows=1 width=6) (actual time=0.040..0.040 rows=0 loops=1)
        Index Cond: ((publicado_em IS NOT NULL) AND (publicado_em < ...))
Planning Time: 0.076 ms
Execution Time: 0.066 ms
```

O que saiu do esquema

`idx_lancamentos_comerciante_data` era prefixo de `idx_lancamentos_comerciante_data_criado` e foi removido na V3. O índice parcial dos pendentes da V1 foi recriado na V3 com a mesma definição.

FONTE: SERVICES/CONSOLIDADO-SERVICE/README.MD

consolidado-service

Projeção de leitura do saldo diário de um comerciante. O serviço não é a fonte da verdade dos lançamentos: consome o evento `LancamentoRegistrado` (SNS → SQS, raw delivery) e serve consultas materializadas. Se este serviço cair, o `lancamentos-service` continua gravando; as mensagens ficam retidas na fila.

Porta local: **8082**.

Responsabilidade

- Aplicar cada lançamento no saldo do dia (`comercianteId` , `dataLancamento`), de forma comutativa.
- Garantir idempotência por `eventId` .
- Responder o saldo do dia, com saldo acumulado, e o relatório de um período de até 366 dias.
- Segurar 50 req/s com cache Redis e pool de conexões, degradando para o PostgreSQL se o Redis sair.

Arquitetura

Hexagonal, pacote base `br.com.fluxocaixa.consolidado` :

```
domain/                 agregado SaldoDiario, VO Dinheiro, relatório — sem Spring
application/            casos de uso e ports
adapters/in/web/        REST, ProblemDetail, comerciante do JWT, correlation id
adapters/in/messaging/  consumidor SQS e camada anticorrupção do evento
adapters/out/persistence/  JDBC + Flyway
adapters/out/cache/     Redis com circuito e timeout
observabilidade/        correlation id e span CONSUMER
config/                 segurança, datasource, SQS
```

O domínio não depende de Spring nem de adaptadores. A aplicação não depende de adaptadores. ArchUnit cobre essas duas regras.

Compatibilidade

Matriz presa ao que o serviço compila e testa. O Resilience4j 2.4.0 publica `resilience4j-spring-boot4` , que depende de `spring-boot-autoconfigure` 4.0.0 e de `resilience4j-spring6` . Por isso o serviço fica no Spring Boot **4.0.9** (última 4.0.x), com Java 25. O starter de AOP do Boot 4.0 é `spring-boot-starter-aspectj` .

Componente	Versão
Java	25
Spring Boot	4.0.9
Spring Framework	7.0.10

Componente	Versão
Resilience4j	2.4.0 (`resilience4j-spring-boot4`)
Jackson	3.1.7 (`tools.jackson`)
Hibernate ORM (gerido pelo Boot; o serviço usa JDBC)	7.2.25.Final
Driver JDBC PostgreSQL	42.7.14
AWS JDBC Wrapper	4.6.0
Lettuce	6.8.2.RELEASE
Testcontainers	2.0.5
AWS SDK for Java v2	2.57.12
Flyway	11.14.1
springdoc-openapi	3.0.3 (linha 3.0, construída sobre Boot 4.0)
Micrometer / Micrometer Tracing	1.16.8 / 1.6.8

Gatilho para o Boot 4.1: subir quando o Resilience4j publicar um artefato compilado e testado contra `spring-boot-autoconfigure` 4.1.x (starter novo ou release que deixe de fixar o autoconfigure 4.0.0). No mesmo movimento, trocar o springdoc para a linha 3.1+ (a 3.1.0 já nasce em cima do Boot 4.1). Até lá, permanecer na última 4.0.x.

Concorrência e idempotência

A aplicação do evento é uma soma. A ordem de chegada não muda o saldo. Por isso o incremento é um UPSERT atômico, e não optimistic locking com retry:

```
INSERT INTO saldos_diarios (...)
VALUES (...)
ON CONFLICT (comerciante_id, data) DO UPDATE SET
    total_creditos = saldos_diarios.total_creditos + EXCLUDED.total_creditos,
    total_debitos = saldos_diarios.total_debitos + EXCLUDED.total_debitos,
    quantidade_lancamentos = saldos_diarios.quantidade_lancamentos +
        EXCLUDED.quantidade_lancamentos,
    versao = saldos_diarios.versao + 1,
    atualizado_em = EXCLUDED.atualizado_em
```

A versão continua sendo gravada (1 no primeiro evento, +1 nos seguintes) para auditoria. Ela não decide o commit.

A idempotência entra na mesma transação, antes do UPSERT:

```
INSERT INTO eventos_processados (event_id, processado_em)
VALUES (?, ?)
ON CONFLICT (event_id) DO NOTHING
```

Se duas entregas do mesmo `eventId` disputam a chave, o PostgreSQL segura a segunda até o commit da primeira. A perdedora recebe 0 linhas, não mexe no saldo e a mensagem é apagada da fila. Eventos diferentes do mesmo dia somam sem lost update. Uma nova tentativa do Resilience4j depois de uma falha transitória repete essa mesma transação: ou reaplica o que não commitou, ou encontra o `eventId` e segue como duplicata.

A mensagem SQS só é apagada depois do commit. Falha transitória, esgotamento do retry ou bulkhead cheio não apagam: a fila reentrega e, após 5 recebimentos, o redrive da infra manda para `consolidado-lancamentos-dlq`. JSON inválido, corpo acima de 8 KiB, `eventType` diferente de `LancamentoRegistrado` ou `eventVersion` diferente de 1 é poison: log de erro sem o corpo e a mensagem também segue para a DLQ.

O consumidor faz long polling de 20s, pede `MessageAttributeNames=All` e `MessageSystemAttributeNames=All`, lotes de até 10 e processa o lote em virtual threads. No desligamento ele para primeiro (`SmartLifecycle` na fase máxima), termina o lote já recebido e só então o restante da aplicação cai.

O cliente HTTP é o `url-connection-client`, o mesmo do lançamentos, sem `apache5-client` e sem `httpcore5` (CVE-2026-54399 e CVE-2026-54428 no 5.3.6 que o SDK 2.57.12 traz). O `HttpURLConnection` encerra o socket no timeout. Com long polling de 20 s, o socket timeout é 25 s, a tentativa da API é 32 s e a chamada é 35 s. Abaixo de 20 s o poll morre antes da fila responder.

Resiliência

Resilience4j 2.4.0, métricas no Micrometer/Prometheus (`resilience4j_circuitbreaker_*`, `resilience4j_retry_*`, `resilience4j_bulkhead_*`, `resilience4j_ratelimiter_*`).

Peça	Nome	Comportamento
CircuitBreaker + TimeLimiter	`redis`	Timeout de 300 ms. Falhas abrem o circuito. Leitura e invalidação caem no PostgreSQL. `allowHealthIndicatorToFail=false`: estado OPEN aparece em `/actuator/health` como `CIRCUIT_OPEN` e não tira o readiness do ar
Bulkhead (semaphore)	`sqs-consumo`	Até 8 processamentos simultâneos, espera de 2 s. Se não houver permissão, a mensagem fica na fila
Retry	`banco-consumo`	3 tentativas, backoff exponencial com jitter, só para falha transitória de banco/conexão. Não retenta poison. Depois de esgotar, a mensagem não é apagada
RateLimiter	`consultas`	50 req/s. **Desligado** por padrão (`APP_RESILIENCE_RATE_LIMITER_HABILITADO=false`). Ligado, responde 429 em `application/problem+json`

O readiness inclui só `readinessState` e `db`. Redis e o circuito ficam de fora. Liveness é só `livenessState`.

Cache

Cache-aside no Redis, TTL de 30s. A chave de dados inclui uma versão por comerciante (`consolidado:versao:{id}`):

`consolidado:dia:{comerciante}:{versao}:{data}` `consolidado:relatorio:{comerciante}:{versao}:{inicio}:{fim}`

Depois do commit, a versão é incrementada. A leitura seguinte não reutiliza o valor antigo. Chaves órfãs expiram no TTL. Se o incremento falhar ou o circuito abrir, a resposta sai do banco. Consistência eventual, janela máxima de 30s quando só a invalidação falha; no caminho feliz a próxima leitura já vê o banco.

O pool Lettuce tem timeout de comando e de conexão de 300 ms. O pool Hikari de escrita tem 10 conexões (tarefa de 1 vCPU; 10 tasks ficam perto de 100 conexões no primário), timeout de conexão de 3 s, `max-lifetime` de 15 min (abaixo da rotação de 30 dias do Secrets Manager) e `statement_timeout` de 3 s em cada conexão nova.

Banco: segredo e réplica

- Sem `DB_SECRET_ARN`: driver `org.postgresql.Driver` com `DB_USERNAME` e `DB_PASSWORD`. Se a URL vier com `jdbc:aws-wrapper:`, o prefixo é removido.
- Com `DB_SECRET_ARN`: a URL efetiva é um único `jdbc:aws-wrapper:postgresql://...`, driver `software.amazon.jdbc.Driver`. Plugins `failover,awsSecretsManager` na instância. No host com `.proxy-` (RDS Proxy), só `awsSecretsManager`. Usuário e senha saem do segredo. Se `AWS_ENDPOINT_URL` estiver preenchido (local), o wrapper usa esse endpoint do Secrets Manager. O Terraform envia `jdbc:postgresql://`.
- `DB_READER_URL` opcional. Quando presente, `@Transactional(readOnly = true)` (consultas da API) vai para a réplica via `LazyConnectionDataSourceProxy` + routing datasource. O Flyway e o consumidor usam o primário. Sem a variável, há um único `DataSource`.

Rastreabilidade

- HTTP e a fila propagam W3C (`traceparent` / `tracestate`) e `X-Amzn-Trace-Id`. O Boot não lê `OTEL_PROPAGATORS`. No perfil `aws`, métricas OTLP vão para o sidecar (`management.otlp.metrics.export.enabled=true`). No local continuam desligadas.
- Header `X-Correlation-Id`: UUID. Se ausente ou inválido, o serviço gera outro, devolve no response e coloca `correlationId` no MDC. Valor inválido não é ecoado.
- Logs JSON (formato logstash) incluem `traceId`, `spanId`, `correlationId` e, no consumo, `eventId` e `comercianteId`.
- O consumidor extrai `traceparent`, `tracestate`, `X-Amzn-Trace-Id` e `correlationId` dos message attributes (raw delivery) e abre um span `CONSUMER` filho do produtor. Mensagem sem atributos de trace inicia um trace novo. O MDC é limpo no fim de cada mensagem, na mesma virtual thread. `/actuator/health` não vira trace.

API

Base `/api/v1`. Erros em `application/problem+json` (RFC 9457). OpenAPI em `/v3/api-docs` e `/swagger-ui.html`. No perfil `local` o Swagger é público. No perfil `aws` o springdoc fica desligado e essas rotas não são públicas — o mesmo contrato do lançamentos. A borda não encaminha `/swagger-ui` nem `/actuator`.

O `comercianteId` sai do JWT, claim `comerciante_id` (`app.security.comerciante-claim`). Escopo `consolidado:read`, com prefixo `app.security.scope-prefix` (vazio no local, `fluxo-caixa/` na AWS). A autoridade fica `SCOPE_<prefixo>consolidado:read`.

CORS fica desligado enquanto `APP_SECURITY_CORS_ALLOWED_ORIGINS` não lista origens. Com a lista, só `GET` / `OPTIONS`, headers `Authorization`, `Content-Type`, `X-Correlation-Id`, `traceparent` e `tracestate`.

Headers de segurança na resposta: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, CSP `default-src 'none'`, `Permissions-Policy` e HSTS.

Dia sem movimento responde 200. Créditos, débitos, saldo e quantidade vêm zerados. `saldoAcumulado` continua sendo a soma dos saldos de todos os dias `<= data`, inclusive os anteriores. `atualizadoEm` fica nulo quando o dia não tem linha.

O relatório do período:

```
{
  "inicio": "2026-11-10",
  "fim": "2026-11-12",
  "dias": [
    {
      "data": "2026-11-10",
      "totalCreditos": 250.00,
      "totalDebitos": 50.00,
      "saldo": 200.00,
      "saldoAcumulado": 200.00,
      "quantidadeLancamentos": 3,
      "atualizadoEm": "2026-11-10T18:00:05Z"
    }
  ]
}
```

Um dia do intervalo sem lançamento entra na lista com zeros e com o acumulado correto. O intervalo máximo é 366 dias, inclusive. Datas fora de `YYYY-MM-DD` respondem 400 sem repetir o valor recebido.

```
curl -s http://localhost:8082/api/v1/consolidados/diario/2026-11-10 \
  -H "Authorization: Bearer $TOKEN" \
  -H "X-Correlation-Id: 11111111-1111-4111-8111-111111111111"

curl -s "http://localhost:8082/api/v1/consolidados/diario?inicio=2026-11-01&fim=2026-11-10" \
  -H "Authorization: Bearer $TOKEN"
```

Actuator liberado sem token: `/actuator/health`, `/actuator/health/liveness`, `/actuator/health/readiness`, `/actuator/info`, `/actuator/prometheus`. O ALB deve usar readiness.

Variáveis

Variável	Uso
`SERVER_PORT`	Porta HTTP. Padrão 8082

Variável	Uso
`DB_URL`	`jdbc:postgresql://` no contrato com o Terraform. Com segredo, o serviço aplica um único prefixo `jdbc:aws-wrapper:` . Os dois formatos são aceitos
`DB_USERNAME` / `DB_PASSWORD`	Credenciais do banco quando `DB_SECRET_ARN` está ausente
`DB_SECRET_ARN`	ARN do segredo do RDS. Liga o wrapper. Failover só fora do endpoint do proxy
`DB_READER_URL`	JDBC opcional da réplica. Roteia transação somente leitura
`AWS_REGION`	Padrão `us-east-1` . Também região do Secrets Manager
`AWS_ENDPOINT_URL`	LocalStack. Vazio na AWS. No profile `local` , o padrão é `http://localhost:4566`
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	Com endpoint override, o padrão local é `test` / `test`
`CONSOLIDADO_QUEUE_URL`	Fila `consolidado-lancamentos-queue` . Obrigatória com o consumo ligado
`JWT_ISSUER_URI`	Emissor do JWT
`JWT_JWK_SET_URI`	Opcional. Quando preenchida, as chaves vêm dessa URL e o `iss` continua sendo validado à parte
`APP_SECURITY_SCOPE_PREFIX`	Prefixo do escopo. Vazio no local; `fluxo-caixa/` na AWS
`APP_SECURITY_COMERCIANTE_CLAIM`	Claim do comerciante. Padrão `comerciante_id`
`APP_SECURITY_CORS_ALLOWED_ORIGINS`	Origens CORS separadas por vírgula. Vazio desliga CORS
`APP_RESILIENCE_RATE_LIMITER_HABILITADO`	`true` liga o limitador das consultas. Padrão `false`
`REDIS_HOST` / `REDIS_PORT` / `REDIS_SSL`	Redis. Padrão `localhost` / `6379` / `false`
`OTEL_EXPORTER_OTLP_ENDPOINT`	Base OTLP. O Boot acrescenta `/v1/traces`
`SPRING_PROFILES_ACTIVE`	`local` ou `aws`
`APP_MESSAGING_CONSUMO_HABILITADO`	Desliga o poller. Os testes de API usam `false`

Profiles: `local` aponta o endpoint AWS para o LocalStack e abre o Swagger. `aws` não define endpoint, usa o prefixo de escopo do Cognito e autentica o Swagger.

Observabilidade

- Logs JSON no formato logstash, com MDC (`traceId` , `spanId` , `correlationId` , `eventId` , `comercianteId`).

- Métricas de domínio: `consolidado.eventos.processados`, `consolidado.eventos.duplicados`, `consolidado.eventos.falhas` e o timer `consolidado.eventos.lag`.
- Traces OTLP. Métricas OTLP ficam desligadas; o scrape é o Prometheus.
- Virtual threads ligadas (`spring.threads.virtual.enabled`).

Planos de execução das consultas críticas: docs/performance.md.

Como rodar

Pré-requisitos locais: Java 25, Docker, e o compose da infra (Postgres, Redis, LocalStack, Keycloak) sobe por outro componente.

```
cd services/consolidado-service
export SPRING_PROFILES_ACTIVE=local
export DB_URL=jdbc:postgresql://localhost:5432/consolidado
export DB_USERNAME=consolidado
export DB_PASSWORD=consolidado
export AWS_REGION=us-east-1
export AWS_ENDPOINT_URL=http://localhost:4566
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export CONSOLIDADO_QUEUE_URL=http://localhost:4566/000000000000/consolidado-lancamentos-queue
export JWT_ISSUER_URI=http://localhost:8080/realms/fluxo-caixa
export JWT_JWK_SET_URI=http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs
export REDIS_HOST=localhost
export REDIS_PORT=6379
export REDIS_SSL=false
export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
./mvnw spring-boot:run
```

Imagem (usuário não-root `app`, com `apt-get upgrade` na build):

```
docker build -t consolidado-service:1.0.0 services/consolidado-service
```

Testes

```
./mvnw verify
```

Sobe PostgreSQL 17, Redis 7 e LocalStack (SQS e, no teste do segredo, Secrets Manager) via Testcontainers. Cobre saldo, idempotência, estorno, isolamento entre comerciantes, traceparent conhecido no log JSON, circuito aberto sem derrubar readiness, seleção do driver e Flyway com o wrapper, e roteamento leitura/escrita.

JaCoCo exige 80% de linhas em `domain` e `application`.

FONTE: SERVICES/CONSOLIDADO-SERVICE/DOCS/PERFORMANCE.MD

Desempenho das consultas do consolidado

Medido em PostgreSQL 17 (imagem `postgres:17-alpine`), cache de buffers quente, sem concorrência. Volume sintético: **20 comerciantes × 1.097 dias** (2023-11-10 a 2026-11-10) = **21.940 linhas**. O script reproduzível está em explain-saldos.sql. Os tempos absolutos são de um container local; o que vale levar para a RDS é o formato do plano.

A PK `(comerciante_id, data)` é o único índice. As duas consultas críticas usam essa chave. Não há N+1: o dia é uma agregação e o período é uma sentença.

Saldo do dia e acumulado

Uma varredura `comerciante_id = ? AND data <= ?`, com `FILTER` para o movimento do dia e `SUM` para o acumulado.

```
Aggregate  (actual time=0.591..0.592 rows=1)
  -> Bitmap Heap Scan on saldos_diarios  (actual time=0.068..0.183 rows=1097)
       Heap Blocks: exact=12
       -> Bitmap Index Scan on pk_saldos_diarios  (actual time=0.058..0.058 rows=1097)
            Index Cond: (comerciante_id = ? AND data <= ?)
Planning Time: 0.643 ms
Execution Time: 0.674 ms
Buffers: shared hit=19
```

1.097 dias de um comerciante, 19 páginas em cache, **0,67 ms**.

Relatório do período

Uma sentença: soma anterior (`data < inicio`) na PK, dias do intervalo na PK, `generate_series` para as lacunas e `SUM(...) OVER (ORDER BY data)` para o saldo acumulado. Período de 31 dias:

```
WindowAgg  (actual time=1.315..1.388 rows=31)
  -> Nested Loop
       -> Merge Left Join
            -> Index Scan using pk_saldos_diarios
                 Index Cond: (comerciante_id = ? AND data >= ? AND data <= ?)
                 (actual time=0.025..0.032 rows=31)
       -> Aggregate  (actual time=1.191..1.193 rows=1 loops=1)
            -> Bitmap Index Scan on pk_saldos_diarios
                 Index Cond: (comerciante_id = ? AND data < ?)
                 (rows=1066)
Planning Time: 0.460 ms
Execution Time: 1.607 ms
Buffers: shared hit=22
```

O intervalo de 31 dias é index scan direto (4 páginas). O custo dominante é a soma de tudo que veio antes (~1,2 ms para 1.066 dias). A window function em si ordena 31 linhas. **1,61 ms** no total.

Pool e timeout

Por task (consolidado em prod é 1 vCPU): Hikari de escrita com no máximo 10 conexões, `connection-timeout` 3 s, `max-lifetime` 15 min, `keepalive` 5 min. Cada conexão nova executa `SET statement_timeout = '3s'`. As consultas medidas ficam duas a três ordens de grandeza abaixo desse teto; o timeout existe para cortar varredura acidental. Com réplica (`DB_READER_URL`), o pool de leitura repete o mesmo tamanho e é `readOnly` .

10 tasks × 10 conexões de escrita ≈ 100 no primário. Um `db.t4g.medium` comporta isso com folga, inclusive se a réplica estiver ligada.

O que não fazer agora

O plano já é range scan na PK. Índice extra, partição ou saldo materializado não mudam o p95 enquanto o cache de 30 s absorve a leitura e o histórico por comerciante cabe em ~1.000 linhas.

Quando o volume crescer

- **Índice de cobertura** `ON saldos_diarios (comerciante_id, data) INCLUDE (total_creditos, total_debitos, quantidade_lancamentos, atualizado_em)` se o `Bitmap Heap Scan` passar a dominar (hoje são 12 páginas). A PK não cobre as colunas somadas, então o plano busca o heap.
- **Snapshot mensal** (`comerciante_id` , `ano_mes` , `saldo_acumulado`) quando um comerciante passar de ~10 mil dias ou o `EXPLAIN` do acumulado passar de ~10 ms. A consulta do dia soma o snapshot do mês anterior mais os dias do mês corrente, em vez do histórico inteiro.
- **Réplica de leitura** (`DB_READER_URL`) quando o primário dividir UPSERT do consumidor com o `SUM` das consultas. O roteamento já manda `@Transactional(readOnly = true)` para a réplica.
- **Particionamento** por mês não ajuda: a PK começa por `comerciante_id` , não por data. Hash de `comerciante_id` só faria sentido com dezenas de milhões de linhas.

FONTE: INFRA/LOCAL/README.MD

Ambiente local

Compose em `infra/local`. Dois perfis:

- `infra` sobe só as dependências, para rodar o serviço pela IDE ou pelo `./mvnw`.
- `completo` sobe dependências e os dois serviços, com build do Dockerfile em `services/<servico>`.

O Compose não tem como expressar "sobe tudo por padrão e um perfil menor por cima". Os alvos do `Makefile` na raiz são o caminho esperado: `make infra-up` e `make up`.

```
make infra-up
make token
make infra-down
```

`make up` falha enquanto `services/*/Dockerfile` não existir.

O que sobe

Serviço	Porta no host	Credencial
postgres 17	5432	bancos `lancamentos` e `consolidado`, usuário = senha = nome do banco. Superusuário `postgres` / `postgres`
redis 7.4	6379	sem senha
localstack	4566	tópico `fluxo-caixa-lancamentos-events`, fila `consolidado-lancamentos-queue`, DLQ com `maxReceiveCount=5`, subscription com raw delivery
keycloak 26.4.7	8080	admin `admin` / `admin`. Realm `fluxo-caixa`
otel-lgtm	3000, 4317, 4318	Grafana `admin` / `admin`

Usuários do realm:

Usuário	Senha	`comerciante_id`
`comerciante1`	`comerciante1`	`11111111-1111-1111-1111-111111111111`
`comerciante2`	`comerciante2`	`22222222-2222-2222-2222-222222222222`

Client público `fluxo-caixa-app`, direct access grants. Escopos default: `lancamentos:write`, `lancamentos:read`, `consolidado:read`. O claim `comerciante_id` vai no access token.

```
infra/local/get-token.sh
infra/local/get-token.sh comerciante2 comerciante2
```

No zsh, `USERNAME=... comando` não funciona: o zsh reserva `USERNAME` para o login do sistema. Use o argumento ou `KC_USERNAME` / `KC_PASSWORD`.

Variáveis dentro do compose

Os serviços usam o hostname da rede do compose (`postgres`, `redis`, `localstack`, `keycloak`, `otel-lgtm`). O issuer continua `http://localhost:8080/realms/fluxo-caixa`, que é o `iss` do token obtido no host. O JWK vem de `JWT_JWK_SET_URI=http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs`. O Keycloak está com `KC_HOSTNAME=http://localhost:8080` e `KC_HOSTNAME_BACKCHANNEL_DYNAMIC=true`.

`AWS_ACCESS_KEY_ID` e `AWS_SECRET_ACCESS_KEY` valem `test`. O LocalStack exige algum valor; isso não existe na task da AWS, que usa a task role.

Na IDE, com `make infra-up`, troque os hostnames por `localhost` e a fila por `http://localhost:4566/000000000000/consolidado-lancamentos-queue`. O resto do contrato está na seção 10 do blueprint.

O script de init do Postgres roda só na primeira subida do volume. Para recriar bancos e realm: `docker compose -f infra/local/docker-compose.yml --profile infra down -v`.

Trace e correlation id no Grafana

Os serviços exportam trace OTLP para o `otel-lgtm` (Tempo). O propagador é o bean da aplicação (W3C e, se o header existir, X-Ray); o Boot não lê `OTEL_PROPAGATORS`. O header `X-Correlation-Id` é da aplicação: volta na resposta e entra no log JSON como `correlationId`, junto com `traceId` e `spanId`.

O Alloy lê o stdout dos containers pelo socket do Docker e entrega no Loki do `otel-lgtm` (`http://otel-lgtm:3100`). No Grafana (`http://localhost:3000`, `admin` / `admin`):

- Tempo: serviço `lancamentos` ou `consolidado`.
- Loki: `{container="fluxo-caixa-lancamentos"} | json | traceId="<id>"` ou `correlationId="<uuid>"`.

O `traceId` do log é o mesmo do Tempo. Sem o Alloy, o trace continua no Tempo e o log fica só no `docker compose logs`.

FONTE: TESTS/LOAD/README.MD

Carga do consolidado

`consolidado-pico.js` segura 50 requisições por segundo no `GET /api/v1/consolidados/diario/{data}` e, em paralelo, um fluxo menor de `POST /api/v1/lancamentos`.

Thresholds:

- `http_req_failed < 0.05` — o RNF de pico. O k6 conta status 0 e `>= 400`.
- p95 do consolidado `< 100` ms — o SLO com cache. Contra um ALB distante, suba o teto: `P95_MS=300`.
- p95 do POST de lançamentos `< 500` ms.

O teste passa se a taxa de falha ficar abaixo de 5% e a latência dentro do teto. Uma falha de threshold no final é o teste reprovando o ambiente, não um erro do script. Olhe `http_req_failed` separado de `http_req_duration`: perda acima de 5% quebra o RNF mesmo com p95 bom. p95 alto com pouca perda aponta cache frio, pool de banco ou pouca task.

```
make load
# ou
k6 run tests/load/consolidado-pico.js
k6 run -e DURATION=5m -e P95_MS=300 -e POST_RATE=5 tests/load/consolidado-pico.js
```

Pelo container, no Mac o host é `host.docker.internal`:

```
docker run --rm -i \
  -e LANCAMENTOS_URL=http://host.docker.internal:8081 \
  -e CONSOLIDADO_URL=http://host.docker.internal:8082 \
  -e TOKEN_URL=http://host.docker.internal:8080/realms/fluxo-caixa/protocol/openid-
       connect/token \
  grafana/k6 run - < tests/load/consolidado-pico.js
```

No Linux, `--network host` e as URLs `localhost` bastam.

Variáveis: `LANCAMENTOS_URL`, `CONSOLIDADO_URL`, `TOKEN_URL`, `DATA_LANCAMENTO`, `DURATION`, `P95_MS`, `POST_RATE`, `USERNAME`, `PASSWORD`, `CLIENT_ID`.

Contra o ALB da AWS, o WAF limita 30000 requisições por IP a cada 5 minutos (cerca de 100 req/s). O pico de 50 req/s de uma única máquina passa. Um teste mais agressivo precisa de mais IPs ou de um limite maior em `waf_rate_limit`.

E2E

`tests/e2e/smoke.sh` pega o token, lança um crédito de 150 e um débito de 40, estorna o débito e espera o saldo do dia fechar em 150.

`tests/e2e/resiliencia.sh` lê o saldo de `2099-01-15`, para o container do consolidado, grava crédito 10 e débito 3 (os dois precisam voltar 201) e, depois de religar, espera o saldo subir 7.

```
make smoke
make resiliencia
```

Os dois precisam dos serviços no ar (`make up`) e de `jq` , `curl` e `uuidgen` .

PARTE 17

Infraestrutura como código (Terraform)

Stack, ambientes dev, hml e prod, bootstrap do state e módulos

FONTE: INFRA/TERRAFORM/README.MD

Terraform — fluxo de caixa na AWS

Terraform `~> 1.14` e provider `hashicorp/aws ~> 6.72`. Região padrão `us-east-1`.

O state de cada ambiente é remoto, no S3, com lock nativo (`use_lockfile = true`). Não há tabela DynamoDB: a partir do Terraform 1.10 o lock é um objeto `.tflock` no próprio bucket. DynamoDB só faria sentido em runtime anterior a 1.10, e cobraria uma tabela à parte sem ganho de isolamento.

Arquitetura

```
Cliente
  -> CloudFront (PriceClass_All, HTTP/2 e HTTP/3, TLS 1.2_2021)
       WAF escopo CLOUDFRONT
       /api/v1/lancamentos* e /api/v1/consolidados* vao ao API Gateway
       o comportamento default devolve 404 na funcao, sem chamar a origem
  -> API Gateway REST regional (stage v1)
       authorizer Cognito com escopo por rota (a Lambda de pre-token v2 grava `comerciante_id`
e os escopos do resource server, então o `USER_PASSWORD_AUTH` passa na borda)
       WAF regional exige X-Origin-Verify (valor atual ou anterior)
       VPC Link v2, HTTP_PROXY, timeout 10 s
  -> ALB interno, sem IP publico
       security group aceita o prefix list do CloudFront e o security group do VPC Link
       rewrite tira o prefixo /v1 que a integracao privada acrescenta
         /api/v1/lancamentos*  -> ECS lancamentos
         /api/v1/consolidados* -> ECS consolidado
  -> tasks Fargate ARM64 nas subnets de aplicacao
       lancamentos -> RDS proprio + SNS
       consolidado -> RDS proprio + SQS + Valkey
  SNS (raw delivery) -> SQS -> DLQ depois de 5 recebimentos
```

`enable_api_gateway = false` tira o gateway, o WAF regional e o VPC Link. O CloudFront volta a usar a VPC Origin no ALB, como na seção 14 do blueprint.

O TLS da internet termina no CloudFront. O trecho API Gateway → ALB é HTTP na VPC: o hostname interno do ALB não cabe num certificado ACM público. A origem do CloudFront para o gateway é HTTPS, com `AllViewerExceptHostHeader`, para o `Host` ser o do `execute-api`. O header `X-Origin-Verify` é custom header da origem e substitui o valor que o cliente mandar.

O provider 6.72 aceita REST API com VPC Link v2 direto no ALB (`aws_apigatewayv2_vpc_link` + `integration_target`). Não há NLB.

A task não recebe `DB_USERNAME` nem `DB_PASSWORD`. Recebe `DB_SECRET_ARN` e `DB_URL=jdbc:postgresql://<host>:5432/<db>`. O serviço acrescenta um único prefixo `jdbc:aws-wrapper:` quando o segredo existe. Se o host for o do RDS Proxy (`.proxy-`), o plugin `failover` fica de fora. A task role lê só esse ARN (`GetSecretValue`, `DescribeSecret`) e decifra a CMK do segredo. `DB_READER_URL` usa o mesmo esquema e só entra quando existe reader.

O sidecar ADOT entrega trace no X-Ray. A aplicação propaga `tracecontext` (W3C) e `xray`.

Árvore

```
infra/terraform
├── bootstrap/                bucket de state, CMK, lifecycle, Object Lock opcional
├── stacks/fluxo-caixa/       composicao. Toda a logica mora aqui.
├── envs/dev|hml|prod/        so chamam a composicao, com tfvars e backend.hcl
└── modules/
    ├── network
    ├── security/kms
    ├── ecr
    ├── ecs-cluster
    ├── ecs-service
    ├── alb
    ├── edge                  CloudFront, VPC Origin e WAF CLOUDFRONT
    ├── api-gateway           REST regional, VPC Link v2, WAF regional, rotacao do header
    ├── rds-postgres
    ├── elasticache
    ├── messaging
    ├── cognito
    └── observability
```

Nome de recurso: `fluxo-caixa-<env>-<recurso>`. Tags padrão do provider: `Project`, `Environment`, `Owner`, `CostCenter`, `ManagedBy`.

Contas separadas são opcionais. `assume_role_arn` no `terraform.tfvars` faz o provider assumir a role da conta do ambiente. O bootstrap roda uma vez em cada conta que guarda state. Sem a variável, o Terraform usa a credencial corrente.

Dimensionamento

Os números da seção 15 do blueprint são o default da composição e estão repetidos no `terraform.tfvars.example` de cada ambiente. Qualquer campo do objeto `sizing` pode ser sobrescrito. Em prod, um `check` recusa plano sem 3 AZs, NAT por AZ, interface endpoints nas 3 AZs, RDS Multi-AZ e deletion protection.

	dev	hml	prod
AZs	2	2	3
NAT	1	1	3
Interface endpoints	0 (sai pelo NAT)	2 AZs	3 AZs
Fargate	Spot, 0,5 vCPU / 1 GB, min 1 max 2	on-demand	on-demand, min 2 max 10
consolidado	0,5 vCPU / 1 GB	1 vCPU / 2 GB, max 3	1 vCPU / 2 GB
RDS (um por serviço)	db.t4g.micro, 20 GB, 1 dia	db.t4g.small, 20 GB, 7 dias	db.t4g.medium, 50 GB, Multi-AZ, 14 dias, deletion protection

	dev	hml	prod
Rotação da senha	90 dias	30 dias	30 dias
Réplica / RDS Proxy	0 / não	0 / não	0 / não, parametrizável
Valkey	cache.t4g.micro, 1 nó	cache.t4g.small, 1 nó	cache.t4g.small, primário + réplica
Logs	7 dias	14 dias	30 dias
Container Insights	desligado	ligado	enhanced
Monitoring do RDS	não	não	60 s

O gateway de S3 existe nos três ambientes. Não tem custo horário e tira a camada de imagem do NAT.

Bootstrap e state

O bootstrap é local de propósito: ele cria o bucket que os outros states vão usar. O bucket tem versionamento, SSE-KMS, block public access, policy que recusa transporte sem TLS, lifecycle das versões antigas e Object Lock opcional (`enable_object_lock` , modo GOVERNANCE). Object Lock só pode ser ligado na criação do bucket.

```
cd infra/terraform/bootstrap
cp terraform.tfvars.example terraform.tfvars
terraform init
terraform apply

terraform output -raw backend_hcl_dev  > ../envs/dev/backend.hcl
terraform output -raw backend_hcl_hml  > ../envs/hml/backend.hcl
terraform output -raw backend_hcl_prod > ../envs/prod/backend.hcl
```

O `backend "s3" {}` do ambiente é parcial. Bucket, key `fluxo-caixa/<env>/terraform.tfstate` , region, `encrypt` , `kms_key_id` e `use_lockfile` ficam no `backend.hcl` , que não entra no git.

```
make tf-init ENV=dev
make tf-plan ENV=dev AWS_PROFILE=aquino-dev
make tf-validate
make tf-test
```

`tf-validate` inicializa cada raiz num `TF_DATA_DIR` temporário e não lê o state, mesmo que o `.terraform/` do ambiente já aponte para o S3. `tf-init` e `tf-plan` usam o backend do ambiente e respeitam `AWS_PROFILE` do comando ou do ambiente. `tf-plan` grava `<env>.tfplan` . Não aplique produção sem esse arquivo revisado e sem aprovação. O bucket de state tem `prevent_destroy` .

Promoção entre ambientes é de imagem, não de state. O CD constrói uma vez (tag = SHA), publica no ECR de dev, assina com cosign e, em hml/prod, copia o mesmo digest. O Terraform de cada ambiente continua dono da task definition; alinhe `image_tag` antes do próximo apply para ele não voltar uma revisão antiga.

Rotação de segredos

O RDS cria o secret (`manage_master_user_password`) e já associa a função de rotação gerenciada. O provider 6.72 não tem agenda no `aws_db_instance` . `aws_secretsmanager_secret_rotation` só ajusta `automatically_after_days` e deixa `rotation_lambda_arn` de fora, para não substituir a função do RDS. `rotate_immediately = false` evita rotacionar a senha em todo apply.

A task recebe `MANAGEMENT_TRACING_SAMPLING_PROBABILITY` : 1.0 em dev, 0.1 em hml e prod. Isso só vale para trace sem pai. O header `X-Amzn-Trace-Id` com `Sampled` continua decidindo o filho. O processo Java não lê `OTEL_PROPAGATORS` .

A CMK do secret é outra, distinta da CMK do storage. Toda CMK do projeto tem `enable_key_rotation = true` . São nove em cada ambiente: storage e secret de cada RDS, ElastiCache, mensageria, logs, ECR e `origin-verify` . O access log do ALB fica em SSE-S3. O ELB não grava nesse bucket com CMK, então não há chave `alb-logs` .

EventBridge escuta `AWS Secrets Manager Rotation Failed` e os eventos de CloudTrail `RotationFailed` / `RotationAbandoned` , e publica no tópico de alertas. O segredo `X-Origin-Verify` entra nesse mapa quando o API Gateway está ligado.

API Gateway

Módulo `modules/api-gateway` . REST API regional, stage `v1` , authorizer Cognito, VPC Link v2 no ALB interno, WAF regional e rotação do header. O POST de lançamento valida corpo e parâmetros. O POST de estorno só valida parâmetros: o corpo é opcional, e `motivo` , quando vem, é limitado no serviço. O detalhe da decisão está no ADR 0031.

Alternar `enable_api_gateway`

O default é `true` nos três ambientes. Para voltar ao caminho CloudFront → VPC Origin → ALB:

```
enable_api_gateway = false
```

no `terraform.tfvars` do ambiente. O módulo do gateway não é criado. O CloudFront manda `/api/v1/*` para a origem `alb` . O listener não reescreve `/v1` .

Cadastrar consumidor com API key

Cliente final usa só JWT. API key é para integração. No tfvars:

```
api_consumers = {
  pdv = {
    description    = "PDV da loja"
    quota_limit    = 100000
    quota_period   = "MONTH"
    throttle_rate  = 10
    throttle_burst = 20
  }
}

api_key_required = {
```

```
  "POST /api/v1/lancamentos" = true
}
```

`quota_period` é `DAY`, `WEEK` ou `MONTH`. A chave é gerada pela AWS e fica no state; o output publica só o id. Sem `api_key_required`, a chave pode ser enviada para cair no usage plan, mas a rota não exige. Com o mapa acima, o POST de lançamento exige a chave e o JWT.

Throttle do consolidado: 20 req/s em dev, 80 em hml, 100 em prod. hml e prod ficam acima do pico de 50. O módulo recusa o contrário quando `require_consolidado_above_peak` está ligado, o que a composição faz fora de dev.

Rotação do `X-Origin-Verify`

O segredo `${name_prefix}/origin-verify` guarda `current` e `previous`, cifrado na CMK `origin-verify`. A Lambda `rotate_origin_verify` implementa `createSecret`, `setSecret`, `testSecret` e `finishSecret`. No `setSecret` ela move o current antigo para `previous`, grava o valor novo e atualiza o custom header da origem `api-gateway` (a distribuição é a que tem a tag `Name` do ambiente) e os dois byte matches da regra `x-origin-verify`. O valor anterior continua aceito até a rotação seguinte. `rotate_immediately = false`. A agenda é `origin_verify_rotation_days`: 90 dias em dev, 30 em hml e prod.

O Terraform não fica preso à versão que criou. O bootstrap usa `ephemeral.random_password` em `secret_string_wo`, com `secret_string_wo_version = 1`. Esse número é o gatilho de reenvio: a senha efêmera muda a cada plan, mas o provider só grava de novo se a versão mudar. Não use `ignore_changes` na distribuição nem na Web ACL: uma regra nova ou um behavior novo precisa continuar aparecendo no plan. CloudFront e o WAF leem `current` e `previous` de `data.aws_secretsmanager_secret_version` com stage `AWSCURRENT`. No primeiro apply o data source espera a versão existir (`depends_on`), sem ciclo. No plan seguinte o valor vigente entra na configuração. Se a Lambda já atualizou a borda, o plan fica vazio. Se só o segredo girou, o apply empurra CloudFront e o WAF para o mesmo par, em vez de reverter para o bootstrap.

O que ainda fica no state: o par lido pelo data source, o custom header da origem do CloudFront e os `search_string` da regra do WAF. O header na distribuição é inevitável. O `secret_string` do resource da versão não fica no state. Não aplique no meio de uma rotação: até o `finishSecret`, `AWSCURRENT` ainda é o valor anterior, e o apply pode desfazer o header que o `setSecret` já gravou.

Réplica e RDS Proxy

`sizing.read_replica_count` default 0. `sizing.enable_rds_proxy` default false.

A AWS não cria read replica de PostgreSQL quando a senha mestre é gerenciada pelo Secrets Manager (limitação). A variável `read_replica_count` valida a combinação, e a instância repete a regra numa precondition. O plano falha com mensagem que aponta o Multi-AZ DB Cluster, o Aurora ou a rotação por Lambda. Enquanto `read_replica_count` for 0, `DB_READER_URL` não é definida e a leitura vai ao primário. O gatilho e a recomendação (Multi-AZ DB Cluster) estão no ADR 0030.

RDS Proxy não depende de réplica. Com `enable_rds_proxy = true`, `DB_URL` aponta para o endpoint do proxy (`jdbc:postgresql://…proxy-…`) e a task alcança o security group do proxy, que por sua vez alcança o banco. O proxy também lê o secret gerenciado pelo RDS. O serviço reconhece `.proxy-` e não liga o plugin `failover` contra esse endpoint.

Contrato da task

`AWS_ENDPOINT_URL` não é definida na AWS. `OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318` fala com o sidecar no mesmo network namespace. O health check do target group é `GET /actuator/health/readiness`.

Validação

```
terraform fmt -recursive -check infra/terraform
make tf-validate
make tf-test
```

Nada disso faz apply. `tflint` e `trivy config` rodam no workflow quando as ferramentas existem. O pre-commit está em `.pre-commit-config.yaml`.

FONTE: INFRA/TERRAFORM/BOOTSTRAP/README.MD

bootstrap

Bucket de state com KMS, versionamento e lock nativo do S3.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "bootstrap" {
  source = "./bootstrap"
}
```

Inputs

Nome	Tipo	Descricao
`aws_region`	`string`	Regiao do bucket de state. Um bucket por conta; a key separa dev, hml e prod.
`assume_role_arn`	`string`	Role opcional assumida pelo provider. Use quando o bootstrap roda a partir de outra conta.
`owner`	`string`	Tag Owner.
`cost_center`	`string`	Tag CostCenter.
`bucket_name`	`string`	Nome globalmente unico do bucket de state. Sugestao: fluxo-caixa-tfstate-<account_id>.
`state_key_deletion_window_days`	`number`	Janela de exclusao da CMK do bucket de state.
`noncurrent_version_expiration_days`	`number`	Dias ate expirar versoes antigas do state. A versao corrente permanece.
`enable_object_lock`	`bool`	Object Lock GOVERNANCE no bucket. So pode ser ligado na criacao do bucket. Padrao false.
`object_lock_retention_days`	`number`	Retencao GOVERNANCE quando enable_object_lock e true. GOVERNANCE ainda permite apagar com permissao explicita, entao um apply quebrado nao prende o state para sempre.

Outputs

Nome	Descricao
`state_bucket`	Bucket S3 do state remoto.

Nome	Descricao
`state_kms_key_arn`	CMK usada pelo bucket de state.
`backend_hcl`	backend.hcl por ambiente. A key isola o state. O bucket pode ser um por conta.
`backend_hcl_dev`	Conteudo sugerido para envs/dev/backend.hcl.
`backend_hcl_hml`	Conteudo sugerido para envs/hml/backend.hcl.
`backend_hcl_prod`	Conteudo sugerido para envs/prod/backend.hcl.

FONTE: INFRA/TERRAFORM/STACKS/FLUXO-CAIXA/README.MD

fluxo-caixa

Composicao da plataforma. Os ambientes so passam sizing e identidade.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "fluxo_caixa" {
  source = "./stacks/fluxo-caixa"
}
```

Inputs

Nome	Tipo	Descricao
`environment`	`string`	Ambiente. Entra no prefixo fluxo-caixa--.
`aws_region`	`string`	Regiao da carga. A borda CloudFront e WAF continua em us-east-1.
`alert_email`	`string`	E-mail dos alarmes. Vazio deixa o topico sem inscricao.
`image_tag`	`string`	Tag imutavel publicada no ECR.
`acm_certificate_arn`	`string`	Certificado ACM us-east-1 do CloudFront. Nulo usa *.cloudfront.net.
`domain_names`	`list(string)`	Aliases do CloudFront. Vazio junto com acm_certificate_arn nulo.
`cognito_domain_prefix`	`string`	Prefixo do dominio hospedado do Cognito. Nulo usa fluxo-caixa--<account_id>.
`adot_image`	`string`	Imagem do ADOT collector, com tag fixa e suporte a ARM64.
`rds_engine_version`	`string`	Versao menor do PostgreSQL 17.
`redis_engine_version`	`string`	Versao do Valkey.
`lancamentos_request_count_target`	`number`	Requests por minuto por task de lancamentos.
`consolidado_request_count_target`	`number`	Requests por minuto por task do consolidado. 1500 cobre cerca de 25 req/s.
`consolidado_queue_backlog_per_task`	`number`	Mensagens visiveis por task antes de subir mais tasks.
`sizing`	`object({`	Sobrescreve o dimensionamento da secao 15. Atributo nulo herda o default do ambiente.

Outputs

Nome	Descricao
`environment`	Ambiente deste state.
`cloudfront_domain_name`	Entrada publica. Use este hostname enquanto nao houver dominio proprio.
`cloudfront_distribution_id`	ID da distribuicao CloudFront.
`alb_dns_name`	DNS interno do ALB. Nao e a URL publica.
`ecr_repository_urls`	Repositorios ECR.
`cognito_user_pool_id`	User pool.
`cognito_user_pool_client_id`	App client.
`cognito_issuer_uri`	Issuer dos access tokens.
`cognito_domain`	Prefixo do dominio hospedado do Cognito.
`cognito_token_endpoint`	Token endpoint OAuth do Cognito.
`lancamentos_topic_arn`	Topico SNS dos eventos de lancamento.
`consolidado_queue_url`	Fila consumida pelo consolidado.
`consolidado_dlq_url`	DLQ do consolidado.
`ecs_cluster_name`	Cluster ECS.
`rds_endpoints`	Hosts que a aplicacao usa. Proxy quando enable_rds_proxy.
`rds_reader_endpoints`	Hosts de leitura. Nulos enquanto read_replica_count for 0.
`rds_secret_arns`	Secrets que a task le via DB_SECRET_ARN.
`redis_endpoint`	Endpoint de escrita do Valkey.
`alert_topic_arn`	Topico dos alarmes e das falhas de rotacao.
`sizing`	Dimensionamento efetivo, depois dos defaults da secao 15 e das sobrescritas.

FONTE: INFRA/TERRAFORM/MODULES/NETWORK/README.MD

network

VPC, subnets publicas/app/dados, NAT e VPC endpoints.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "network" {
  source = "./modules/network"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo dos recursos de rede. Ex.: fluxo-caixa-dev.
`region`	`string`	Regiao dos VPC endpoints.
`vpc_cidr`	`string`	CIDR da VPC. O desenho usa /24 derivados deste bloco.
`az_count`	`number`	Quantidade de AZs. Dev e hml usam 2. Prod usa 3.
`nat_gateway_per_az`	`bool`	true cria um NAT por AZ (prod). false cria um unico NAT (dev).
`interface_endpoint_az_count`	`number`	AZs com interface endpoint. 0 faz a task sair para as APIs AWS pelo NAT. Hml usa 2 e prod usa 3.
`logs_kms_key_arn`	`string`	CMK dos log groups, inclusive o de VPC flow logs.
`log_retention_days`	`number`	Retencao dos flow logs.
`flow_log_traffic_type`	`string`	Tipo de trafego do flow log. REJECT reduz custo; ALL e mais util em prod.

Outputs

Nome	Descricao
`vpc_id`	ID da VPC.
`vpc_cidr`	CIDR da VPC.

Nome	Descricao
`availability_zones`	AZs usadas, na ordem estavel do slice.
`public_subnet_ids`	Subnets publicas (ALB e NAT), uma por AZ.
`app_subnet_ids`	Subnets privadas de aplicacao (ECS).
`data_subnet_ids`	Subnets privadas de dados, sem rota para a internet.
`endpoint_security_group_id`	Security group dos interface endpoints. Nulo quando interface_endpoint_az_count e 0.

FONTE: INFRA/TERRAFORM/MODULES/SECURITY/KMS/README.MD

kms

CMKs com rotacao anual. Nove chaves, iguais em dev, hml e prod: storage e secret de cada RDS, ElastiCache, mensageria, logs, ECR e origin-verify. O access log do ALB usa SSE-S3 e nao tem CMK. Uma chave separa o secret da senha do storage do RDS.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "kms" {
  source = "./modules/security/kms"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo dos aliases. Ex.: fluxo-caixa-dev.
`region`	`string`	Regiao usada nas conditions kms:ViaService e logs.
`deletion_window_days`	`number`	Janela de exclusao das CMKs. 7 em dev, 30 em prod.

Outputs

Nome	Descricao
`key_arns`	ARNs das CMKs, indexados por finalidade.
`key_ids`	IDs das CMKs, indexados por finalidade.

FONTE: INFRA/TERRAFORM/MODULES/ECR/README.MD

ecr

Repositorios imutaveis, scan on push, criptografia KMS.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "ecr" {
  source = "./modules/ecr"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo dos repositorios. Ex.: fluxo-caixa-dev.
`repository_names`	`list(string)`	Nomes logicos dos repositorios, sem o prefixo.
`kms_key_arn`	`string`	CMK das imagens.
`image_count_to_keep`	`number`	Quantidade de imagens mantidas pela lifecycle policy.

Outputs

Nome	Descricao
`repository_urls`	URLs dos repositorios, indexadas pelo nome logico.
`repository_arns`	ARNs dos repositorios, indexados pelo nome logico.

FONTE: INFRA/TERRAFORM/MODULES/ECS-CLUSTER/README.MD

ecs-cluster

Cluster Fargate com FARGATE e FARGATE_SPOT e Container Insights configuravel.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "ecs_cluster" {
  source = "./modules/ecs-cluster"
}
```

Inputs

Nome	Tipo	Descricao
`name`	`string`	Nome do cluster ECS.
`container_insights`	`string`	containerInsights do cluster: disabled em dev, enabled em hml, enhanced em prod.

Outputs

Nome	Descricao
`cluster_id`	ID do cluster.
`cluster_name`	Nome do cluster.
`cluster_arn`	ARN do cluster.

FONTE: INFRA/TERRAFORM/MODULES/ECS-SERVICE/README.MD

ecs-service

Task ARM64, ADOT, security group minimo, scaling e circuit breaker.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "ecs_service" {
  source = "./modules/ecs-service"
}
```

Inputs

Nome	Tipo	Descricao
`name`	`string`	Nome do servico ECS e prefixo das roles. Ex.: fluxo-caixa-dev-lancamentos.
`container_name`	`string`	Nome do container da aplicacao. O CD usa este nome no render da task definition.
`cluster_arn`	`string`	ARN do cluster.
`cluster_name`	`string`	Nome do cluster, usado no scaling da fila.
`vpc_id`	`string`	VPC das tasks.
`vpc_cidr`	`string`	CIDR da VPC, liberado apenas para DNS.
`subnet_ids`	`list(string)`	Subnets de aplicacao. Uma task por AZ quando desired_count acompanha a quantidade de AZs.
`container_image`	`string`	Imagem no ECR, com tag imutavel.
`container_port`	`number`	Porta HTTP do Spring Boot.
`cpu`	`number`	CPU da task Fargate. 256, 512, 1024...
`memory`	`number`	Memoria da task em MiB, compativel com a CPU.
`desired_count`	`number`	Tasks iniciais. O autoscaling assume o valor depois; o Terraform ignora mudancas posteriores.
`min_capacity`	`number`	Minimo do autoscaling. Em prod, 2 para espalhar entre AZs.
`max_capacity`	`number`	Maximo do autoscaling.
`environment`	`map(string)`	Variaveis da aplicacao, sem segredo. O Boot nao le `OTEL_PROPAGATORS`; o bean Java compoe W3C e X-Ray. O sidecar ADOT continua com `tracecontext,baggage,xray`.

Nome	Tipo	Descricao
secrets	list(object({	Segredos injetados pelo ECS a partir do Secrets Manager. valueFrom pode apontar para uma chave JSON.
secret_arns	list(string)	ARNs puros dos secrets, para o IAM da execution role.
execution_kms_key_arns	list(string)	CMKs que a execution role precisa decifrar (secret do RDS e logs).
task_policy_statements	list(object({	Permissoes extras da task role. Lancamentos publica no SNS; consolidado consome a fila.
log_group_name	string	Log group da aplicacao e do sidecar.
log_group_arn	string	ARN do log group da aplicacao.
metrics_log_group_name	string	Log group das metricas EMF.
metrics_log_group_arn	string	ARN do log group de metricas.
aws_region	string	Regiao usada pelo awslogs e pelo ADOT.
target_group_arn	string	Target group do ALB.
alb_security_group_id	string	Security group do ALB.
vpc_endpoint_security_group_id	string	Security group dos interface endpoints. Nulo abre HTTPS 443 para a internet (NAT), usado em dev.
capacity_provider_strategy	list(object({	Capacity providers da task. Dev usa FARGATE_SPOT. Hml e prod usam FARGATE.
task_secret_arns	list(string)	Secrets que a task role le em runtime (AWS JDBC Wrapper). Nao vao para a execution role.
task_secret_kms_key_arns	list(string)	CMKs dos secrets lidos pela task role. A policy concede apenas kms:Decrypt.
data_plane_access	list(object({	Saida para RDS ou Redis do proprio servico. O modulo tambem abre o ingress no SG de destino.
readonly_root_filesystem	bool	Filesystem raiz somente leitura, com /tmp montado a parte. No Fargate esse volume nasce como root; a imagem faz chmod 1777 e desce para o usuario app.
enable_execute_command	bool	Liga o ECS Exec. Util em dev, desligado em prod.
enable_request_count_scaling	bool	Target tracking de ALBRequestCountPerTarget.
request_count_target	number	Requisicoes por minuto por task. 1500 equivale a cerca de 25 req/s.
alb_resource_label	string	arn_suffix do ALB + / + arn_suffix do target group.

Nome	Tipo	Descricao
`cpu_target`	`number`	Percentual de CPU alvo do target tracking.
`queue_name`	`string`	Fila usada no scaling por backlog. Nulo desliga esse scaling.
`queue_backlog_per_task`	`number`	Mensagens visiveis por task. Nulo desliga o scaling da fila.
`adot_image`	`string`	Imagem do ADOT collector, com tag fixa e suporte a ARM64.
`health_check_grace_period_seconds`	`number`	Tempo para o Spring Boot subir antes do ALB contar falha de readiness.

Outputs

Nome	Descricao
`service_name`	Nome do servico ECS.
`service_arn`	ARN do servico ECS.
`task_definition_arn`	ARN da task definition gerenciada pelo Terraform.
`task_role_arn`	Task role. O nome e previsivel para a policy de SNS/SQS.
`task_role_name`	Nome da task role.
`security_group_id`	Security group das tasks.

FONTE: INFRA/TERRAFORM/MODULES/ALB/README.MD

alb

ALB interno. O security group aceita so o prefix list do CloudFront.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "alb" {
  source = "./modules/alb"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo do ALB, dos target groups e do bucket de logs.
`vpc_id`	`string`	VPC do ALB.
`subnet_ids`	`list(string)`	Subnets privadas do ALB interno, uma por AZ. O CloudFront chega por VPC Origin.
`enable_waf`	`bool`	WAF regional no ALB. Padrao false: o ALB e interno e o WAF de borda (escopo CLOUDFRONT) ja inspeciona o cliente. Um WAF regional veria so o IP do CloudFront e cobraria de novo.
`deletion_protection`	`bool`	Protege o ALB contra delete. true em prod.
`access_logs_bucket_name`	`string`	Nome globalmente unico do bucket de access logs. O bucket usa SSE-S3: o ALB nao grava em bucket com CMK.
`access_log_expiration_days`	`number`	Dias ate expirar o access log.
`waf_rate_limit`	`number`	Requisicoes por IP em 5 minutos. 50 req/s sustentados geram 15000, entao o teto fica acima disso.
`services`	`map(object({`	Target groups e regras de caminho.

Outputs

Nome	Descricao
`alb_arn`	ARN do ALB.
`alb_arn_suffix`	Sufixo usado em metricas e no scaling por request count.

Nome	Descricao
`alb_dns_name`	DNS interno do ALB. O CloudFront usa este nome na VPC Origin.
`alb_zone_id`	Zone ID do ALB, para um alias no Route 53.
`alb_url`	URL HTTP interna do ALB. A URL publica e a do CloudFront.
`security_group_id`	Security group do ALB.
`target_group_arns`	ARNs dos target groups, indexados pelo servico.
`target_group_arn_suffixes`	Sufixos dos target groups.
`waf_arn`	ARN do Web ACL regional. Nulo quando enable_waf e false.
`access_logs_bucket`	Bucket dos access logs.

FONTE: INFRA/TERRAFORM/MODULES/EDGE/README.MD

edge

CloudFront, VPC Origin, WAF CLOUDFRONT e headers de seguranca.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "edge" {
  source = "./modules/edge"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo dos recursos de borda. Ex.: fluxo-caixa-dev.
`alb_arn`	`string`	ARN do ALB interno usado como VPC Origin.
`alb_dns_name`	`string`	DNS do ALB interno. O CloudFront usa este hostname na origem.
`acm_certificate_arn`	`string`	Certificado ACM em us-east-1. Nulo usa o certificado padrao *.cloudfront.net.
`domain_names`	`list(string)`	Aliases da distribuicao. Vazio quando o certificado padrao do CloudFront e usado.
`logs_kms_key_arn`	`string`	CMK do log group do WAF. A chave precisa estar em us-east-1, mesma regiao do escopo CLOUDFRONT.
`log_retention_days`	`number`	Retencao dos logs do WAF.
`waf_rate_limit`	`number`	Requisicoes por IP em 5 minutos no WAF de borda.
`anonymous_ip_count_only`	`bool`	AnonymousIpList em count (dev) em vez de block. Count deixa o trafego passar e so mede.

Outputs

Nome	Descricao
`distribution_id`	ID da distribuicao CloudFront.
`distribution_domain_name`	Dominio *.cloudfront.net. E a URL publica quando nao ha alias.
`distribution_arn`	ARN da distribuicao.

Nome	Descricao
`waf_arn`	ARN do Web ACL de escopo CLOUDFRONT.

FONTE: INFRA/TERRAFORM/MODULES/API-GATEWAY/README.MD

api-gateway

REST API regional na frente do ALB interno. CloudFront é a URL pública. Este módulo é o authorizer, o throttle, o validator e a trava contra quem chama o `execute-api` direto.

terraform-docs não estava instalado. Esta página lista o contrato do módulo.

O que cria

- REST API `REGIONAL`, stage `v1`, só as rotas da seção 7 do blueprint.
- Authorizer `COGNITO_USER_POOLS` com escopo por método.
- Models JSON Schema para o POST de lançamento e o POST de estorno.
- VPC Link v2 (`aws_apigatewayv2_vpc_link`) com `integration_target` no ARN do ALB. Timeout 10 s. Sem NLB.
- Usage plan e API key quando `api_consumers` não é vazio.
- WAF regional no stage: managed rules e a regra `x-origin-verify`.
- Segredo no Secrets Manager, CMK recebida de fora, Lambda de rotação.

Integração privada

O provider 6.72 aceita VPC Link v2 na REST API. `connection_id` é o id do `aws_apigatewayv2_vpc_link`. `integration_target` é o ARN do ALB. A URI define o Host e o path, inclusive `{id}` e `{data}`.

A AWS inclui o nome do stage no path que chega ao backend. A composição liga `stage_path_prefix = "v1"` no ALB, e o listener reescreve `^/v1(/.*)$` para `$1`. Este módulo não faz essa reescrita.

O security group do VPC Link só sai na porta 80 para o security group do ALB. O módulo abre essa entrada no ALB.

Rotação

O valor inicial nasce em `ephemeral.random_password` e entra no Secrets Manager por `secret_string_wo`. `secret_string_wo_version` fica em 1: a senha efêmera não é regravada a cada plan. CloudFront e o WAF não leem essa versão. Leem `data.aws_secretsmanager_secret_version` em `AWSCURRENT`, que depois da Lambda é o par vigente. Não há `ignore_changes` na distribuição nem na Web ACL.

O header continua no state da distribuição, o par continua no state do data source e nas strings da regra do WAF. O resource da versão do segredo não guarda `secret_string`.

Estorno

O validator do POST de estorno é só de parâmetros. Corpo ausente passa. O model documenta `motivo` opcional; quem limita o tamanho é o serviço.

Inputs principais

Nome	Descrição
`name_prefix`	Prefixo. Ex.: `fluxo-caixa-dev`.
`alb_arn`, `alb_dns_name`, `alb_security_group_id`	ALB interno.
`subnet_ids`	Subnets do VPC Link, no mínimo duas.
`cognito_user_pool_arn`	Authorizer.
`scope_prefix`	Padrão `fluxo-caixa/`.
`secret_kms_key_arn`	CMK do header.
`stage_throttling`, `route_throttling`	Rate e burst. Fora de dev o consolidado precisa passar de 50 req/s.
`cache_enabled`	Padrão `false`.
`canary`	Nulo não cria. O canary usa o deployment corrente.
`api_key_required`	Mapa `METHOD /path` → bool. Padrão false.
`api_consumers`	Usage plan. Padrão vazio.
`rotation_days`	Agenda do header.

Outputs

Nome	Descrição
`invoke_url`	URL regional do stage. Não é a URL pública.
`api_id`	ID da REST API.
`stage_name`	`v1`.
`regional_domain_name`	Host que o CloudFront usa como origem.
`origin_verify_header_value`	Valor `current`. Sensível.
`origin_verify_secret_arn`	ARN do segredo.
`api_key_ids`	IDs das chaves. O valor fica no state.

FONTE: INFRA/TERRAFORM/MODULES/RDS-POSTGRES/README.MD

rds-postgres

PostgreSQL 17, senha no Secrets Manager, parametros de observabilidade, replica e proxy opcionais.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "rds_postgres" {
  source = "./modules/rds-postgres"
}
```

Inputs

Nome	Tipo	Descricao
`identifier`	`string`	Identificador da instancia RDS.
`db_name`	`string`	Database inicial. Um banco por servico.
`username`	`string`	Usuario mestre. A senha fica no Secrets Manager (manage_master_user_password).
`engine_version`	`string`	Versao menor do PostgreSQL 17 disponivel na regiao.
`instance_class`	`string`	Classe da instancia. Dev usa burstable pequena; prod pode subir sem trocar o modulo.
`allocated_storage`	`number`	Storage inicial em GiB. gp3 exige pelo menos 20.
`max_allocated_storage`	`number`	Teto do autoscaling de storage.
`multi_az`	`bool`	Multi-AZ. true em prod, false em dev.
`deletion_protection`	`bool`	Impede o delete da instancia. true em prod.
`skip_final_snapshot`	`bool`	Pula o snapshot final. true somente em dev.
`backup_retention_days`	`number`	Retencao de backup. PITR fica ativo com valor maior que zero. Meta de RPO: 5 minutos.
`apply_immediately`	`bool`	Aplica mudancas na hora. true em dev, false em prod.
`kms_key_arn`	`string`	CMK do storage, do Performance Insights e do secret mestre.
`subnet_ids`	`list(string)`	Subnets de dados.
`vpc_id`	`string`	VPC do security group.
`secret_kms_key_arn`	`string`	CMK exclusiva do secret da senha mestre. Distinta da CMK do storage.

Nome	Tipo	Descricao
`manage_master_user_password`	`bool`	RDS cria o secret e rotaciona a senha. Permanece true para a task usar DB_SECRET_ARN.
`master_password_rotation_days`	`number`	Intervalo da rotacao do secret mestre. Dev 90, hml e prod 30. O RDS ja rotaciona; este valor ajusta a agenda.
`read_replica_count`	`number`	Replicas de leitura. Padrao 0. PostgreSQL com senha gerenciada pelo RDS nao aceita replica (limitacao da AWS).
`replica_instance_class`	`string`	Classe da replica. Nulo reutiliza instance_class.
`enable_rds_proxy`	`bool`	RDS Proxy na frente da instancia. Padrao false. Nao exige replica.
`monitoring_interval`	`number`	Enhanced Monitoring em segundos. 0 desliga. Prod usa 60.
`performance_insights_retention_days`	`number`	Retencao do Performance Insights. 7 dias esta no free tier.
`log_min_duration_statement_ms`	`number`	log_min_duration_statement em milissegundos. Consultas mais lentas vao para o log do PostgreSQL.
`statement_timeout_ms`	`number`	statement_timeout padrao do parameter group, em milissegundos.

Outputs

Nome	Descricao
`address`	Hostname da instancia primaria.
`endpoint`	Host que a aplicacao usa em DB_URL. O proxy, quando enable_rds_proxy; senao a instancia.
`reader_endpoint`	Host de DB_READER_URL. Nulo sem replica.
`client_security_group_id`	Security group que a task deve alcancar. O do proxy, se existir; senao o do banco.
`port`	Porta PostgreSQL.
`db_name`	Nome do database.
`master_user_secret_arn`	Secret gerenciado pelo RDS com username e password.
`security_group_id`	Security group do banco. O servico abre o ingress 5432.
`identifier`	Identificador da instancia, usado nos alarmes.

FONTE: INFRA/TERRAFORM/MODULES/ELASTICACHE/README.MD

elasticache

Valkey com TLS. Sem senha, porque o contrato nao tem REDIS_PASSWORD.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "elasticache" {
  source = "./modules/elasticache"
}
```

Inputs

Nome	Tipo	Descricao
`name`	`string`	replication_group_id. Maximo de 40 caracteres, sem underscore.
`description`	`string`	Descricao do replication group.
`engine_version`	`string`	Versao do Valkey. 8.x evita a licenca do Redis e atende o contrato REDIS_SSL.
`node_type`	`string`	Tipo do no. cache.t4g.micro em dev, cache.t4g.small em prod.
`num_cache_clusters`	`number`	Primario mais replicas. 1 em dev, 2 em prod.
`automatic_failover_enabled`	`bool`	Promove a replica se o primario cair. Exige num_cache_clusters >= 2.
`multi_az_enabled`	`bool`	Replica em outra AZ. Exige failover automatico.
`snapshot_retention_limit`	`number`	Dias de snapshot. 0 desliga.
`apply_immediately`	`bool`	Aplica mudancas na hora.
`kms_key_arn`	`string`	CMK da criptografia em repouso.
`subnet_ids`	`list(string)`	Subnets de dados. Pelo menos duas AZs, mesmo com um unico no.
`vpc_id`	`string`	VPC do security group.

Outputs

Nome	Descricao
`primary_endpoint`	Endpoint de escrita. REDIS_HOST.

Nome	Descricao
`port`	Porta. Com TLS continua 6379.
`security_group_id`	Security group do Redis. O consolidado abre o ingress.

FONTE: INFRA/TERRAFORM/MODULES/MESSAGING/README.MD

messaging

SNS, SQS e DLQ. Raw delivery e maxReceiveCount 5.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "messaging" {
  source = "./modules/messaging"
}
```

Inputs

Nome	Tipo	Descricao
`topic_name`	`string`	Nome do topico SNS. No ambiente local o nome e fixo; na AWS leva o prefixo do ambiente.
`queue_name`	`string`	Fila consumida pelo consolidado.
`dlq_name`	`string`	DLQ da fila do consolidado.
`kms_key_arn`	`string`	CMK de SNS e SQS.
`publisher_role_arn`	`string`	Task role de lancamentos. ARN montado pelo caller para nao criar ciclo com o modulo ECS.
`consumer_role_arn`	`string`	Task role de consolidado.
`max_receive_count`	`number`	Tentativas antes da DLQ.

Outputs

Nome	Descricao
`topic_arn`	ARN do topico de lancamentos. LANCAMENTOS_TOPIC_ARN.
`topic_name`	Nome do topico.
`queue_url`	URL da fila do consolidado. CONSOLIDADO_QUEUE_URL.
`queue_arn`	ARN da fila do consolidado.

Nome	Descricao
`queue_name`	Nome da fila, usado nos alarmes e no scaling.
`dlq_arn`	ARN da DLQ.
`dlq_name`	Nome da DLQ.
`dlq_url`	URL da DLQ.

FONTE: INFRA/TERRAFORM/MODULES/COGNITO/README.MD

cognito

User pool Essentials, dominio hospedado, resource server fluxo-caixa e Lambda de pre-token v2. A Lambda copia `comerciante_id` e acrescenta os escopos recebidos em `TOKEN_SCOPES`, para o `USER_PASSWORD_AUTH` passar no authorizer.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "cognito" {
  source = "./modules/cognito"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo do user pool, do client e da Lambda.
`deletion_protection`	`string`	ACTIVE em prod, INACTIVE em dev.
`log_retention_days`	`number`	Retencao do log da Lambda de pre-token.
`logs_kms_key_arn`	`string`	CMK do log group da Lambda.
`domain_prefix`	`string`	Prefixo do dominio hospedado. Nulo usa `<name_prefix>-<account_id>`.
`callback_urls`	`list(string)`	Callbacks do app client. O fluxo principal dos servicos e o password grant, mas o client deixa o authorization code preparado.
`logout_urls`	`list(string)`	URLs de logout do app client.

Outputs

Nome	Descricao
`user_pool_id`	ID do user pool.
`user_pool_arn`	ARN do user pool.
`user_pool_client_id`	App client publico do comerciante.
`issuer_uri`	JWT_ISSUER_URI. O prefixo de escopo na AWS e fluxo-caixa/.

Nome	Descricao
`resource_server_identifier`	Identificador do resource server.
`domain`	Prefixo do dominio hospedado.
`token_endpoint`	Token endpoint OAuth desse dominio.

FONTE: INFRA/TERRAFORM/MODULES/OBSERVABILITY/README.MD

observability

Logs, dashboard, alarmes e eventos de falha de rotacao.

terraform-docs nao estava instalado. Esta pagina lista as variaveis e os outputs do modulo.

Uso

```
module "observability" {
  source = "./modules/observability"
}
```

Inputs

Nome	Tipo	Descricao
`name_prefix`	`string`	Prefixo dos log groups, do dashboard e dos alarmes.
`region`	`string`	Regiao do dashboard.
`log_retention_days`	`number`	Retencao dos logs de aplicacao e de metricas EMF.
`logs_kms_key_arn`	`string`	CMK dos log groups.
`alerts_kms_key_arn`	`string`	CMK do topico de alertas. A key policy precisa autorizar o CloudWatch.
`alert_email`	`string`	E-mail inscrito no topico. Vazio nao cria inscricao. A AWS envia um e-mail de confirmacao.
`cluster_name`	`string`	Cluster ECS das metricas de CPU e memoria.
`alb_arn_suffix`	`string`	Sufixo do ALB.
`services`	`map(object({`	Servicos monitorados. O nome da task ECS precisa bater com ecs_service_name.
`queue_name`	`string`	Fila principal do consolidado.
`dlq_name`	`string`	DLQ do consolidado.
`queue_visible_threshold`	`number`	Alarme de profundidade da fila.
`queue_age_threshold_seconds`	`number`	Alarme da idade da mensagem mais antiga. Acima do SLO de 5s para nao disparar em atraso curto.
`rotation_secret_arns`	`map(string)`	Secrets de banco cuja falha de rotacao deve ir para o topico de alertas. O mapa entra na descricao da regra para o apply esperar o secret existir.

Nome	Tipo	Descricao
`rds_instances`	`map(object({`	Instancias RDS monitoradas.

Outputs

Nome	Descricao
`log_group_names`	Nomes dos log groups, incluindo metrics.
`log_group_arns`	ARNs dos log groups.
`alert_topic_arn`	Topico SNS dos alarmes.
`dashboard_name`	Nome do dashboard operacional.